

SPINKS (Spin Inversion Kinetics Suite): A Fully Automated Python-ORCA Framework for End-to-End Computation of Spin-Crossing Probabilities, Microcanonical and Canonical Reaction Rate Constants:

Supporting Information Part S2

Hamed Barzinmehr^a, Simon U. Okafor^a, Gabriele Pinto^a, and Darrin J. Bellert^{a*}

^aBaylor University, Dept of Chemistry and Biochemistry, One Bear Place #97348, Waco, TX 76798-7151, United States

*Email: Darrin_Bellert@baylor.edu

Table of Contents

Part I. User Guide	10
1. Software Overview	10
2. Software Requirements and Setup	11
2.1 Python environment	11
2.2 Required Python packages	11
2.3 ORCA quantum-chemistry package	12
2.4 Microsoft MPI for parallel execution on Windows (optional)	12
3. Running the SPINKS Workflow	13
3.1 Running SPINKS on a Local PC	14
3.1.1 User input	14
3.1.2 Review of the shared ORCA settings	18
3.1.3 Interactive editing during the workflow	22
3.1.4 Energy range and rate calculations	26
3.1.5 Output files, directory structure, and interactive energy analysis	28
3.2 Running SPINKS on an HPC Cluster	31
3.2.1 Cluster job monitoring, existing outputs, and resubmission	35
Part II. Detailed Computational Implementation of SPINKS	37
4. Step-by-Step Implementation of the SPINKS Workflow	37
4.1 Step 1: Minima Optimization, Interpolation Scan, and MECP Optimization	37
4.1.1 Workflow initialization	38
4.1.2 Collection of molecular and computational input	39
4.1.3 Project-directory organization	40
4.1.4 Construction and review of ORCA inputs	41

4.1.5 Optimization of the two spin-state minima.....	42
4.1.6 Automatic selection of the reactant reference minimum	44
4.1.7 Construction of the Cartesian interpolation	45
4.1.8 Single-point energies on both spin surfaces	45
4.1.9 Selection of the MECP starting geometry	46
4.1.10 Interpolation-energy visualization	47
4.1.11 Construction of the MECP input	48
4.1.12 Protected MECP optimization	48
4.1.13 Step 1 output variables	49
4.1.14 Files generated by step 1	50
4.1.15 Summary of the step 1 algorithm	51
4.2 Step 2: Orbital Preparation and SOC Calculation at the MECP	52
4.2.1 Initialization.....	52
4.2.2 High-spin ROHF orbital preparation	54
4.2.3 Automatic SOMO detection	55
4.2.4 UHF/UNO orbital preparation	55
4.2.5 Localized SOMO analysis	57
4.2.6 Selection of the orbital source	58
4.2.7 Automatic construction of the SOC input	59
4.2.8 Extraction of the SOC matrix.....	60
4.2.9 Construction of the M_s -resolved SOC matrix.....	60
4.2.10 Effective, intermediate, and M_s -specific SOCs	61
4.2.11 User override of the effective SOC.....	62
4.2.12 Variables exported by step 2.....	62
4.2.13 Summary of the step 2 algorithm	63
4.3 Step 3: MECP Energy Gradients	66
4.3.1 Import of quantities from the preceding steps	66
4.3.2 Preparation of the gradient-calculation method.....	68
4.3.3 Directory preparation	69
4.3.4 Construction of the ORCA Engrad inputs	70
4.3.5 Generation of the two spin-state jobs.....	71
4.3.6 Execution and protected input repair.....	71
4.3.7 Location and parsing of the ORCA .engrad files	73

4.3.8 Storage of the spin-state gradients	74
4.3.9 Gradient norms	75
4.3.10 Cartesian gradient-difference vector	76
4.3.11 Crossing-seam normal direction	76
4.3.12 Assignment of the gradient-gap magnitude	77
4.3.13 Files generated by step 3	77
4.3.14 Variables exported by step 3	79
4.3.15 Summary of the step 3 algorithm	80
4.4 Step 4: Gradient Analysis and Crossing Classification	82
4.4.1 Import of the two MECP gradients.....	83
4.4.2 Removal of stale projected-gradient variables	83
4.4.3 Surface-gradient-difference convention	84
4.4.4 Reverse surface-gradient convention.....	84
4.4.5 Gradient-difference magnitude	85
4.4.6 Individual gradient norms.....	86
4.4.7 Cartesian geometric-mean gradient diagnostic	86
4.4.8 Crossing classification	87
4.4.9 Effective-Hessian weighting parameter	88
4.4.10 Normalized gradient-difference direction	89
4.4.11 Reverse-gradient Cartesian direction	89
4.4.12 MECP-only workflow enforcement	90
4.4.13 Separation of step 4 and the final projected gradient mean.....	91
4.4.14 Files generated by step 4	91
4.4.15 Variables exported by step 4	93
4.4.16 Summary of the step 4 algorithm	93
4.5 Step 5: MECP Hessians and Rotational Constants	96
4.5.1 Import of quantities from the preceding steps	97
4.5.2 Energy-unit conversion	98
4.5.3 Reuse of reference-state data from step 1.....	98
4.5.4 Extraction of reference rotational constants and symmetry number	99
4.5.5 Extraction and selection of the reference frequency block	100
4.5.6 Independent calculation of the reference ZPE from frequencies	101
4.5.7 Preparation of the MECP frequency directory	101

4.5.8 Construction of the SurfCrossNumFreq inputs	102
4.5.9 Generation of both spin-surface orderings	103
4.5.10 Execution and protected repair of the MECP frequency jobs.....	104
4.5.11 Collection of the two Hessian files	106
4.5.12 Extraction of MECP rotational constants and symmetry number	107
4.5.13 Extraction of diagnostic PES2 frequencies.....	108
4.5.14 Electronic MECP barrier bookkeeping	108
4.5.15 Preparation of downstream variables	110
4.5.16 Distinction between reference frequencies and MECP diagnostic frequencies	111
4.5.17 Files generated by step 5	111
4.5.18 Variables exported by step 5	113
4.5.19 Summary of the step 5 algorithm	114
4.6 Step 6: Effective Hessian and Reduced Mass.....	116
4.6.1 Import of gradient and Hessian data.....	117
4.6.2 MECP geometry parsing and atomic-mass assignment.....	117
4.6.3 Validation of the gradient dimensions.....	118
4.6.4 Gradient-difference and gradient-sum vectors.....	119
4.6.5 Gradient-difference magnitude and normalized crossing direction.....	120
4.6.6 Projected gradient components and the final projected gradient mean	121
4.6.7 Direct Cartesian gradient-direction reduced mass	123
4.6.8 Crossing classification	123
4.6.9 ORCA Hessian parsing	124
4.6.10 Mass weighting of the Hessians.....	125
4.6.11 Construction of the peaked-crossing effective Hessian.....	125
4.6.12 Construction of the sloped-crossing effective Hessian.....	126
4.6.13 Selection of the active effective Hessian.....	126
4.6.14 Construction of the translational and rotational subspace	127
4.6.15 Generation of candidate reaction-coordinate seeds.....	128
4.6.16 Removal of rigid-body contamination from each seed	129
4.6.17 Projected zero-mode reaction-coordinate analysis	129
4.6.18 Direct reaction-coordinate reduced mass	130
4.6.19 Automatic selection of the final reaction coordinate.....	131
4.6.20 Distinction among the three reduced masses.....	132

4.6.21 Projection of the effective Hessian into the transverse subspace.....	133
4.6.22 Diagonalization of the projected effective Hessian	133
4.6.23 Conversion of effective-Hessian eigenvalues to frequencies	134
4.6.24 Expected number of effective-Hessian frequencies.....	135
4.6.25 Storage of the two original Hessians	135
4.6.26 Reaction-coordinate candidate diagnostics.....	136
4.6.27 Atomic contributions to the selected reaction coordinate	137
4.6.28 Files generated by step 6	137
4.6.29 Variables exported by step 6	138
4.6.30 Summary of the step 6 algorithm	139
4.7 Step 7: Reaction Coordinate Energy and Velocity Grid	143
4.7.1 Import of quantities from the preceding steps	144
4.7.2 Recovery of the reference zero-point energy	144
4.7.3 Validation of the effective-Hessian MECP frequencies	145
4.7.4 Effective-Hessian MECP zero-point energy	145
4.7.5 MECP–reference zero-point-energy difference	146
4.7.6 Final ZPE-corrected MECP barrier	146
4.7.7 Barrier-source labeling	147
4.7.8 User definition of the maximum excess energy	147
4.7.9 Definition of the common energy spacing	148
4.7.10 Construction of the total-energy grid	148
4.7.11 Identification of the MECP threshold bin	149
4.7.12 Reaction-coordinate excess-energy grid	150
4.7.13 Why the velocity is zero at and below the MECP	151
4.7.14 Conversion of excess energy to joules	152
4.7.15 Conversion of the reduced mass to kilograms	152
4.7.16 Reaction-coordinate velocity in meters per second.....	153
4.7.17 Velocity in bohr per second.....	153
4.7.18 Numerically safe velocity array.....	154
4.7.19 Common-grid consistency with later steps.....	154
4.7.20 Velocity-grid plot	156
4.7.21 Step 7 output directory	156
4.7.22 Energy and velocity summary file	157

4.7.23 Complete energy and velocity grid file	158
4.7.24 Variables exported by step 7	158
4.7.25 Summary of the step 7 algorithm	160
4.8 Step 8: Landau-Zener and Weak Coupling Probabilities	162
4.8.1 Import of active quantities	163
4.8.2 Reuse of the common Step 7 grid	164
4.8.3 Validation of the energy-grid dimensions	164
4.8.4 Verification of the uniform common energy grid	165
4.8.5 Verification of the MECP threshold bin	165
4.8.6 Active gradient quantities	166
4.8.7 Reduced-mass conversions	166
4.8.8 Independent velocity consistency check	167
4.8.9 Effective spin-orbit coupling	168
4.8.10 Conversion of the SOC to joules	168
4.8.11 Energy mask used for the LZ probability	168
4.8.12 Time derivative of the diabatic energy difference	169
4.8.13 Implemented Landau-Zener parameter	170
4.8.14 Implemented Landau-Zener probability	170
4.8.15 Energy dependence of the LZ probability	172
4.8.16 Conversion of SOC and excess energy to atomic units	172
4.8.17 Weak coupling prefactor	173
4.8.18 Weak coupling Airy scale	173
4.8.19 Evaluation of the Airy function	174
4.8.20 Subthreshold WC probabilities	175
4.8.21 Effective Landau-Zener probability	175
4.8.22 Effective weak coupling probability	176
4.8.23 Selection of the SOC matrix	176
4.8.24 Validation of the M_S dimensions	177
4.8.25 Individual SOC channel definition	177
4.8.26 Individual channel probability curves	178
4.8.27 No sum over M_S-specific channels	179
4.8.28 Dependence of individual curves on $ H_{SO} $	180
4.8.29 Grouping equal- $ H_{SO} $ channels for plotting	180

4.8.30 Interpretation of grouped channel labels.....	181
4.8.31 Grouping by M_S	181
4.8.32 Row-resolved intermediate SOC magnitude	182
4.8.33 Intermediate SOC for an M_S group	182
4.8.34 Intermediate probability curves	183
4.8.35 No weighted sum of intermediate curves.....	184
4.8.36 Plotting ranges	184
4.8.37 Landau–Zener probability plot	184
4.8.38 Weak coupling probability plot.....	185
4.8.39 Step 8 summary file	185
4.8.40 Effective probability grid file	186
4.8.41 Intermediate probability summary	187
4.8.42 M_S-resolved channel summary	187
4.8.43 Local and cluster output handling	188
4.8.44 Effective probability variables and compatibility aliases.....	189
4.8.45 Intermediate and channel-specific variables	190
4.8.46 Exported probability functions	190
4.8.47 Summary of the step 8 algorithm	190
4.9 Step 9: Microcanonical Rate Constants	193
4.9.1 Import of quantities from the preceding steps	193
4.9.2 Selection of the active ZPE-corrected MECP threshold	195
4.9.3 Construction of the common total-energy grid	198
4.9.4 Evaluation of the reactant and MECP rovibrational densities of states	200
4.9.5 Independent evaluation of the rovibrational densities of states.....	203
4.9.6 Evaluation of the Landau-Zener transition probability.....	206
4.9.7 Calculation of the effective number of crossing states and microcanonical rate constants ..	209
4.9.8 Calculation of the effective Landau-Zener and weak coupling microcanonical rate constants	211
4.9.9 Calculation of the intermediate microcanonical rate constants	214
4.9.10 Calculation of M _s -specific microcanonical rate constants.....	216
4.9.11 Construction of symmetry-collapsed M _s -specific rate curves.....	218
4.9.12 Generation of final rate arrays for canonical calculations	220
4.9.13 Summary of the step 9 algorithm	222

4.10 Step 10: Canonical Rate Constants	224
4.10.1 Import of step 9 microcanonical quantities	224
4.10.2 Construction of the canonical temperature grid	226
4.10.3 Canonical Boltzmann averaging	228
4.10.4 Evaluation of the effective canonical Landau-Zener and weak coupling rate constants	230
4.10.5 Evaluation of the intermediate canonical Landau-Zener and weak coupling rate constants	232
4.10.6 Evaluation of the symmetry-collapsed M_S -specific canonical rate constants	234
4.10.7 Generation of canonical rate plots	237
4.10.8 Export of canonical results and output files	239
4.10.9 Summary of the step 10 algorithm	240
4.11 Step 11: Conditional WC Threshold-Truncation Correction	242
4.11.1 Import of required quantities and conditional workflow setup	242
4.11.2 Reconstruction of the raw weak coupling probability	244
4.11.3 Application of the weak coupling threshold-truncation rule	246
4.11.4 Conditional evaluation of the threshold-corrected effective weak coupling rate	248
4.11.5 Correction of intermediate weak coupling probabilities and rates	250
4.11.6 Correction of symmetry-collapsed M_S -specific weak coupling channels	252
4.11.7 Recalculation of canonical weak coupling rate constants	254
4.11.8 Diagnostics, output files, plots, and exported variables	256
4.11.9 Summary of the step 11 algorithm	259
4.12 Step 12: Final Results Reporting, Diagnostics, and Optional NAST Interoperability Export	261
4.12.1 Initialization and helper functions	261
4.12.2 Automatic selection of the active weak coupling results	265
4.12.3 Reporting of canonical rate constants	267
4.12.4 Interactive microcanonical diagnostic	268
4.12.5 Construction of the final output directory	272
4.12.6 Collection of final physical quantities	273
4.12.7 Determination of global probability and rate extrema	277
4.12.8 Collection or reconstruction of intermediate spin-orbit couplings	279
4.12.9 Construction of the final SPINKS report	281
4.12.10 Validation of quantities required for NAST input	285
4.12.11 Construction of the NAST input template	285
4.12.12 Generation of the two NAST barrier conventions	288

4.12.13 Optional cluster upload and global export	289
4.12.14 Summary of the step 12 algorithm	290
References	293

Part I. User Guide

1. Software Overview

SPINKS is distributed in two implementations: a Local PC version intended for execution on a personal computer and an HPC cluster version intended for use with a remote high-performance computing environment. Both implementations use the same scientific workflow, ORCA input-construction logic, electronic-structure procedures, spin-crossing equations, and kinetic calculations; they differ in how ORCA jobs are executed and how files are managed. The Local PC version launches ORCA directly on the local computer, whereas the HPC cluster version connects to a remote system by SSH/SFTP, generates batch-submission scripts, submits calculations to the queueing system, monitors their status, and retrieves the resulting files. Very small numerical differences between Local PC and HPC results can nevertheless arise from the underlying ORCA execution environment, including processor architecture, parallel execution, MPI or numerical-library implementation, and finite-precision operation ordering; these effects can lead to slightly different optimization trajectories and final converged structures. The current SPINKS implementation was developed and tested on Windows using Python 3.13 in the Spyder integrated development environment and ORCA 5.0.4. The source code uses platform-independent Python path and subprocess handling and is designed to support execution on Linux and macOS; however, those local operating systems were not directly tested in the present work. The complete source code for both implementations is freely available through the official SPINKS GitHub repository:

https://github.com/Hamed-Barzinmehr/SPINKS

2. Software Requirements and Setup

2.1 Python environment

SPINKS was developed and tested using the Spyder integrated development environment distributed through the Anaconda Python distribution. The latest Anaconda distribution can be downloaded from:

```
https://www.anaconda.com/download
```

Because the workflow is organized as sequential Spyder code cells that preserve variables between execution steps, users are strongly encouraged to run SPINKS in Spyder. After installing Anaconda, launch the **Anaconda Prompt** and install **Spyder** if it is not already included with your installation. Spyder may also be installed from the Anaconda Prompt using:

```
conda install spyder -y
```

The workflow described in this work was developed and tested using Python 3.13 in the Spyder IDE distributed with Anaconda. A dedicated SPINKS environment may be created by pasting the following commands in the Anaconda Prompt:

```
conda create -n SPINKS python=3.13 -y  
conda activate SPINKS
```

2.2 Required Python packages

The SPINKS workflow requires the following Python packages:

- **NumPy** (numpy)
- **SciPy** (scipy)
- **Matplotlib** (matplotlib)
- **Spyder** (spyder)
- **Paramiko** (paramiko, required only for the cluster version)

Install the required packages by pasting the following commands in Anaconda Prompt:

```
conda install -n SPINKS numpy scipy matplotlib spyder -y  
conda install -n SPINKS -c conda-forge paramiko -y
```

The **Paramiko** package is required only for the cluster version. Installing it for the Local PC version is harmless and does not affect the operation of the workflow.

2.3 ORCA quantum-chemistry package

SPINKS interfaces directly with the **ORCA** quantum-chemistry package¹ to perform all electronic-structure calculations. Consequently, a working ORCA installation is required before executing the SPINKS workflow.

For the **PC version**, ORCA must be installed locally on the user's computer.

ORCA is freely available for academic use. It can be obtained by registering through the official ORCA website:

<https://orcaforum.kofo.mpg.de/>

After creating an account and agreeing to the license conditions, users may download the appropriate ORCA version for their operating system from the ORCA download portal. ORCA distributions are available for Windows, Linux, and macOS. For local execution of SPINKS, users must install the ORCA distribution appropriate for their operating system. The present SPINKS implementation was validated with ORCA on Windows; use with Linux or macOS requires a compatible ORCA installation and may require operating-system-specific configuration of executable permissions, environment variables, and the MPI runtime.

For the **HPC cluster version**, ORCA must be installed on the remote high-performance computing (HPC) cluster. Before using the cluster workflow, users are encouraged to verify whether ORCA is already available on their system. One convenient way is to search for the ORCA executable from the Linux command line using:

`which orca`

If the command returns the location of the ORCA executable (for example, `/usr/bin/orca`), ORCA is available in the user's environment. If no path is returned, ORCA may not be installed or may not be loaded into the current environment, and the user should consult the HPC administrator or load the appropriate software module.

2.4 Microsoft MPI for parallel execution on Windows (optional)

Microsoft MPI (MS-MPI) is an optional requirement for the **Windows PC version** of SPINKS. It allows ORCA calculations executed locally on a Windows computer to use more than one processor.

Without Microsoft MPI, the PC version of SPINKS can still be used, but ORCA calculations must be performed using a single processor. When Microsoft MPI is installed and available, users may specify multiple processors and thereby reduce the execution time of suitable ORCA calculations.

Microsoft MPI can be downloaded from the official Microsoft Download Center:

<https://www.microsoft.com/en-us/download/details.aspx?id=105289>

After installation, the mpiexec.exe executable is typically located in:

[C:\Program Files\Microsoft MPI\Bin\mpiexec.exe](#)

Microsoft MPI is not required for the HPC cluster version because parallel execution on the cluster is handled through the MPI implementation provided by the HPC system.

3. Running the SPINKS Workflow

Once the required software has been installed and configured as described in Section 2, the SPINKS workflow is ready to be executed. SPINKS is distributed as two independent Python scripts, both of which are freely available from the official GitHub repository:

<https://github.com/Hamed-Barzinmehr/SPINKS>

- **SPINKS_PC_version.py** – for local execution using a locally installed ORCA distribution. The implementation was developed and tested on Windows and is designed to support Linux and macOS through platform-independent path handling and automatic detection of the corresponding ORCA executable. Linux and macOS execution has not yet been directly validated by the authors.
- **SPINKS_HPC_Cluster_version.py** – for execution on a remote HPC cluster through SSH/SFTP. The local client implementation was developed and tested on Windows and is designed to support local computers running Linux or macOS; however, these local operating systems have not yet been directly validated by the authors.

After either script is launched, SPINKS guides the user through the entire workflow using a sequence of interactive prompts displayed in the Spyder console. At each stage, the program requests the information required for the next calculation and provides clear instructions regarding the available options, allowing the workflow to be completed without prior knowledge of the internal implementation. The following sections provide a detailed

description of each stage of the interactive workflow, the corresponding user prompts, and the calculations performed by SPINKS.

3.1 Running SPINKS on a Local PC

The instructions and screenshots in this section describe the Windows environment used during development and validation. The local implementation also includes operating-system-aware detection of the ORCA executable and platform-independent file handling intended to support Linux and macOS. Because complete SPINKS calculations were not performed on those systems during the present validation, Linux and macOS users may need to adapt environment activation, ORCA installation paths, executable permissions, and MPI configuration to their local setup.

On Windows, open the Anaconda Prompt. On Linux or macOS, open a terminal in which Conda has been initialized. Activate the SPINKS Conda environment by typing:

```
conda activate SPINKS
```

Next, launch the Spyder integrated development environment by typing:

```
spyder
```

Once Spyder has opened, select **File** → **Open** and open the following script:

```
SPINKS_PC_version.py
```

Now the user may execute the script from the beginning. The program will then guide the user through a sequence of prompts to collect the information required to initiate the automated workflow.

3.1.1 User input

The first prompt requests the ORCA installation/extraction directory. This should normally be the directory containing the ORCA executable. The Local PC code also accepts a direct path to an executable named orca or orca.exe. If a parent directory is supplied and it contains a single ORCA executable in a subdirectory, SPINKS can locate that executable recursively. For example, if ORCA was extracted to:

```
C:\ORCA
```

then the user should enter:

```
C:\ORCA
```

On Linux or macOS, the user should instead provide the directory containing the local orca executable. For example:

```
Linux: /home/<username>/orca  
macOS: /Users/<username>/Library/Orca
```

Next, the program asks the user to provide a **project name**. The chosen name is automatically appended to the prefix **Spin_Inversion_** to create the project directory. For example, if the user enters:

```
Iron_cluster
```

the project directory created by SPINKS will be:

```
Spin_Inversion_Iron_cluster
```

On the validated Windows configuration used in this work, the complete project directory would be:

```
C:\ORCA\Spin_Inversion_Iron_cluster
```

On Linux or macOS, the corresponding project directory is created inside the ORCA installation directory supplied by the user. This behavior is implemented using platform-independent Python path handling but has not been directly validated by the authors on those operating systems.

All input files, output files, intermediate calculations, figures, and final results generated during the workflow are organized automatically within this project directory.

The program then prompts the user to specify the two spin multiplicities to be investigated. The multiplicities should be entered as two comma-separated integers. For example:

```
1,3
```

requests calculations for the singlet and triplet spin states.

The multiplicity convention adopted by SPINKS follows the standard quantum-chemical definition:

Multiplicity	Spin state
1	Singlet
2	Doublet
3	Triplet
4	Quartet
5	Quintet
6	Sextet
7	Septet
8	Octet
...	Higher multiplicities

Next, the user is asked to specify the molecular charge.

It is important to emphasize that not every combination of charge and spin multiplicity corresponds to a physically meaningful electronic state. The user is therefore responsible for selecting chemically appropriate values. Incorrect charge or multiplicity assignments may cause subsequent ORCA calculations to terminate unsuccessfully.

The program then requests the desired **electronic-structure method (level of theory)**. Because SPINKS interfaces directly with ORCA, any electronic method supported by ORCA may be specified, including density-functional methods, Hartree–Fock, and correlated wavefunction methods. Examples include:

- B3LYP
- PBE
- HF
- MP2

The default electronic method is PBE. Therefore, pressing Enter without typing a value automatically selects PBE.

The user is subsequently asked to specify the basis set. Any basis set supported by ORCA may be used, for example:

- 6-31G
- 6-31G(d)
- def2-SVP
- def2-TZVP
- cc-pVTZ

The default basis set is def2-TZVP. Pressing Enter without entering a value automatically selects this basis.

The program next requests the **number of processors** and the **memory per processor** to be allocated for ORCA calculations. On Windows systems, multiple processors may be used only if **Microsoft MPI** has been installed as described in Section 2.4. When Microsoft MPI is detected, the requested number of processors is passed directly to ORCA. If Microsoft MPI is not available, SPINKS automatically forces the calculation to use a single processor, regardless of the number entered by the user. The memory requested at this stage corresponds to the ORCA %maxcore value specified for each processor.

Finally, the program requests the **initial Cartesian geometry** of the molecular system. This is the only molecular input that must be supplied manually by the user. SPINKS displays the following example:

```
atom_label  x_coordinate  y_coordinate  z_coordinate
Example:
O  0.0000000000000000  0.00000000818129  0.11690363084563
H  0.0000000000000000  0.76528672468584  -0.46747581229803
H  0.0000000000000000 -0.76528673286712  -0.46747581854760

Press ENTER on an empty line when the geometry is complete.
```

The user should simply paste the complete Cartesian coordinate block and then press **Enter** on an empty line to indicate that the geometry is complete. Depending on the terminal or Spyder console, the pasted coordinates may not remain perfectly aligned in columns. This formatting change is purely cosmetic and has no effect on the execution of the program. The coordinates should therefore be pasted directly without attempting to manually realign them.

The initial geometry may be generated using molecular editors such as **Avogadro**, **GaussView**, or similar software. A fully optimized structure is **not** required. An approximate chemically reasonable geometry is sufficient because SPINKS automatically performs all required geometry

optimizations, interpolation calculations, minimum-energy crossing-point searches, frequency calculations, and subsequent electronic-structure calculations throughout the workflow.

3.1.2 Review of the shared ORCA settings

After the molecular geometry and the basic calculation parameters have been entered, SPINKS automatically generates the ORCA input files required for optimization and vibrational-frequency analysis of the two requested spin-state minima. Before any calculations are run, the complete inputs for both multiplicities are displayed in the Spyder console, and the user is asked whether the shared ORCA settings should be edited.

An example generated input for a singlet minimum is shown below:

Complete generated minimum input | Mn_by_singlet_min.inp

! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq

%pal nprocs 12 end
%maxcore 8000

%scf
MaxIter 1500
SOSCFStart 0.01
LevelShift 0.5
end

%output
PrintLevel 3
End

* xyz 0 1
C -3.20587267 0.34191104 -0.13279854
C -1.09651374 1.22323127 -0.59633772
C -1.52218711 2.49658112 -0.17654073
C -2.84503891 2.67749915 0.26178877
C -3.70673693 1.57500652 0.28606120
C -3.98685429 -0.97151299 -0.12878342
C -3.76823299 -3.24253787 -0.61535409
C -5.09006267 -3.47359606 -0.19085037
C -5.87057810 -2.39848438 0.26626241
C -5.30707130 -1.11885242 0.29861025
H -0.09177117 1.07410194 -0.93288653
H -0.84262804 3.32262814 -0.19206589
H -3.18719275 3.64270672 0.57298589
H -4.71897750 1.67395907 0.61849059
H -3.16456312 -4.05311650 -0.96744128
H -5.49883731 -4.46180399 -0.21651210
H -6.87946876 -2.55603710 0.58558413
H -5.87178134 -0.27793621 0.64254377
N -3.27212686 -1.99876936 -0.57741987

N	-1.96036161	0.19967109	-0.57427770
Mn	-1.54166454	-1.54008291	-1.14614303
C	0.17668970	-0.95226802	-1.82635724
C	-1.04823068	-3.37946219	-1.51176563
C	-0.82807151	-1.80542306	0.63583876
Br	-2.39248738	-1.22371848	-3.27082327
O	-0.40587730	-1.96457005	1.65865916
O	1.16574646	-0.61677431	-2.21580124
O	-0.76681196	-4.43787311	-1.73578431
*			

The corresponding input for the second multiplicity is generated using the same computational settings but the appropriate spin multiplicity. For example, the triplet input contains:

 Complete generated minimum input | Mn_by_triplet_min.inp

! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq

%pal nprocs 12 end
 %maxcore 8000

%scf
 MaxIter 1500
 SOSCFstart 0.01
 LevelShift 0.5
 end

%output
 PrintLevel 3
 end

* xyz 0 3

C	-3.20587267	0.34191104	-0.13279854
C	-1.09651374	1.22323127	-0.59633772
C	-1.52218711	2.49658112	-0.17654073
C	-2.84503891	2.67749915	0.26178877
C	-3.70673693	1.57500652	0.28606120
C	-3.98685429	-0.97151299	-0.12878342
C	-3.76823299	-3.24253787	-0.61535409
C	-5.09006267	-3.47359606	-0.19085037
C	-5.87057810	-2.39848438	0.26626241
C	-5.30707130	-1.11885242	0.29861025
H	-0.09177117	1.07410194	-0.93288653
H	-0.84262804	3.32262814	-0.19206589
H	-3.18719275	3.64270672	0.57298589
H	-4.71897750	1.67395907	0.61849059
H	-3.16456312	-4.05311650	-0.96744128
H	-5.49883731	-4.46180399	-0.21651210
H	-6.87946876	-2.55603710	0.58558413
H	-5.87178134	-0.27793621	0.64254377

N	-3.27212686	-1.99876936	-0.57741987
N	-1.96036161	0.19967109	-0.57427770
Mn	-1.54166454	-1.54008291	-1.14614303
C	0.17668970	-0.95226802	-1.82635724
C	-1.04823068	-3.37946219	-1.51176563
C	-0.82807151	-1.80542306	0.63583876
Br	-2.39248738	-1.22371848	-3.27082327
O	-0.40587730	-1.96457005	1.65865916
O	1.16574646	-0.61677431	-2.21580124
O	-0.76681196	-4.43787311	-1.73578431
*			

After displaying both inputs, SPINKS presents the following prompt:

Edit the shared ORCA settings block before minimum runs? (Y/N, default N):

If the user answers N, or simply presses Enter, SPINKS proceeds with the generated inputs. The default DFT SCF block uses MaxIter 1500, SOSCFStart 0.01, and LevelShift 0.5. The method, basis set, number of processors, memory allocation, molecular charge, spin multiplicities, and Cartesian geometry are taken from the values supplied earlier in Step 1.

If the user answers Y, SPINKS displays the shared pre-geometry portion of the ORCA input and allows it to be modified before the calculations begin. Only the section preceding the * xyz line is editable at this stage. In an ORCA input, the * xyz line marks the beginning of the molecular charge, spin multiplicity, and Cartesian-coordinate block. The molecular charge, spin multiplicities, and Cartesian geometry are protected because they were defined earlier and must remain consistent throughout the workflow.

The editing stage is intended for users who wish to customize the electronic-structure calculations while maintaining consistent computational settings. For example, the user may:

- modify or remove the default %scf block;
- change SCF convergence parameters such as MaxIter, SOSCFStart, or LevelShift;
- add implicit-solvent settings such as %cpcm or SMD;
- add dispersion corrections or other ORCA keywords to the ! line;
- introduce custom basis-set or effective-core-potential definitions through a %basis block;
- add integration-grid, method, output, or other transferable ORCA blocks.

For example, the default header:

! B3LYP def2-TZVP TightSCF SlowConv Opt Freq

may be changed to:

```
! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq
```

to include a dispersion correction. Similarly, an implicit-solvent treatment may be introduced using:

```
%cpcm  
smd true  
SMDsolvent "CHLOROFORM"  
End
```

A custom basis set or effective-core potential may be added, for example:

```
%basis  
NewECP Mn "SDD" end  
End
```

Any changes made to the reviewed shared settings are automatically propagated to subsequent DFT jobs. SPINKS adapts only the job-specific keyword on the ! line while reusing the reviewed transferable percent blocks. The reviewed settings are therefore used consistently for the following calculations. If an individual ORCA job later fails, the user may edit that failed job during the repair cycle; such a failure-specific edit applies only to that job and does not silently redefine the shared settings for the other jobs.

- spin-state minimum optimizations and frequency calculations;
- interpolation single-point calculations;
- minimum-energy crossing-point optimization;
- energy-gradient calculations; and
- numerical-frequency and Hessian calculations.

The SOC calculation is treated separately because it uses a CASSCF-based input with its own job-specific settings and output requirements. The transferable electronic-structure settings are incorporated where appropriate, but the complete SOC input is displayed and reviewed independently, as described in Section 3.1.3.

To preserve the integrity of the automated workflow, the Opt and Freq keywords cannot be removed from the minimum inputs because they are required for geometry optimization and vibrational-frequency analysis of the two spin-state minima. The optimized geometries are

used in the subsequent interpolation and MECP searches, while the calculated frequencies provide the zero-point energies required to determine the ZPE-corrected MECP barrier height.

3.1.3 Interactive editing during the workflow

Once the shared ORCA settings have been accepted or modified, SPINKS automatically performs the optimizations of the two spin-state minima, constructs the interpolation pathway between them, identifies the interpolation geometry with the smallest spin-state energy gap, performs the MECP optimization, and extracts the optimized MECP geometry.

If an ORCA calculation does not terminate normally during the minimum optimizations, interpolation calculations, MECP optimization, orbital-preparation calculations, SOC calculation, energy-gradient calculations, or numerical-frequency calculations, SPINKS pauses the Local PC workflow, prints the final portion of the ORCA output, and asks whether the corresponding input should be edited and rerun. Failed-attempt files are archived before the corrected job is executed again.

For a general failed ORCA calculation, the program displays a message of the following form:

```
=====
ORCA INPUT REVISION REQUIRED
=====
WARNING: ORCA did not terminate normally.
Failed calculation: <calculation label>
```

Current input is shown below:

Paste the corrected input and finish with EOF.

The current ORCA input is then printed in the console. The user may paste a corrected version and terminate the edited input with a line containing only:

EOF

For interpolation calculations, only the pre-geometry ORCA settings are editable because the interpolation geometry is generated automatically. SPINKS displays:

```
Only the pre-geometry ORCA settings block is editable for interpolation jobs.
The geometry, charge, and multiplicity are protected because interpolation
uses automatically generated geometries.

Current editable block:

! B3LYP D3 def2-TZVP TightSCF SlowConv

%pal nprocs 12 end
```

```
%maxcore 8000
```

```
%scf
```

```
MaxIter 1500
```

```
SOSCFStart 0.01
```

```
LevelShift 0.5
```

```
end
```

```
%output
```

```
PrintLevel 3
```

```
end
```

Modify convergence/header settings only.

Paste the corrected pre-geometry block and finish with EOF.

This allows the user to change convergence settings, SCF parameters, grid settings, or other ORCA options without altering the automatically generated interpolation geometry.

If a revised input violates one of the workflow requirements, SPINKS rejects it and displays:

INPUT VALIDATION FAILED:

<description of the protected setting that was changed>

Please edit again. Finish with EOF.

During MECP optimization, the method, basis set, molecular charge, multiplicities, generated MECP starting geometry, %mecp block, and required SurfCrossOpt keyword are protected. These quantities cannot be removed or altered because they define the crossing-point calculation.

After successful optimization of the MECP, SPINKS performs high-spin ROHF and UHF/UNO orbital analyses and allows the user to choose the orbital reference used to construct the SOC input. The program displays the available options and then requests:

Before the orbital-source choice is requested, the ROHF output is parsed for singly occupied orbitals with OCC approximately equal to 1.0000, while the UHF/UNO output is parsed for natural-orbital occupations. UNO orbitals with $0.02 < \text{occupation} < 1.98$ are proposed as the UHF/UNO active space. SPINKS also attempts an orca_loc localized-SOMO diagnostic and reports the detected transition metal together with a diagnostic metal-population threshold of 0.93. This localization analysis is diagnostic only: if orca_loc fails, Step 2 continues using the ROHF SOMO and UHF/UNO information already obtained, and the user still makes the final ROHF or UHF/UNO orbital-source choice.

Use which orbitals for the generated SOC input?

Enter ROHF or UHF (default ROHF):

ROHF is the recommended default for most systems, whereas UHF/UNO is provided as an advanced option for difficult open-shell systems.

If a completed SOC output file is already present when Step 2 is entered, SPINKS reuses that output directly, skips the ROHF/UHF orbital-source question and SOC rerun, and proceeds to parse the existing SOC result.

SPINKS then generates the CASSCF SOC input and displays it for optional editing. The corresponding prompt has the following form:

```
===== GENERATED Mn_by_SOC.inp =====  
  
! ROHF def2-TZVP TightSCF SlowConv  
  
%pal nprocs 12 end  
  
%maxcore 8000  
  
%scf  
  MaxIter 1500  
  SOSCFStart 0.01  
  LevelShift 0.5  
end  
  
%rel  
  SOType 3  
  SOCFlags 1,4,3,0  
  SOCMaxCenter 4  
end  
  
%casscf  
  maxiter 1  
  mult 1,3  
  nroots 1,1  
  nel 2  
  nOrb 2  
  
  rel  
  dosoc true  
  dosscc false  
  PrintLevel 3  
  TPrint 0.01  
  end  
end  
  
%output  
  PrintLevel 3  
end  
  
* xyz 0 3  
C   -3.169581589396   0.319564681057  -0.090043621650
```


C	-1.154250839700	1.338323631375	-0.606033868267
C	-1.634613314236	2.584679389638	-0.248831796464
C	-2.942986431434	2.688461536098	0.208458567920
C	-3.715177775868	1.543018336537	0.289409603261
C	-3.901711002999	-0.949268926711	-0.059039276249
C	-3.787575531764	-3.224115865901	-0.488578734228
C	-5.096758283324	-3.417407196690	-0.092248957324
C	-5.832156481102	-2.317110358155	0.334978063424
C	-5.225712739628	-1.074554817138	0.352765447572
H	-0.145890512990	1.215367003953	-0.972730826814
H	-0.995883250846	3.451923818270	-0.336987810927
H	-3.354551306477	3.647316895030	0.493486913722
H	-4.736101148133	1.599705482476	0.635594668079
H	-3.185425954029	-4.049121584614	-0.839145348221
H	-5.528315464363	-4.407440120852	-0.124773426946
H	-6.862138047075	-2.427980561706	0.646199471103
H	-5.778507689104	-0.206397112976	0.678544845337
N	-3.195538041581	-2.022969256159	-0.479446672524
N	-1.895776265226	0.227623784656	-0.532362588631
Mn	-1.354744300958	-1.617281089802	-1.268305531258
C	0.360949696042	-1.189370574908	-1.899874750833
C	-0.874853230279	-3.316707766472	-1.830258914102
C	-0.640950296555	-1.928524679190	1.123775383562
Br	-2.519316775345	-0.943404656145	-3.444165874538
O	-1.246722227647	-2.486046094930	1.904505754869
O	1.399120475295	-0.953105540011	-2.313725258284
O	-0.512396051277	-4.330701306730	-2.222283491588

*

=====
 Edit Mn_by_SOC.inp before local ORCA run? (Y/N, default N):

If the user answers N, the generated SOC input is run without modification. If the user answers Y, SPINKS displays:

Paste the full modified text below.
 Finish with a line containing only EOF.

Unlike the protected DFT repair paths, the generated SOC input is intentionally fully editable before the local ORCA run and again if the SOC job fails. The user may modify the active space, orbital reference, %casscf settings, number of roots, convergence parameters, relativistic settings, the ! line, or additional ORCA blocks. SPINKS does not impose the protected-input validation rules on this SOC input. For a scientifically consistent SPINKS calculation, however, the user should normally retain the intended molecular charge, spin multiplicities, MECP geometry, and compatible basis-set treatment unless a deliberate alternative SOC protocol is being tested.

After the ORCA SOC matrix has been parsed, SPINKS constructs the M_S -resolved complex SOC matrix and derives the effective, intermediate, and M_S -specific SOC quantities used by the later probability and rate calculations. It also reports the diagnostic full-matrix norm and explains the ORCA spin-orbit operator convention used in the calculation. For example:

Diagnostic full-matrix norm:

H_SO_full_matrix_norm_cm = 28.923281 cm⁻¹

=====

SOC OPERATOR CONVENTION

=====

The ORCA SOC value printed above is obtained with the ORCA mean-field spin-orbit treatment requested by the %rel block, here SOCType 3 with SOCFlags 1,4,3,0 and SOCMaxCenter 4.

The exact Breit-Pauli spin-orbit operator contains one-electron and two-electron terms. ORCA does not evaluate the full Breit-Pauli operator directly in this workflow; instead, it uses an effective one-electron mean-field approximation that includes the dominant Coulomb and exchange contributions to the two-electron SOC operator.

Therefore, ORCA and GAMESS SOC values may differ if GAMESS is using a full Breit-Pauli treatment or a different two-electron SOC convention.

Use a user-supplied SOC value instead of the ORCA effective SOC?
(Y/N, default N):

The user may therefore retain the effective SOC calculated by ORCA or replace it with a non-negative externally calculated SOC value. In this effective-SOC-only mode, SPINKS calculates only the effective probability and rate curves; intermediate and M_S -specific results are not generated.

The subsequent energy-gradient and numerical-frequency calculations are generated automatically from the reviewed Step 1 DFT settings and are not displayed for optional editing before their initial execution. If an Engrad job fails, repair validation preserves the Engrad keyword, method, basis set, molecular charge, multiplicity, and MECP geometry. If a SurfCrossNumFreq job fails, SPINKS additionally preserves the required SurfCrossNumFreq keyword and the appropriate %mecp multiplicity block.

3.1.4 Energy range and rate calculations

Following completion of the electronic-structure calculations and construction of the effective-Hessian reaction coordinate, SPINKS proceeds to the nonadiabatic statistical analysis.

The program first asks the user to specify the maximum reaction-coordinate energy above the ZPE-corrected MECP barrier. The following message is displayed:

=====

COMMON TOTAL-ENERGY GRID

=====

The entered value is the maximum reaction-coordinate energy above the ZPE-corrected MECP barrier.

Step 7 converts this value into the common 1 cm⁻¹ total-energy grid used by Steps 8 and 9.

Maximum reaction-coordinate energy above MECP, cm⁻¹, e.g. 5000:

The entered value does not represent the total energy relative to the reference minimum. Instead, it specifies how far the energy grid should extend beyond the ZPE-corrected MECP barrier.

For example, if the ZPE-corrected MECP barrier is:

10663.867 cm⁻¹

and the user enters:

5000

the total-energy grid extends from 0 cm⁻¹, relative to the selected reference minimum, to approximately:

15663.867 cm⁻¹

The total-energy grid is generated using a spacing of 1 cm⁻¹. SPINKS then calculates the reaction-coordinate kinetic energy and velocity at each accessible energy, followed by the Landau-Zener and weak coupling spin-crossing probabilities and the corresponding microcanonical rate constants.

After completion of the microcanonical calculations, SPINKS requests the temperature range for canonical averaging. The program displays:

CANONICAL TEMPERATURE GRID

Canonical averaging formula:

$$k(T) = \frac{\sum k(E) \rho_R(E) \exp[-E/(kBT)] \Delta E}{\sum \rho_R(E) \exp[-E/(kBT)] \Delta E}$$

Energies are relative to the reactant/reference minimum in cm^{-1} .

Minimum temperature for canonical rates, K, e.g. 100:

Maximum temperature for canonical rates, K, e.g. 1000:

Number of temperature points, e.g. 100:

The user specifies the minimum temperature, maximum temperature, and number of temperature points. SPINKS then generates an evenly spaced temperature grid including both endpoints.

For example, the entries

Minimum temperature for canonical rates, K, e.g. 100: 300

Maximum temperature for canonical rates, K, e.g. 1000: 350

Number of temperature points, e.g. 100: 6

produce the temperature grid:

300, 310, 320, 330, 340, and 350 K

The canonical Landau-Zener and weak coupling rate constants are subsequently evaluated at each of these temperatures using the previously calculated microcanonical rate constants and reference-state densities of states.

3.1.5 Output files, directory structure, and interactive energy analysis

Throughout execution, SPINKS automatically organizes intermediate and final results into a structured directory hierarchy within the project directory. The principal folders are listed below. The Step 11 folder `WC_threshold_truncation_correction` is created only when the raw WC probability reaches or exceeds unity and the threshold-truncation procedure is therefore required.

- Minima;
- Interpolation_scan;
- MECP;

- SOC;
- Engrad;
- Frequencies;
- MECP_gradient_definitions;
- Effective_Hessian_RC;
- RC_energy_velocity_grid;
- LZ_WC_probabilities;
- Microcanonical_rates;
- Canonical_rates;
- WC_threshold_truncation_correction;
- final_output_and_interoperability_exports.

These directories contain the ORCA input and output files, extracted geometries, Hessians, gradients, SOC matrices, intermediate numerical data, textual summaries, energy and velocity grids, probability tables, rate-constant tables, and publication-quality figures generated during the workflow.

After all probability and rate calculations have been completed, SPINKS reports the active ZPE-corrected MECP barrier and starts an interactive microcanonical diagnostic. For example:

```

===== INTERACTIVE MICROCANONICAL DIAGNOSTIC =====

Active MECP barrier: 10663.867220 cm-1

Enter an energy in cm-1 to print LZ/WC probabilities and rates.
Type 'stop' to finish.

Energy in cm-1, or 'stop':
The energy entered at this stage is the total energy relative to the reference minimum, which is
assigned an energy of 0 cm-1. The user may enter any energy from 0 cm-1 to the maximum
value included in the common total-energy grid.
For example, the entry
Energy in cm-1, or 'stop': 11663.867
is mapped to the nearest 1 cm-1 energy bin, and SPINKS displays:
===== MICROCANONICAL COMPARISON AT 11664.000 cm-1 =====

Grid energy           = 11664.000 cm-1
Requested energy      = 11663.867 cm-1
Energy index          = 11664
Crossing-energy bin index = 10664
MECP                  = 10663.867220 cm-1

```

```

WC source                = Original clipped WC

Density of states:
Reactant state density    = 5.984835003877828e+03
Shifted crossing state density = 1.342193005282502e+02
Internal crossing state density = 1.342193005282502e+02

Effective probabilities:
P_LZ                      = 5.335188251202716e-03
P_WC                      = 6.514828516034694e-03

Effective cumulative states:
N_LZ                      = 3.470165216947797e-03
N_WC                      = 3.688691535043188e-03

Microcanonical rates:
k_LZ                      = 3.815074311094852e+09 s^-1
k_WC                      = 4.055320550205378e+09 s^-1
log10(k_LZ)               = 9.581503002
log10(k_WC)               = 9.608025188

```

In this output, the crossing-energy bin index identifies the discrete total-energy-grid bin corresponding to the MECP threshold. The reactant state density is evaluated on the common total-energy grid, the shifted crossing state density is the crossing-region density mapped onto that grid, and the internal crossing state density is expressed relative to the crossing threshold.

The diagnostic identifies the requested energy, the actual discrete energy bin used by the calculation, the MECP barrier, the reactant and crossing-region densities of states, the effective Landau-Zener and weak coupling probabilities, the corresponding effective numbers of states, and the resulting microcanonical rate constants.

Energies below the MECP may also be entered because the total-energy reference is the selected reference minimum, not the crossing point. In this region the LZ probability and LZ microcanonical rate are zero because the classical crossing coordinate is not energetically accessible, whereas the WC probability and WC rate may remain finite through the Airy-function tunneling treatment. The internal crossing-state density is reported only for positive internal crossing energies and may therefore appear as nan below or at the crossing threshold.

After displaying the requested data, SPINKS presents the energy prompt again:

```
Energy in cm^-1, or 'stop':
```

The user may repeat the analysis for as many energies as desired.

Entering:

```
stop
```

terminates the interactive diagnostic and causes SPINKS to proceed with generation of the final output files.

After completion of the workflow, SPINKS creates the directory:

final_output_and_interoperability_exports

This directory contains the comprehensive FINAL_OUTPUT.txt report and two automatically generated NAST-compatible interoperability inputs: NAST_zpe1_electronic_barrier.inp and NAST_zpe0_ZPE_corrected_barrier.inp. The final report records the M_s -specific, intermediate, and effective SOC information available to the workflow; gradient and reduced-mass quantities; rotational and vibrational data; electronic and ZPE-corrected barriers; global probability and rate extrema; canonical rates; and any microcanonical energies selected interactively by the user. Step 12 automatically uses the threshold-truncated WC arrays when Step 11 has generated them; otherwise it uses the original clipped effective WC arrays.

3.2 Running SPINKS on an HPC Cluster

The HPC implementation of SPINKS follows the same computational workflow described in Section 3.1, including interactive editing of ORCA input files, automated execution of geometry optimizations, interpolation scans, MECP optimization, spin-orbit coupling calculations, gradient and frequency calculations, and the subsequent evaluation of spin-crossing probabilities and reaction rate constants. Consequently, only the features that differ from the local PC implementation are described below.

In addition to the remote working directories created on the HPC cluster, the cluster version creates a local directory on the computer from which SPINKS is executed. This local directory stores downloaded ORCA files, intermediate data, figures, summaries, and final results. The location is selected automatically according to the operating system:

Windows:

`C:\SPINKS\Spin_Inversion_<project_name>_cluster_local`

macOS:

`/Users/<username>/SPINKS/Spin_Inversion_<project_name>_cluster_local`

Linux:

`/home/<username>/SPINKS/Spin_Inversion_<project_name>_cluster_local`

On macOS and Linux, the local path is designed to be constructed automatically from the current user's home directory. This behavior follows the platform-independent Path.home() implementation in the source code. The Windows path was directly validated in the present work,

whereas local execution of the cluster client on macOS and Linux has not yet been experimentally tested by the authors. The remote cluster directory is created separately under the remote home directory supplied by the HPC system. Thus, the local operating system determines only where downloaded and processed SPINKS files are stored; the ORCA calculations themselves are executed on the remote HPC cluster. The default remote execution workflow was developed and tested for a Linux-based OpenPBS/PBS cluster. Use with other schedulers or cluster environments requires modification of the submission template, queue-monitoring commands, MPI module, and ORCA executable path.

Unlike the PC version, the cluster implementation first requests the information required to establish a secure connection to the remote computing system. Specifically, the user is prompted to provide the cluster hostname (or login URL), username, and password. These credentials are used to establish an SSH connection and to transfer the required input files automatically throughout the calculation.

Before the first calculations are submitted, SPINKS also displays the PBS batch script that will be used to execute the ORCA jobs on the cluster. An example is shown below:

```
#!/bin/bash
#PBS -N Mn_by_singlet_min
#PBS -l nodes=1:ppn=12
#PBS -o Mn_by_singlet_min.pbs.out
#PBS -e Mn_by_singlet_min.pbs.err

cd $PBS_O_WORKDIR

echo "=====
echo "Job name: Mn_by_singlet_min"
echo "Working directory:"
pwd
echo "Start time:"
date
echo "=====

module load openmpi/4.1.5
export PATH=/usr/local/apps/orca/5.0.4:$PATH

/usr/local/apps/orca/5.0.4/orca Mn_by_singlet_min.inp > Mn_by_singlet_min.out

echo "=====
echo "End time:"
date
echo "=====
```

Before using SPINKS on a new computing cluster, users should verify that the cluster-specific commands in the batch script are consistent with their local computing environment. In

particular, the two most important settings that typically require modification are **(i)** the MPI module specified by the module load command and **(ii)** the ORCA installation directory specified by the export PATH command. These settings are cluster-dependent and should be updated to match the software environment available on the target system.

The MPI module specified in the batch script:

```
module load openmpi/4.1.5
```

should be replaced with the MPI module available on the target cluster. If the appropriate module name is unknown, users can list the available OpenMPI modules by executing:

```
module avail openmpi
```

which may produce output similar to:

```
----- /usr/local/Modules/modulefiles -----  
openmpi/4.1.1 openmpi/4.1.1-aocc openmpi/4.1.5 openmpi/5.0.2 openmpi/5.0.8  
openmpi/5.0.8-borked  
  
----- /cm/shared/modulefiles -----  
openmpi/gcc/64/4.1.8 openmpi4/gcc/4.1.8
```

Users should then select the OpenMPI module appropriate for their computing environment and update the module load command in the batch script accordingly. For example, if openmpi/5.0.2 is available on the target cluster instead of openmpi/4.1.5, replace:

```
module load openmpi/4.1.5
```

with:

```
module load openmpi/5.0.2
```

The ORCA installation directory specified by:

```
export PATH=/usr/local/apps/orca/5.0.4:$PATH
```

should also be verified before running SPINKS on a new computing cluster. Users can first determine the available ORCA version(s) on their system using:

```
module avail orca
```

which may produce output similar to:

```
----- /usr/local/Modules/modulefiles -----  
orca/6.1.1
```

After identifying the desired ORCA version, the corresponding installation directory can be obtained by displaying the module information. For example:

```
module show orca/6.1.1
```

may contain an entry such as:

```
prepend-path PATH=/opt/software/orca/6.1.1:$PATH
```

indicating that the ORCA installation directory is **/opt/software/orca/6.1.1**. The export PATH command in the batch script should then be modified accordingly by replacing:

```
export PATH=/usr/local/apps/orca/5.0.4:$PATH
```

with:

```
export PATH=/opt/software/orca/6.1.1:$PATH
```

Based on the example above, where the available MPI module is **openmpi/5.0.2** and the ORCA installation directory is **/opt/software/orca/6.1.1**, the batch script should be modified as follows:

```
#!/bin/bash
#PBS -N Mn_by_singlet_min
#PBS -l nodes=1:ppn=12
#PBS -o Mn_by_singlet_min.pbs.out
#PBS -e Mn_by_singlet_min.pbs.err

cd $PBS_O_WORKDIR

echo "=====
echo "Job name: Mn_by_singlet_min"
echo "Working directory:"
pwd
echo "Start time:"
date
echo "=====

module load openmpi/5.0.2
export PATH=/opt/software/orca/6.1.1:$PATH

/opt/software/orca/6.1.1/orca Mn_by_singlet_min.inp > Mn_by_singlet_min.out

echo "=====
echo "End time:"
date
echo "=====
```

SPINKS separately reviews the queue-submission command used to submit each generated bash file. The default submission-command prefix is `qsub -l nodes=1:ppn=<nprocs>`. The

user may replace this prefix when required by the target scheduler; the generated .sh filename should not be entered because SPINKS appends it automatically.

The reviewed bash template and submission-command prefix are established once in Step 1 and reused by later cluster steps. For recognized scheduler syntaxes, SPINKS normalizes the node count to one and updates the processor count to the nprocs value selected by the user.

3.2.1 Cluster job monitoring, existing outputs, and resubmission

The HPC implementation uses the PBS queue state as the authority for whether a submitted job is alive. ORCA output-file size, modification time, printed progress, or apparent inactivity are not used to declare a running job stalled. The .out file is examined only for terminal success or explicit ORCA failure markers.

Before a new qsub submission is made, SPINKS first checks whether a normally terminated ORCA output already exists. If so, the completed calculation is reused. Otherwise, SPINKS performs a bulk qstat -f -u \$USER query and looks for an exact match of both PBS Job_Name and PBS_O_WORKDIR, allowing an already submitted Q, R, H, S, or E job to be adopted instead of duplicated.

- Q: the job is queued and SPINKS waits. The queue is checked every 15 seconds.
- R: the job is running and SPINKS waits indefinitely for the scheduler state or a terminal ORCA output; output-file inactivity is not treated as a failure signal.
- H: the job is held. SPINKS waits for three monitoring cycles (45 seconds total). If the H state persists, the job is cancelled with qdel, its files are archived, and the same ORCA input is resubmitted.
- S: the job is suspended. SPINKS applies the same three-cycle (45 seconds) waiting period and then cancels, archives, and resubmits the same input if the S state persists.
- E: the job is exiting. If the ORCA output has not already terminated normally, SPINKS treats the E state as a queue/execution-state condition rather than as proof of an ORCA-input failure. The exiting job is cancelled with qdel when possible, its files are archived, and the same ORCA input is resubmitted. If a replacement job also enters E, the same recovery procedure is repeated.
- If the stored PBS job ID disappears from the ordinary qstat view, SPINKS first tries to rediscover the job by the exact Job_Name + PBS_O_WORKDIR pair. PBS server suffixes are not hard-coded; only the leading numeric job identifier is used for ID matching.

- If no matching job is rediscovered, SPINKS requires three consecutive scheduler-only confirmations, separated by the 15 s monitoring interval, before treating the PBS job as gone. This avoids reacting to a transient queue-query gap.
- Once PBS has repeatedly confirmed that the job is gone, a normally terminated .out is accepted as success. An explicit ORCA failure causes SPINKS to display the final 100 lines and, when interactive repair is allowed, ask the user whether to edit and resubmit the input. An incomplete .out with no explicit ORCA failure marker is treated as a cluster interruption and the same input is archived and resubmitted. If no .out exists, the same input is resubmitted as a scheduler/submission recovery.
- The explicit ORCA failure classification is intentionally conservative and is triggered by case-insensitive matches to ORCA finished by error termination, finished with error, or aborting the run. An existing .out that contains neither a normal-termination marker nor one of these explicit failure markers is classified as INCOMPLETE rather than as an input failure.
- Before every automatic recycle or user-directed resubmission, files from the unsuccessful attempt are moved into a timestamped archive directory so that earlier calculations are not overwritten.

Transient SSH/SFTP interruptions are handled by automatic reconnection and retry logic. Once the connection is restored, SPINKS continues queue monitoring, file transfer, and workflow execution from the current stage.

This monitoring strategy allows the cluster workflow to resume completed or already-running jobs, wait safely for queued and active calculations, recover automatically from scheduler-level interruptions, and reserve user input editing for genuine ORCA failures or deliberate manual intervention.

Part II. Detailed Computational Implementation of SPINKS

4. Step-by-Step Implementation of the SPINKS Workflow

The theoretical quantities used in the later SPINKS steps, including the NAST gradient conventions, effective-Hessian treatment, weak-coupling formulation, density-of-states calculations, and statistical rate expressions, follow the NAST framework described in Ref. 2.² The present implementation couples those kinetic procedures to an automated ORCA workflow. The Local PC and HPC versions use the same scientific sequence and equations; differences between the scripts in Steps 1 and 2 concern ORCA execution, queue/file handling, and platform-specific bookkeeping rather than the underlying spin-crossing methodology.

This section is intended for users who wish to examine the internal implementation of SPINKS and understand how each stage of the workflow is carried out in the source code. The discussion therefore follows the program logic step by step and relates the mathematical expressions and computational procedures to the corresponding Python implementation. Readers may compare the descriptions given here directly with the source-code lines available in the official SPINKS GitHub repository: <https://github.com/Hamed-Barzinmehr/SPINKS>

4.1 Step 1: Minima Optimization, Interpolation Scan, and MECP Optimization

Step 1 performs the complete sequence required to obtain the optimized minimum-energy crossing point from a single user-supplied Cartesian geometry. The principal operations are:

1. collection and validation of molecular and computational input;
2. generation and optional editing of the ORCA inputs;
3. optimization and frequency analysis of the two requested spin-state minima;
4. automatic selection of the reactant reference minimum;
5. construction of a linear Cartesian interpolation between the optimized minima;
6. evaluation of both spin-state energies at every interpolation geometry;
7. selection of the interpolation point with the smallest absolute energy gap;
8. MECP optimization using the selected geometry; and

9. storage of the geometries, energies, settings, and metadata required by later steps.

The Local PC and HPC implementations use the same scientific Step 1 sequence: the same ORCA input-construction logic, two-state minimum optimizations, electronic-plus-ZPE reference selection, Cartesian interpolation, single-point gap analysis, and SurfCrossOpt MECP search. Their differences are operational. The Local PC version locates a local ORCA executable and runs jobs directly as blocking subprocesses in one local project tree. The HPC version establishes SSH/SFTP access, maintains separate remote and local project trees, submits ORCA jobs through an OpenPBS-style queue, monitors scheduler state, and transfers selected files between the two locations. These execution-specific differences are identified explicitly below and do not alter the scientific definitions used to choose the reference minimum, interpolation point, or MECP.

4.1.1 Workflow initialization

The common scientific workflow is initialized in MECP-only mode. RUN_MODE identifies only the execution backend:

```
RUN_MODE = "LOCAL"    # Local PC version
RUN_MODE = "CLUSTER"  # HPC version
workflow_mode = "MECP_ONLY"
run_mecp_optimization = True
```

Thus, both scripts optimize two spin-state minima, construct the same Cartesian interpolation, and locate the MECP. RUN_MODE changes the execution and file-management pathway rather than the scientific objective.

In the Local PC implementation, Step 1 first asks for the ORCA installation or extraction directory and resolves the executable through:

```
ORCA_EXE, ORCA_DIR = ask_orca_installation()
find_orca_executable(orca_dir_text)
```

The executable search accepts the platform-appropriate ORCA binary in the supplied directory and can also search a parent directory for a unique ORCA executable. ORCA_EXE stores the complete executable path and ORCA_DIR stores its installation directory.

Local processor selection is handled by choose_local_nprocs(). On Windows, microsoft_mpi_available() searches the standard Microsoft MPI directories and the system PATH. If Microsoft MPI is not detected, a request for more than one processor is reset automatically to nprocs = 1. On Linux and macOS, the code does not perform an equivalent OpenMPI compatibility test, so the user is responsible for providing the MPI environment required by the local ORCA distribution.

In the HPC implementation, Step 1 instead opens a Paramiko SSH connection, creates an SFTP session, and obtains the authenticated remote home directory from \$HOME. The

cluster code also defines automatic SSH/SFTP reconnection helpers so interrupted remote commands or file reads can be retried without changing the scientific job input.

The HPC bash template and queue-submission command are reviewed when cluster submission is first required. The default template is OpenPBS-style and contains the ORCA executable path, MPI module, processor count, and generated input/output filenames; the user may modify the template or submission-command prefix for a different cluster environment. These settings are reused by later cluster jobs.

4.1.2 Collection of molecular and computational input

The user supplies the following quantities:

- jobname
- multiplicities
- charge
- method
- basis
- nprocs
- mem_gb
- Initial_geometry

The variable jobname defines the project identifier and is incorporated into the project-directory name and ORCA job names. Fixed SPINKS summary/metadata filenames are used where specified later in this section. The two spin multiplicities are parsed from a comma-separated entry:

```
multiplicities = parse_int_list(...)
```

SPINKS requires exactly two multiplicities because Step 1 treats a crossing between two spin surfaces.

The lower and higher numerical multiplicities are subsequently stored as:

```
low_spin_mult = min(multiplicities)
high_spin_mult = max(multiplicities)
```

The user also specifies the molecular charge, electronic-structure method, basis set, processor count, and ORCA memory allocation. In the HPC script, nprocs is accepted directly as a positive integer. In the Local PC script, the requested processor count passes through choose_local_nprocs() and may be forced to one on Windows when Microsoft MPI is unavailable. The default electronic-structure values are:

```
default_method = "PBE"
default_basis = "def2-TZVP"
```

The entered mem_gb value represents the requested ORCA %maxcore memory per processor in GB. It is converted to megabytes per processor as:

```
maxcore_mb = mem_gb * 1000
```

The initial geometry is read as a multiline Cartesian block and stored in:

Initial_geometry

This initial geometry is used for optimization of both spin-state minima. It does not need to be a stationary point, but the atom order must remain identical for both optimized structures because the subsequent Cartesian interpolation assumes direct correspondence between atomic coordinates.

4.1.3 Project-directory organization

The directory structure is scientifically equivalent in both implementations but differs in whether a true remote filesystem is present.

For Local PC, the project directory is created inside the resolved ORCA installation directory:

```
local_base = os.path.join(
    str(ORCA_DIR),
    f"Spin_Inversion_{jobname}"
)
```

The three calculation subdirectories are:

```
local_minima = os.path.join(local_base, "Minima")
local_interp = os.path.join(local_base, "Interpolation_scan")
local_mecp = os.path.join(local_base, "MECP")
```

Because no remote filesystem exists in Local PC mode, later remote-style names are retained only as compatibility aliases:

```
remote_base = local_base
remote_minima = local_minima
remote_interp = local_interp
remote_mecp = local_mecp
```

For HPC, a true remote project tree is created under the authenticated cluster home directory using POSIX paths:

```
remote_base = posixpath.join(
    remote_home,
    f"Spin_Inversion_{jobname}"
)
remote_minima = posixpath.join(remote_base, "Minima")
remote_interp = posixpath.join(remote_base, "Interpolation_scan")
remote_mecp = posixpath.join(remote_base, "MECP")
```

These remote directories are created with `remote_mkdir_p(...)`. A separate local mirror is created on the computer running SPINKS. Windows uses `C:/SPINKS`, whereas Linux and macOS use a SPINKS directory under `Path.home()`:


```
if os.name == "nt":
    local_output_root = Path("C:/SPINKS")
else:
    local_output_root = Path.home() / "SPINKS"
```

The HPC local project directory is then:

```
local_base = local_output_root / (
    f"Spin_Inversion_{jobname}_cluster_local"
)
```

with local Minima, Interpolation_scan, and MECP subdirectories. After creation, the HPC local Path objects are converted to strings for compatibility with the later SPINKS steps. Remote paths contain the files used for cluster execution, whereas the local mirror contains transferred outputs, processed data, figures, and metadata.

4.1.4 Construction and review of ORCA inputs

Both implementations define minimum, interpolation single-point, and MECP input templates. The minimum template begins with:

```
! {method} {basis} TightSCF SlowConv Opt Freq
```

and the default transferable settings include %pal, %maxcore, %scf, and %output blocks. The current default SCF block is:

```
%scf
  MaxIter 1500
  SOSCFStart 0.01
  LevelShift 0.5
end
```

Minimum calculations must retain both Opt and Freq. This is enforced by:

```
enforce_minimum_input_requirements(inp_text, job_label)
```

because the optimized geometries are required for interpolation and the frequency calculations provide the zero-point energies used in reference-state selection and the later barrier correction.

For the requested lower and higher multiplicities, both scripts first generate and display the complete minimum input files. The user is then given one shared editing opportunity for the pre-geometry ORCA settings block. The * xyz line, charge, multiplicity, Cartesian geometry, and closing * are excluded from that editable block.

```
default_pre_xyz, _, _ = split_orca_xyz_block(
    minima_inputs[low_spin_mult]["inp_text"]
)
reviewed_pre_xyz = default_pre_xyz
```

If the user edits this block, the same reviewed settings are applied to both minimum inputs with their appropriate protected charge, multiplicity, and initial geometry. The finalized shared settings are stored as:

```
DFT_REVIEWED_PRE_XYZ = reviewed_pre_xyz.rstrip() + "\n"
```

All transferable ORCA percent blocks are then extracted into:

```
ORCA_TRANSFER_BLOCKS_ALL
ORCA_TRANSFER_BLOCKS_NO_OUTPUT
```

where the second form excludes %output for later jobs that require their own fixed output block.

The reusable ! line is stripped of job-specific keywords and rebuilt by `adapt_bang_line_for_job(...)`. `make_dft_pre_xyz(...)` and `apply_dft_settings_to_input(...)` therefore preserve the reviewed method/basis and transferable percent blocks while changing only the calculation-specific instructions: minimum adds Opt Freq, single_point removes optimization/frequency keywords, mecp adds SurfCrossOpt, engrad adds Engrad, and freq adds SurfCrossNumFreq.

The final minimum inputs are rebuilt from the reviewed pre-geometry block and checked again for Opt and Freq before execution. This review and propagation logic is the same in Local PC and HPC.

One bookkeeping string differs between the present scripts. Local PC records `ORCA_INPUT_TEMPLATE_SOURCE` as a user-entered method/basis with one shared reviewed DFT pre-geometry block, whereas the current HPC code retains an older descriptive source string referring to default PBE/def2-TZVP templates and individual minimum review. The actual HPC control flow nevertheless uses the same single shared pre-geometry review described above; this label does not alter any generated scientific input.

Separately from the ORCA-input review, the HPC version reviews its cluster bash template and submission-command prefix when the first remote submission is needed. Local PC has no bash, qsub, SSH, or SFTP submission layer.

4.1.5 Optimization of the two spin-state minima

A separate optimization and frequency calculation is generated for each requested multiplicity. The job name is:

```
job_i = f'{jobname}_{spin_name}_min'
```

where multiplicity_name(mult) supplies labels such as singlet, triplet, or quintet.

In Local PC mode, the two minimum calculations are executed sequentially through:

```
submit_and_monitor_orca_job(  
    local_minima,  
    job_i,  
    inp_text,  
    f'{spin_name} minimum Opt/Freq',  
    repair_policy="free_minimum"  
)
```

Each call is blocking: the local ORCA subprocess finishes before the next minimum job is started. A normally terminated existing output is reused. A pre-existing non-normal output is archived before rerun. If the new local run does not terminate normally, SPINKS prints the final 100 output lines and can allow the user to edit and rerun the input. The free_minimum repair policy allows broad editing but requires Opt and Freq to remain present.

In HPC mode, the two minimum jobs are prepared or adopted first and then monitored together:

```
job_record = submit_orca_job_no_monitor(...)  
submitted_minima_jobs = monitor_submitted_orca_jobs(  
    remote_minima,  
    submitted_minima_jobs,  
    allow_interactive_repair=True  
)
```

Before a new qsub submission, the cluster code checks for a normally terminated .out and for an existing PBS job with the exact Job_Name and PBS_O_WORKDIR. Existing jobs in Q, R, H, S, or E are adopted rather than duplicated.

PBS state controls liveness. Q and R are waited on. H and S are allowed three 15 s monitoring cycles and are then cancelled, archived, and resubmitted with the same ORCA input. E is treated as a queue/execution-state problem and is likewise recycled with the same input. If a stored job ID disappears, SPINKS first attempts exact Job_Name + PBS_O_WORKDIR rediscovery; only after three consecutive scheduler-only confirmations of absence is the job treated as gone.

The ORCA .out file is not used to infer a stall from file size, modification time, or printed progress. Its content is used only for terminal classification. ORCA TERMINATED NORMALLY completes the job; explicit ORCA failure markers can trigger interactive input repair; an incomplete or missing output with no active PBS job is treated as a cluster interruption and resubmitted with the same input.

After successful completion, both implementations extract the electronic energy, zero-point energy, and optimized Cartesian geometry. Local PC reads the local .out and .xyz files directly. HPC reads the corresponding remote files through SFTP and writes local copies of each final .out, .xyz, and .inp into the local Minima mirror.

For each multiplicity the common scientific results are stored in minima_data[mult], including Electronic_Eh, ZPE_Eh, E_plus_ZPE_Eh, and geometry. The path fields differ operationally: Local PC stores local out_path/xyz_path values, whereas HPC stores the remote output and XYZ paths used to extract the results.

4.1.6 Automatic selection of the reactant reference minimum

The reactant reference is selected according to the electronic-plus-zero-point energy rather than the electronic energy alone.

The multiplicities are ordered using:

```
sorted_mults = sorted(  
    multiplicities,  
    key=lambda m: minima_data[m]["E_plus_ZPE_Eh"]  
)
```

The lower-energy state becomes:

```
reference_multiplicity = sorted_mults[0]
```

and the second state becomes:

```
other_multiplicity = sorted_mults[1]
```

The optimized geometry of the reference state is stored as:

```
Reference_geometry
```

and its multiplicity is stored as:

```
Reference_multiplicity
```

The reference-selection description is stored in:

```
Reference_source
```

The energy comparison is based on Equation S1:

$$E_i^{\text{ref}} = E_{\text{elec},i} + E_{\text{ZPE},i} \quad (\text{S1})$$

The relative energies are calculated using the lowest electronic-plus-ZPE energy, E_0 , as the zero:

```
E0 = minima_data[reference_multiplicity]["E_plus_ZPE_Eh"]
```

For each multiplicity:

```
dEh = minima_data[mult]["E_plus_ZPE_Eh"] - E0
```

is converted into reciprocal centimetres and kilocalories per mole using:

```
hartree_to_cm = 219474.6  
hartree_to_kcal = 627.509474
```

This reference minimum defines the zero of energy for the later MECP barrier, density-of-states, microcanonical-rate, and canonical-rate calculations.

4.1.7 Construction of the Cartesian interpolation

Step 1 constructs the same linear Cartesian interpolation between the two optimized minimum geometries in both implementations.

The default number of interpolation points is:

```
n_interp = 10
```

For interpolation point i , the interpolation parameter is:

```
lam = i / (n_interp - 1)
```

which spans 0 to 1 and includes both optimized minima. The interpolated coordinates are evaluated according to Equation S2:

$$\mathbf{R}(\lambda) = (1 - \lambda)\mathbf{R}_A + \lambda\mathbf{R}_B \quad (\text{S2})$$

The implementation is:

```
interpolate_geometry(geom_a, geom_b, lam)
```

Before interpolation, `interpolate_geometry(...)` verifies both atom ordering and coordinate-array dimensions. A mismatch raises an error because direct Cartesian interpolation requires one-to-one correspondence between atoms.

The generated XYZ geometry is written locally in both implementations. The stored path metadata then differs by execution mode. Local PC keeps both geometry references local:

```
"geometry_file": local_xyz  
"local_geometry_file": local_xyz
```

HPC writes the same XYZ file into the local interpolation mirror, uploads it by SFTP to `remote_interp`, and stores both locations:

```
"geometry_file": remote_xyz  
"local_geometry_file": local_xyz
```

The geometry, interpolation parameter, and per-spin job definitions are otherwise identical.

4.1.8 Single-point energies on both spin surfaces

For every interpolation geometry, SPINKS evaluates one single-point energy for each multiplicity. With `n_interp = 10`, the scientific scan therefore contains 20 ORCA single-point calculations in either implementation.

The job-specific input is rebuilt from the reviewed Step 1 DFT settings using:

```
apply_dft_settings_to_input(sp_inp, "single_point")
```

which removes optimization/frequency job keywords while preserving the reviewed method, basis set, SCF settings, processor and memory directives, solvent/dispersion settings when present, and other transferable percent blocks.

The interpolation repair policy is common to both versions:

```
repair_policy = "interp_header_only"
```

Only the pre-geometry ORCA settings block is presented for editing; the automatically generated * xyz block is carried forward unchanged. The validator also requires the selected method and basis to remain present.

Execution scheduling differs. Local PC processes one interpolation geometry at a time. For each point, the two spin-state jobs are run through the blocking local submit_and_monitor_orca_job(...), checked for normal termination, and their local output files are parsed before the code advances to the next interpolation geometry.

HPC uses:

```
interp_batch_geometries = 2
```

so two interpolation geometries are handled per batch. Because each geometry has two spin states, up to four single-point jobs are prepared/adopted with submit_orca_job_no_monitor(...) and then monitored simultaneously with monitor_submitted_orca_jobs(...). Energies are read from the remote .out files after the batch has completed.

For either implementation, the final electronic energy is extracted from the last occurrence of FINAL SINGLE POINT ENERGY. For interpolation point i:

```
gap = energies[mult_a] - energies[mult_b]
gap_abs_Eh = abs(gap)
gap_cm1 = gap * hartree_to_cm
gap_abs_cm1 = abs(gap * hartree_to_cm)
```

The record added to interp_records contains the interpolation index, lambda, geometry, both electronic energies, signed and absolute gaps, and the gap in kcal mol⁻¹. HPC additionally retains both remote and local geometry-file paths in each record.

4.1.9 Selection of the MECP starting geometry

The interpolation geometry with the smallest absolute electronic-energy difference is selected identically in both implementations:

```
best_record = min(
    interp_records,
    key=lambda r: r["gap_abs_Eh"]
)
MECP_start_geometry = best_record["geometry"]
```

The associated interpolation index, lambda value, gap in hartree, and gap in cm^{-1} remain available through best_record. This geometry is only the closest-to-degeneracy point found on the linear interpolation; it is used as the initial structure for the later MECP optimization and is not assumed to be the final crossing point.

The complete interpolation table is written to the fixed filename:

```
interpolation_scan_summary.txt
```

Local PC writes this summary only into local_interp. HPC writes the summary to the local interpolation mirror and uploads the same file to remote_interp by SFTP.

The selected starting geometry is also written to:

```
<jobname>_best_MECP_crossing_guess.xyz
```

after the interpolation plot is generated. Local PC keeps this file locally and exposes a local compatibility alias; HPC uploads it to the remote interpolation directory as well.

4.1.10 Interpolation-energy visualization

Both implementations generate the same interpolation-energy visualization from the electronic single-point energies. The common zero is the lowest electronic energy found anywhere on either interpolated spin surface:

```
interpolation_reference_Eh = min(
    np.min(low_spin_energies_Eh),
    np.min(high_spin_energies_Eh)
)
```

No ZPE correction is applied to this plot because the interpolation scan consists of electronic single-point calculations. The relative energies are converted to kcal mol^{-1} using the same hartree_to_kcal factor for both spin surfaces.

The resulting figure is saved as:

```
interpolation_energy_scan.png
interpolation_energy_scan.pdf
```

The PNG is written at 600 dpi and the PDF retains vector graphics. The plotting syntax differs slightly (Local PC uses the pyplot interface; HPC uses an explicit fig/ax object), but the plotted quantities and labels are the same.

Local PC stores the plot files in local_interp only. HPC stores them in the local interpolation mirror and then uploads both files to remote_interp. The HPC script therefore exports

remote_interpolation_plot_png and remote_interpolation_plot_pdf in addition to the local plot paths.

4.1.11 Construction of the MECP input

The main multiplicity used in the ORCA MECP input is the reference-state multiplicity:

```
mult_main = reference_multiplicity  
mult_other = other_multiplicity
```

The MECP input is generated using the selected interpolation geometry and the ORCA keyword:

```
SurfCrossOpt
```

The second multiplicity is defined through the %mecp block:

```
%mecp Mult <mult_other>  
end
```

Programmatically, the input is constructed using:

```
mecp_inp = apply_dft_settings_to_input(  
    mec_inp,  
    "mecp",  
    job_specific_blocks=  
        f"%mecp Mult {mult_other}\\nend"  
)
```

This procedure preserves the reviewed DFT settings while introducing the MECP-specific keyword and multiplicity block.

4.1.12 Protected MECP optimization

The MECP calculation uses the same restricted scientific input in both implementations. Local PC executes it directly in local_mecp through submit_and_monitor_orca_job(...), whereas HPC calls the same high-level helper on remote_mecp, where the job is submitted/adopted and monitored through the PBS layer.

Both versions use:

```
repair_policy = "mecp_restricted"
```

with the locked quantities:


```
locked = {  
  "method": method,  
  "basis": basis,  
  "charge": charge,  
  "mult": mult_main,  
  "mecp_mult": mult_other,  
  "geom": MECP_start_geometry  
}
```

A repaired MECP input must therefore retain the selected method and basis, SurfCrossOpt, molecular charge, main multiplicity, second multiplicity in the %mecp block, and the automatically selected starting geometry.

After normal termination, both scripts first look for <jobname>.xyz. If the expected file is absent, Local PC searches the local MECP directory for an available XYZ file, whereas HPC performs the corresponding remote directory search through SSH. The selected XYZ file is parsed and stored as:

MECP_geometry

This optimized crossing geometry is then used as the common nuclear configuration in the subsequent SOC, gradient, Hessian, reduced-mass, and kinetic calculations.

4.1.13 Step 1 output variables

The principal scientific variables passed from Step 1 are common to both implementations:

```
Initial_geometry  
Reference_geometry  
MECP_start_geometry  
MECP_geometry
```

representing the user-supplied starting structure, the optimized electronic-plus-ZPE reference minimum, the best interpolated MECP starting structure, and the optimized MECP, respectively.

The energetic and spin-state information is retained in:

```
minima_data  
Reference_multiplicity  
reference_multiplicity  
Reference_source  
reference_source  
mult_main  
mult_other
```

The interpolation information is retained in interp_records and best_record, and the reviewed DFT settings are retained in:

```
DFT_REVIEWED_PRE_XYZ
ORCA_TRANSFER_BLOCKS_ALL
ORCA_TRANSFER_BLOCKS_NO_OUTPUT
```

These scientific settings are common to later Engrad and SurfCrossNumFreq jobs. The two implementations additionally retain different execution-specific objects.

Local PC exports ORCA_EXE, ORCA_DIR, MPI_AVAILABLE, and MPI_DIR together with local_base/local_minima/local_interp/local_mecp. The remote_* directory variables are local-path aliases. At the end of Step 1, the cluster template variables are explicitly disabled and ssh/sftp are set to None as compatibility placeholders.

HPC exports the live ssh/sftp connection objects, cluster host information, true remote and local directory trees, the reviewed cluster bash/submission settings, remote plot paths, queue-monitoring helpers, and the monitoring thresholds. These variables control remote execution only; they do not change the common geometries, energies, multiplicities, or reviewed DFT settings.

4.1.14 Files generated by step 1

The principal scientific files generated by Step 1 are:

```
Reference_geometry.txt
Reference_info.txt
MECP_geometry.txt
MECP_start_interpolated_geometry.txt
<jobname>_final_MECP_input.inp
interpolation_scan_summary.txt
<jobname>_best_MECP_crossing_guess.xyz
interpolation_energy_scan.png
interpolation_energy_scan.pdf
step1_orca_input_templates.txt
```

The content and transfer behavior of the metadata files differ slightly between the implementations.

In Local PC mode, step1_orca_input_templates.txt records the reviewed DFT pre-geometry block, ORCA_TRANSFER_BLOCKS_ALL, ORCA_TRANSFER_BLOCKS_NO_OUTPUT, and the minimum, single-point, and MECP templates. Reference_info.txt records Run mode = LOCAL, the reference-state energetics, interpolation selection, ORCA executable and installation directory, and the ORCA-input source description. All Step 1 files remain in the local project tree.

The HPC implementation writes four additional local metadata files:

```
cluster_sh_template.txt
DFT_reviewed_pre_xyz_settings.txt
ORCA_transfer_blocks_all.txt
ORCA_transfer_blocks_no_output.txt
```

In the current HPC code, step1_orca_input_templates.txt contains the three base template definitions, while the reviewed pre-geometry settings and the two transfer-block strings are written to the separate files above. Reference_info.txt records Run mode = CLUSTER, the same reference-state energetics and interpolation selection, the ORCA-input source label, and the cluster bash-template source; it does not contain a local ORCA executable/directory entry.

HPC also mirrors the final minimum .out/.xyz/.inp files locally after remote execution. The interpolation summary, interpolation plots, best-crossing-guess XYZ, Reference_geometry.txt, Reference_info.txt, MECP geometry/start files, final MECP input, cluster_sh_template.txt, and step1_orca_input_templates.txt are uploaded or maintained in the remote project tree as appropriate. The separate DFT_reviewed_pre_xyz_settings.txt and ORCA_transfer_blocks_*.txt files are written to the local metadata tree by the current code; their corresponding variables remain available in memory for subsequent SPINKS steps.

4.1.15 Summary of the step 1 algorithm

The complete scientific sequence implemented in Step 1 can be summarized as:

This sequence is identical for Local PC and HPC. Local PC executes each ORCA calculation directly on the local machine, whereas HPC may run independent jobs concurrently through the queue (including simultaneous minima and two-geometry interpolation batches) and mirrors selected remote results into the local SPINKS output tree.

```
Initial geometry → { Minimum optimization and frequency, state A
                    { Minimum optimization and frequency, state B
→ lowest ( $E_{\text{elec}} + E_{\text{ZPE}}$ ) reference
→ Cartesian interpolation
→  $E_A(\lambda), E_B(\lambda)$ 
→  $\lambda_{\min} = \arg \min_{\lambda} |E_A(\lambda) - E_B(\lambda)|$ 
→ SurfCrossOpt MECP optimization
→ optimized MECP geometry.
```

This automated procedure provides all geometries, energies, reference-state assignments, and electronic-structure settings required for the subsequent SPINKS calculations.

4.2 Step 2: Orbital Preparation and SOC Calculation at the MECP

Step 2 converts the optimized MECP geometry into the spin-orbit-coupling quantities required by the later probability and rate calculations. SPINKS prepares high-spin ROHF and UHF/UNO reference calculations, determines candidate active spaces, performs a localized-SOMO diagnostic, and—when a completed SOC output is not already available—constructs an editable CASSCF SOC input. The ORCA SOC output is parsed into the complete M_s -resolved complex matrix, from which the effective scalar SOC and channel-resolved quantities are obtained.

The principal operations performed in Step 2 are:

1. preparation of ROHF and UHF/UNO reference wavefunctions,
2. automatic determination of active orbitals,
3. localized-orbital analysis of the ROHF SOMOs,
4. generation of the SOC input,
5. construction or reuse of the CASSCF SOC calculation,
6. extraction of the complex SOC matrix,
7. construction of the M_s -resolved SOC matrix, and
8. evaluation of the effective, classified component, and M_s -specific SOC quantities.

The Local PC and HPC implementations perform the same scientific orbital preparation, active-space analysis, SOC-matrix parsing, and downstream SOC definitions. Their differences are operational. The Local PC version runs ORCA and `orca_loc` directly in the local SOC directory and retains several remote-style path names only as compatibility aliases. The HPC version creates separate local and remote SOC directories, reuses the cluster submission settings established in Step 1, executes ORCA and `orca_loc` through SSH/PBS, transfers files by SFTP, and writes diagnostic results to both locations. A few status, diagnostic, and bookkeeping variables also have implementation-specific names; these differences are identified explicitly below.

4.2.1 Initialization

Step 2 imports the optimized MECP geometry, spin multiplicities, basis and resource settings, and the execution helpers established in Step 1. The core scientific inputs are common to both implementations, but the path and execution dependencies differ.

In the Local PC implementation, the required-variable check is:

```
required_vars_step2 = [
    "jobname", "MECP_geometry", "charge", "mult_main", "mult_other",
    "multiplicities", "basis", "nprocs", "mem_gb", "maxcore_mb",
    "local_base", "local_mecp", "ORCA_EXE", "ORCA_DIR",
    "submit_and_monitor_orca_job"
]
```

The Local PC version sets `workflow_mode = "MECP_ONLY"` and `RUN_MODE = "LOCAL"` by default. For compatibility with later code that uses remote-style names, local paths are also exposed through aliases:

```
remote_base = local_base
remote_mecp = local_mecp
```

SSH, SFTP, and cluster-submission objects are intentionally unused in this mode; their compatibility variables are set to disabled or empty values.

In the HPC implementation, the required-variable check additionally includes the remote connection, file-system, and cluster-execution helpers:

```
required_vars_step2 = [
    "jobname", "MECP_geometry", "charge", "mult_main", "mult_other",
    "multiplicities", "basis", "nprocs", "mem_gb", "maxcore_mb",
    "ssh", "sftp", "remote_base", "local_base",
    "remote_mkdir_p", "remote_file_exists", "remote_read_text",
    "run_ssh", "submit_and_monitor_orca_job"
]
```

The HPC version also verifies that `CLUSTER_SH_TEMPLATE` and the cluster submission-command settings selected in Step 1 are still available. No new bash-template or submission-command review is performed in Step 2.

The SOC directory is also implementation-specific. Local PC creates one local directory and uses it for both local and remote-style names:

```
local_soc = os.path.join(local_base, "SOC")
os.makedirs(local_soc, exist_ok=True)
remote_soc = local_soc
```

HPC instead maintains a true remote directory together with a local mirror:

```
remote_soc = posixpath.join(remote_base, "SOC")
remote_mkdir_p(sftp, remote_soc)
local_soc = os.path.join(local_base, "SOC")
os.makedirs(local_soc, exist_ok=True)
```

Both implementations check for an already completed SOC output before a new SOC input is generated, but the status helpers and flag names differ. Local PC uses `local_text_status(...)`, aliases it as `remote_text_status(...)`, and stores the completed-output condition as:

```
SOC_OUTPUT_ALREADY_COMPLETE = (soc_status_initial == "OK")
```

The local status helper returns MISSING, OK, ERROR, or FAILED from the local .out file. HPC reads the remote output through SFTP using remote_text_status(...) and stores the corresponding condition as:

```
soc_output_already_completed = (soc_status_preexisting == "OK")
```

The HPC status helper returns MISSING, OK, ERROR, or INCOMPLETE. In both implementations, an already completed SOC output causes the later ROHF/UHF orbital-source question for SOC generation to be skipped and the existing SOC result to be parsed directly.

4.2.2 High-spin ROHF orbital preparation

The first electronic-structure calculation performed in Step 2 is a restricted open-shell Hartree-Fock calculation on the higher-spin state.

The multiplicity is selected automatically as:

```
soc_orbital_mult = max(multiplicities)
```

and the corresponding ORCA input is generated using:

```
! ROHF {basis} TightSCF SlowConv
```

The optimized MECP geometry from Step 1 is inserted directly into the ROHF input. Both implementations reuse the transferable percent blocks captured from the reviewed Step 1 settings through step2_transfer_blocks(...); the electronic-structure header itself is explicitly ROHF rather than the DFT method used in Step 1.

Before execution, both versions check whether a normally terminated ROHF output and the corresponding GBW file already exist. If both are present, the calculation is skipped and the existing wavefunction is reused.

In Local PC mode, remote_soc is an alias of local_soc and the existence checks resolve to ordinary local-file operations. If a new calculation is required, submit_and_monitor_orca_job(...) runs ORCA directly on the local machine.

In HPC mode, the same logical checks are performed against remote_orb_out and remote_orb_gbw in the remote SOC directory. If execution is needed, submit_and_monitor_orca_job(...) submits and monitors the job through the cluster machinery established in Step 1.

The repair policy is the same in both implementations:

```
repair_policy = "rohf_restricted"
```

The ROHF method, basis set, molecular charge, high-spin multiplicity, and MECF geometry are protected if a failed job is edited. Both versions verify the output and GBW files before proceeding to SOMO analysis.

4.2.3 Automatic SOMO detection

After the ROHF calculation has completed, SPINKS parses the **ORBITAL ENERGIES** section of the ORCA output to identify singly occupied molecular orbitals.

The implementation searches for orbitals satisfying the criterion in Equation S3:

$$\text{Occupation} \approx 1.0000 \quad (\text{S3})$$

using the function:

```
parse_singly_occupied_orbitals_from_rohf_out(...)
```

The detected SOMO indices are stored in:

```
rohf_somo_indices
```

For example:

```
Detected ROHF SOMOs:  
[97,98]
```

would automatically produce:

```
first_somo = 97  
last_somo = 98
```

The corresponding active space is then defined as:

```
nel_soc = len(rohf_somo_indices)  
norb_soc = len(rohf_somo_indices)
```

yielding

$$\text{CAS}(n_{\text{SOMO}}, n_{\text{SOMO}}).$$

Thus, a system containing two SOMOs automatically generates a CAS(2,2) active space, whereas four SOMOs produce CAS(4,4), without requiring any user intervention.

4.2.4 UHF/UNO orbital preparation

Although ROHF is used by default, SPINKS also performs an unrestricted Hartree–Fock calculation with unrestricted natural orbitals.

The corresponding input contains:

```
! UHF {basis} TightSCF SlowConv UNO
```

together with:

```
Print[P_UNO_OccNum] 1  
Print[P_UNO_AtPopMO_M] 1
```

which form the required UHF/UNO-specific output block. For this job, Step 1 transferable percent blocks are reused with the Step 1 %output block excluded so that the required UNO occupation printing cannot be replaced.

The UHF/UNO calculation follows the same scientific input in both implementations but uses the implementation-specific execution layer described above. Local PC runs or reuses the job in local_soc, whereas HPC runs or reuses it in remote_soc.

The completed-output checks differ slightly in form. Local PC skips a rerun when local status is OK, while HPC requires both an OK remote status and the expected remote output file before skipping submission. Both implementations subsequently verify that the UHF/UNO output exists before parsing it.

The UNO occupation numbers are extracted using:

```
parse_uno_occupations_from_uhf_out(...)
```

The active orbitals are then selected according to the occupation criterion in Equation S4:

$$0.02 < n_i < 1.98 \quad (\text{S4})$$

implemented by:

```
select_uno_active_orbitals(  
  low=0.02,  
  high=1.98  
)
```

The resulting active space is

$$\text{CAS}(N_{\text{active}}, M_{\text{active}}),$$

where N_e is obtained by summing the UNO occupations within the selected window and rounding to the nearest integer.

The selected UHF/UNO active-space variables are uhf_uno_active_indices, uhf_uno_active_electrons, and uhf_uno_active_orbitals. The active-electron count is obtained by summing the selected occupations and rounding to the nearest integer.

The occupation window itself is identical in both versions. The current Local PC code additionally requires both uhf_uno_active_orbitals > 0 and uhf_uno_active_electrons > 0, whereas the HPC code explicitly checks only that uhf_uno_active_orbitals > 0. For a valid

nonempty active space, this validation difference does not change the selected orbitals or active-electron count.

4.2.5 Localized SOMO analysis

SPINKS next evaluates whether the ROHF SOMOs are predominantly metal centered. The previously generated ROHF GBW file is supplied to `orca_loc`, which localizes the contiguous SOMO window identified in the preceding section.

The `orca_loc` input is common to both implementations and contains the GBW filename, localization-output filename, first SOMO, last SOMO, and the final localization flag. The localized orbital compositions are parsed using:

```
parse_lmo_compositions_from_orca_loc_output(...)
```

For every SOMO, the contribution from the detected transition-metal atom is evaluated. For example:

```
MO 97  
Fe population = 0.961
```

indicates that approximately 96% of the localized orbital resides on the metal atom. The diagnostic threshold is:

```
metal_population_threshold = 0.93
```

If every localized SOMO is metal dominated and exceeds this threshold, the diagnostic records that condition; otherwise ligand character or a subthreshold metal population is reported.

The execution of `orca_loc` differs between the two implementations. Local PC locates the executable inside `ORCA_DIR` (`orca_loc.exe`, `orca_loc.bat`, or `orca_loc` depending on the operating system), prepares a local execution environment, and invokes it with `subprocess.run(...)` in the local SOC directory. If an MPI directory is available, it is added to `PATH` for the local process.

```
result = subprocess.run(  
    [orca_loc_command, locinp_filename],  
    cwd=str(local_dir),  
    ...  
)
```

HPC instead determines the cluster `orca_loc` command from the ORCA path in `CLUSTER_SH_TEMPLATE` when possible, with `/usr/local/apps/orca/5.0.4/orca_loc` as the fallback. The `locinp` file is uploaded to the remote SOC directory, and `orca_loc` is executed there through SSH:

```
run_ssh(ssh, cmd)
```

The captured remote output is read back through SFTP, and the generated .loc file is downloaded when available. Thus, both versions analyze the same localized-orbital information, but one executes locally and the other through the remote cluster filesystem and shell.

If orca_loc is unavailable or the localization/parsing step fails, both implementations report the failure and continue because the ROHF SOMO list and UHF/UNO active-space information were obtained independently. Local PC additionally stores explicit bookkeeping flags orca_loc_failed and orca_loc_failure_reason.

The localized-SOMO analysis remains diagnostic and does not automatically choose the SOC active space. In the current source codes, choose_soc_protocol_from_lmo_records(...) returns different unused protocol-label strings in the Local PC and HPC implementations; however, the caller discards those first two return values and retains only lmo_diagnostics:

```
_, _, lmo_diagnostics = choose_soc_protocol_from_lmo_records(...)
```

Consequently, those helper-label differences do not alter the generated SOC calculation or the downstream SOC definitions.

4.2.6 Selection of the orbital source

Following completion of the ROHF and UHF/UNO calculations, both implementations summarize the two automatically generated active-space options. If a new SOC calculation is required, the user is asked whether the SOC input should use ROHF orbitals (recommended/default) or UHF/UNO orbitals.

For a new calculation, the selected source determines the active indices and active-space dimensions:

```
soc_active_indices  
soc_nel  
soc_norb
```

The selected values are assigned to nel_soc, norb_soc, and soc_active_indices. ROHF uses rohfsomo_indices and CAS(len(SOMOs), len(SOMOs)); UHF/UNO uses the orbitals selected from the $0.02 < \text{occupation} < 1.98$ window and the rounded active-electron count.

When a completed SOC output already exists, neither implementation asks the orbital-source question for SOC generation. The existing output is parsed directly and ROHF-derived active-space values are retained only for bookkeeping/diagnostic reporting.

The reuse-path bookkeeping differs slightly. Local PC uses:

```
SOC_OUTPUT_ALREADY_COMPLETE
soc_protocol = "EXISTING_SOC_OUTPUT_REUSED"
selected_soc_keyword = "EXISTING"
```

whereas HPC uses:

```
soc_output_already_completed
soc_protocol = "EXISTING_SOC_OUTPUT_PARSED_DIRECTLY"
selected_soc_keyword = None
```

These labels do not affect the parsed SOC matrix. When a new calculation is generated, both implementations use the common protocol label CASSCF_MAXITER_1_SOC_<orbital source> and the same selected active-space quantities.

4.2.7 Automatic construction of the SOC input

When a new SOC calculation is required, both implementations construct the same CASSCF SOC input from the selected ROHF or UHF/UNO reference and active space.

The fixed relativistic block is:

```
%rel
  SOType 3
  SOCFlags 1,4,3,0
  SOCMaxCenter 4
end
```

This is followed by a %casscf block containing maxiter 1, the two crossing multiplicities, one root per multiplicity, nel, nOrb, and the relativistic SOC settings:

```
%casscf
  maxiter 1
  mult {mult_list}
  nroots {nroots_list}
  nel {nel_soc}
  nOrb {norb_soc}
  rel
    dosoc true
    dosscc false
    PrintLevel 3
    TPrint 0.01
  end
end
```

For SOC construction, Step 1 transferable percent blocks are reused with %output excluded, after which Step 2 inserts its own fixed %output block. No %moinp/MoRead reference is inserted by SPINKS. The Cartesian block uses the optimized MECP geometry and max(soc_mults) as the xyz multiplicity.

The generated SOC input is fully editable before execution. If the SOC job subsequently fails, both implementations use:

```
repair_policy = "free_full"
```

with no locked fields, allowing an advanced user to deliberately modify the active space, roots, relativistic settings, or other SOC-specific options.

The completed-output reuse path is implemented differently only at the filesystem/bookkeeping level. Local PC uses `soc_status_initial` and reads the existing local SOC input when present; if that input file is absent, it stores a short local placeholder noting that the completed SOC output was reused. HPC uses `soc_status_preexisting` and attempts to read the remote SOC input through SFTP; if it is unavailable, a descriptive placeholder is retained.

If no completed SOC output exists, Local PC executes the editable input directly in the local SOC directory through `submit_and_monitor_orca_job(...)`, whereas HPC submits and monitors the same logical job in the remote SOC directory. If the SOC output was already complete, both versions skip the new SOC run and parse the existing output instead.

4.2.8 Extraction of the SOC matrix

After the CASSCF calculation has completed, SPINKS parses the section:

```
NONZERO SOC MATRIX ELEMENTS (cm**-1)
```

from the ORCA output.

Each printed element is stored as a complex number formed from its real and imaginary components, as defined in Equation S5:

$$H_{ij} = H_{ij}^{\text{real}} + iH_{ij}^{\text{imag}} \quad (\text{S5})$$

The corresponding spin quantum numbers are stored as:

```
bra_S  
bra_Ms  
ket_S  
ket_Ms
```

using:

```
parse_orca_soc_matrix_elements(...)
```

Only intermultiplicity elements are subsequently placed into the low-spin/high-spin M_s matrix; the raw parsed records remain available in `soc_values`.

4.2.9 Construction of the M_s -resolved SOC matrix

The extracted SOC elements are reorganized into the matrix:

SOC_Ms_matrix_cm1

whose rows correspond to the lower-spin M_s values and whose columns correspond to the higher-spin M_s values.

The implementation is performed by:

build_ms_resolved_soc_matrix(...)

The associated absolute-value matrix is stored as SOC_Ms_abs_cm1, and every nonzero matrix element is also collected in H_SO_channels_cm1 using an internal S/ M_s -resolved channel key.

4.2.10 Effective, intermediate, and M_s -specific SOC

The effective ORCA spin-orbit coupling is defined as the Euclidean (Frobenius) norm of the complete intermultiplicity M_s -resolved SOC matrix given in Equation S6:

$$H_{SO} = \sqrt{\sum_{M_S, M'_S} |\langle S, M_S | H_{SO} | S', M'_S \rangle|^2} \quad (S6)$$

The implementation evaluates this norm with compute_effective_soc_norm (SOC_Ms_matrix_cm1).

The same norm is calculated in both implementations. Local PC stores the diagnostic value as H_SO_full_matrix_norm_cm, whereas the current HPC code uses the temporary name H_SO_all_matrix_elements_cm. Local PC additionally exports H_SO_full_matrix_norm_cm; the HPC temporary name is not separately exported. Both versions assign the physically active ORCA-derived value to the common downstream variable:

H_SO_ORCA_effective_cm

No RMS averaging or empirical factor is applied. H_SO_cm is initially assigned this ORCA-derived effective value, and H_SO_source is set to "ORCA SOC calculation".

All nonzero M_s -specific complex elements are stored in H_SO_channels_cm1. A second dictionary is used to classify these elements:

H_SO_intermediate_components_cm1

This dictionary classifies individual nonzero matrix elements by spin-projection change: $\Delta M_s = 0$ components receive z labels, $|\Delta M_s| = 1$ components receive ib labels, and any other nonzero components are retained under other labels.

For recognized singlet-triplet, doublet-quartet, triplet-quintet, and quartet-sextet pairs, the same classified component dictionary is additionally exposed through H_SO_ST_components_cm1, H_SO_DQ_components_cm1, H_SO_TQ_components_cm1, or H_SO_QS_components_cm1, respectively.

The Step 2 classified components are individual complex matrix elements. They must not be confused with the intermediate scalar SOC values generated later in Step 8, where symmetry-related M_s rows are combined through row norms for the probability calculations.

SOC_CONVENTION_FACTOR is fixed at 1.0 in both implementations. SOC_CONVENTION_LABEL uses slightly different wording in the two scripts, but both labels specify the same full M_s -resolved matrix norm with no RMS averaging. Thus, the implementation-specific text label does not change the numerical SOC convention.

4.2.11 User override of the effective SOC

Although H_SO_cm is initially the ORCA-derived effective SOC, SPINKS allows the user to provide a non-negative external scalar SOC for downstream calculations.

If selected, the user-defined scalar:

H_SO_cm

The user-defined scalar replaces the active downstream value. The original ORCA effective value remains stored in H_SO_ORCA_effective_cm for reference. Because a single user-supplied scalar does not define an M_s -resolved matrix, SPINKS sets SOC_EFFECTIVE_ONLY = True and SOC_MATRIX_CHANNELS_AVAILABLE = False, zeros the active SOC matrix arrays, and clears the channel/intermediate component dictionaries.

If the ORCA SOC table contains no usable nonzero channels, the code similarly enters effective-only mode with H_SO_cm = 0.0. Otherwise the full matrix and channel-resolved information remain available.

H_SO_source identifies whether the active scalar came from the ORCA SOC calculation, a user-supplied value, or the zero-SOC fallback.

4.2.12 Variables exported by step 2

The principal scientific quantities exported for later steps are common to both implementations:

```
H_SO_cm  
H_SO_ORCA_effective_cm  
H_SO_user_cm  
H_SO_source  
SOC_Ms_matrix_cm1  
SOC_Ms_abs_cm1  
H_SO_channels_cm1  
H_SO_intermediate_components_cm1  
SOC_EFFECTIVE_ONLY  
SOC_MATRIX_CHANNELS_AVAILABLE
```

Both versions also export `H_SO_raw_ORCA_norm_cm`, the scaled-matrix compatibility aliases, the spin-pair-specific component dictionaries, `SOC_CONVENTION_FACTOR/SOC_CONVENTION_LABEL`, the parsed SOC records, ROHF/UNO active-space metadata, `soc_orbital_source`, `soc_active_indices`, `soc_nel`, `soc_norb`, `soc_mults`, `soc_protocol`, `soc_protocol_reason`, `lmo_diagnostics`, the detected metal symbol, and the Step 2 diagnostic-file paths.

A few implementation-specific bookkeeping names remain. Local PC exports the UNO occupations as `uhf_uno_occupations` and additionally exports `H_SO_full_matrix_norm_cm`, `orca_loc_failed`, and `orca_loc_failure_reason`. The current HPC implementation exports the UNO occupation list as `uno_occupations`; its `H_SO_all_matrix_elements_cm` variable is used only as a temporary diagnostic before `H_SO_ORCA_effective_cm` is assigned.

The path variables `SOC_FOLDER_REMOTE`, `SOC_OUT_REMOTE`, `SOC_INP_REMOTE`, `SOC_GBW_REMOTE`, `SOC_LOC_REMOTE`, `SOC_LOCINP_REMOTE`, and `SOC_ORCA_LOC_STDOUT_REMOTE` are retained in both implementations. In Local PC mode these "REMOTE" names point to local files for backward compatibility; in HPC mode they are genuine remote paths.

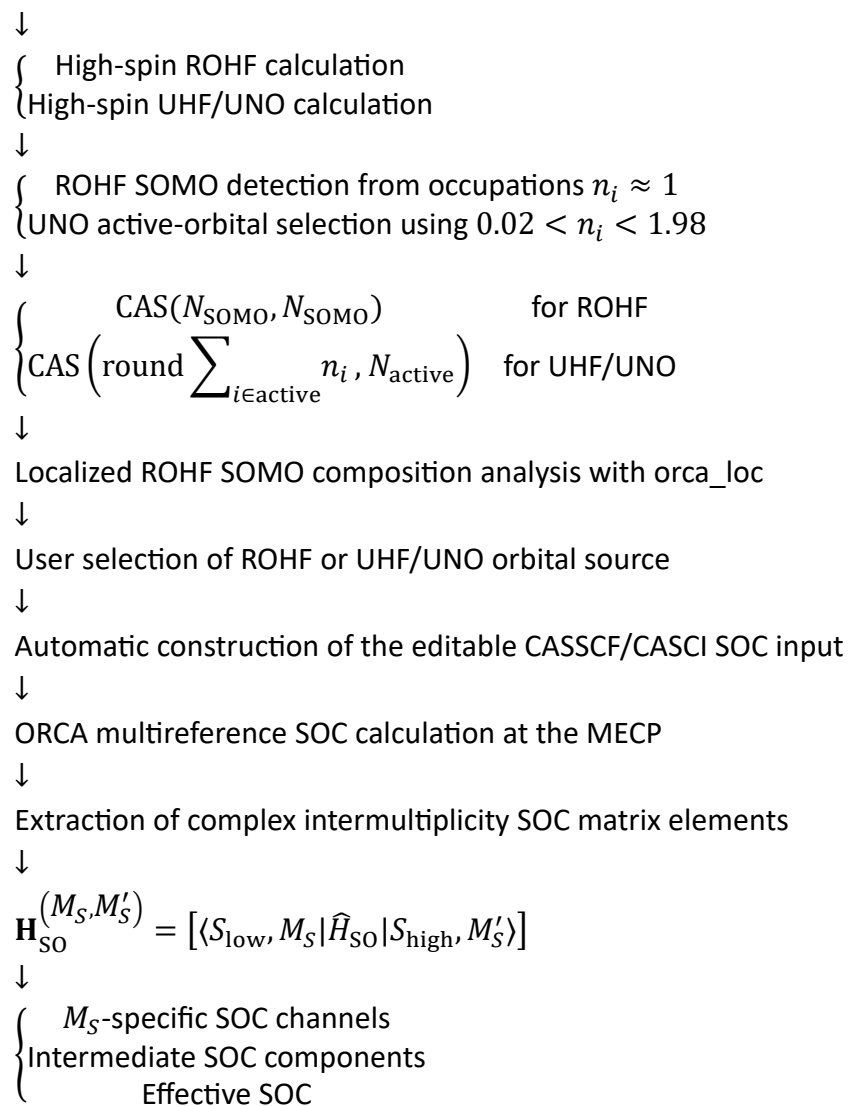
Output-file handling also differs. In Local PC mode, the ORCA and diagnostic files are already present in `local_soc`, so no transfer step is required. In HPC mode, important remote ROHF, UHF/UNO, localization, and SOC files are downloaded to `local_soc` when available. The locally generated diagnostic text files are then uploaded back to the remote SOC directory by SFTP.

4.2.13 Summary of the step 2 algorithm

The complete Step 2 sequence is summarized below:

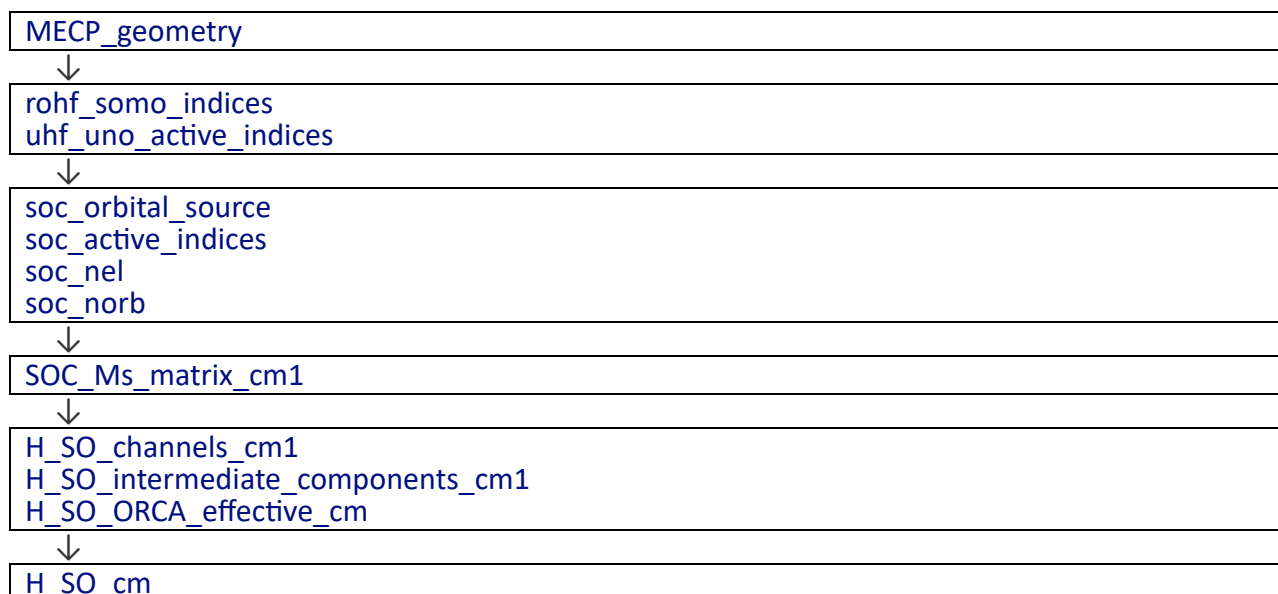
The scientific sequence below is common to both implementations. Local PC carries it out entirely on the local filesystem, whereas HPC performs the electronic-structure and localization execution in the remote SOC directory and mirrors the resulting files locally.

Optimized MECP geometry from Step 1



The effective ORCA scalar is the full M_S -resolved matrix norm described above.

The principal variable flow is:



If the ORCA-derived SOC is retained, SOC_EFFECTIVE_ONLY is False and the complete M_s -resolved matrix and channel representations remain available. If a scalar override is supplied—or no usable ORCA channels exist—the code switches to effective-only mode and later steps use H_SO_cm without M_s -resolved channel analysis.

Step 2 therefore converts the optimized MECP geometry (or a previously completed SOC output at that geometry) into the effective scalar SOC and, when available, the channel-resolved SOC information required downstream.

$$\text{MECP geometry} \rightarrow \text{reference orbitals} \rightarrow \text{active space} \rightarrow \text{SOC matrix} \rightarrow \left\{ H_{\text{SO}}^{\text{eff}}, H_{\text{SO}}^{\text{int}}, H_{\text{SO}}^{M_S, M'_S} \right\}$$

These quantities are used in Step 8 to evaluate Landau–Zener and weak-coupling probabilities using the effective SOC. When SOC_MATRIX_CHANNELS_AVAILABLE is True, Step 8 additionally evaluates the corresponding intermediate and M_s -specific probability curves.

4.3 Step 3: MECP Energy Gradients

Step 3 evaluates the analytical energy gradients of the two spin-state potential-energy surfaces at the optimized MECP geometry obtained in Step 1. The resulting Cartesian gradient vectors provide the nuclear derivative information used to define the surface-gradient difference, its normalized Cartesian directions, and the gradient diagnostics passed to Step 4. The final reaction-coordinate-projected gradient mean is not defined in Step 3; it is calculated later from the selected effective-Hessian reaction coordinate.

The principal operations performed in Step 3 are:

1. verification and import of the MECP geometry and computational settings;
2. construction of one ORCA Engrad input for each spin multiplicity;
3. propagation of the reviewed DFT settings from Step 1;
4. execution and monitoring of the two analytical-gradient calculations;
5. extraction of the $3N$ -dimensional Cartesian gradient vectors from the ORCA .engrad files;
6. assignment of the lower- and higher-spin gradients;
7. construction of both Cartesian gradient-difference orientations;
8. normalization of the gradient-difference vectors and definition of the Cartesian MECP seam-normal/reaction-direction aliases; and
9. storage of the gradients, Cartesian gradient diagnostics, and associated source-file metadata for later steps.

The Local PC and HPC implementations use the same scientific Engrad protocol, gradient parser, and downstream numerical definitions. Their differences are operational. The Local PC version executes ORCA jobs in the local Engrad directory and retains variables with remote-style names as local compatibility aliases. The HPC version creates separate local and remote Engrad directories, submits both gradient jobs to the remote scheduler, monitors them together, retrieves the generated files, and uploads the final gradient summary. The implementation-specific details are identified explicitly below.

4.3.1 Import of quantities from the preceding steps

Step 3 requires the optimized MECP geometry and the electronic-structure settings established in Step 1. These dependencies are verified before any calculation is initiated:

The core scientific inputs are common to both implementations:

```
required_vars_step3 = [  
    "jobname",  
    "MECP_geometry",  
    "charge",  
    "method",  
    "basis",  
    "nprocs",  
    "mem_gb",  
    "maxcore_mb",  
    "multiplicities",  
    "mult_main",  
    "mult_other",  
    "local_base",  
    "remote_base",  
    ...  
]
```

The Local PC required-variable check additionally lists the local job-record helpers:

```
"submit_orca_job_no_monitor",  
"monitor_submitted_orca_jobs"
```

In the current Local PC Step 3 code, the actual job-execution block calls `submit_and_monitor_orca_job(...)` for each Engrad calculation and later calls `monitor_submitted_orca_jobs(...)` on the resulting records. Thus, the execution path uses `submit_and_monitor_orca_job(...)` even though that function is not explicitly included in the Local PC `required_vars_step3` list.

The HPC implementation instead requires the remote connection, file-system, and cluster-submission helpers in addition to the common scientific variables:

```
"ssh",  
"sftp",  
"remote_mkdir_p",  
"remote_file_exists",  
"remote_read_text",  
"run_ssh",  
"submit_orca_job_no_monitor",  
"monitor_submitted_orca_jobs"
```

The HPC version also verifies that the cluster shell template and submission-command settings established in Step 1 are available, and then marks those settings as already reviewed. Both implementations set `workflow_mode = "MECP_ONLY"` and terminate if `MECP_geometry` is missing or empty.

The most important imported quantity is:

```
MECP_geometry
```

which contains the optimized Cartesian geometry of the crossing point. Both spin-state gradients must be evaluated at the same nuclear configuration. Step 3 therefore terminates if MECP_geometry is missing or empty.

The two multiplicities participating in the crossing are obtained from:

```
mult_main  
mult_other
```

and placed in numerical order:

```
engrad_mults = sorted([  
    int(mult_main),  
    int(mult_other)  
])
```

The smaller multiplicity is later identified as the low-spin state and the larger multiplicity as the high-spin state.

4.3.2 Preparation of the gradient-calculation method

The method string supplied in Step 1 may contain job-specific ORCA keywords such as Opt, Freq, SurfCrossOpt, or Engrad. These keywords must not be propagated indiscriminately into the new gradient input.

SPINKS removes incompatible job keywords using:

```
def clean_method_for_engrad(method_text):  
    excluded = {  
        "surfcrossopt",  
        "surfcrossnumfreq",  
        "numfreq",  
        "freq",  
        "opt",  
        "engrad"  
    }
```

The resulting electronic-structure method is stored as:

```
engrad_method
```

For example, a reviewed Step 1 header containing:

```
! B3LYP D3 def2-TZVP TightSCF SlowConv Opt Freq
```

is adapted for Step 3 by removing Opt and Freq and introducing the required Engrad keyword.

This prevents incompatible calculation types from being included simultaneously while preserving the selected functional, dispersion correction, basis set, convergence settings, solvent model, and other transferable ORCA options.

4.3.3 Directory preparation

All Step 3 files are placed in a dedicated Engrad directory:

```
local_engrad = os.path.join(  
    local_base,  
    "Engrad"  
)
```

This directory contains:

- the two generated ORCA input files;
- the corresponding ORCA output files;
- the binary and text files generated by ORCA;
- the two .engrad files;
- revised inputs from any failed-job recovery; and
- the final gradient-summary file.

In the Local PC implementation, the remote-style directory name is only a compatibility alias:

```
remote_engrad = local_engrad
```

No remote directory is created in Local PC mode.

In the HPC implementation, a true remote Engrad directory is created independently of the local copy:

```
remote_engrad = posixpath.join(remote_base, "Engrad")  
remote_mkdir_p(sftp, remote_engrad)  
local_engrad = os.path.join(local_base, "Engrad")  
os.makedirs(local_engrad, exist_ok=True)
```

Accordingly, Local PC keeps all Step 3 files in one local directory, whereas HPC maintains parallel local and remote Engrad directories.

4.3.4 Construction of the ORCA Engrad inputs

A separate analytical-gradient calculation is generated for each spin multiplicity. Both implementations are designed to propagate the reviewed Step 1 DFT settings, but the code path used to construct the input differs.

In the Local PC implementation, `make_engrad_input(...)` first checks whether the reviewed Step 1 helper is available:

```
if "make_dft_pre_xyz" in globals():
    new_pre = make_dft_pre_xyz("engrad")
    return f'''{new_pre.rstrip()}

* xyz {charge} {mult}
{MECP_geometry}
*
'''
```

This is the preferred Local PC route and directly reuses the reviewed pre-geometry ORCA settings. If `make_dft_pre_xyz` is unavailable in an older session, the Local PC code falls back to constructing the Engrad header, `%pal`, `%maxcore`, `SCF`, `%output`, and `xyz` blocks explicitly.

The HPC implementation contains fallback definitions of the Step 1 DFT-settings propagation helpers when they are not already present. Its `make_engrad_input(...)` first builds a complete Engrad template and then applies the reviewed settings through:

```
return apply_dft_settings_to_input(
    inp,
    job_type="engrad"
)
```

In both implementations, the fallback SCF settings for the first Engrad attempt use `MaxIter 1500`, `SOSCFStart 0.01`, and `LevelShift 0.5`. The reviewed Step 1 settings remain authoritative whenever the propagation helpers are available.

Thus, both spin surfaces are evaluated using:

- the same electronic-structure method;
- the same basis set;
- the same SCF settings;
- the same solvent and dispersion treatment, where applicable;
- the same molecular charge;
- the same MECP geometry; and
- their respective spin multiplicities.

A representative header is:

```
! Engrad B3LYP D3 def2-TZVP TightSCF SlowConv
```

The consistency of the geometry and electronic-structure settings is important because the gradient difference must represent the local difference between the two potential-energy surfaces at one common nuclear configuration.

4.3.5 Generation of the two spin-state jobs

The program creates one job definition for each multiplicity:

```
for mult in engrad_mults:
    spin_name = multiplicity_name(mult)

    engrad_jobname = (
        f"{jobname}_Engrad_{spin_name}"
    )
```

For a singlet–triplet crossing, for example, the generated job names are:

```
<jobname>_Engrad_singlet
<jobname>_Engrad_triplet
```

Each job definition contains the multiplicity, spin-state name, input text, job name, and descriptive label:

```
engrad_job_definitions.append({
    "mult": mult,
    "spin_name": spin_name,
    "jobname": engrad_jobname,
    "inp_text": inp_text,
    "job_label":
        f"{spin_name} DFT Engrad at MECF"
})
```

This data structure preserves the correspondence between each generated file and its associated spin state.

4.3.6 Execution and protected input repair

The two implementations use different execution paths while enforcing the same scientific repair restrictions.

Local PC executes each Engrad calculation through the local job wrapper:

```

status, final_inp = submit_and_monitor_orca_job(
    remote_dir=local_engrad,
    jobname_i=job["jobname"],
    inp_text=job["inp_text"],
    job_label=job["job_label"],
    allow_interactive_repair=True,
    repair_policy="engrad_restricted",
    locked={...}
)

```

The returned final input is stored in a local job record with short_id = "LOCAL". After both local records have been created, the current code makes a further monitoring call:

```

submitted_engrad_jobs = monitor_submitted_orca_jobs(
    local_engrad,
    submitted_engrad_jobs,
    allow_interactive_repair=True
)

```

HPC instead submits both jobs without waiting, attaches the same repair policy and locked scientific quantities to each job record, and only then monitors the two calculations together:

```

job_record = submit_orca_job_no_monitor(
    remote_engrad,
    job["jobname"],
    job["inp_text"],
    job["job_label"],
    attempt=1
)
submitted_engrad_jobs = monitor_submitted_orca_jobs(
    remote_engrad,
    submitted_engrad_jobs,
    allow_interactive_repair=True
)

```

For both implementations, the repair policy is "engrad_restricted" and the protected quantities are:

```

locked = {
    "method": engrad_method,
    "basis": basis,
    "charge": charge,
    "mult": job["mult"],
    "geom": MECP_geometry
}

```

A revised input must therefore retain:

- the required Engrad keyword;
- the selected method;
- the selected basis set;

- the molecular charge;
- the appropriate multiplicity; and
- the optimized MECP geometry.

The user may modify convergence-related settings or add compatible ORCA options if a calculation fails, but cannot alter the quantities that define the physical gradient calculation.

After execution, both implementations require every stored job record to have status "OK" before Step 3 proceeds. In the HPC workflow, the two remote jobs are submitted first and monitored together; in the Local PC workflow, each job is run through the local wrapper and the resulting records are subsequently checked again by the monitoring routine.

4.3.7 Location and parsing of the ORCA .engrad files

ORCA writes analytical gradients to .engrad files. The two implementations locate these files differently because one works on the local file system and the other on the remote cluster.

Local PC uses `local_find_engrad_file(...)`. It first checks the expected local path and, if necessary, searches the local Engrad directory for a matching filename:

```
expected = local_dir / f"{jobname_i}.engrad"
candidates = sorted(local_dir.glob(f"{jobname_i}*.engrad"))
```

HPC uses `remote_find_engrad_file(...)`. It first tests the expected remote path through `remote_file_exists(...)`; if that file is absent, it issues an SSH `ls` command for the first matching remote .engrad file and returns the corresponding POSIX path.

The gradient is extracted using:

```
extract_gradient_from_engrad_text(...)
```

The parser first reads the number of atoms, N , from the section marked:

```
Number of atoms
```

and then locates the section:

```
The current gradient in Eh/bohr
```

Exactly $3N$ Cartesian gradient components are collected:

```
expected = 3 * natoms
```

The parser raises an error unless the number of extracted values is exactly $3N$. This validation prevents incomplete or malformed gradient files from being used in subsequent calculations.

The extracted gradient is returned as a one-dimensional NumPy array:

```
gradient = np.array(
    gradient_values,
    dtype=float
)
```

In Local PC mode, the located file is read directly from disk. In HPC mode, the .engrad text is read remotely, copied into the local Engrad directory, and the corresponding .inp, .out, and .gbw files are downloaded when present. The gradient parser itself is identical in both implementations.

The ordering is given in Equation S7:

$$\mathbf{g} = \left(\frac{\partial E}{\partial x_1}, \frac{\partial E}{\partial y_1}, \frac{\partial E}{\partial z_1}, \dots, \frac{\partial E}{\partial z_N} \right) \quad (\text{S7})$$

The units are Hartree per bohr.

4.3.8 Storage of the spin-state gradients

The extracted gradients are stored by both multiplicity and spin-state name:

```
gradients_by_multiplicity[mult] = gradient
gradients_by_name[spin_name] = gradient
```

They are also exported as dynamically named variables:

```
globals()[f"g_{spin_name}"] = gradient
```

The source-file bookkeeping is implementation-specific. Local PC initializes both engrad_files_local and engrad_files_remote and stores the same local .engrad path in both dictionaries. HPC initializes engrad_files_remote only; its entries are the true remote .engrad paths. The downloaded local copies in the HPC Engrad directory are not collected into a separate engrad_files_local dictionary in the current cluster Step 3 code.

For example, a singlet–triplet calculation produces variables equivalent to:

```
g_LS
g_HS
```

The two general-purpose gradient variables used by later steps are assigned as:

```
g_LS = gradients_by_multiplicity[mult_LS]
g_HS = gradients_by_multiplicity[mult_HS]
```

where:

```
mult_LS = available_mults[0]
mult_HS = available_mults[-1]
```

The code verifies that both vectors have identical dimensions:

```
if g_LS.shape != g_HS.shape:
    raise RuntimeError(...)
```

This confirms that both gradients correspond to the same number of atoms and the same Cartesian coordinate ordering.

4.3.9 Gradient norms

The first Cartesian gradient norm is given in Equation S8:

$$\| \mathbf{g}_{\text{LS}} \| = \sqrt{\sum_{k=1}^{3N} g_{\text{LS},k}^2} \quad (\text{S8})$$

The corresponding expression is given in Equation S9:

$$\| \mathbf{g}_{\text{HS}} \| = \sqrt{\sum_{k=1}^{3N} g_{\text{HS},k}^2} \quad (\text{S9})$$

The corresponding implementation is:

```
g_LS_norm_Eh_per_Bohr = float(
    np.linalg.norm(g_LS)
)

g_HS_norm_Eh_per_Bohr = float(
    np.linalg.norm(g_HS)
)
```

These values are retained for the gradient analysis performed in Step 4.

Step 3 also evaluates the Cartesian geometric mean of the two gradient norms:

```
cartesian_geometric_mean_gradient_Eh_per_Bohr = float(
    np.sqrt(
        g_LS_norm_Eh_per_Bohr *
        g_HS_norm_Eh_per_Bohr
    )
)
```

Equivalently, the expression can be written as Equation S10:

$$g_{\text{mean}} = \sqrt{\| \mathbf{g}_{\text{LS}} \| \| \mathbf{g}_{\text{HS}} \|} \quad (\text{S10})$$

This quantity is stored as CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR and is only a full-Cartesian diagnostic. PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is intentionally not defined in Step 3; the projected quantity is calculated later from the selected effective-Hessian reaction coordinate.

4.3.10 Cartesian gradient-difference vector

The selected-sign gradient-difference vector is constructed as:

```
surface_gradient_difference_cart = g_LS - g_HS
```

The corresponding expression is given in Equation S11:

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \quad (\text{S11})$$

This vector describes the first-order difference between the two surface gradients in Cartesian coordinate space. Its norm is calculated as:

```
GRADIENT_GAP_NORM_EH_PER_BOHR = float(  
    np.linalg.norm(surface_gradient_difference_cart)  
)
```

Equivalently, the expression can be written as Equation S12:

$$\|\Delta \mathbf{g}\| = \sqrt{\sum_{k=1}^{3N} (g_{\text{HS},k} - g_{\text{LS},k})^2} \quad (\text{S12})$$

The code rejects nonfinite or nonpositive values:

```
if (  
    not np.isfinite(  
        GRADIENT_GAP_NORM_EH_PER_BOHR  
    )  
    or GRADIENT_GAP_NORM_EH_PER_BOHR <= 0.0  
):  
    raise RuntimeError(...)
```

A zero gradient difference would prevent definition of a unique crossing-normal direction and would indicate an invalid or numerically degenerate result.

4.3.11 Crossing-seam normal direction

Step 3 retains both normalized sign conventions. The selected-sign direction is $\text{gradient_difference_unit_vector} = \text{surface_gradient_difference_cart} / \text{GRADIENT_GAP_NORM_EH_PER_BOHR}$, while the opposite direction is:

```
reverse_gradient_unit_vector = (  
    reverse_surface_gradient_difference_cart /  
    GRADIENT_GAP_NORM_EH_PER_BOHR  
)
```

The corresponding expression is given in Equation S13:

$$\mathbf{n}_{\text{cart}} = \frac{\Delta \mathbf{g}}{\|\Delta \mathbf{g}\|} \quad (\text{S13})$$

The reverse normalized direction is also copied to the compatibility aliases:

```
MECP_seam_normal_cart = reverse_gradient_unit_vector.copy()
```

By construction, Equation S14 holds:

$$\|\mathbf{n}_{\text{cart}}\| = 1 \quad (\text{S14})$$

SPINKS prints the norm as a diagnostic to verify numerical normalization.

At this stage these are Cartesian gradient-difference directions. The final reaction-coordinate direction used for projected quantities is selected later from the effective Hessian in Step 6.

4.3.12 Assignment of the gradient-gap magnitude

Step 3 assigns the explicit downstream aliases:

```
gradient_gap_magnitude = GRADIENT_GAP_NORM_EH_PER_BOHR  
gradient_gap_norm_Eh_per_Bohr = GRADIENT_GAP_NORM_EH_PER_BOHR
```

This gives Equation S15:

$$\Delta g_{\parallel} = \|\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}\| \quad (\text{S15})$$

GRADIENT_GAP_NORM_EH_PER_BOHR is the explicit unit-bearing scalar retained for the later spin-crossing calculations.

The uppercase variable name:

```
GRADIENT_GAP_NORM_EH_PER_BOHR
```

is retained throughout the remaining workflow to identify the active value and its units explicitly.

Step 4 recalculates and consolidates these Cartesian quantities, determines the crossing classification and effective-Hessian mixing coefficient, and again leaves the final reaction-coordinate-projected gradient mean for the effective-Hessian analysis.

4.3.13 Files generated by step 3

The complete gradient information is written to:

```
Engrad_gradients_summary.txt
```

In Local PC mode, the summary remains local and the remote-style summary variable is an alias:

```
remote_grad_file = local_grad_file
```

In HPC mode, the summary is first written locally and then uploaded to the remote Engrad directory:

```
remote_grad_file = posixpath.join(
    remote_engrad,
    os.path.basename(local_grad_file)
)
sftp.put(local_grad_file, remote_grad_file)
```

The Local PC summary labels the source paths as local Engrad files, whereas the HPC summary records the remote Engrad files. Apart from this path distinction, both summaries contain the same scientific gradient quantities and vector components.

The file records:

- electronic-structure method;
- basis set;
- molecular charge;
- both multiplicities;
- identities of the low- and high-spin surfaces;
- $\| \mathbf{g}_{\text{LS}} \| g_{\text{LS_norm_Eh_per_Bohr}}$;
- $\| \mathbf{g}_{\text{HS}} \| g_{\text{HS_norm_Eh_per_Bohr}}$;
- CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR;
- $\| \Delta \mathbf{g} \| \text{GRADIENT_GAP_NORM_EH_PER_BOHR}$;
- both Cartesian gradient-difference orientations;
- the relevant .engrad source paths (local paths in Local PC mode and remote paths in HPC mode);
- all components of $\mathbf{g}_{\text{LS}}$;
- all components of $\mathbf{g}_{\text{HS}}$;
- all components of gradient_difference_unit_vector; and $\Delta \mathbf{g}$

- all components of `reverse_gradient_unit_vector`.

The individual gradient vectors are written using full numerical precision so that the subsequent gradient analysis can be reproduced independently.

The HPC workflow additionally keeps local copies of the remote `.engrad` text and, when available, the associated `.inp`, `.out`, and `.gbw` files in `local_engrad`.

4.3.14 Variables exported by step 3

The principal variables passed to Step 4 are:

```
g_LS
g_HS
surface_gradient_difference_cart
reverse_surface_gradient_difference_cart
gradient_difference_unit_vector
reverse_gradient_unit_vector
```

together with the scalar quantities:

```
g_LS_norm_Eh_per_Bohr
g_HS_norm_Eh_per_Bohr
GRADIENT_GAP_NORM_EH_PER_BOHR
gradient_gap_magnitude
CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR
```

Both implementations export the core bookkeeping dictionaries:

```
gradients_by_multiplicity
gradients_by_name
natoms_by_multiplicity
engrad_files_remote
```

For Local PC, `engrad_files_remote` contains local compatibility paths, and the code additionally exports:

```
engrad_files_local
```

In the HPC implementation, `engrad_files_remote` contains the actual remote `.engrad` paths; the current cluster Step 3 code does not create or export an `engrad_files_local` dictionary.

Both implementations also retain `engrad_job_records`, `local_engrad`, `remote_engrad`, `engrad_method`, `engrad_mults`, the low- and high-spin multiplicities and names, the complete gradient vectors and diagnostics, and `local_grad_file` and `remote_grad_file`. The aliases `MECP_seam_normal_cart` and `reaction_direction_cart` are retained as copies of `reverse_gradient_unit_vector`.

4.3.15 Summary of the step 3 algorithm

The complete scientific sequence implemented in Step 3 can be summarized as:

Optimized MECP geometry from Step 1

↓

{ DFT analytical gradient on the low-spin surface
 { DFT analytical gradient on the high-spin surface

↓

{ $\mathbf{g}_{\text{LS}} = \nabla E_{\text{LS}}(\mathbf{R}_{\text{MECP}})$
 { $\mathbf{g}_{\text{HS}} = \nabla E_{\text{HS}}(\mathbf{R}_{\text{MECP}})$

↓

Validation that both vectors contain exactly $3N$ Cartesian components

↓

$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$

↓

$\Delta g_{\parallel} = \|\Delta \mathbf{g}\|$

↓

$\mathbf{n}_{\text{cart}} = \frac{\Delta \mathbf{g}}{\|\Delta \mathbf{g}\|}$

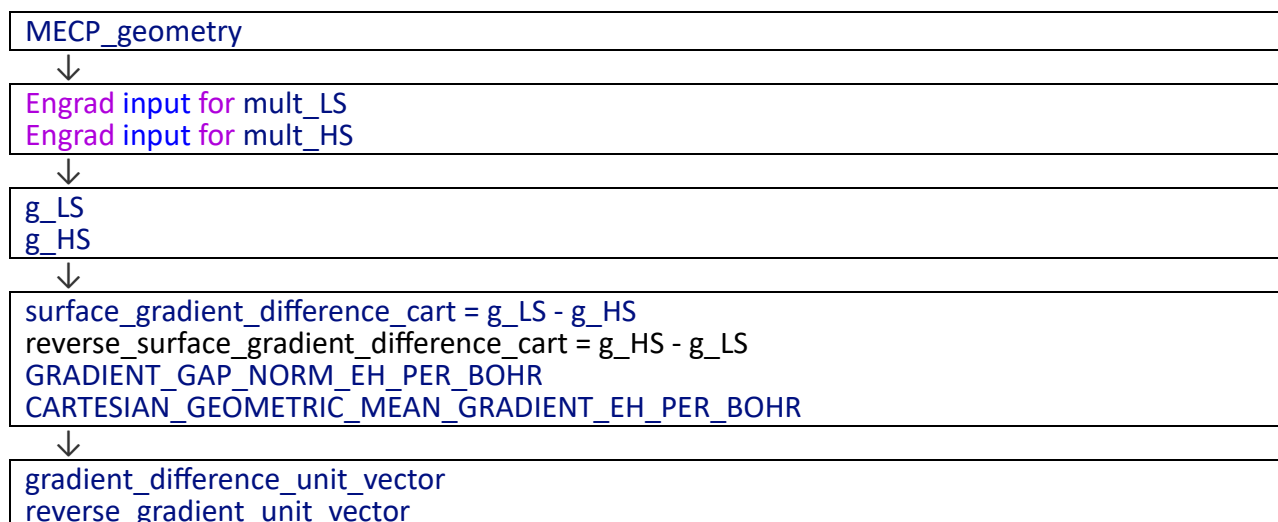
together with the diagnostic gradient magnitudes

$\|\mathbf{g}_{\text{LS}}\|, \|\mathbf{g}_{\text{HS}}\|,$

The corresponding expression is given in Equation S16:

$$g_{\text{mean}} = \sqrt{\|\mathbf{g}_{\text{LS}}\| \|\mathbf{g}_{\text{HS}}\|} \quad (\text{S16})$$

The corresponding variable flow is:



The scientific variable flow is identical in Local PC and HPC. Local PC uses local ORCA execution and local path aliases; HPC uses separate remote directories, remote file discovery, simultaneous cluster submission and monitoring, and SSH/SFTP transfer. Local PC additionally exports engrad_files_local, whereas HPC uses engrad_files_remote for the true cluster paths.

Therefore, Step 3 transforms the optimized MECP geometry into the two complete Cartesian energy-gradient vectors and their normalized difference direction:

$$\mathbf{R}_{\text{MECP}} \rightarrow \{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}\} \rightarrow \Delta \mathbf{g} \rightarrow \{\Delta g_{\parallel}, \mathbf{n}_{\text{cart}}\}$$

These quantities are subsequently re-evaluated in Step 4 to determine the crossing classification and effective-Hessian mixing coefficient. The final projected gradient mean and selected effective-Hessian reaction coordinate are obtained later in Step 6.

4.4 Step 4: Gradient Analysis and Crossing Classification

Step 4 reformulates the two Cartesian gradient vectors obtained in Step 3 into the gradient and directional conventions required by the downstream statistical treatment of the crossing point. No additional ORCA calculation is performed in this step. Instead, Step 4 operates entirely on the previously extracted low-spin and high-spin gradients.

The Local PC and HPC implementations perform the same scientific gradient analysis in Step 4. Because no new ORCA job is launched in this step, the differences are limited to bookkeeping and output handling: the Local PC version keeps the Step 4 summary entirely on the local filesystem, whereas the HPC version constructs a remote Step 4 path and, when the remote helpers are available, creates the remote directory and uploads the local summary file by SFTP.

The principal operations performed in Step 4 are:

1. verification and reshaping of the two Cartesian gradient vectors;
2. construction of the surface-gradient-difference vector;
3. preservation of the opposite sign convention used by later workflow sections;
4. evaluation of the gradient-difference magnitude;
5. calculation of the individual gradient norms;
6. evaluation of a Cartesian geometric-mean gradient diagnostic;
7. classification of the crossing as peaked or sloped;
8. calculation of the effective-Hessian weighting parameter;
9. construction of normalized Cartesian crossing directions; and
10. export of the resulting quantities for the effective-Hessian and kinetic calculations.

A critical distinction is made between the Cartesian diagnostic

$$\sqrt{\| \mathbf{g}_{\text{LS}} \| \| \mathbf{g}_{\text{HS}} \|}$$

calculated in Step 4 and the final reaction-coordinate-projected quantity `PROJECTED_GRADIENT_MEAN_EH_PER_BOHR`, which is evaluated later after selection of the effective-Hessian reaction coordinate.

4.4.1 Import of the two MECP gradients

Step 4 requires the same scientific and path variables in both implementations:

```
required_vars_step4 = [  
    "jobname",  
    "MECP_geometry",  
    "Reference_geometry",  
    "g_LS",  
    "g_HS",  
    "mult_LS",  
    "mult_HS",  
    "spin_name_LS",  
    "spin_name_HS",  
    "local_base",  
    "remote_base"  
]
```

If a required quantity is missing, the Local PC error message directs the user to run Local PC Steps 1–3 first, whereas the HPC error message directs the user to run cluster Steps 1–3 first. This wording difference does not alter the scientific inputs.

The two gradients are converted explicitly to one-dimensional floating-point arrays in both implementations:

```
g_LS = np.asarray(g_LS, dtype=float).reshape(-1)  
g_HS = np.asarray(g_HS, dtype=float).reshape(-1)
```

Their dimensions are then compared:

```
if g_LS.shape != g_HS.shape:  
    raise RuntimeError(...)
```

This check ensures that both vectors contain the same Cartesian components and correspond to the same MECP geometry and atom ordering.

The run-mode default is implementation-specific:

```
RUN_MODE = globals().get("RUN_MODE", "LOCAL")    # Local PC  
RUN_MODE = globals().get("RUN_MODE", "CLUSTER")  # HPC
```

Both versions then set `workflow_mode = "MECP_ONLY"`.

4.4.2 Removal of stale projected-gradient variables

Before defining the Step 4 quantities, both implementations remove the two projected-gradient aliases that could otherwise be mistaken for a current Step 4 result:

```
globals().pop("PROJECTED_GRADIENT_MEAN_EH_PER_BOHR", None)  
globals().pop("projected_gradient_mean_Eh_per_Bohr", None)
```

The Local PC implementation additionally removes the older compatibility alias:

```
globals().pop("projected_gradient_mean", None)
```

The current HPC Step 4 code does not include this third pop statement. In both implementations, the purpose is the same: Step 4 must not carry forward a stale projected gradient mean, because the authoritative PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is calculated later by projecting the gradients onto the selected effective-Hessian reaction coordinate.

4.4.3 Surface-gradient-difference convention

The SPINKS and effective-Hessian convention used in Step 4 defines the gradient-difference vector as Equation S17.

$$\Delta \mathbf{G}_{\text{diff}} = \mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}} \quad (\text{S17})$$

The implementation is:

```
surface_gradient_difference_cart = g_LS - g_HS
```

The vector surface_gradient_difference_cart is therefore the authoritative selected-sign gradient-difference vector.

Its Cartesian components are given in Equation S18:

$$\Delta G_k = g_{\text{LS},k} - g_{\text{HS},k}, k = 1, \dots, 3N \quad (\text{S18})$$

The sign determines the orientation of the normalized vector but does not affect its norm.

4.4.4 Reverse surface-gradient convention

Earlier workflow sections and some later calculations use the opposite gradient difference given in Equation S19:

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \quad (\text{S19})$$

The reverse surface-gradient-difference vector is retained as:

```
reverse_surface_gradient_difference_cart = g_HS - g_LS
```

The two gradient-difference vectors have opposite orientations and equal norms, as shown in Equation S20:

$$\Delta \mathbf{g} = -\Delta \mathbf{G}_{\text{diff}} \quad (\text{S20})$$

Thus, Equation S21 follows:

$$\|\Delta \mathbf{g}\| = \|\Delta \mathbf{G}_{\text{diff}}\| \quad (\text{S21})$$

Retaining both orientations makes the sign convention explicit while preserving the same gradient-gap magnitude.

4.4.5 Gradient-difference magnitude

Both implementations evaluate the magnitude of the same selected-sign surface-gradient-difference vector. The Local PC version stores the initial result in an explicitly unit-bearing internal variable:

```
gradient_gap_norm_Eh_per_Bohr = float(
    np.linalg.norm(surface_gradient_difference_cart)
)
```

The HPC version uses the shorter internal alias:

```
gradient_gap_norm = float(np.linalg.norm(surface_gradient_difference_cart))
```

The physical quantity is identical in both implementations. The corresponding expression is given in Equation S22:

$$\|\Delta \mathbf{g}_{\text{diff}}\| = \sqrt{\sum_{k=1}^{3N} (g_{\text{LS},k} - g_{\text{HS},k})^2} \quad (\text{S22})$$

Each implementation verifies that its internal norm is finite and positive before continuing.

In the Local PC implementation:

```
if (
    not np.isfinite(gradient_gap_norm_Eh_per_Bohr)
    or gradient_gap_norm_Eh_per_Bohr <= 0.0
):
    raise RuntimeError(...)
GRADIENT_GAP_NORM_EH_PER_BOHR = gradient_gap_norm_Eh_per_Bohr
gradient_gap_magnitude = GRADIENT_GAP_NORM_EH_PER_BOHR
```

In the HPC implementation:

```
if not np.isfinite(gradient_gap_norm) or gradient_gap_norm <= 0.0:
    raise RuntimeError(...)
gradient_gap_magnitude = gradient_gap_norm
GRADIENT_GAP_NORM_EH_PER_BOHR = gradient_gap_norm
```

Thus, both versions export the same implementation-independent gradient-gap magnitude through `GRADIENT_GAP_NORM_EH_PER_BOHR` and `gradient_gap_magnitude`. Therefore, Equation S23 follows:

$$\Delta g_{\parallel} = \|\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}\| \quad (\text{S23})$$

The units are Hartree per bohr.

`GRADIENT_GAP_NORM_EH_PER_BOHR` is the principal variable passed to the Landau-Zener and weak coupling probability calculations.

4.4.6 Individual gradient norms

The two implementations evaluate the same low-spin and high-spin Cartesian gradient norms, but the HPC code first stores them in shorter internal variables.

In the Local PC implementation:

```
g_LS_norm_Eh_per_Bohr = float(np.linalg.norm(g_LS))  
g_HS_norm_Eh_per_Bohr = float(np.linalg.norm(g_HS))
```

In the HPC implementation:

```
norm_grad_LS = float(np.linalg.norm(g_LS))  
norm_grad_HS = float(np.linalg.norm(g_HS))  
g_LS_norm_Eh_per_Bohr = norm_grad_LS  
g_HS_norm_Eh_per_Bohr = norm_grad_HS
```

The resulting physical values are therefore identical. Equivalently, the low-spin expression can be written as Equation S24:

$$g_{\text{LS}}^{\text{norm}} = \| \mathbf{g}_{\text{LS}} \| \quad (\text{S24})$$

The corresponding expression is given in Equation S25:

$$g_{\text{HS}}^{\text{norm}} = \| \mathbf{g}_{\text{HS}} \| \quad (\text{S25})$$

The corresponding variables are:

```
g_LS_norm_Eh_per_Bohr  
g_HS_norm_Eh_per_Bohr
```

These values are used in the Cartesian diagnostic and are also retained for later verification of the effective-Hessian projections.

4.4.7 Cartesian geometric-mean gradient diagnostic

Step 4 evaluates the same geometric mean of the two Cartesian gradient norms in both implementations.

The Local PC implementation uses the explicit unit-bearing norm variables:

```
cartesian_geometric_mean_gradient_Eh_per_Bohr = float(  
    np.sqrt(g_LS_norm_Eh_per_Bohr * g_HS_norm_Eh_per_Bohr)  
)
```

The HPC implementation uses its internal norm aliases:

```
cartesian_geometric_mean_gradient_Eh_per_Bohr = float(
    np.sqrt(norm_grad_LS * norm_grad_HS)
)
```

Both expressions are numerically equivalent. The corresponding expression is given in Equation S26:

$$g_{\text{mean}}^{\text{Cartesian}} = \sqrt{\| \mathbf{g}_{\text{LS}} \| \| \mathbf{g}_{\text{HS}} \|} \quad (\text{S26})$$

The value is stored as:

```
CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR
```

and:

```
cartesian_geometric_mean_gradient_Eh_per_Bohr
```

This quantity is retained only as a diagnostic of the full Cartesian gradients.

It is not the final projected gradient mean used in the weak coupling probability. The final value is obtained later from the gradient components projected along the effective-Hessian reaction coordinate.

This distinction is explicitly recorded in the output:

The output explicitly records that the final reaction-coordinate-projected PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is calculated later.

4.4.8 Crossing classification

The local crossing topology is classified using the scalar product of the two surface gradients:

```
gradient_dot_product = float(
    np.dot(g_LS, g_HS)
)
```

Equivalently, the expression can be written as Equation S27:

$$G_{\text{slope}} = \mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} \quad (\text{S27})$$

The classification is then assigned as:

```
if gradient_dot_product < 0.0:
    intersection_type = "peaked"
else:
    intersection_type = "sloped"
```

Thus, Equation S28 follows:

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} < 0 \Rightarrow \text{peaked crossing} \quad (\text{S28})$$

For the alternative case, Equation S29 gives:

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} \geq 0 \Rightarrow \text{sloped crossing} \quad (\text{S29})$$

At a peaked crossing, the two gradients point generally in opposing directions. At a sloped crossing, they possess a nonnegative mutual projection and therefore point more nearly in the same direction.

The classification is stored as:

```
intersection_type
```

for reporting and later diagnostics.

4.4.9 Effective-Hessian weighting parameter

Step 4 calculates the same effective-Hessian mixing coefficient in both implementations; only the internal name of the gradient-gap norm differs.

In the Local PC implementation:

```
effective_hessian_mix_coefficient = float(
    np.dot(surface_gradient_difference_cart, g_LS)
    / gradient_gap_norm_Eh_per_Bohr**2
)
```

In the HPC implementation:

```
effective_hessian_mix_coefficient = float(
    np.dot(surface_gradient_difference_cart, g_LS)
    / (gradient_gap_norm ** 2)
)
```

Because `gradient_gap_norm_Eh_per_Bohr` and `gradient_gap_norm` represent the same norm in their respective implementations, the physical definition is identical. In mathematical form, this is written as Equation S30:

$$\lambda = \frac{\Delta \mathbf{G}_{\text{diff}} \cdot \mathbf{g}_{\text{LS}}}{\|\Delta \mathbf{G}_{\text{diff}}\|^2} \quad (\text{S30})$$

The relation used here is stated in Equation S31:

$$\Delta \mathbf{G}_{\text{diff}} = \mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}} \quad (\text{S31})$$

The parameter may equivalently be written as shown in Equation S32:

$$\lambda = \frac{(\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}) \cdot \mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}\|^2} \quad (\text{S32})$$

This value is exported as:


```
effective_hessian_mix_coefficient
```

and is used later to form the effective crossing-point Hessian from the two spin-state Hessians.

Step 4 only calculates the weighting parameter. The Hessians themselves are obtained in Step 5 and combined in Step 6.

4.4.10 Normalized gradient-difference direction

The normalized selected-sign gradient-difference vector is constructed from the same numerator in both implementations.

In the Local PC implementation:

```
gradient_difference_unit_vector = (  
    surface_gradient_difference_cart /  
    gradient_gap_norm_Eh_per_Bohr  
)
```

In the HPC implementation:

```
gradient_difference_unit_vector = (  
    surface_gradient_difference_cart / gradient_gap_norm  
)
```

These definitions are equivalent because the denominators are the same physical norm. The corresponding expression is given in Equation S33:

$$\mathbf{n}_{\text{diff}} = \frac{\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}}{\|\mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}}\|} \quad (\text{S33})$$

By construction, Equation S34 holds:

$$\|\mathbf{n}_{\text{diff}}\| = 1 \quad (\text{S34})$$

The norm is printed as a numerical validation:

```
np.linalg.norm(gradient_difference_unit_vector)
```

This vector defines the SPINKS orientation of the Cartesian gradient-difference direction.

4.4.11 Reverse-gradient Cartesian direction

The opposite normalized Cartesian direction is retained in both implementations, again using the implementation-specific internal norm alias.

In the Local PC implementation:

```
reverse_gradient_unit_vector = (  
    reverse_surface_gradient_difference_cart /  
    gradient_gap_norm_Eh_per_Bohr  
)
```

In the HPC implementation:

```
reverse_gradient_unit_vector = (  
    reverse_surface_gradient_difference_cart / gradient_gap_norm  
)
```

Equivalently, the common physical expression can be written as Equation S35:

$$\mathbf{n}_{\text{cart}} = \frac{\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}\|} \quad (\text{S35})$$

Therefore, Equation S36 follows:

$$\mathbf{n}_{\text{cart}} = -\mathbf{n}_{\text{diff}} \quad (\text{S36})$$

The code also assigns:

```
MECP_seam_normal_cart = reverse_gradient_unit_vector.copy()  
reaction_direction_cart = reverse_gradient_unit_vector.copy()
```

These variables preserve the direction convention used by later parts of the workflow.

At this stage `reaction_direction_cart` remains a Cartesian gradient-difference alias. The final reaction-coordinate vector used for projected quantities is selected later from the effective Hessian in Step 6.

4.4.12 MECP-only workflow enforcement

Step 4 explicitly disables variables associated with transition-state and intrinsic-reaction-coordinate calculations:

```
IRC_frames = None  
IRC_energies_hartree = None  
coords_all = None  
irc = None  
symbols_irc = None  
TS_geometry = None  
irc_tangent = None  
TS_index = None
```

This prevents an IRC tangent or transition-state geometry from being used accidentally in the MECP-only workflow.

The reaction-coordinate definitions in SPINKS are therefore based exclusively on:

- the two spin-state gradients at the MECP;

- the corresponding spin-state Hessians; and
- the effective-Hessian eigenvectors.

No conventional transition-state IRC is required.

4.4.13 Separation of step 4 and the final projected gradient mean

The code intentionally does not define:

`PROJECTED_GRADIENT_MEAN_EH_PER_BOHR`

in Step 4.

Instead, Step 4 defines only:

`CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR`

This separation prevents two physically different quantities from being confused.

The Step 4 Cartesian diagnostic is defined in Equation S37:

$$g_{\text{mean}}^{\text{Cartesian}} = \sqrt{\| \mathbf{g}_{\text{LS}} \| \| \mathbf{g}_{\text{HS}} \|} \quad (\text{S37})$$

The later projected quantity is based on the components of $\mathbf{g}_{\text{LS}}$ and $\mathbf{g}_{\text{HS}}$ along the selected effective-Hessian reaction coordinate $\mathbf{e}_{\text{RC}}$ (Equation S38).

$$g_{\text{LS},\parallel} = \mathbf{g}_{\text{LS}} \cdot \mathbf{e}_{\text{RC}}, g_{\text{HS},\parallel} = \mathbf{g}_{\text{HS}} \cdot \mathbf{e}_{\text{RC}} \quad (\text{S38})$$

The final projected gradient mean is then obtained from those directional components rather than from the full Cartesian norms.

Accordingly, the quantity used downstream in the validated weak coupling probability is calculated only after the effective-Hessian reaction coordinate has been determined.

4.4.14 Files generated by step 4

Both implementations create the local Step 4 directory and write the same summary file:

```
local_step4 = os.path.join(local_base, "MECP_gradient_definitions")
os.makedirs(local_step4, exist_ok=True)
local_step4_file = os.path.join(
    local_step4,
    "MECP_gradient_definitions.txt"
)
```

The Local PC implementation retains compatibility aliases that point to these same local paths:

```
remote_step4 = local_step4
remote_step4_file = local_step4_file
```

The HPC implementation instead constructs a true remote directory with POSIX path handling and creates it when the remote helpers are available:

```
remote_step4 = posixpath.join(remote_base, "MECP_gradient_definitions")
if "remote_mkdir_p" in globals() and "sftp" in globals():
    remote_mkdir_p(sftp, remote_step4)
```

After the local summary is written, the HPC version constructs `remote_step4_file` and attempts to upload the file with `sftp.put(...)`. If that upload raises an exception, `remote_step4_file` is set to `None`.

The file records:

- identities and multiplicities of both spin surfaces;
- the SPINKS and opposite gradient-difference conventions;
- the low-spin gradient norm, $||g_{LS}||$;
- the high-spin gradient norm, $||g_{HS}||$;
- the gradient dot product, $g_{LS} \cdot g_{HS}$;
- the peaked or sloped classification;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR;
- effective_hessian_mix_coefficient;
- all components of g_{LS} ;
- all components of g_{HS} ;
- all components of both sign conventions for the gradient difference; and
- all components of both normalized crossing directions.

The output also explicitly states that the projected PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is not defined in Step 4.

Thus, the contents of `MECP_gradient_definitions.txt` are scientifically the same in both versions. The distinction is only where the file is retained: Local PC uses local compatibility aliases, whereas HPC preserves the local copy and, when possible, an uploaded remote copy.

4.4.15 Variables exported by step 4

The principal vectors exported to later steps are common to both implementations:

```
surface_gradient_difference_cart
reverse_surface_gradient_difference_cart
gradient_difference_unit_vector
reverse_gradient_unit_vector
MECP_seam_normal_cart
reaction_direction_cart
```

Both versions also export the Step 4 directory and summary-file paths:

```
local_step4
remote_step4
local_step4_file
remote_step4_file
```

For Local PC, the variables carrying the remote names are local aliases. For HPC, they refer to the constructed remote directory and remote summary path, with `remote_step4_file` becoming `None` if the optional SFTP upload fails.

The principal scalar quantities common to both implementations are:

```
gradient_gap_magnitude
GRADIENT_GAP_NORM_EH_PER_BOHR
cartesian_geometric_mean_gradient_Eh_per_Bohr
CARTESIAN_GEOMETRIC_MEAN_GRADIENT_EH_PER_BOHR
gradient_dot_product
intersection_type
effective_hessian_mix_coefficient
```

The implementation-specific internal gradient-gap alias is also exported:

```
gradient_gap_norm_Eh_per_Bohr  # Local PC
gradient_gap_norm               # HPC
```

The original surface gradients and their unit-bearing norm aliases remain available in both versions:

```
g_LS
g_HS
g_LS_norm_Eh_per_Bohr
g_HS_norm_Eh_per_Bohr
```

The HPC implementation additionally keeps `norm_grad_LS` and `norm_grad_HS` as ordinary top-level Step 4 variables because they are created before the `globals().update({...})` block, although they are not explicitly listed as keys in that export dictionary. These bookkeeping differences do not change the shared quantities used by the subsequent effective-Hessian analysis.

4.4.16 Summary of the step 4 algorithm

The complete sequence implemented in Step 4 can be summarized by Equation S39:

$$\{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}\} \downarrow \Delta \mathbf{G}_{\text{diff}} = \mathbf{g}_{\text{LS}} - \mathbf{g}_{\text{HS}} \quad (\text{S39})$$

The reverse surface-gradient convention is given in Equation S40:

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} = -\Delta \mathbf{G}_{\text{diff}} \quad (\text{S40})$$

The common gradient-difference magnitude is defined in Equation S41:

$$\Delta g_{\parallel} = \|\Delta \mathbf{G}_{\text{diff}}\| = \|\Delta \mathbf{g}\| \quad (\text{S41})$$

The first normalized directional vector is given in Equation S42:

$$\mathbf{n}_{\text{diff}} = \frac{\Delta \mathbf{G}_{\text{diff}}}{\|\Delta \mathbf{G}_{\text{diff}}\|} \quad (\text{S42})$$

The second normalized directional vector is given in Equation S43:

$$\mathbf{n}_{\text{cart}} = -\mathbf{n}_{\text{diff}} \quad (\text{S43})$$

The crossing topology is classified using Equation S44:

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} \begin{cases} < 0, & \text{peaked} \\ \geq 0, & \text{sloped} \end{cases} \quad (\text{S44})$$

The effective-Hessian weighting parameter is defined in Equation S45:

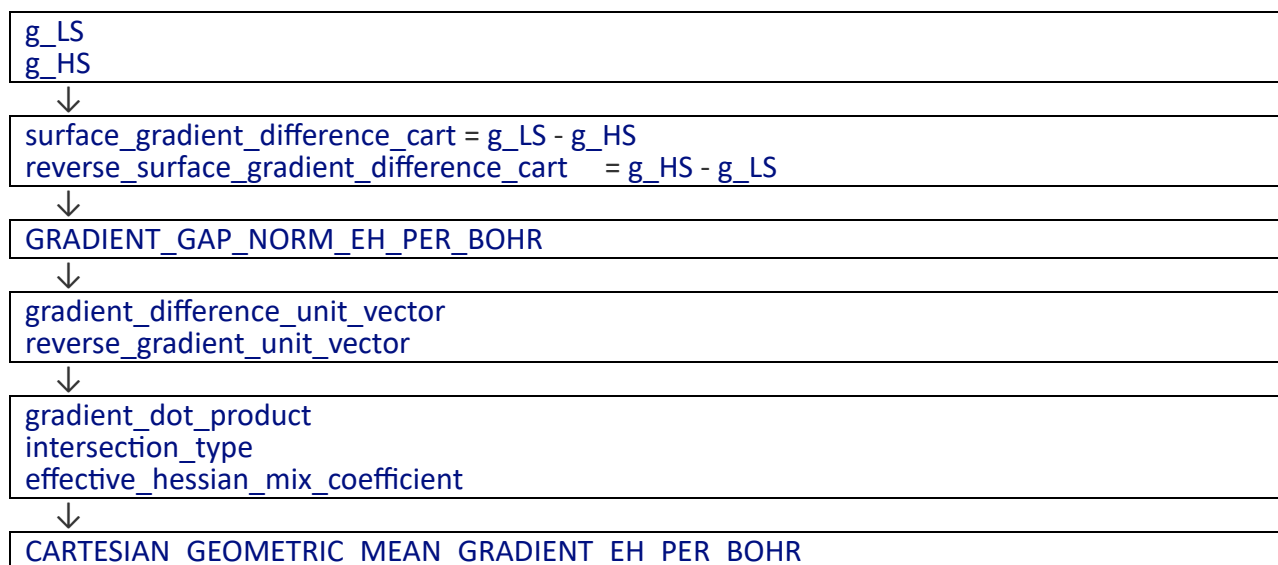
$$\lambda = \frac{\Delta \mathbf{G}_{\text{diff}} \cdot \mathbf{g}_{\text{LS}}}{\|\Delta \mathbf{G}_{\text{diff}}\|^2} \quad (\text{S45})$$

The Cartesian diagnostic gradient mean is defined in Equation S46:

$$g_{\text{mean}}^{\text{Cartesian}} = \sqrt{\|\mathbf{g}_{\text{LS}}\| \|\mathbf{g}_{\text{HS}}\|} \quad (\text{S46})$$

but the final projected SPINKS PROJECTED_GRADIENT_MEAN_EH_PER_BOHR is intentionally deferred until the effective-Hessian reaction coordinate has been obtained.

The corresponding variable flow is:



The scientific variable flow is therefore identical for Local PC and HPC. The implementation differences are limited to the internal gradient-gap alias ($gradient_gap_norm_Eh_per_Bohr$ versus $gradient_gap_norm$), the additional stale compatibility-alias cleanup in the Local PC code, and local-versus-remote handling of the Step 4 summary file.

Therefore, Step 4 transforms the raw Cartesian gradients from Step 3 into the selected-sign gradient difference, crossing-normal directions, crossing classification, and effective-Hessian weighting parameter:

$$\{g_{LS}, g_{HS}\} \rightarrow \{\Delta G_{diff}, \Delta g_{\parallel}, \mathbf{n}_{diff}, \lambda, \text{crossing type}\}$$

These quantities are subsequently combined with the two spin-state Hessians to construct the effective Hessian, identify the final reaction-coordinate mode, calculate the reduced mass, and evaluate $PROJECTED_GRADIENT_MEAN_EH_PER_BOHR$.

4.5 Step 5: MECP Hessians and Rotational Constants

Step 5 prepares the vibrational, rotational, symmetry, energy, and Hessian information required for the density-of-states treatment and for construction of the effective MECP Hessian in Step 6.

The reference-state vibrational data are reused directly from the optimized minimum obtained in Step 1. At the MECP, two separate SurfCrossNumFreq calculations are performed so that the Hessian information associated with both spin-surface orderings is available, as described in Equation S47:

$$\text{Job 1: xyz multiplicity} = m_{\text{main}}, \text{ partner multiplicity} = m_{\text{other}} \quad (\text{S47})$$

The second SurfCrossNumFreq ordering is given in Equation S48:

$$\text{Job 2: xyz multiplicity} = m_{\text{other}}, \text{ partner multiplicity} = m_{\text{main}} \quad (\text{S48})$$

Here, **main** denotes the multiplicity placed on the ORCA * xyz line, whereas **other** denotes the partner multiplicity specified in the %mecp block. The labels main/other and other/main therefore identify the two reversed spin-surface orderings used to obtain both MECP Hessians.

The raw ORCA PES2 frequencies generated by these calculations are retained only as diagnostics. They are not used as the final MECP frequencies in the downstream calculation. The final crossing-point frequencies are generated in Step 6 from the projected effective Hessian.

The Local PC and HPC implementations perform the same scientific operations in Step 5. They differ only in execution and file handling: the Local PC version runs ORCA directly in the local project directory, whereas the HPC version prepares remote and local frequency directories, submits the two SurfCrossNumFreq jobs to the cluster, monitors them together, and retrieves the required output and Hessian files.

The principal operations performed in Step 5 are:

1. import of the reference minimum and MECP data from the preceding steps;
2. reuse of the reference electronic energy and zero-point energy from Step 1;
3. extraction of reference vibrational frequencies, rotational constants, and symmetry number;
4. construction of two SurfCrossNumFreq inputs at the MECP;
5. execution and validation of both MECP frequency calculations;
6. collection of the two MECP Hessian files;
7. extraction of MECP rotational constants and symmetry number;

8. extraction of the ORCA PES2 frequency block for diagnostic purposes;
9. calculation of the provisional electronic MECP barrier;
10. preparation of the downstream variables required by later steps; and
11. storage of all reference and MECP metadata.

4.5.1 Import of quantities from the preceding steps

Step 5 verifies the scientific quantities required from Steps 1–4 together with the execution helpers required by the selected implementation. The scientific inputs are common to both versions, while the job-execution and file-access helpers differ.

In the Local PC implementation, the required-variable check is:

```
required_vars_step5 = [
    "jobname", "MECP_geometry", "Reference_multiplicity", "minima_data",
    "local_base", "remote_base", "method", "basis", "charge",
    "mult_main", "mult_other", "nprocs", "maxcore_mb",
    "submit_and_monitor_orca_job", "make_dft_pre_xyz",
]
```

In the HPC implementation, the corresponding check includes the cluster communication, remote-file, submission, and monitoring helpers:

```
required_vars_step5 = [
    "jobname",
    "MECP_geometry",
    "Reference_multiplicity",
    "minima_data",
    "local_base",
    "remote_base",
    "method",
    "basis",
    "charge",
    "mult_main",
    "mult_other",
    "nprocs",
    "maxcore_mb",
    "ssh",
    "sftp",
    "remote_mkdir_p",
    "remote_file_exists",
    "remote_read_text",
    "run_ssh",
    "submit_orca_job_no_monitor",
    "monitor_submitted_orca_jobs"
]
```

The principal scientific quantities imported by both implementations are:

```
Reference_multiplicity
minima_data
MECP_geometry
mult_main
mult_other
```

Reference_multiplicity identifies the lower-energy optimized minimum selected in Step 1, while minima_data contains its optimized geometry, electronic energy, zero-point energy, ORCA output path, and XYZ file path.

MECP_geometry contains the optimized crossing geometry at which the two Hessian calculations are performed. The Local PC implementation additionally verifies that MECP_geometry is a non-empty string before proceeding.

4.5.2 Energy-unit conversion

Step 5 defines the conversion factor:

```
hartree_to_cm = 219474.6313705
```

which is used to convert electronic-energy differences from Hartree to wavenumbers (Equation S49):

$$\Delta E (\text{cm}^{-1}) = \Delta E (E_h) \times 219474.6313705 \quad (\text{S49})$$

This conversion is used later when the electronic MECP barrier is calculated relative to the selected reference minimum.

4.5.3 Reuse of reference-state data from step 1

The reference-state record is obtained directly from the minima_data dictionary:

```
ref_data = minima_data[Reference_multiplicity]
```

The optimized reference geometry is then assigned as:

```
Reference_geometry = ref_data["geometry"]
```

The reference electronic energy is extracted using either of the supported field names:

```
if "Electronic_Eh" in ref_data:
    Ele_REF_hartree = float(ref_data["Electronic_Eh"])
elif "Ele_hartree" in ref_data:
    Ele_REF_hartree = float(ref_data["Ele_hartree"])
```

Similarly, the Step 1 zero-point energy is recovered as:

```
if "ZPE_Eh" in ref_data:
    ZPE_REF_thermo_hartree = float(ref_data["ZPE_Eh"])
elif "ZPE_hartree" in ref_data:
    ZPE_REF_thermo_hartree = float(ref_data["ZPE_hartree"])
```

The corresponding aliases are:

```
E_ref_hartree = Ele_REF_hartree
ZPE_REF_hartree = ZPE_REF_thermo_hartree
ZPE_ref_hartree = ZPE_REF_thermo_hartree
```

Thus, Step 5 does not rerun the reference minimum frequency calculation. It reuses the already validated Step 1 output, avoiding unnecessary duplication and ensuring that the same optimized minimum defines the energetic and vibrational reference throughout the workflow.

4.5.4 Extraction of reference rotational constants and symmetry number

The validated Step 1 reference output is accessed according to the selected implementation.

In the Local PC implementation:

```
local_ref_out = ref_data["out_path"]
local_ref_xyz = ref_data["xyz_path"]
ref_out_text = read_local_text_file(local_ref_out)
```

In the HPC implementation:

```
remote_ref_out = ref_data["out_path"]
remote_ref_xyz = ref_data["xyz_path"]
ref_out_text = read_remote_text_file(remote_ref_out)
```

Both implementations then parse ref_out_text using:

```
extract_symmetry_and_rot_constants_from_text(...)
```

The parser searches for the ORCA line containing:

```
Symmetry Number:
```

and for:

```
Rotational constants in cm-1
```

The three rotational constants are stored as:

```
Reference_rot_constants_cm1
```

and the rotational symmetry number as:

```
Reference_symmetry_number
```

If ORCA does not explicitly print a symmetry number, the code assigns:

```
symmetry_number = 1
```

This value is used as the default in both implementations.

The resulting quantities are used later in the rotational density-of-states correction (Equation S50):

$$\sigma_R = \text{reference rotational symmetry number} \quad (\text{S50})$$

The corresponding second rotational relation is given in Equation S51:

$$\mathbf{B}_R = (B_{R,A}, B_{R,B}, B_{R,C}) \quad (\text{S51})$$

4.5.5 Extraction and selection of the reference frequency block

An ORCA output can contain more than one vibrational-frequency section. SPINKS therefore identifies all blocks beginning with:

```
VIBRATIONAL FREQUENCIES
```

The frequency blocks are parsed using:

```
extract_orca_frequency_blocks_from_text(...)
```

Each frequency block is separated into:

```
block["imag"]
block["zero"]
block["real"]
```

The corresponding relation is given in Equation S52:

$$\nu_i < 0, \nu_i \approx 0, \nu_i > 0 \quad (\text{S52})$$

The reference frequency block is selected using:

```
select_reference_frequency_block_from_text(...)
```

The selection procedure is:

1. identify all frequency blocks;
2. retain blocks containing no imaginary frequencies;
3. among those, choose the block containing the largest number of real frequencies.

If all blocks contain imaginary frequencies, the code selects the block with the fewest imaginary modes and issues a warning.

The selected arrays are stored as:

```
freq_reactant_cm1_all
freq_reactant_real_cm1
freq_reactant_imag_cm1
freq_reactant_zero_cm1
```

The real frequencies are subsequently exported under the SPINKS name:

```
reference_vibrational_frequencies_cm1
```

and are used as the reactant vibrational frequencies in the density-of-states calculation.

4.5.6 Independent calculation of the reference ZPE from frequencies

Although the reference ZPE has already been obtained from the Step 1 thermochemistry output, Step 5 independently calculates it from the extracted positive frequencies:

```
def zpe_from_freqs_cm1(freqs_cm1):  
    freqs = freqs[  
        np.isfinite(freqs) & (freqs > 0.0)  
    ]  
    return 0.5 * float(np.sum(freqs))
```

Therefore, Equation S53 follows:

$$\text{ZPE}_R^{(\text{cm}^{-1})} = \frac{1}{2} \sum_{\nu_i > 0} \nu_i \quad (\text{S53})$$

The corresponding Hartree value is:

```
ZPE_REF_from_freq_hartree = (  
    ZPE_REF_from_freq_cm1 / hartree_to_cm  
)
```

Equivalently, the expression can be written as Equation S54:

$$\text{ZPE}_R^{(\text{Eh})} = \frac{\text{ZPE}_R^{(\text{cm}^{-1})}}{219474.6313705} \quad (\text{S54})$$

The two reference ZPE values are therefore kept separately:

```
ZPE_REF_thermo_hartree  
ZPE_REF_from_freq_hartree
```

The first is the ZPE extracted from the Step 1 thermochemistry data, whereas the second is reconstructed directly from the selected positive vibrational frequencies.

This provides a useful internal consistency check.

4.5.7 Preparation of the MECF frequency directory

Step 5 creates a Frequencies directory for the MECF SurfCrossNumFreq calculations and their Hessian files. The path handling differs between the two implementations.

In the Local PC implementation:

```
local_hess_dir = os.path.join(local_base, "Frequencies")  
os.makedirs(local_hess_dir, exist_ok=True)  
remote_hess_dir = local_hess_dir
```

Thus, `remote_hess_dir` is only a compatibility alias for the local frequency directory.

In the HPC implementation:

```
remote_hess_dir = posixpath.join(remote_base, "Frequencies")
remote_mkdir_p(sftp, remote_hess_dir)
local_hess_dir = os.path.join(local_base, "Frequencies")
os.makedirs(local_hess_dir, exist_ok=True)
```

The HPC version therefore maintains a true remote cluster directory for execution and a local Frequencies directory for retrieved files and metadata.

The Step 5 frequency directories contain, as appropriate to the implementation:

- both generated SurfCrossNumFreq inputs;
- both ORCA outputs;
- both Hessian files;
- copied or retrieved reference information;
- diagnostic PES2 frequency information; and
- rotational and symmetry metadata.

4.5.8 Construction of the SurfCrossNumFreq inputs

The MECP frequency inputs are generated using:

```
build_mecp_freq_input(
    geom_text,
    xyz_mult,
    mec_p_mult
)
```

The required MECP partner multiplicity is inserted through:

```
mecp_block = f"""%mecp Mult {int(mecp_mult)}
end
"""
```

The reviewed Step 1 DFT header and all transferable percent blocks are then propagated using:

```
pre_xyz = make_dft_pre_xyz(
    "freq",
    job_specific_blocks=mecp_block
)
```

The final ORCA input is constructed as:

```
return build_orca_input_from_pre_xyz(  
    pre_xyz,  
    charge,  
    xyz_mult,  
    geom_text  
)
```

This approach preserves user-reviewed settings such as:

- the DFT functional;
- the basis set;
- dispersion corrections;
- solvent models;
- custom basis or ECP blocks;
- integration-grid settings;
- SCF settings;
- parallelization settings; and
- memory settings.

At the same time, incompatible job keywords are removed and the required SurfCrossNumFreq specification is introduced.

4.5.9 Generation of both spin-surface orderings

Two separate inputs are generated:

```
mecp_main_other_inp = build_mecp_freq_input(  
    MECP_geometry,  
    xyz_mult=mult_main,  
    mecp_mult=mult_other  
)
```

The reverse ordering is generated as:

```
mecp_other_main_inp = build_mecp_freq_input(  
    MECP_geometry,  
    xyz_mult=mult_other,  
    mecp_mult=mult_main  
)
```

The two job definitions are therefore:

```
freq_jobs = [
    {
        "xyz_mult": mult_main,
        "mecp_mult": mult_other
    },
    {
        "xyz_mult": mult_other,
        "mecp_mult": mult_main
    }
]
```

The use of both orderings ensures that Hessian information is available for both electronic surfaces at the same MECP geometry.

The generated files are named:

```
<jobname>_Freq_MECP_main_other.inp
<jobname>_Freq_MECP_other_main.inp
```

This explicit naming preserves the relationship between the primary XYZ multiplicity and the partner MECP multiplicity.

4.5.10 Execution and protected repair of the MECP frequency jobs

The two implementations use the same protected scientific definition of each SurfCrossNumFreq job but different execution paths.

In the Local PC implementation, each job is executed and monitored locally through:

```
status, final_inp = submit_and_monitor_orca_job(
    remote_dir=local_hess_dir,
    jobname_i=job["jobname"],
    inp_text=job["inp_text"],
    job_label=job["job_label"],
    allow_interactive_repair=True,
    repair_policy="freq_restricted",
    locked={
        "method": clean_method_for_frequency(method),
        "basis": basis,
        "charge": charge,
        "mult": job["xyz_mult"],
        "mecp_mult": job["mecp_mult"],
        "geom": MECP_geometry,
    }
)
```

In the HPC implementation, both jobs are first submitted without waiting for either one to finish:

```
rec = submit_orca_job_no_monitor(
    remote_hess_dir,
```



```
job["jobname"],  
job["inp_text"],  
job["job_label"],  
attempt=1  
)
```

The same restricted repair policy and locked scientific quantities are attached to each submitted job record:

```
rec["repair_policy"] = "freq_restricted"  
rec["locked"] = {  
    "method": clean_method_for_frequency(method),  
    "basis": basis,  
    "charge": charge,  
    "mult": job["xyz_mult"],  
    "mecp_mult": job["mecp_mult"],  
    "geom": MECP_geometry,  
}
```

After both submissions have been created, the HPC version monitors them together:

```
submitted_freq_jobs = monitor_submitted_orca_jobs(  
    remote_hess_dir,  
    submitted_freq_jobs,  
    allow_interactive_repair=True  
)
```

Thus, the Local PC implementation runs and checks the two jobs locally, whereas the HPC implementation submits both jobs first and then monitors the pair simultaneously.

In both implementations, any repaired frequency input must preserve:

- the selected method;
- the selected basis set;
- the molecular charge;
- the XYZ multiplicity;
- the partner MECP multiplicity;
- the optimized MECP geometry; and
- the required frequency job type.

Convergence-related settings may be edited during an interactive repair, but the scientifically defining quantities remain protected. Step 5 proceeds only after both frequency jobs return successful status.

4.5.11 Collection of the two Hessian files

After successful completion, SPINKS locates the two ORCA output files and two Hessian files using implementation-specific file helpers.

In the Local PC implementation:

```
local_mecp_main_other_out = local_find_file(local_hess_dir,
f"{freq_mecp_main_other_label}.out")
local_mecp_other_main_out = local_find_file(local_hess_dir,
f"{freq_mecp_other_main_label}.out")
local_mecp_main_other_hess = local_find_file(local_hess_dir,
f"{freq_mecp_main_other_label}.hess")
local_mecp_other_main_hess = local_find_file(local_hess_dir,
f"{freq_mecp_other_main_label}.hess")
```

In the HPC implementation, the authoritative cluster files are first located remotely:

```
remote_mecp_main_other_out = remote_find_file(remote_hess_dir,
f"{freq_mecp_main_other_label}.out")
remote_mecp_other_main_out = remote_find_file(remote_hess_dir,
f"{freq_mecp_other_main_label}.out")
remote_mecp_main_other_hess = remote_find_file(remote_hess_dir,
f"{freq_mecp_main_other_label}.hess")
remote_mecp_other_main_hess = remote_find_file(remote_hess_dir,
f"{freq_mecp_other_main_label}.hess")
```

The HPC outputs are read from the remote paths, local output copies are written, and the two Hessian files are downloaded into local_hess_dir. By contrast, in the Local PC implementation the remote_mecp_* names are assigned as aliases of the corresponding local paths.

The primary Hessian paths exported for Step 6 are:

```
MECP_hess_main_other_path
MECP_hess_other_main_path
```

For compatibility, both implementations also provide Freq_MECP.out and Freq_MECP.hess based on the main/other calculation. In the Local PC version these are local copies; in the HPC version the main/other output is written locally and its Hessian is downloaded. The explicitly ordered Hessian paths remain the authoritative pair used in Step 6.

These two ordered Hessian paths are the central Step 5 Hessian outputs used in Step 6.

Conceptually, the first ordered Hessian corresponds to one spin-surface ordering:

$$\mathbf{H}_{\text{main/other}}$$

The second ordered Hessian corresponds to the reverse spin-surface ordering:

$$\mathbf{H}_{\text{other/main}}$$

Step 6 subsequently combines these two Hessians using the effective-Hessian weighting parameter obtained in Step 4.

4.5.12 Extraction of MECP rotational constants and symmetry number

Both implementations extract the MECP rotational constants and symmetry number from `mecp_main_other_out_text`. The source label records the implementation-specific path.

In the Local PC implementation:

```
MECP_symmetry_number, MECP_rot_constants_cm1 = (
    extract_symmetry_and_rot_constants_from_text(
        mecp_main_other_out_text,
        source_label=local_mecp_main_other_out
    )
)
```

In the HPC implementation:

```
MECP_symmetry_number, MECP_rot_constants_cm1 = (
    extract_symmetry_and_rot_constants_from_text(
        mecp_main_other_out_text,
        source_label=remote_mecp_main_other_out
    )
)
```

The resulting variables are:

```
MECP_symmetry_number
MECP_rot_constants_cm1
```

and are exported under the SPINKS names:

```
crossing_symmetry_factor
crossing_rotational_constants_cm1
```

Thus, Equation S55 follows:

$$\sigma_X = \text{MECP rotational symmetry number} \quad (\text{S55})$$

The second rotational quantity is given in Equation S56:

$$\mathbf{B}_X = (B_{X,A}, B_{X,B}, B_{X,C}) \quad (\text{S56})$$

These quantities are later used in evaluating the rotational contribution to the crossing-state density of states.

4.5.13 Extraction of diagnostic PES2 frequencies

The ORCA SurfCrossNumFreq output can contain several vibrational-frequency blocks. Step 5 searches specifically for blocks whose title contains:

```
PES2
```

The PES2 frequency blocks are selected using:

```
select_mecp_pes2_frequency_block_from_text(...)
```

If more than one PES2 block is present, the preferred block is the one with:

1. no imaginary frequencies; and
2. the largest number of real frequencies.

If all PES2 blocks contain imaginary modes, the block with the fewest imaginary frequencies is retained and a warning is printed.

The extracted arrays are stored as:

```
freq_MECP_PES2_real_cm1_diagnostic  
freq_MECP_PES2_imag_cm1_diagnostic  
freq_MECP_PES2_zero_cm1_diagnostic
```

These quantities are explicitly labeled as diagnostic. They are not assigned as the final MECP vibrational frequencies used in SPINKS:

Step 5 does not assign the final MECP frequencies. The final frequencies must come from the projected effective Hessian constructed in Step 6.

This separation is important because the final crossing-point modes must be derived from the effective Hessian constructed from both spin surfaces, not from a single raw PES2 frequency block.

4.5.14 Electronic MECP barrier bookkeeping

Step 5 next determines the electronic energy of the optimized MECP. The preferred source is the original MECP optimization output from Step 1.

In the Local PC implementation, the primary candidate is constructed as:

```
candidate = os.path.join(  
    local_mecp,  
    f"{jobname}.out"  
)
```

If local_mecp is not available, the Local PC code can also test the compatible remote_mecp path as a local filesystem path.

In the HPC implementation, the corresponding cluster candidate is:

```
candidate = posixpath.join(remote_mecp, f"{jobname}.out")
```

and its existence is checked remotely through `remote_file_exists(sftp, candidate)`. If the original MECP optimization output is unavailable, both implementations fall back to the final single-point energy in the main/other `SurfCrossNumFreq` output.

The MECP electronic energy is extracted using:

```
extract_final_sp_energy_from_text(...)
```

which selects the final occurrence of:

```
FINAL SINGLE POINT ENERGY
```

The electronic MECP barrier relative to the reference minimum is then:

```
E_MECP_electronic_hartree = (  
    Ele_MECP_hartree - Ele_REF_hartree  
)
```

Equivalently, the expression can be written as Equation S57:

$$\Delta E_{\text{MECP}}^{\text{el}} = E_{\text{MECP}}^{\text{el}} - E_R^{\text{el}} \quad (\text{S57})$$

The barrier in wavenumbers is:

```
VaG_MECP_electronic_cm1 = (  
    E_MECP_electronic_hartree *  
    hartree_to_cm  
)
```

Equivalently, the expression can be written as Equation S58:

$$V_{\text{MECP}}^{\text{el}} = (E_{\text{MECP}}^{\text{el}} - E_R^{\text{el}}) \times 219474.6313705 \quad (\text{S58})$$

At this point, the active barrier is provisionally assigned as:

```
VaG_MECP_cm1 = VaG_MECP_electronic_cm1
```

This is an electronic barrier only. The final zero-point-corrected barrier is determined later after the effective-Hessian frequencies have been obtained.

The corresponding provisional value in kJ mol^{-1} is then calculated as:

```
VaG_MECP_kJmol = (  
    VaG_MECP_cm1 * 0.01196266  
)
```

4.5.15 Preparation of downstream variables

The reference and MECP data are reorganized into the variable names used by the subsequent downstream calculations. The symmetry factors and ordered Hessian paths are common to both implementations:

```
reference_symmetry_factor = float(Reference_symmetry_number)
crossing_symmetry_factor = float(MECP_symmetry_number)
MECP_hess_main_other_path = local_mecp_main_other_hess
MECP_hess_other_main_path = local_mecp_other_main_hess
```

In the Local PC implementation, the vibrational and rotational arrays are explicitly normalized to one-dimensional NumPy arrays:

```
reference_vibrational_frequencies_cm1 = np.asarray(
    freq_reactant_real_cm1,
    dtype=float
).reshape(-1)
reference_rotational_constants_cm1 = np.asarray(
    Reference_rot_constants_cm1,
    dtype=float
).reshape(-1)
crossing_rotational_constants_cm1 = np.asarray(
    MECP_rot_constants_cm1,
    dtype=float
).reshape(-1)
```

In the HPC implementation, the equivalent downstream aliases are assigned directly from the already parsed NumPy arrays:

```
reference_vibrational_frequencies_cm1 = freq_reactant_real_cm1
reference_rotational_constants_cm1 = Reference_rot_constants_cm1
crossing_rotational_constants_cm1 = MECP_rot_constants_cm1
```

In both implementations, MECP_hess_main_other_path and MECP_hess_other_main_path point to the local Hessian files used by Step 6. The remote counterparts are retained separately through MECP_hess_main_other_remote and MECP_hess_other_main_remote.

The first resulting physical quantity is given in Equation S59:

$$\{\nu_{R,i}\} = \text{reference real vibrational frequencies}, \mathbf{B}_R = \text{reference rotational constants}, \mathbf{B}_X = \text{MECP rotational constants}, \sigma_R = \text{reference symmetry number} \quad (\text{S59})$$

The second resulting physical quantity is given in Equation S60:

$$\sigma_X = \text{MECP symmetry number} \quad (\text{S60})$$

The final MECP effective frequencies, $\{\nu_{X,i}\}$, are not yet defined in Step 5.

4.5.16 Distinction between reference frequencies and MECP diagnostic frequencies

The reference frequencies:

[reference_vibrational_frequencies_cm1](#)

These are final quantities because they originate from the validated optimized-minimum frequency calculation.

In contrast, the MECP PES2 frequencies:

[freq_MECP_PES2_real_cm1_diagnostic](#)

These are retained only as provisional diagnostics.

This distinction can be summarized as follows.

Reference frequencies from Step 1 → used directly in SPINKS

The raw MECP PES2 frequencies from Step 5 are used only as diagnostics:

Raw MECP PES2 frequencies from Step 5 → diagnostic only

The projected effective-Hessian frequencies from Step 6 are the final MECP frequencies used in SPINKS:

Projected effective-Hessian frequencies from Step 6 → final MECP frequencies used in SPINKS

This prevents the density-of-states calculation from using frequencies belonging to only one raw crossing surface rather than the combined effective crossing-point Hessian.

4.5.17 Files generated by step 5

Both implementations create the local metadata file:

[rotational_constants_and_symmetry.txt](#)

This file records:

- the reference output and XYZ paths;
- the reference multiplicity;
- the reference symmetry number;

- the reference rotational constants;
- both MECP output paths;
- both MECP Hessian paths;
- the XYZ and partner multiplicities for both jobs;
- the MECP symmetry number;
- the MECP rotational constants;
- the reference electronic energy;
- the MECP electronic energy;
- the source of the MECP energy;
- the electronic MECP barrier;
- the provisional active barrier;
- the numbers of reference real, imaginary, and zero frequencies;
- the reference ZPE reconstructed from frequencies; and
- the diagnostic MECP PES2 frequency counts.

The file explicitly notes that the final MECP frequencies are generated later from the projected effective Hessian. The Local PC metadata records local output and Hessian paths. The HPC metadata records the remote reference paths and both remote and local MECP output/Hessian paths.

For Local PC, the remote metadata path is only an alias:

```
remote_rot_info_file = rot_info_file
```

For HPC, a remote metadata path is constructed and the local file is uploaded:

```
remote_rot_info_file = posixpath.join(
    remote_hess_dir,
    os.path.basename(rot_info_file)
)
sftp.put(rot_info_file, remote_rot_info_file)
```

Step 5 also creates a local convenience copy of the reference minimum output under the name:

```
Reference_minimum.out
```

The Local PC version writes this copy from `ref_out_text`, whereas the HPC version downloads `remote_ref_out` into the local Frequencies directory.

4.5.18 Variables exported by step 5

The principal reference-state variables exported by both implementations are:

```
Ele_REF_hartree  
ZPE_REF_thermo_hartree  
ZPE_REF_from_freq_cm1  
ZPE_REF_from_freq_hartree  
freq_reactant_real_cm1  
reference_vibrational_frequencies_cm1  
Reference_rot_constants_cm1  
Reference_symmetry_number  
reference_symmetry_factor
```

The principal MECP variables exported by both implementations are:

```
Ele_MECP_hartree  
VaG_MECP_electronic_cm1  
VaG_MECP_cm1  
MECP_rot_constants_cm1  
MECP_symmetry_number  
crossing_symmetry_factor  
MECP_hess_main_other_path  
MECP_hess_other_main_path
```

The diagnostic MECP frequency variables are:

```
freq_MECP_PES2_real_cm1_diagnostic  
freq_MECP_PES2_imag_cm1_diagnostic  
freq_MECP_PES2_zero_cm1_diagnostic
```

The rotational constants used later are exported as:

```
reference_rotational_constants_cm1  
crossing_rotational_constants_cm1
```

Both implementations also export the ordered MECP output paths, local/remote Hessian aliases, multiplicity bookkeeping, and the local and remote rotational/symmetry metadata paths. In the Local PC implementation, the variables carrying remote-path names are local compatibility aliases.

In its `explicit globals().update({...})` block, the Local PC version additionally includes the keys:

```
freq_job_records  
mecp_main_other_inp  
mecp_other_main_inp
```

The current HPC `globals().update({...})` block does not include these three keys. However, `mecp_main_other_inp` and `mecp_other_main_inp` are still created earlier as top-level Step 5 variables in the HPC code; the compatibility key `freq_job_records` is not created there.

4.5.19 Summary of the step 5 algorithm

The complete sequence implemented in Step 5 can be summarized as:

$$\begin{array}{c} \text{Reference minimum data from Step 1} \\ \downarrow \\ \{E_R^{\text{el}}, \text{ZPE}_R, \{\nu_{R,i}\}, \mathbf{B}_R, \sigma_R\} \end{array}$$

The relevant definition is given in Equation S61:

$$\text{ZPE}_R = \frac{1}{2} \sum_{\nu_{R,i} > 0} \nu_{R,i} \quad (\text{S61})$$

In parallel, the optimized MECP geometry is used to generate two Hessian calculations:

$$\begin{array}{c} \mathbf{R}_{\text{MECP}} \\ \downarrow \\ \{\text{SurfCrossNumFreq}(m_{\text{main}}, m_{\text{other}}), \\ \text{SurfCrossNumFreq}(m_{\text{other}}, m_{\text{main}})\}. \end{array}$$

The two calculations produce the ordered spin-surface Hessians:

$$\{\mathbf{H}_{\text{main/other}}, \mathbf{H}_{\text{other/main}}\},$$

They also provide the MECP rotational constants and symmetry number:

$$\{\mathbf{B}_X, \sigma_X\}.$$

The provisional electronic crossing barrier is then calculated according to Equation S62.

$$V_{\text{MECP}}^{\text{el}} = (E_{\text{MECP}}^{\text{el}} - E_R^{\text{el}}) \times 219474.6313705 \quad (\text{S62})$$

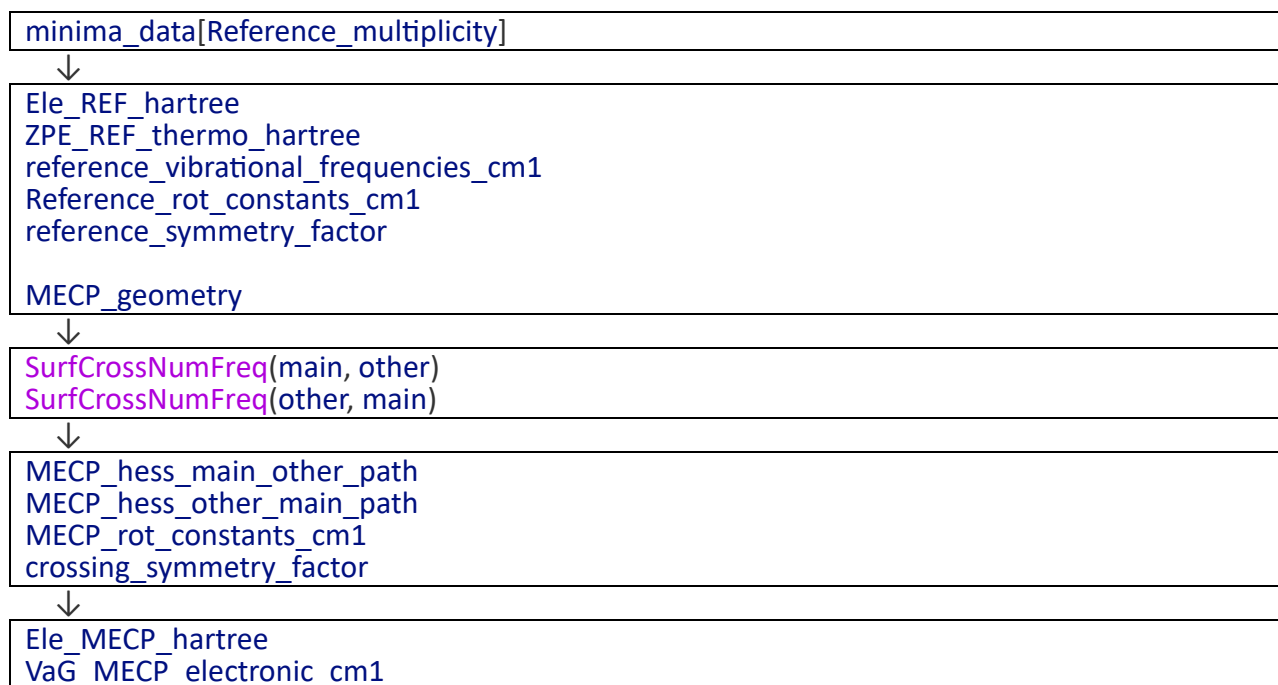
The raw ORCA PES2 frequencies are extracted only for diagnostic purposes:

$$\{\nu_{X,i}^{\text{PES2}}\} \text{ diagnostic only.}$$

The final MECP frequencies remain undefined until Step 6, where they are obtained according to Equation S63:

$$\{\nu_{X,i}^{\text{eff}}\} = \text{eigenfrequencies of the projected effective Hessian} \quad (\text{S63})$$

The corresponding variable flow is:



The ordered Hessian data are then passed to Step 6 for effective-Hessian construction. The scientific variable flow is therefore identical in the two implementations, while execution differs only in job and file handling. Local PC executes the SurfCrossNumFreq jobs in the local Frequencies directory with submit_and_monitor_orca_job. HPC submits both jobs to remote_hess_dir with submit_orca_job_no_monitor, monitors them together with monitor_submitted_orca_jobs, and retrieves the required output and Hessian files to local_hess_dir.

Therefore, Step 5 transforms the validated reference minimum and optimized MECP geometry into the vibrational, rotational, symmetry, energy, and Hessian data required for the downstream statistical treatment:

$$\{\text{reference minimum, MECP geometry}\} \rightarrow \{\{\nu_{R,i}\}, \mathbf{B}_R, \sigma_R, \mathbf{H}_1, \mathbf{H}_2, \mathbf{B}_X, \sigma_X, V_{\text{MECP}}^{\text{el}}\}$$

The two MECP Hessians are subsequently combined in Step 6 using the effective-Hessian mixing coefficient obtained in Step 4. The projected effective Hessian then provides the final reaction-coordinate mode, reduced mass, transverse MECP vibrational frequencies, and reaction-coordinate-projected gradient mean used by the later probability and rate calculations. The effective-Hessian frequencies are subsequently used for the final ZPE correction of the MECP barrier.

4.6 Step 6: Effective Hessian and Reduced Mass

Step 6 combines the two MECP Hessians generated in Step 5 with the two spin-state gradients obtained in Step 3 to construct the SPINKS effective Hessian, identify the crossing reaction coordinate, calculate the corresponding reduced mass, and generate the transverse MECP vibrational frequencies used in the subsequent density-of-states calculation. The Local PC and HPC versions perform the same numerical operations, although a small number of internal variable aliases differ between the two scripts.

No additional ORCA calculation is performed in this step. All operations consist of numerical analysis of the previously generated Cartesian gradients, molecular geometry, atomic masses, and Hessian matrices.

The principal operations performed in Step 6 are:

1. parsing the MECP geometry and assigning atomic masses;
2. reconstructing the gradient-difference and force quantities;
3. evaluating the final projected gradient mean,
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR;
4. reading and validating the two ORCA Hessian files;
5. transforming both Hessians to mass-weighted coordinates;
6. constructing peaked- and sloped-crossing effective Hessians;
7. removing translational and rotational motion;
8. generating several candidate reaction-coordinate vectors;
9. selecting the validated reaction-coordinate convention;
10. evaluating alternative and final reduced masses;
11. projecting translation, rotation, and the reaction coordinate from the effective Hessian;
12. diagonalizing the transverse effective Hessian;
13. converting the eigenvalues into MECP frequencies; and
14. exporting the final gradient, mass, reaction-coordinate, and frequency quantities.

The principal downstream outputs are:

```
GRADIENT_GAP_NORM_EH_PER_BOHR
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
reduced_mass_amu
freq_MECP_effhess_real_cm1
reaction_direction_hessian_cart
reaction_direction_hessian_mw
```

4.6.1 Import of gradient and Hessian data

Step 6 requires the two Cartesian gradient vectors, the optimized MECP geometry, and the two Hessian files generated in Step 5:

```
required_vars_step6 = [
    "jobname",
    "MECP_geometry",
    "g_LS",
    "g_HS",
    "mult_LS",
    "mult_HS",
    "spin_name_LS",
    "spin_name_HS",
    "MECP_hess_main_other_path",
    "MECP_hess_other_main_path",
    "local_base",
    "remote_base"
]
```

The two Hessian paths are:

```
MECP_hess_main_other_path
MECP_hess_other_main_path
```

and correspond to the two SurfCrossNumFreq calculations in which the multiplicities were interchanged between the primary XYZ state and the MECP partner state.

Step 6 terminates if any required gradient, geometry, or Hessian quantity is unavailable.

4.6.2 MECP geometry parsing and atomic-mass assignment

The Cartesian geometry is parsed atom by atom from:

```
MECP_geometry
```

to obtain the element symbols and coordinates:

```

symbols_mecp.append(sym)

coords_mecp.append([
    float(parts[1]),
    float(parts[2]),
    float(parts[3])
])

```

The resulting arrays are:

```

symbols_mecp
coords_mecp

```

For a system containing N atoms, the Cartesian dimension is given by Equation S64.

$$N_{\text{cart}} = 3N \quad (\text{S64})$$

The code therefore defines:

```

natoms = len(symbols_mecp)
num_cart = 3 * natoms

```

Each atom is assigned a standard isotopic-average atomic mass from the internal `atomic_masses` dictionary. The atomic masses are stored as:

```

masses

```

and expanded into a Cartesian mass vector:

```

mass_vector = np.repeat(masses, 3)

```

The Cartesian mass vector is defined by Equation S65:

$$\mathbf{m} = (m_1, m_1, m_1, m_2, m_2, m_2, \dots, m_N, m_N, m_N) \quad (\text{S65})$$

The square-root mass vector is also generated:

```

sqrt_mass_vector = np.sqrt(mass_vector)

```

These quantities are used for mass weighting, projector construction, and reduced-mass evaluation.

4.6.3 Validation of the gradient dimensions

The low-spin and high-spin gradients are converted to one-dimensional NumPy arrays:

```

g_LS = np.asarray(g_LS, dtype=float).reshape(-1)
g_HS = np.asarray(g_HS, dtype=float).reshape(-1)

```

Each gradient must contain exactly $3N$ components:

```

if g_LS.size != num_cart:
    raise RuntimeError(
        f"g_LS length {g_LS.size} does not match 3N = {num_cart}."
    )

if g_HS.size != num_cart:
    raise RuntimeError(
        f"g_HS length {g_HS.size} does not match 3N = {num_cart}."
    )

```

This validation ensures that both gradients correspond exactly to the parsed MECP geometry and the Hessian dimensions.

The individual gradient norms and their dot product are then calculated. In the Local PC version these quantities are stored as:

```

g_LS_norm_Eh_per_Bohr = float(np.linalg.norm(g_LS))
g_HS_norm_Eh_per_Bohr = float(np.linalg.norm(g_HS))
gradient_dot_product = float(np.dot(g_LS, g_HS))

```

The corresponding gradient norms are given by Equation S66:

$$\| \mathbf{g}_{\text{LS}} \| = \sqrt{\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{LS}}}, \| \mathbf{g}_{\text{HS}} \| = \sqrt{\mathbf{g}_{\text{HS}} \cdot \mathbf{g}_{\text{HS}}} \quad (\text{S66})$$

The HPC version uses the equivalent aliases:

```

gLS_norm
gHS_norm
grad_dot

```

4.6.4 Gradient-difference and gradient-sum vectors

Both implementations use the same physical gradient-difference direction, $\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$, for the downstream reaction direction. The Local PC and HPC scripts assign the two opposite orientations to different internal aliases, so the implementation-specific definitions are listed explicitly below.

In the Local PC version:

```

surface_gradient_difference_cart = g_LS - g_HS
reverse_surface_gradient_difference_cart = g_HS - g_LS
surface_gradient_sum_cart = g_HS + g_LS

```

In the HPC version:

```

surface_gradient_difference_cart = g_HS - g_LS
surface_gradient_sum_cart = g_HS + g_LS
reverse_surface_gradient_difference_cart = g_LS - g_HS

```

The physical gradient-difference vector is defined by Equation S67:

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \quad (\text{S67})$$

The gradient-sum vector is defined by Equation S68:

$$\mathbf{g}_{\text{sum}} = \mathbf{g}_{\text{HS}} + \mathbf{g}_{\text{LS}} \quad (\text{S68})$$

The opposite orientation of the gradient-difference vector is defined by Equation S69:

$$\Delta \mathbf{G}_{\text{diff}} = -\Delta \mathbf{g} \quad (\text{S69})$$

Thus, the two implementations use different internal aliases for the two orientations, but the physical gradient-difference and gradient-sum vectors are the same.

4.6.5 Gradient-difference magnitude and normalized crossing direction

The magnitude of the Cartesian gradient difference is evaluated from `surface_gradient_difference_cart`. Because the Euclidean norm is sign independent, the Local PC and HPC definitions give the same scalar magnitude:

```
gradient_gap_norm_Eh_per_Bohr = float(np.linalg.norm(surface_gradient_difference_cart))
```

The gradient-gap magnitude is given by Equation S70:

$$\Delta g_{\parallel} = \|\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}\| \quad (\text{S70})$$

```
GRADIENT_GAP_NORM_EH_PER_BOHR = gradient_gap_norm_Eh_per_Bohr
```

The implementation-specific gradient-gap aliases are:

Local PC:

```
gradient_gap_magnitude
```

HPC:

```
gradient_gap_magnitude_Eh_per_Bohr
```

Both implementations assign the normalized $\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$ direction to `reaction_direction_cart` and `MECP_seam_normal_cart`.

In the Local PC version:

```
reverse_gradient_unit_vector = (
    reverse_surface_gradient_difference_cart
    / gradient_gap_norm_Eh_per_Bohr
)
reaction_direction_cart = reverse_gradient_unit_vector.copy()
MECP_seam_normal_cart = reverse_gradient_unit_vector.copy()
```

In the HPC version, the same physical direction is stored using:

The normalized physical reaction direction is given by Equation S71:

$$\mathbf{n} = \frac{\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}\|} \quad (\text{S71})$$


```

gradient_gap_unit_vector = (
    surface_gradient_difference_cart / gradient_gap_norm_Eh_per_Bohr
)
reaction_direction_cart = gradient_gap_unit_vector.copy()
MECP_seam_normal_cart = gradient_gap_unit_vector.copy()

```

The opposite normalized direction is defined by Equation S72:

$$\mathbf{n}_{\text{diff}} = -\mathbf{n} \quad (\text{S72})$$

In the Local PC version, the opposite normalized direction is retained as:

```

gradient_difference_unit_vector = (
    surface_gradient_difference_cart
    / gradient_gap_norm_Eh_per_Bohr
)

```

In the HPC version, the opposite normalized direction is retained as:

```

reverse_gradient_gap_unit_vector = (
    reverse_surface_gradient_difference_cart
    / gradient_gap_norm_Eh_per_Bohr
)

```

4.6.6 Projected gradient components and the final projected gradient mean

Step 6 projects each spin-state gradient onto the normalized $\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$ reaction direction. The exact internal direction variable differs between the two implementations.

Local PC:

```

low_spin_projected_gradient_signed = float(
    np.dot(g_LS, reaction_direction_cart)
)
high_spin_projected_gradient_signed = float(
    np.dot(g_HS, reaction_direction_cart)
)

```

HPC:

```

low_spin_projected_gradient_signed = float(
    np.dot(g_LS, gradient_gap_unit_vector)
)
high_spin_projected_gradient_signed = float(
    np.dot(g_HS, gradient_gap_unit_vector)
)

```

The signed projections are given by Equation S73:

$$g_{\text{LS}}^{\text{signed}} = \mathbf{g}_{\text{LS}} \cdot \mathbf{n}, g_{\text{HS}}^{\text{signed}} = \mathbf{g}_{\text{HS}} \cdot \mathbf{n} \quad (\text{S73})$$

```

low_spin_projected_gradient_magnitude = abs(low_spin_projected_gradient_signed)

```

```
high_spin_projected_gradient_magnitude = abs(high_spin_projected_gradient_signed)
```

The HPC version stores the corresponding magnitudes as:

```
low_spin_projected_gradient_Eh_per_Bohr = abs(low_spin_projected_gradient_signed)
high_spin_projected_gradient_Eh_per_Bohr = abs(high_spin_projected_gradient_signed)
```

The authoritative projected mean used downstream is common to both implementations:

The projected-gradient magnitudes are given by Equation S74:

$$g_{LS,\parallel} = |\mathbf{g}_{LS} \cdot \mathbf{n}|, g_{HS,\parallel} = |\mathbf{g}_{HS} \cdot \mathbf{n}| \quad (\text{S74})$$

In the Local PC implementation, the projected mean is evaluated as:

```
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR = float(np.sqrt(
    low_spin_projected_gradient_magnitude
    * high_spin_projected_gradient_magnitude
))
```

The final projected gradient mean is defined by Equation S75:

$$\text{GRADMEAN} = \sqrt{g_{LS,\parallel} g_{HS,\parallel}} \quad (\text{S75})$$

HPC:

```
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR = float(
    np.sqrt(
        low_spin_projected_gradient_Eh_per_Bohr
        * high_spin_projected_gradient_Eh_per_Bohr
    )
)
```

This projected mean is distinct from the earlier full-Cartesian geometric-mean gradient diagnostic because only the components parallel to the selected $\mathbf{g}_{HS}$ - $\mathbf{g}_{LS}$ direction enter the Step 6 quantity.

The earlier full-Cartesian diagnostic is:

$$\sqrt{\|\mathbf{g}_{LS}\| \|\mathbf{g}_{HS}\|}.$$

The backward-compatible projected-mean aliases are:

Both implementations:

```
projected_gradient_mean_Eh_per_Bohr
```

Local PC additionally:

```
projected_gradient_mean
```

4.6.7 Direct Cartesian gradient-direction reduced mass

Before the effective-Hessian reaction coordinate is constructed, Step 6 calculates a diagnostic reduced mass directly from the normalized g_HS - g_LS Cartesian direction.

Local PC:

```
inverse_gradient_direction_mass_amu_inv = float(
    np.sum(
        (reaction_direction_cart ** 2)
        / mass_vector
    )
)
reduced_mass_gradient_direction_amu = (
    1.0 / inverse_gradient_direction_mass_amu_inv
)
```

HPC:

```
inverse_mass_along_gradient_gap = float(
    np.sum((gradient_gap_unit_vector ** 2) / mass_vector)
)
gradient_gap_reduced_mass_amu = 1.0 / inverse_mass_along_gradient_gap
```

The inverse mass associated with the normalized gradient-difference direction is given by Equation S76:

$$\frac{1}{\mu_{\Delta g}} = \sum_{i=1}^{3N} \frac{n_i^2}{m_i} \quad (\text{S76})$$

The corresponding Cartesian gradient-direction reduced mass is given by Equation S77:

$$\mu_{\Delta g} = \left[\sum_{i=1}^{3N} \frac{n_i^2}{m_i} \right]^{-1} \quad (\text{S77})$$

This Cartesian gradient-direction mass is retained only as a diagnostic and is not automatically assigned as the final reduced_mass_amu used downstream.

4.6.8 Crossing classification

The dot product of the two surface gradients is used to classify the crossing. The Local PC script stores this scalar as gradient_dot_product, whereas the HPC script uses grad_dot.

Local PC:

```
intersection_type = (
    "peaked"
    if gradient_dot_product < 0.0
    else "sloped"
)
```

HPC:

```

if grad_dot < 0.0:
    intersection_type = "peaked"
else:
    intersection_type = "sloped"

```

The peaked-crossing condition is given by Equation S78:

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} < 0 \Rightarrow \text{peaked} \quad (\text{S78})$$

The sloped-crossing condition is given by Equation S79:

$$\mathbf{g}_{\text{LS}} \cdot \mathbf{g}_{\text{HS}} \geq 0 \Rightarrow \text{sloped} \quad (\text{S79})$$

4.6.9 ORCA Hessian parsing

Each ORCA .hess file is read using:

```

read_orca_hessian(
    hess_path,
    expected_dim
)

```

The parser first attempts to interpret the file as a plain numerical matrix. If that fails, it locates the ORCA

```
$hessian
```

block and reconstructs the matrix from the blockwise column format.

The parser verifies that the Hessian dimension is exactly

$$3N \times 3N.$$

The reconstructed matrix is symmetrized as:

$$\mathbf{H} = 0.5 * (\mathbf{H} + \mathbf{H}^T)$$

The Hessian is symmetrized according to Equation S80:

$$\mathbf{H} = \frac{1}{2}(\mathbf{H} + \mathbf{H}^T) \quad (\text{S80})$$

The parser also rejects a matrix containing only zeros.

The two Cartesian Hessians are stored as:

```

H_mecp_main_other_cart
H_mecp_other_main_cart

```

4.6.10 Mass weighting of the Hessians

The mass-weighting matrix is constructed as:

```
mass_weight_matrix = (
    1.0 /
    np.sqrt(
        np.outer(mass_vector, mass_vector)
    )
)
```

For Cartesian indices i and j , the mass-weighting matrix is defined by Equation S81:

$$W_{ij} = \frac{1}{\sqrt{m_i m_j}} \quad (\text{S81})$$

The two mass-weighted Hessians are therefore:

```
H_mecp_main_other_mw = (
    H_mecp_main_other_cart *
    mass_weight_matrix
)

H_mecp_other_main_mw = (
    H_mecp_other_main_cart *
    mass_weight_matrix
)
```

The corresponding mass-weighted Hessian elements are given by Equation S82:

$$H_{ij}^{\text{mw}} = \frac{H_{ij}^{\text{cart}}}{\sqrt{m_i m_j}} \quad (\text{S82})$$

All effective-Hessian construction, projection, and diagonalization are subsequently performed in mass-weighted coordinates.

4.6.11 Construction of the peaked-crossing effective Hessian

For a peaked crossing, Step 6 constructs the gradient-norm-weighted sum. In the Local PC implementation, the corresponding code is:

```
H_eff_mw_plus = (
    g_HS_norm_Eh_per_Bohr * H_mecp_main_other_mw
    + g_LS_norm_Eh_per_Bohr * H_mecp_other_main_mw
) / (g_LS_norm_Eh_per_Bohr + g_HS_norm_Eh_per_Bohr)
```

The peaked-crossing effective Hessian is calculated according to Equation S83:

$$\mathbf{H}_{\text{eff}}^{(+)} = \frac{\|\mathbf{g}_{\text{HS}}\| \mathbf{H}_1 + \|\mathbf{g}_{\text{LS}}\| \mathbf{H}_2}{\|\mathbf{g}_{\text{LS}}\| + \|\mathbf{g}_{\text{HS}}\|} \quad (\text{S83})$$

The HPC implementation uses the same expression with the following gradient-norm aliases:

```
gHS_norm
gLS_norm
```

This expression provides a weighted combination of the two surface Hessians in which the weights are determined by the corresponding gradient magnitudes.

4.6.12 Construction of the sloped-crossing effective Hessian

For a sloped crossing, Step 6 constructs the gradient-norm-weighted difference. In the Local PC implementation, the corresponding code is:

```
H_eff_mw_minus = (
    g_HS_norm_Eh_per_Bohr * H_mecp_main_other_mw
    - g_LS_norm_Eh_per_Bohr * H_mecp_other_main_mw
) / (g_HS_norm_Eh_per_Bohr - g_LS_norm_Eh_per_Bohr)
```

The sloped-crossing effective Hessian is calculated according to Equation S84:

$$\mathbf{H}_{\text{eff}}^{(-)} = \frac{\|\mathbf{g}_{\text{HS}}\|\mathbf{H}_1 - \|\mathbf{g}_{\text{LS}}\|\mathbf{H}_2}{\|\mathbf{g}_{\text{HS}}\| - \|\mathbf{g}_{\text{LS}}\|} \quad (\text{S84})$$

If the two gradient norms are numerically equal within 1.0×10^{-14} , the denominator approaches zero. The Local PC implementation therefore applies the following safeguard:

```
if abs(g_HS_norm_Eh_per_Bohr - g_LS_norm_Eh_per_Bohr) > 1.0e-14:
    H_eff_mw_minus = (
        g_HS_norm_Eh_per_Bohr * H_mecp_main_other_mw
        - g_LS_norm_Eh_per_Bohr * H_mecp_other_main_mw
    ) / (g_HS_norm_Eh_per_Bohr - g_LS_norm_Eh_per_Bohr)
else:
    H_eff_mw_minus = H_eff_mw_plus.copy()
```

The HPC implementation applies the same safeguard using:

```
gHS_norm
gLS_norm
```

This safeguard is present in both implementations.

4.6.13 Selection of the active effective Hessian

The final effective Hessian is selected according to the crossing classification:

```

if intersection_type == "peaked":
    H_eff_mw_selected = H_eff_mw_plus.copy()
    effective_hessian_choice = "plus/peaked"
else:
    H_eff_mw_selected = H_eff_mw_minus.copy()
    effective_hessian_choice = "minus/sloped"

```

The active effective Hessian is selected according to Equation S85:

$$\mathbf{H}_{\text{eff}} = \begin{cases} \mathbf{H}_{\text{eff}}^{(+)} & \text{peaked crossing} \\ \mathbf{H}_{\text{eff}}^{(-)} & \text{sloped crossing} \end{cases} \quad (\text{S85})$$

For diagnostic purposes, the arithmetic average is also stored:

```

H_eff_mw_average = 0.5 * (
    H_mecp_main_other_mw +
    H_mecp_other_main_mw
)

```

However, the selected gradient-weighted effective Hessian is the matrix used for reaction-coordinate and frequency analysis.

4.6.14 Construction of the translational and rotational subspace

The center of mass is calculated as:

```

R_cm = (
    np.sum(
        coords_mecp * masses[:, None],
        axis=0
    ) / np.sum(masses)
)

```

and the geometry is shifted to the center-of-mass frame:

```

coords_centered = coords_mecp - R_cm

```

A $3N \times 6$ matrix, $\mathbf{Z}_{\text{raw}}$, is then constructed from three mass-weighted translational vectors and three mass-weighted rotational vectors.

The translational vectors have components proportional to

$$\sqrt{m_a}$$

along each Cartesian axis.

The rotational vectors are generated from the centered atomic coordinates and atomic masses.

The independent rigid-body vectors are orthonormalized using:

```
Z = orth(Z_raw)
```

The translational/rotational projector is:

```
P_TR = Z @ Z.T
```

and the complementary vibrational-space projector is:

```
P_RT = np.eye(num_cart) - P_TR
```

The translational/rotational projector is given by Equation S86:

$$\mathbf{P}_{\text{TR}} = \mathbf{Z}\mathbf{Z}^T \quad (\text{S86})$$

The complementary nonrigid-space projector is given by Equation S87:

$$\mathbf{P}_{\text{RT}} = \mathbf{I} - \mathbf{P}_{\text{TR}} \quad (\text{S87})$$

The code reports the projector idempotency errors for $\mathbf{P}_{\text{TR}}$ and $\mathbf{P}_{\text{RT}}$ as numerical diagnostics.

For a nonlinear molecule, the rigid-body subspace normally has dimension six. For a linear molecule, linear dependence among the rotational vectors may reduce this dimension to five automatically through orthogonalization.

4.6.15 Generation of candidate reaction-coordinate seeds

Step 6 evaluates four Cartesian seed definitions. The physical seed set is the same in both implementations, but the alias used for $\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$ differs.

Local PC:

```
reverse_surface_gradient_difference_cart  
surface_gradient_sum_cart  
g_HS_unit - g_LS_unit  
g_HS_unit + g_LS_unit
```

HPC:

```
surface_gradient_difference_cart  
surface_gradient_sum_cart  
g_HS_unit - g_LS_unit  
g_HS_unit + g_LS_unit
```

The first three mathematical seed definitions are summarized by Equation S88:

$$\mathbf{s}_1 = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}, \mathbf{s}_2 = \mathbf{g}_{\text{HS}} + \mathbf{g}_{\text{LS}}, \mathbf{s}_3 = \frac{\mathbf{g}_{\text{HS}}}{\|\mathbf{g}_{\text{HS}}\|} - \frac{\mathbf{g}_{\text{LS}}}{\|\mathbf{g}_{\text{LS}}\|} \quad (\text{S88})$$

The fourth seed definition is given by Equation S89:

$$\mathbf{s}_4 = \frac{\mathbf{g}_{HS}}{\|\mathbf{g}_{HS}\|} + \frac{\mathbf{g}_{LS}}{\|\mathbf{g}_{LS}\|} \quad (\text{S89})$$

Each seed is tested under two mass-weighting conventions:

```
seed_mw_div = seed_cart / sqrt_mass_vector
seed_mw_mul = seed_cart * sqrt_mass_vector
```

The seed/sqrt(m) convention is given by Equation S90:

$$\mathbf{s}_{\text{div}}^{\text{mw}} = \mathbf{M}^{-1/2} \mathbf{s} \quad (\text{S90})$$

The seed*sqrt(m) convention is given by Equation S91:

$$\mathbf{s}_{\text{mul}}^{\text{mw}} = \mathbf{M}^{1/2} \mathbf{s} \quad (\text{S91})$$

This produces eight candidate reaction-coordinate definitions for diagnostic comparison.

4.6.16 Removal of rigid-body contamination from each seed

Each mass-weighted seed is projected into the nonrigid subspace:

```
seed_mw = P_RT @ seed_mw_raw
```

The projected seed is then normalized:

```
seed_mw = normalize_vector(seed_mw, label + " " + convention)
```

This removes any translational or rotational contamination from the candidate direction.

The reaction-coordinate projector associated with the seed is then:

```
P_RC_seed = np.outer(seed_mw, seed_mw)
```

The reaction-coordinate projector is given by Equation S92:

$$\mathbf{P}_{\text{RC}} = \mathbf{s}_{\text{mw}} \mathbf{s}_{\text{mw}}^T \quad (\text{S92})$$

The combined projector over translation, rotation, and the reaction coordinate is:

```
P_TOTAL = P_TR + P_RC_seed
```

The combined translation/rotation/reaction-coordinate projector is given by Equation S93:

$$\mathbf{P}_{\text{TOTAL}} = \mathbf{P}_{\text{TR}} + \mathbf{P}_{\text{RC}} \quad (\text{S93})$$

4.6.17 Projected zero-mode reaction-coordinate analysis

For each candidate, the effective Hessian is projected to remove translation, rotation, and the seed reaction-coordinate direction:

```
H_proj = (
    (np.eye(num_cart) - P_TOTAL)
    @ H_eff_mw_selected
    @ (np.eye(num_cart) - P_TOTAL)
)
```

The projected effective Hessian is given by Equation S94:

$$\mathbf{H}_{\text{proj}} = (\mathbf{I} - \mathbf{P}_{\text{TOTAL}}) \mathbf{H}_{\text{eff}} (\mathbf{I} - \mathbf{P}_{\text{TOTAL}}) \quad (\text{S94})$$

The projected Hessian is diagonalized:

```
eigvals, eigvecs = np.linalg.eigh(H_proj)
```

The eigenvectors associated with the smallest absolute eigenvalues are examined to identify the removed reaction-coordinate zero mode.

Each candidate eigenvector is scored using:

- overlap with the seed reaction coordinate;
- overlap with the translation/rotation subspace; and
- magnitude of its eigenvalue.

The preferred zero-mode vector has high reaction-coordinate overlap, low rigid-body overlap, and a near-zero eigenvalue.

The corresponding zero-mode reduced mass is stored in each candidate dictionary under:

```
cand["mu_zero_amu"]
```

This value is reported as the zero-mode reduced mass.

4.6.18 Direct reaction-coordinate reduced mass

Step 6 also constructs the one-dimensional reaction-coordinate Hessian block:

```
H_RC = (
    P_RC_seed
    @ H_eff_mw_selected
    @ P_RC_seed
)
```

The one-dimensional reaction-coordinate Hessian block is given by Equation S95:

$$\mathbf{H}_{\text{RC}} = \mathbf{P}_{\text{RC}} \mathbf{H}_{\text{eff}} \mathbf{P}_{\text{RC}} \quad (\text{S95})$$

The corresponding reaction-coordinate eigenvector is identified as the eigenvector with the largest overlap with the seed:

```
idx_rc = int(
    np.argmax(
        np.abs(eigvecs_rc.T @ seed_mw)
    )
)
```

The resulting vector is stored as:

```
k_rc_direct
```

and converted into a Cartesian-like displacement vector by:

```
k_cart_mass_unweighted = (
    k_mw / sqrt_mass_vector
)
```

The direct reduced mass and its Cartesian-like vector are returned by:

```
mu_direct, k_direct_cart_like = rc_mass_from_vector(k_rc_direct)
```

The corresponding reduced-mass expression is given by Equation S96:

$$\mu_{\text{RC}} = \left[\sum_{i=1}^{3N} \left(\frac{k_i^{\text{mw}}}{\sqrt{m_i}} \right)^2 \right]^{-1} \quad (\text{S96})$$

This value is described in the workflow as direct reaction-coordinate mass.

4.6.19 Automatic selection of the final reaction coordinate

Although eight candidate definitions are evaluated, the final selection rule is fixed explicitly in both implementations:

```
if (
    cand["label"].startswith("gradient difference")
    and cand["convention"] == "seed/sqrt(m)"
):
    selected_rc = cand
```

The selected Cartesian seed is defined by Equation S97:

$$\mathbf{s} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \quad (\text{S97})$$

Its seed/sqrt(m) mass-weighted form is given by Equation S98:

$$\mathbf{s}^{\text{mw}} = \mathbf{M}^{-1/2} \mathbf{s} \quad (\text{S98})$$

```
reduced_mass_amu = float(selected_rc["mu_direct_amu"])
```

The selected mass-weighted seed is exported as `reaction_direction_hessian_mw`. The direct reaction-coordinate eigenvector is mass-unweighted and normalized before being exported as `reaction_direction_hessian_cart`.

The final downstream reduced mass is summarized by Equation S99:

$$\mu_{\text{final}} = \mu_{\text{direct}}[\Delta \mathbf{g}/\sqrt{\mathbf{m}}] \quad (\text{S99})$$

The two diagnostic effective-Hessian masses are retained separately as:

`reduced_mass_effhess_zero_selected_amu`
`reduced_mass_effhess_direct_selected_amu`

4.6.20 Distinction among the three reduced masses

Step 6 produces three related reduced-mass quantities:

Cartesian gradient-direction mass:

In the Local PC implementation:

`reduced_mass_gradient_direction_amu`

In the HPC implementation:

`gradient_gap_reduced_mass_amu`

Both quantities are calculated directly from the normalized $\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}$ Cartesian direction.

Effective-Hessian zero-mode mass:

`reduced_mass_effhess_zero_selected_amu`

calculated from the near-zero eigenvector recovered after projection for the selected candidate.

Effective-Hessian direct mass:

`reduced_mass_effhess_direct_selected_amu`

calculated from the direct reaction-coordinate block of the selected effective Hessian.

The final downstream value is:

`reduced_mass_amu = reduced_mass_effhess_direct_selected_amu`

This final assignment is summarized by Equation S100:

$$\mu_{\text{final}} = \mu_{\text{direct}} \quad (\text{S100})$$

The other two values remain available as diagnostics.

4.6.21 Projection of the effective Hessian into the transverse subspace

After selection of the final reaction coordinate, Step 6 removes translation, rotation, and reaction-coordinate motion from the effective Hessian.

The retained transverse-space projector is constructed as:

```
P_REMOVE = P_TOTAL  
P_KEEP = np.eye(num_cart) - P_REMOVE
```

The equivalent projector relation is given by Equation S101:

$$\mathbf{P}_{\text{keep}} = \mathbf{I} - \mathbf{P}_{\text{TR}} - \mathbf{P}_{\text{RC}} \quad (\text{S101})$$

The transverse effective Hessian is then:

```
H_eff_projected_transverse = (  
    P_KEEP  
    @ H_eff_mw_selected  
    @ P_KEEP  
)
```

The transverse effective Hessian is given by Equation S102:

$$\mathbf{H}_{\text{eff}}^{\perp} = \mathbf{P}_{\text{keep}} \mathbf{H}_{\text{eff}} \mathbf{P}_{\text{keep}} \quad (\text{S102})$$

The matrix is symmetrized before diagonalization.

This projected Hessian contains only vibrational motion transverse to the crossing reaction coordinate.

4.6.22 Diagonalization of the projected effective Hessian

The transverse effective Hessian is diagonalized using:

```
eigvals_eff, eigvecs_eff = np.linalg.eigh(  
    H_eff_projected_transverse  
)
```

The diagonalization is represented by Equation S103:

$$\mathbf{H}_{\text{eff}}^{\perp} \mathbf{v}_i = \lambda_i \mathbf{v}_i \quad (\text{S103})$$

The eigenvalues are sorted in ascending order.

Each eigenvector is then tested for residual overlap with:

- the rigid-body basis, Z; and
- the selected reaction-coordinate vector reaction_direction_hessian_mw.

The overlaps are calculated as:

```
rt_overlap_eff  
rc_overlap_eff
```

A mode is classified as zero-like if either overlap exceeds the numerical threshold:

```
zero_like_mask = (  
    (rt_overlap_eff > 1.0e-6)  
    | (rc_overlap_eff > 1.0e-6)  
)
```

This prevents translational, rotational, or reaction-coordinate modes from being misclassified as physical transverse vibrations.

4.6.23 Conversion of effective-Hessian eigenvalues to frequencies

The nonzero eigenvalues are converted to wavenumbers using:

```
conversion_factor = 5140.48
```

The effective-Hessian frequencies are then evaluated as:

```
freq_eff_cm1[j] = (  
    np.sign(val)  
    * np.sqrt(abs(val))  
    * conversion_factor  
)
```

The frequency conversion is given by Equation S104:

$$\nu_i = \text{sgn}(\lambda_i) \sqrt{|\lambda_i|} \times 5140.48 \quad (\text{S104})$$

Positive eigenvalues produce real frequencies according to Equation S105:

$$\lambda_i > 0 \Rightarrow \nu_i > 0 \quad (\text{S105})$$

Negative eigenvalues produce signed imaginary frequencies according to Equation S106:

$$\lambda_i < 0 \Rightarrow \nu_i < 0 \quad (\text{S106})$$

Modes identified as translational, rotational, or reaction-coordinate-like are assigned zero frequency.

The final arrays are:

```
freq_MECP_effhess_real_cm1  
freq_MECP_effhess_imag_cm1  
freq_MECP_effhess_zero_cm1
```

The aliases used in later steps are:

freq_MECP_real_cm1 freq_MECP_imag_cm1 freq_MECP_zero_cm1
--

The real transverse frequencies are the authoritative MECP frequencies used in the density-of-states calculation.

4.6.24 Expected number of effective-Hessian frequencies

For a nonlinear molecule with N atoms, the full Cartesian space contains $3N$ coordinates.

After removing:

- three translations;
- three rotations; and
- one reaction-coordinate direction,

the expected number of transverse vibrational modes is

$$3N - 7.$$

For a linear molecule, the expected number is

$$3N - 6,$$

because only two independent rotational modes are present.

The implementation does not hard-code these numbers. Instead, it constructs the rigid-body subspace numerically and identifies zero-like modes through projector overlaps. This allows the same procedure to accommodate both linear and nonlinear geometries.

4.6.25 Storage of the two original Hessians

The Cartesian Hessians are stored together in the three-dimensional array:

hessian_data_MECP

Its dimensions are:

$$(3N) \times (3N) \times 2.$$

The first slice contains:

[H_mecp_main_other_cart](#)

and the second contains:

[H_mecp_other_main_cart](#)

The mass-weighted forms are stored similarly in:

[mw_hessian_data_MECP](#)

These arrays preserve the original surface-specific Hessian information for later diagnostics or independent analysis.

4.6.26 Reaction-coordinate candidate diagnostics

All eight candidate reaction-coordinate analyses are retained in:

[rc_candidates](#)

For each candidate, SPINKS records:

- seed definition;
- mass-weighting convention;
- zero-mode reduced mass;
- direct reduced mass;
- selected near-zero eigenvalue;
- overlap with the seed reaction coordinate; and
- overlap with the translation/rotation subspace.

A typical diagnostic record contains:

```
{  
  "label": ...,  
  "convention": ...,  
  "mu_zero_amu": ...,  
  "mu_direct_amu": ...,  
  "best_zero_eigval": ...,  
  "rc_overlap_zero": ...,  
  "rt_overlap_zero": ...  
}
```

Although only one convention is used downstream, retaining all candidates makes the reaction-coordinate selection transparent and allows comparison among alternative definitions.

4.6.27 Atomic contributions to the selected reaction coordinate

The Cartesian reaction-coordinate vector is divided into three-component atomic displacements. The amplitude associated with atom *a* is calculated according to Equation S107:

$$A_a = \sqrt{k_{ax}^2 + k_{ay}^2 + k_{az}^2} \quad (\text{S107})$$

The implementation is:

```
amp = float(
    np.linalg.norm(
        k_cart_like[idx:idx + 3]
    )
)
```

The atoms are ranked by decreasing amplitude.

This analysis identifies which atoms contribute most strongly to the effective crossing reaction coordinate and provides a chemically interpretable description of the spin-crossing motion.

4.6.28 Files generated by step 6

Step 6 creates the directory:

`Effective_Hessian_RC`

and writes three principal output files.

Effective-Hessian and reduced-mass summary:

`Step6_effective_hessian_RC_summary.txt`

This file contains:

- both gradient norms;
- the gradient dot product;
- crossing classification;
- GRADIENT_GAP_NORM_EH_PER_BOHR;
- both projected gradient magnitudes;
- PROJECTED_GRADIENT_MEAN_EH_PER_BOHR;
- both Hessian paths;
- the selected effective-Hessian expression;
- all three reduced masses;

- the selected reaction-coordinate definition;
- zero-mode and overlap diagnostics;
- frequency counts;
- all reaction-coordinate candidates; and
- ranked atomic contributions.

Effective-Hessian frequency file:

Step6_effective_hessian_frequencies.txt

This file lists separately:

- real transverse frequencies;
- imaginary transverse frequencies; and
- zero-like frequencies.

Reaction-coordinate vector file:

Step6_reaction_coordinate_vectors.txt

This file contains:

- [reaction_direction_hessian_cart](#)
- [reaction_direction_hessian_mw](#)

- In the Local PC version, the normalized gradient-difference vectors are stored as:

[reverse_gradient_unit_vector](#)
[gradient_difference_unit_vector](#)

- In the HPC version, the normalized gradient-difference vectors are stored as:

[gradient_gap_unit_vector](#)
[reverse_gradient_gap_unit_vector](#)

4.6.29 Variables exported by step 6

The primary gradient quantities common to both implementations are:

[GRADIENT_GAP_NORM_EH_PER_BOHR](#)
[PROJECTED_GRADIENT_MEAN_EH_PER_BOHR](#)
[projected_gradient_mean_Eh_per_Bohr](#)

The individual projected-gradient magnitudes are also exported. The Local PC names are `low_spin_projected_gradient_magnitude` and `high_spin_projected_gradient_magnitude`; the HPC names are `low_spin_projected_gradient_Eh_per_Bohr` and `high_spin_projected_gradient_Eh_per_Bohr`.

The primary reduced-mass and reaction-coordinate variables are:

```
reduced_mass_amu  
reaction_direction_hessian_cart  
reaction_direction_hessian_mw
```

The Cartesian gradient-direction diagnostic is exported under the implementation-specific names:

In the Local PC implementation:

```
reduced_mass_gradient_direction_amu
```

In the HPC implementation:

```
gradient_gap_reduced_mass_amu
```

The effective-Hessian diagnostic masses are:

```
reduced_mass_effhess_zero_selected_amu  
reduced_mass_effhess_direct_selected_amu
```

The primary Hessian variables are:

```
H_eff_mw_selected  
H_eff_mw_plus  
H_eff_mw_minus  
H_eff_projected_transverse
```

The final MECP frequency variables are:

```
freq_MECP_effhess_real_cm1  
freq_MECP_effhess_imag_cm1  
freq_MECP_effhess_zero_cm1
```

The associated projectors and rigid-body basis retained in both implementations are:

```
Z  
P_TR  
P_RT  
P_RC  
P_TOTAL
```

The original Cartesian and mass-weighted surface Hessians are also retained as `hessian_data_MECP` and `mw_hessian_data_MECP`. Together, these quantities provide the gradient, effective-Hessian, reaction-coordinate, reduced-mass, and MECP-frequency information required by the subsequent energetic, probability, and density-of-states calculations.

4.6.30 Summary of the step 6 algorithm

The complete sequence implemented in Step 6 can be summarized as follows.

The two Cartesian gradients and Hessians are imported:

$$\{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}, \mathbf{H}_1, \mathbf{H}_2\}.$$

The gradient-difference direction is constructed according to Equation S108:

$$\Delta \mathbf{g} = \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}, \mathbf{n} = \frac{\Delta \mathbf{g}}{\|\Delta \mathbf{g}\|} \quad (\text{S108})$$

The gradient-gap magnitude is given by Equation S109:

$$\Delta g_{\parallel} = \|\Delta \mathbf{g}\| \quad (\text{S109})$$

The two surface gradients are projected onto this direction according to Equation S110:

$$g_{\text{LS},\parallel} = |\mathbf{g}_{\text{LS}} \cdot \mathbf{n}|, g_{\text{HS},\parallel} = |\mathbf{g}_{\text{HS}} \cdot \mathbf{n}| \quad (\text{S110})$$

The final projected gradient mean is then obtained from Equation S111:

$$\text{GRADMEAN} = \sqrt{g_{\text{LS},\parallel} g_{\text{HS},\parallel}} \quad (\text{S111})$$

The two Cartesian Hessians are mass weighted according to Equation S112:

$$\mathbf{H}_i^{\text{mw}} = \mathbf{M}^{-1/2} \mathbf{H}_i \mathbf{M}^{-1/2} \quad (\text{S112})$$

The effective Hessian is selected according to the crossing topology using Equation S113:

$$\mathbf{H}_{\text{eff}} = \begin{cases} \frac{\|\mathbf{g}_{\text{HS}}\| \mathbf{H}_1 + \|\mathbf{g}_{\text{LS}}\| \mathbf{H}_2}{\|\mathbf{g}_{\text{HS}}\| + \|\mathbf{g}_{\text{LS}}\|}, & \text{peaked} \\ \frac{\|\mathbf{g}_{\text{HS}}\| \mathbf{H}_1 - \|\mathbf{g}_{\text{LS}}\| \mathbf{H}_2}{\|\mathbf{g}_{\text{HS}}\| - \|\mathbf{g}_{\text{LS}}\|}, & \text{sloped} \end{cases} \quad (\text{S113})$$

The translational and rotational subspace is removed according to Equation S114:

$$\mathbf{P}_{\text{RT}} = \mathbf{I} - \mathbf{P}_{\text{TR}} \quad (\text{S114})$$

The selected reaction-coordinate seed is given by Equation S115:

$$\mathbf{s}_{\text{RC}} = \mathbf{M}^{-1/2} (\mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}}) \quad (\text{S115})$$

Its projector is given by Equation S116:

$$\mathbf{P}_{\text{RC}} = \mathbf{s}_{\text{RC}} \mathbf{s}_{\text{RC}}^{\text{T}} \quad (\text{S116})$$

The final reduced mass is obtained from the direct effective-Hessian reaction-coordinate vector according to Equation S117:

$$\mu_{\text{RC}} = \left[\sum_i \left(\frac{k_i^{\text{mw}}}{\sqrt{m_i}} \right)^2 \right]^{-1} \quad (\text{S117})$$

The effective Hessian is then projected into the transverse subspace according to Equation S118:

$$\mathbf{H}_{\text{eff}}^{\perp} = (\mathbf{I} - \mathbf{P}_{\text{TR}} - \mathbf{P}_{\text{RC}}) \mathbf{H}_{\text{eff}} (\mathbf{I} - \mathbf{P}_{\text{TR}} - \mathbf{P}_{\text{RC}}) \quad (\text{S118})$$

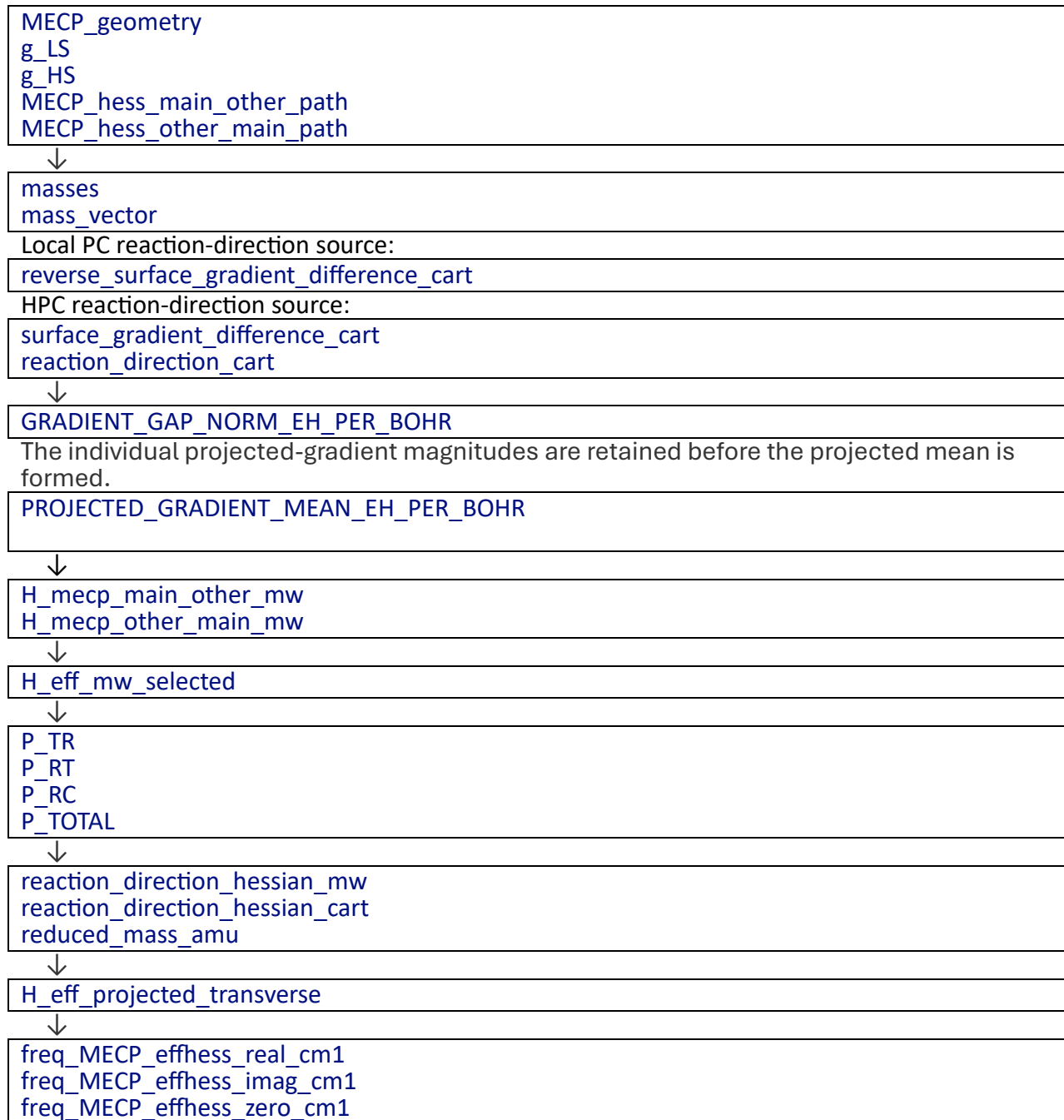
The projected effective Hessian is diagonalized according to Equation S119:

$$\mathbf{H}_{\text{eff}}^{\perp} \mathbf{v}_i = \lambda_i \mathbf{v}_i \quad (\text{S119})$$

The transverse MECP frequencies are obtained from Equation S120:

$$\nu_i = \text{sgn}(\lambda_i) \sqrt{|\lambda_i|} \times 5140.48 \quad (\text{S120})$$

The corresponding variable flow is:



Therefore, Step 6 transforms the two MECP gradients and the two surface Hessians into the authoritative gradient-gap magnitude, projected gradient mean, effective-Hessian reaction coordinate, final reduced mass, and transverse crossing-point frequencies.

The overall transformation is represented schematically below:

$$\{\mathbf{g}_{\text{LS}}, \mathbf{g}_{\text{HS}}, \mathbf{H}_1, \mathbf{H}_2\} \rightarrow \{\Delta g_{\parallel}, \text{GRADMEAN}, \mathbf{k}_{\text{RC}}, \mu_{\text{RC}}, \{v_{X,i}^{\text{eff}}\}\}$$

These quantities are subsequently used to finalize the zero-point-corrected MECP barrier, calculate the energy-dependent crossing velocity, evaluate Landau-Zener and weak coupling probabilities, and construct the MECP density and sum of states.

4.7 Step 7: Reaction Coordinate Energy and Velocity Grid

Step 7 uses the effective-Hessian frequencies and reduced mass obtained in Step 6 to finalize the zero-point-energy-corrected MECP barrier and construct the common microcanonical energy and velocity grids used in the subsequent probability and rate calculations.

No additional electronic-structure calculation is performed in this step. The calculations are numerical and use:

- the electronic MECP barrier from Step 5;
- the reference-state vibrational frequencies from Step 5;
- the projected effective-Hessian MECP frequencies from Step 6;
- the reaction-coordinate reduced mass from Step 6;
- the gradient-gap magnitude from Step 6; and
- the projected gradient mean from Step 6.

The principal operations performed in Step 7 are:

1. calculation of the effective-Hessian MECP zero-point energy;
2. calculation of the MECP–reference ZPE difference;
3. replacement of the provisional electronic barrier with the active ZPE-corrected MECP barrier;
4. user definition of the maximum reaction-coordinate excess energy;
5. construction of a common 1 cm^{-1} total-energy grid;
6. identification of the MECP threshold bin;
7. calculation of the energy available along the reaction coordinate;
8. conversion of the reaction-coordinate excess energy to SI units;
9. evaluation of the corresponding crossing velocity;
10. generation of the velocity plot;
11. storage of the complete energy and velocity arrays; and
12. export of the common grid for the probability and rate calculations.

4.7.1 Import of quantities from the preceding steps

Step 7 verifies that the principal energetic, gradient, frequency, and reduced-mass quantities are available:

```
required_vars_step7 = [  
    "jobname",  
    "VaG_MECP_electronic_cm1",  
    "reduced_mass_amu",  
    "GRADIENT_GAP_NORM_EH_PER_BOHR",  
    "PROJECTED_GRADIENT_MEAN_EH_PER_BOHR",  
    "freq_MECP_effhess_real_cm1",  
    "local_base",  
    "remote_base"  
]
```

The most important inputs are:

```
VaG_MECP_electronic_cm1  
freq_MECP_effhess_real_cm1  
ZPE_REF_from_freq_cm1  
reduced_mass_amu
```

These quantities represent, respectively, the electronic MECP barrier relative to the selected reference minimum, the real transverse frequencies from the projected effective Hessian, the reference zero-point energy, and the reaction-coordinate reduced mass.

4.7.2 Recovery of the reference zero-point energy

The preferred reference ZPE variable is:

```
ZPE_REF_from_freq_cm1
```

which was calculated in Step 5 directly from the positive reference-state frequencies.

If this variable is unavailable, Step 7 reconstructs it from one of the available Hartree values:

```
if "ZPE_REF_thermo_hartree" in globals():  
    ZPE_REF_from_freq_cm1 = (  
        float(ZPE_REF_thermo_hartree)  
        * hartree_to_cm  
    )  
elif "ZPE_REF_hartree" in globals():  
    ZPE_REF_from_freq_cm1 = (  
        float(ZPE_REF_hartree)  
        * hartree_to_cm  
    )
```

The Hartree-to-wavenumber conversion is given by Equation S121:

$$\text{ZPE}_R(\text{cm}^{-1}) = \text{ZPE}_R(E_h) \times 219474.6313705 \quad (\text{S121})$$

This fallback ensures that the ZPE correction can still be evaluated if the frequency-derived reference variable is absent but the thermochemical ZPE is available.

4.7.3 Validation of the effective-Hessian MECP frequencies

The Step 6 frequencies are converted to a one-dimensional numerical array:

```
freq_MECP_effhess_real_cm1 = np.asarray(  
    freq_MECP_effhess_real_cm1,  
    dtype=float  
).reshape(-1)
```

Only finite, positive values are retained:

```
freq_MECP_effhess_real_cm1 = (  
    freq_MECP_effhess_real_cm1[  
        np.isfinite(freq_MECP_effhess_real_cm1)  
        & (freq_MECP_effhess_real_cm1 > 0.0)  
    ]  
)
```

Thus, translational, rotational, reaction-coordinate, imaginary, and invalid modes do not contribute to the MECP ZPE.

The code terminates if no usable positive frequencies remain:

```
if freq_MECP_effhess_real_cm1.size == 0:  
    raise RuntimeError(...)
```

This safeguard prevents construction of the final barrier from an invalid effective-Hessian analysis.

4.7.4 Effective-Hessian MECP zero-point energy

The crossing-point ZPE is calculated from the positive transverse effective-Hessian frequencies:

```
ZPE_MECP_effhess_cm1 = (  
    0.5  
    * float(  
        np.sum(freq_MECP_effhess_real_cm1)  
    )  
)
```

The effective-Hessian MECP ZPE is given by Equation S122:

$$\text{ZPE}_X^{\text{eff}} = \frac{1}{2} \sum_i v_{X,i}^{\text{eff}} \quad (\text{S122})$$

Only the real transverse modes are included.

This quantity differs from the raw ORCA PES2 ZPE because the frequencies originate from the projected effective Hessian rather than from one uncombined spin surface.

4.7.5 MECP–reference zero-point-energy difference

The ZPE correction relative to the reference minimum is:

```
Delta_ZPE_MECP_minus_REF_cm1 = (  
  ZPE_MECP_effhess_cm1  
  - float(ZPE_REF_from_freq_cm1)  
)
```

The MECP-reference ZPE difference is given by Equation S123:

$$\Delta ZPE_{X-R} = ZPE_X^{\text{eff}} - ZPE_R \quad (\text{S123})$$

A positive value raises the active barrier relative to the electronic barrier, whereas a negative value lowers it.

The variable is stored as:

```
Delta_ZPE_MECP_minus_REF_cm1
```

for later reporting and validation.

4.7.6 Final ZPE-corrected MECP barrier

The active MECP barrier is calculated as:

```
VaG_MECP_ZPE_corrected_cm1 = (  
  float(VaG_MECP_electronic_cm1)  
  + Delta_ZPE_MECP_minus_REF_cm1  
)
```

The ZPE-corrected MECP barrier is given by Equation S124:

$$V_{\text{MECP}}^{\text{ZPE}} = V_{\text{MECP}}^{\text{el}} + ZPE_X^{\text{eff}} - ZPE_R \quad (\text{S124})$$

The result replaces the provisional electronic barrier:

```
VaG_MECP_cm1 = float(  
  VaG_MECP_ZPE_corrected_cm1  
)
```

The active barrier used downstream is given by Equation S125:

$$V_{\text{MECP}} = V_{\text{MECP}}^{\text{el}} + \Delta ZPE_{X-R} \quad (\text{S125})$$

The corresponding value in kJ mol^{-1} is:

```
VaG_MECP_kJmol = (  
  VaG_MECP_cm1 * 0.01196266  
)
```

The general MECP-energy aliases are then updated:

```
E_MECP = VaG_MECP_cm1  
E_MECP_cm1 = VaG_MECP_cm1
```

From this point onward, the active MECP barrier variable shown above refers to the ZPE-corrected barrier rather than the electronic barrier.

4.7.7 Barrier-source labeling

The definition of the active barrier is recorded explicitly in:

```
VaG_MECP_barrier_label
```

as:

```
ZPE-corrected MECP barrier = electronic barrier  
+ ZPE_effhess(MECP) - ZPE(reference)
```

This label is propagated to the final reports so that the energetic origin of the threshold is unambiguous.

The electronic and ZPE-corrected barriers remain available separately:

```
VaG_MECP_electronic_cm1  
VaG_MECP_ZPE_corrected_cm1  
VaG_MECP_cm1
```

This separation allows the contribution of the ZPE correction to be examined directly.

4.7.8 User definition of the maximum excess energy

The user is asked to specify the highest reaction-coordinate energy above the ZPE-corrected MECP barrier. The two implementations store this entered value under different variable names.

In the Local PC implementation:

```
maximum_reaction_coordinate_energy_cm1 = ask_float(  
    "Maximum reaction-coordinate energy above MECP, cm-1, e.g. 5000: ",  
    minimum=1.0e-12  
)
```

In the HPC implementation:

```
epsilon_rc_max_cm1 = ask_float(  
    "Maximum reaction-coordinate energy above MECP, cm-1, e.g. 5000: ",  
    minimum=1.0e-12  
)
```

Both variables represent the same physical maximum excess energy.

The entered value is related to the requested total-energy limit by Equation S126:

$$\epsilon_{\text{RC,max}} = E_{\text{max}} - E_{\text{MECP}} \quad (\text{S126})$$

The requested maximum total energy is then formed by adding the entered excess energy to the active MECP barrier.

In the Local PC implementation:

```
requested_maximum_total_energy_cm1 = (
    float(VaG_MECP_cm1)
    + float(maximum_reaction_coordinate_energy_cm1)
)
```

In the HPC implementation:

```
E_max_requested_cm1 = (
    float(VaG_MECP_cm1) + float(epsilon_rc_max_cm1)
)
```

The common total-energy relation is summarized by Equation S127:

$$E_{\text{max}}^{\text{requested}} = V_{\text{MECP}} + \epsilon_{\text{RC,max}} \quad (\text{S127})$$

4.7.9 Definition of the common energy spacing

The microcanonical energy spacing is fixed as:

```
energy_step_cm1 = 1.0
```

The fixed energy spacing is given by Equation S128:

$$\Delta E = 1 \text{ cm}^{-1} \quad (\text{S128})$$

This is the common energy resolution used by the subsequent probability, density-of-states, sum-of-states, microcanonical-rate, and canonical-rate calculations.

Using a common integer-spaced grid ensures consistent indexing and direct alignment with the SPINKS implementation.

4.7.10 Construction of the total-energy grid

The highest required integer energy-bin index is calculated from the requested maximum total energy.

In the Local PC implementation:

```
maximum_energy_bin_index = int(  
    np.ceil(  
        requested_maximum_total_energy_cm1  
        / energy_step_cm1  
    )  
)
```

In the HPC implementation:

```
maximum_energy_bin_index = int(np.ceil(E_max_requested_cm1 / energy_step_cm1))
```

Both implementations then construct the common total-energy grid as:

```
total_energy_grid_cm1 = (  
    np.arange(maximum_energy_bin_index + 1, dtype=float)  
    * energy_step_cm1  
)
```

The total-energy grid is represented mathematically by Equation S129:

$$E_j = j\Delta E, \quad j = 0, 1, \dots, n_{\max} \quad (\text{S129})$$

The grid spacing appearing in that expression is given by Equation S130:

$$\Delta E = 1 \text{ cm}^{-1} \quad (\text{S130})$$

The grid begins at zero energy relative to the selected reference minimum and extends to the smallest integer-spaced grid point that reaches or exceeds the requested maximum.

The actual maximum grid energy is the final element of the total-energy grid:

```
E_max_cm1 = float(total_energy_grid_cm1[-1])
```

Additional aliases in the Local PC implementation are:

```
maximum_total_energy_cm1 = E_max_cm1  
interoperability_maximum_energy_cm1 = E_max_cm1
```

The HPC implementation additionally stores:

```
E_max_grid_cm1 = E_max_cm1
```

The actual maximum grid energy is summarized by Equation S131:

$$E_{\max} = \left\lceil E_{\max}^{\text{requested}} \right\rceil \quad (\text{S131})$$

4.7.11 Identification of the MECP threshold bin

The MECP bin is defined as:

```
crossing_energy_bin_index = int(  
    np.ceil(E_MECP / energy_step_cm1)  
)
```

For the fixed 1 cm^{-1} spacing, the threshold-bin definition is given by Equation S132:

$$j_{\text{MECP}} = \lceil V_{\text{MECP}} \rceil \quad (\text{S132})$$

Because the ZPE-corrected MECP energy is generally noninteger, the first grid point above the exact MECP energy is the threshold bin.

For example, consider the ZPE-corrected MECP energy in Equation S133:

$$V_{\text{MECP}} = 10663.867 \text{ cm}^{-1} \quad (\text{S133})$$

The corresponding threshold-bin index is then given by Equation S134:

$$j_{\text{MECP}} = 10664 \quad (\text{S134})$$

This distinction is important because the exact MECP energy itself is not necessarily a member of the integer grid.

4.7.12 Reaction-coordinate excess-energy grid

The signed energy available relative to the crossing barrier is calculated directly from the common total-energy grid.

In the Local PC implementation:

```
reaction_coordinate_excess_energy_cm1 = (  
    total_energy_grid_cm1 - E_MECP  
)
```

In the HPC implementation:

```
epsilon_rc_cm1 = total_energy_grid_cm1 - E_MECP
```

The common physical definition of the signed excess energy is given by Equation S135:

$$\epsilon_{\text{RC}}(E) = E - V_{\text{MECP}} \quad (\text{S135})$$

The nonnegative excess-energy array is also stored under implementation-specific names.

In the Local PC implementation:

```
positive_reaction_coordinate_energy_cm1 = np.maximum(  
    reaction_coordinate_excess_energy_cm1,  
    0.0  
)
```

In the HPC implementation:

```
epsilon_rc_positive_cm1 = np.maximum(
    epsilon_rc_cm1,
    0.0
)
```

- The signed reaction-coordinate excess energy is negative below the MECP;
- it is zero only if the exact barrier lies on a grid point; and
- it is positive above the MECP.

For operations requiring nonnegative energy, the definition is summarized by Equation S136:

$$\epsilon_{\text{RC}}^+ = \max(E - V_{\text{MECP}}, 0) \quad (\text{S136})$$

Additional signed-excess aliases are retained for compatibility.

In the Local PC implementation:

```
energy_above_crossing_cm1 = reaction_coordinate_excess_energy_cm1.copy()
energy_relative_to_crossing_cm1 = reaction_coordinate_excess_energy_cm1.copy()
```

In the HPC implementation:

```
E_above_MECP_cm1 = epsilon_rc_cm1.copy()
dE_cm1 = epsilon_rc_cm1.copy()
```

These aliases preserve the same signed excess-energy array.

4.7.13 Why the velocity is zero at and below the MECP

The classical reaction-coordinate velocity is evaluated only when the energy exceeds the crossing barrier.

For energies at or below the MECP threshold, Equation S137:

$$E \leq V_{\text{MECP}} \quad (\text{S137})$$

the available reaction-coordinate kinetic energy is nonpositive, as shown in Equation S138:

$$\epsilon_{\text{RC}} \leq 0 \quad (\text{S138})$$

The implementation therefore assigns zero velocity, as given by Equation S139:

$$v(E) = 0 \quad (\text{S139})$$

For energies above the MECP threshold, Equation S140:

$$E > V_{\text{MECP}} \quad (\text{S140})$$

the reaction-coordinate velocity is given by Equation S141:

$$v(E) = \sqrt{\frac{2\epsilon_{RC}}{\mu_{RC}}} \quad (S141)$$

Consequently, if the exact MECP energy lies between integer grid points, the first nonzero velocity occurs at the first grid bin above the MECP.

4.7.14 Conversion of excess energy to joules

The signed excess energy in wavenumbers is converted to joules. The two implementations use the same conversion but reference their implementation-specific signed excess-energy arrays differently.

In the Local PC implementation:

```
E_excess_J = (
  reaction_coordinate_excess_energy_cm1
  * 100.0
  * h_SI
  * c_SI
)
```

In the HPC implementation:

```
E_excess_J = (
  (total_energy_grid_cm1 - E_MECP)
  * 100.0
  * h_SI
  * c_SI
)
```

The reciprocal-centimetre to joule conversion is given by Equation S142:

$$\epsilon_{RC}(J) = \epsilon_{RC}(\text{cm}^{-1}) \times 100 \times h \times c \quad (S142)$$

The factor of 100 converts inverse centimetres to inverse metres.

The physical constants used in this conversion are:

```
h_SI = 6.62607015e-34
c_SI = 2.99792458e8
```

These values correspond to Planck's constant and the speed of light, respectively.

4.7.15 Conversion of the reduced mass to kilograms

The reaction-coordinate reduced mass from Step 6 is converted from atomic mass units to kilograms:


```
mu_kg = (
    reduced_mass_amu
    * amu_to_kg
)
```

with:

```
amu_to_kg = 1.66053906660e-27
```

The reduced-mass conversion is given by Equation S143:

$$\mu_{\text{kg}} = \mu_{\text{amu}} \times 1.66053906660 \times 10^{-27} \quad (\text{S143})$$

The code verifies that the converted mass is finite and positive before calculating the velocity.

4.7.16 Reaction-coordinate velocity in meters per second

The velocity array is initialized as zero:

```
v_m_s = np.zeros_like(
    total_energy_grid_cm1,
    dtype=float
)
```

The positive-energy mask is:

```
mask_energy = E_excess_J > 0.0
```

and the velocity is evaluated only for those bins:

```
v_m_s[mask_energy] = np.sqrt(
    2.0
    * E_excess_J[mask_energy]
    / mu_kg
)
```

The piecewise velocity expression is given by Equation S144:

$$v(E) = \begin{cases} 0, & E \leq V_{\text{MECP}} \\ \sqrt{\frac{2(E - V_{\text{MECP}})}{\mu_{\text{RC}}}}, & E > V_{\text{MECP}} \end{cases} \quad (\text{S144})$$

Above the barrier, the square-root dependence is summarized by Equation S145:

$$v \propto \sqrt{E - V_{\text{MECP}}} \quad (\text{S145})$$

It is therefore largest at the highest energy included in the selected grid.

4.7.17 Velocity in bohr per second

The SI velocity is converted to bohr per second using:

```
v_bohr_s = v_m_s / bohr_m
```

with:

```
bohr_m = 5.29177210903e-11
```

The bohr-per-second conversion is given by Equation S146:

$$v_{\text{bohr/s}} = \frac{v_{\text{m/s}}}{5.29177210903 \times 10^{-11}} \quad (\text{S146})$$

The converted velocity array is used directly in the probability equations because the gradient quantities are expressed in atomic units involving bohr.

4.7.18 Numerically safe velocity array

At and below the crossing threshold, the velocity is exactly zero. Some subsequent probability equations contain inverse powers of velocity, so direct division by zero must be prevented.

Step 7 therefore creates:

```
v_bohr_s_safe = np.copy(v_bohr_s)
```

and replaces all values below the floor with an extremely small positive number:

```
velocity_floor = 1.0e-12
```

```
v_bohr_s_safe[  
    v_bohr_s_safe <= velocity_floor  
] = 1.0e-300
```

This array is used only for numerically safe evaluation of expressions involving division by velocity.

The physical velocity remains stored separately in:

```
v_bohr_s
```

and is still exactly zero at and below the MECP.

The probability code subsequently applies the appropriate energy masks so that subthreshold bins remain physically inactive.

4.7.19 Common-grid consistency with later steps

The following total-energy and velocity variables are common to both implementations:

```
total_energy_grid_cm1
probability_energy_grid_cm1
v_m_s
v_bohr_s
v_bohr_s_safe
```

The signed reaction-coordinate excess-energy array is implementation specific.
In the Local PC implementation:

```
reaction_coordinate_excess_energy_cm1
```

In the HPC implementation:

```
epsilon_rc_cm1
```

The nonnegative excess-energy array is also implementation specific.
In the Local PC implementation:

```
positive_reaction_coordinate_energy_cm1
```

In the HPC implementation:

```
epsilon_rc_positive_cm1
```

Additional compatibility aliases for the signed excess-energy array are:
In the Local PC implementation:

```
energy_above_crossing_cm1
energy_relative_to_crossing_cm1
```

In the HPC implementation:

```
E_above_MECP_cm1
dE_cm1
```

The common probability-grid alias is:

```
probability_energy_grid_cm1 = total_energy_grid_cm1.copy()
```

This ensures that the Landau-Zener probabilities are evaluated on exactly the same total-energy grid used for:

- weak coupling probabilities;
- intermediate SOC channels;
- M_S-specific SOC channels;
- densities of states;
- sums of states;
- microcanonical rates; and
- canonical integrations.

This prevents interpolation errors or bin misalignment between different parts of the workflow.

4.7.20 Velocity-grid plot

Step 7 generates a plot of velocity against total energy:

```
plt.plot(
    total_energy_grid_cm1,
    v_m_s,
    linewidth=2
)
```

The ZPE-corrected MECP barrier is marked by a vertical dashed line:

```
plt.axvline(
    E_MECP,
    linestyle="--",
    linewidth=1.5,
    label="ZPE-corrected MECP"
)
```

The horizontal axis is defined in Equation S147:

$$x = \text{total energy relative to the reference minimum} \quad (\text{S147})$$

The vertical axis is defined in Equation S148:

$$y = \text{reaction-coordinate velocity at the MECP} \quad (\text{S148})$$

Below the barrier, the plotted velocity remains zero; above the barrier, the square-root dependence is summarized by Equation S149:

$$v(E) \propto \sqrt{E - V_{\text{MECP}}} \quad (\text{S149})$$

The plot is saved as:

```
Step7_velocity_grid.png
```

The figure is written at 300 dpi.

4.7.21 Step 7 output directory

The common local output directory is:

```
local_step7 = os.path.join(
    local_base,
    "RC_energy_velocity_grid"
)
```

In the Local PC implementation, the remote-directory name is retained only as a compatibility alias:

```
remote_step7 = local_step7
```

In the HPC implementation, the remote directory is:

```
remote_step7 = posixpath.join(  
    remote_base,  
    "RC_energy_velocity_grid"  
)
```

When an active SFTP connection is available, the HPC summary, grid, and plot files are uploaded to the remote project directory.

Failure of the optional HPC upload does not invalidate the local Step 7 calculation.

4.7.22 Energy and velocity summary file

The file:

```
Step7_energy_velocity_summary.txt
```

records:

- the electronic MECP barrier;
- the reference ZPE;
- the effective-Hessian MECP ZPE;
- the MECP–reference ZPE difference;
- the final ZPE-corrected barrier;
- the barrier in kJ mol^{-1} ;
- the barrier-source label;
- energy spacing;
- number of bins;
- MECP bin index;
- requested maximum excess energy;
- actual maximum grid energy;
- reduced mass;

- reduced mass in kilograms;
- the gradient-gap magnitude;
- the projected gradient mean;
- minimum and maximum velocities; and
- paths to the generated files.

This file provides a compact record of the energy-grid construction.

4.7.23 Complete energy and velocity grid file

The file:

Step7_energy_velocity_grid.txt

contains one row for every 1 cm⁻¹ energy bin.

The output-column labels differ between the Local PC and HPC files.

In the Local PC implementation:

total_energy_cm1 reaction_coordinate_excess_energy_cm1 positive_reaction_coordinate_energy_cm1 excess_energy_J velocity_m_s velocity_bohr_s velocity_bohr_s_safe
--

In the HPC implementation:

total_energy_grid_cm1 epsilon_rc_cm1 epsilon_rc_positive_cm1 E_excess_J v_m_s v_bohr_s v_bohr_s_safe
--

For every total-energy grid point, the file records the signed and nonnegative excess energies, the excess energy in joules, and the corresponding velocities in SI and bohr-per-second units.

This complete tabulation permits independent reproduction of the probability calculations.

4.7.24 Variables exported by step 7

The principal energetic variables exported are:

VaG_MECP_electronic_cm1
ZPE_REF_from_freq_cm1
ZPE_MECP_effhess_cm1
Delta_ZPE_MECP_minus_REF_cm1
VaG_MECP_ZPE_corrected_cm1
VaG_MECP_cm1
VaG_MECP_kJmol

The following grid variables are common to both implementations:

energy_step_cm1
maximum_energy_bin_index
crossing_energy_bin_index
total_energy_grid_cm1
probability_energy_grid_cm1

Additional Local PC grid variables are:

maximum_reaction_coordinate_energy_cm1
requested_maximum_total_energy_cm1
reaction_coordinate_excess_energy_cm1
positive_reaction_coordinate_energy_cm1
energy_above_crossing_cm1
energy_relative_to_crossing_cm1
minimum_total_energy_cm1
E_max_cm1
maximum_total_energy_cm1
interoperability_maximum_energy_cm1

Additional HPC grid variables are:

epsilon_rc_max_cm1
E_max_requested_cm1
epsilon_rc_cm1
epsilon_rc_positive_cm1
E_above_MECP_cm1
dE_cm1
E_min_cm1
E_max_cm1
E_max_grid_cm1

The principal velocity and SI-conversion variables common to both implementations are:

mu_kg
E_excess_J
v_m_s
v_bohr_s
v_bohr_s_safe
epsilon_rc_J
velocity_floor

The validated gradient quantities retained for later steps are:

GRADIENT_GAP_NORM_EH_PER_BOHR
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR

These arrays and scalar quantities are consumed directly by the probability calculations in Step 8 and the rate calculations in the following steps.

4.7.25 Summary of the step 7 algorithm

The complete Step 7 sequence is scientifically identical in the Local PC and HPC implementations. Both compute the effective-Hessian MECP ZPE from the positive transverse frequencies, form the MECP-reference ZPE correction, replace the active barrier with the ZPE-corrected value, construct the same 1 cm^{-1} total-energy grid, and evaluate the same reaction-coordinate velocity. The implementations differ only in several compatibility variable names and in local-versus-remote file handling.

The effective-Hessian MECP ZPE is calculated from the positive transverse frequencies according to Equation S150:

$$\text{ZPE}_X^{\text{eff}} = \frac{1}{2} \sum_i v_{X,i}^{\text{eff}} \quad (\text{S150})$$

The ZPE difference relative to the reference minimum is given by Equation S151:

$$\Delta\text{ZPE}_{X-R} = \text{ZPE}_X^{\text{eff}} - \text{ZPE}_R \quad (\text{S151})$$

The active crossing barrier is then given by Equation S152:

$$V_{\text{MECP}} = V_{\text{MECP}}^{\text{el}} + \Delta\text{ZPE}_{X-R} \quad (\text{S152})$$

The user supplies a maximum reaction-coordinate excess energy, $\epsilon_{\text{RC,max}}$.

Step 7 then constructs the total-energy grid according to Equation S153:

$$E_j = j, j = 0, 1, \dots, [V_{\text{MECP}} + \epsilon_{\text{RC,max}}] \quad (\text{S153})$$

The grid energies are expressed in cm^{-1} .

The reaction-coordinate excess energy is given by Equation S154:

$$\epsilon_{\text{RC}}(E_j) = E_j - V_{\text{MECP}} \quad (\text{S154})$$

The reaction-coordinate velocity is evaluated according to Equation S155:

$$v(E_j) = \begin{cases} 0, & \epsilon_{\text{RC}} \leq 0 \\ \sqrt{\frac{2\epsilon_{\text{RC}}}{\mu_{\text{RC}}}}, & \epsilon_{\text{RC}} > 0 \end{cases} \quad (\text{S155})$$

The excess-energy conversion to joules is given by Equation S156:

$$\epsilon_{\text{RC}}(\text{J}) = \epsilon_{\text{RC}}(\text{cm}^{-1}) \times 100hc \quad (\text{S156})$$

The reduced-mass conversion is given by Equation S157:

$$\mu_{\text{kg}} = \mu_{\text{amu}} \times 1.66053906660 \times 10^{-27} \quad (\text{S157})$$

The corresponding variable flow is:

VaG_MECP_electronic_cm1
freq_MECP_effhess_real_cm1
ZPE_REF_from_freq_cm1

These quantities determine the effective-Hessian MECP ZPE and the MECP-reference ZPE correction:

ZPE_MECP_effhess_cm1
Delta_ZPE_MECP_minus_REF_cm1

These values define the active ZPE-corrected MECP barrier:

VaG_MECP_ZPE_corrected_cm1
VaG_MECP_cm1

The requested excess-energy range is stored under implementation-specific names, while the grid spacing is common.

In the Local PC implementation:

maximum_reaction_coordinate_energy_cm1
requested_maximum_total_energy_cm1

In the HPC implementation:

epsilon_rc_max_cm1
E_max_requested_cm1

The common spacing is:

energy_step_cm1 = 1.0

These values define the common total-energy grid and threshold index.

total_energy_grid_cm1
crossing_energy_bin_index

The signed excess-energy array is stored as:

In the Local PC implementation:

reaction_coordinate_excess_energy_cm1

In the HPC implementation:

epsilon_rc_cm1

The excess energy and reduced mass are then used to evaluate the reaction-coordinate velocity:

E_excess_J
reduced_mass_amu

The resulting velocity arrays are:

v_m_s
v_bohr_s
v_bohr_s_safe

These arrays are passed directly to Step 8 and to the subsequent rate calculations.

Therefore, Step 7 transforms the electronic barrier, effective-Hessian frequencies, reference ZPE, and reduced mass into the final active MECP threshold and a common energy-dependent velocity grid.

These quantities are subsequently used in Step 8 to calculate the effective, intermediate, and M_S-specific Landau-Zener and weak coupling spin-crossing probabilities on the same common energy grid.

4.8 Step 8: Landau-Zener and Weak Coupling Probabilities

Step 8 evaluates energy-dependent spin-crossing probabilities on the common total-energy grid generated in Step 7. Both the Landau-Zener (LZ) and weak-coupling (WC) expressions use the same active ZPE-corrected MECP barrier, reduced mass, gradient quantities, and Step 7 velocity arrays.

Three levels of spin-orbit-coupling resolution are considered:

1. Effective probabilities, calculated using the scalar effective SOC;
2. Intermediate probabilities, calculated from SOC values grouped according to $|M_S|$ on the lower-spin surface; and
3. Individual M_S -resolved probabilities, calculated from every nonzero element of the spin-orbit-coupling matrix.

No separate energy or velocity grid is created in Step 8. The Step 7 arrays are imported directly and independently verified. Individual M_S -specific probabilities are never summed in this step; channels with numerically equivalent $|H_{SO}|$ values are grouped only to avoid duplicate plotted curves.

The principal operations performed in Step 8 are:

1. import and verification of the Step 7 energy and velocity grids;
2. validation of the active MECP, gradient, SOC, and reduced-mass quantities;
3. evaluation of the effective Landau-Zener probability;
4. evaluation of the effective weak coupling probability;
5. extraction of individual nonzero SOC matrix elements;
6. calculation of one LZ and WC probability curve for every nonzero M_S -specific channel;
7. grouping of symmetry-equivalent or numerically equal- $|H_{SO}|$ curves for plotting only;
8. construction of intermediate SOC quantities grouped by $|M_S|$ on the lower-spin surface;
9. calculation of intermediate LZ and WC probability curves;
10. generation of probability plots;

11. storage of complete numerical probability arrays; and
12. export of Step 9-compatible probability variables.

4.8.1 Import of active quantities

Step 8 verifies the availability of the effective SOC, MECP threshold, gradient quantities, reduced mass, energy grid, and velocity arrays:

```
required_vars_step8 = [
    "jobname",
    "H_SO_cm",
    "GRADIENT_GAP_NORM_EH_PER_BOHR",
    "PROJECTED_GRADIENT_MEAN_EH_PER_BOHR",
    "VaG_MECP_cm1",
    "total_energy_grid_cm1",
    "v_m_s",
    "v_bohr_s",
    "energy_step_cm1",
    "maximum_energy_bin_index",
    "crossing_energy_bin_index",
    "reduced_mass_amu",
    "local_base",
    "remote_base"
]
```

The principal physical inputs are:

```
H_SO_cm
GRADIENT_GAP_NORM_EH_PER_BOHR
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
VaG_MECP_cm1
reduced_mass_amu
```

The effective spin-orbit coupling obtained in Step 2 is supplied through the first variable listed above.

The gradient-difference magnitude used in the Landau-Zener treatment is defined by Equation S158:

$$\Delta g_{\parallel} = \| \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \| \quad (\text{S158})$$

The projected gradient mean obtained in Step 6 and used in the weak-coupling treatment is represented by Equation S159:

$$\bar{g} = \sqrt{g_{\text{LS},\parallel} g_{\text{HS},\parallel}} \quad (\text{S159})$$

The active ZPE-corrected crossing barrier and the selected reaction-coordinate reduced mass are supplied by the variables listed above.

4.8.2 Reuse of the common Step 7 grid

The total-energy grid is imported and converted to a one-dimensional array:

```
total_energy_grid_cm1 = np.asarray(  
    total_energy_grid_cm1,  
    dtype=float  
).reshape(-1)
```

The probability-grid alias is defined as:

```
probability_energy_grid_cm1 = total_energy_grid_cm1.copy()
```

Therefore, Equation S160 follows:

$$E_{\text{LZ},j} = E_j \quad (\text{S160})$$

and no separate Landau-Zener grid is constructed.

The Step 7 velocity arrays are imported as:

```
v_m_s_step7  
v_bohr_s_step7
```

and later copied into the authoritative Step 8 arrays:

```
v_m_s  
v_bohr_s
```

Consequently, every effective, intermediate, and M_S-specific probability is evaluated at the same total energies and velocities used in the subsequent microcanonical-rate calculation.

4.8.3 Validation of the energy-grid dimensions

Step 8 checks that the total-energy array contains exactly one more point than the maximum energy-bin index:

```
if total_energy_grid_cm1.size != maximum_energy_bin_index + 1:  
    raise RuntimeError(...)
```

It also verifies that both imported velocity arrays have the same length as the energy grid:

```
if v_m_s_step7.size != total_energy_grid_cm1.size:  
    raise RuntimeError(...)
```

```
if v_bohr_s_step7.size != total_energy_grid_cm1.size:  
    raise RuntimeError(...)
```

These checks prevent probabilities from being paired with velocities belonging to different energy bins.

4.8.4 Verification of the uniform common energy grid

The expected grid is reconstructed as:

```
expected_grid = (  
    np.arange(maximum_energy_bin_index + 1, dtype=float)  
    * energy_step_cm1  
)
```

and compared with the imported array:

```
if not np.allclose(  
    total_energy_grid_cm1,  
    expected_grid,  
    rtol=0.0,  
    atol=1.0e-12  
):  
    raise RuntimeError(...)
```

Thus, Step 8 requires the condition in Equation S161:

$$E_j = j\Delta E \quad (\text{S161})$$

The corresponding uniform spacing is given in Equation S162:

$$\Delta E = E_{\text{step}} \quad (\text{S162})$$

For the default common grid, the spacing is given by Equation S163:

$$\Delta E = 1 \text{ cm}^{-1} \quad (\text{S163})$$

4.8.5 Verification of the MECP threshold bin

The expected MECP bin is independently reconstructed in both implementations.

In the Local PC implementation:

```
expected_binX = int(np.ceil(E_MECP_cm1 / energy_step_cm1))
```

In the HPC implementation:

```
expected_crossing_energy_bin_index = int(np.ceil(E_MECP_cm1 / energy_step_cm1))
```

The stored MECP-bin index from Step 7 must equal the independently reconstructed value.

In the Local PC implementation:

```
if crossing_energy_bin_index != expected_binX:
```

In the HPC implementation:

```
if crossing_energy_bin_index != expected_crossing_energy_bin_index:
```

Therefore, Equation S164 follows:

$$i_{\text{MECP}} = \left\lceil \frac{V_{\text{MECP}}}{\Delta E} \right\rceil \quad (\text{S164})$$

This ensures that Step 8 and Step 7 use the same crossing threshold and the same first integer energy bin above the MECP.

4.8.6 Active gradient quantities

The two active gradient quantities are converted to positive scalar magnitudes:

```
gradient_gap_magnitude = abs(
    float(GRADIENT_GAP_NORM_EH_PER_BOHR)
)
```

```
projected_gradient_mean = abs(
    float(PROJECTED_GRADIENT_MEAN_EH_PER_BOHR)
)
```

The code verifies that both are finite and positive.

The active Landau-Zener gradient quantity is defined by Equation S165:

$$\Delta g_{\parallel} = \| \mathbf{g}_{\text{HS}} - \mathbf{g}_{\text{LS}} \| \quad (\text{S165})$$

The active weak-coupling gradient quantity is the projected value generated in Step 6 and is defined by Equation S166:

$$\bar{g} = \sqrt{g_{\text{LS},\parallel} g_{\text{HS},\parallel}} \quad (\text{S166})$$

Step 8 therefore uses the authoritative projected gradient mean generated in Step 6, not the Cartesian diagnostic calculated in Step 4.

4.8.7 Reduced-mass conversions

The reduced mass is converted to both kilograms and atomic units:

```
mu_kg = (
    float(reduced_mass_amu)
    * amu_to_kg
)
```

```
mu_au = (
    float(reduced_mass_amu)
    * amu_to_au
)
```

with:

```
amu_to_kg = 1.66053906660e-27
amu_to_au = 1822.8884853323708
```

Thus, Equation S167 follows:

$$\mu_{\text{kg}} = \mu_{\text{amu}} \times 1.66053906660 \times 10^{-27} \quad (\text{S167})$$

The second reduced-mass conversion is given in Equation S168:

$$\mu_{\text{au}} = \mu_{\text{amu}} \times 1822.8884853323708 \quad (\text{S168})$$

The SI mass is used for the independent velocity check, whereas the atomic-unit mass is used in the weak coupling probability.

4.8.8 Independent velocity consistency check

Although Step 8 uses the Step 7 velocity arrays directly, it independently reconstructs the same velocity definition for validation.

The excess energy is recalculated as:

```
E_excess_J_check = (
    (total_energy_grid_cm1 - E_MECP_cm1)
    * 100.0
    * h_SI
    * c_SI
)
```

The check arrays are initialized and evaluated as:

```
v_m_s_check = np.zeros_like(total_energy_grid_cm1)
mask_velocity_check = E_excess_J_check > 0.0
v_m_s_check[mask_velocity_check] = np.sqrt(
    2.0 * E_excess_J_check[mask_velocity_check] / mu_kg
)
v_bohr_s_check = v_m_s_check / bohr_m
```

Therefore, Equation S169 follows:

$$v(E) = \begin{cases} 0, & E \leq V_{\text{MECP}} \\ \sqrt{\frac{2(E - V_{\text{MECP}})}{\mu}}, & E > V_{\text{MECP}} \end{cases} \quad (\text{S169})$$

The reconstructed arrays are compared with the imported Step 7 arrays using strict numerical tolerances.

The two comparisons are:

```
np.allclose(v_m_s_step7, v_m_s_check, rtol=1.0e-12, atol=1.0e-12)
```

The bohr-per-second velocity is checked independently using:

```
np.allclose(v_bohr_s_step7, v_bohr_s_check, rtol=1.0e-12, atol=1.0e-6)
```

Step 8 terminates if either comparison fails.

This guarantees that the probability and rate sections use an identical velocity definition.

4.8.9 Effective spin–orbit coupling

The effective SOC used for the principal probability curves is:

`H_SO_cm`

This scalar may originate from:

- the complete M_S-resolved SOC matrix;
- a scaled SOC matrix;
- an effective ORCA SOC value; or
- a user-supplied replacement.

The probability functions use the absolute SOC magnitude:

`H_cm = float(abs(H_cm))`

because both implemented probability expressions depend on H_{SO}^2 . The complex phase or sign of a matrix element therefore does not affect its individual probability curve.

4.8.10 Conversion of the SOC to joules

For the Landau-Zener calculation, an SOC supplied in cm^{-1} is converted to joules:

```
H_J = (  
  h_SI  
  * c_SI  
  * H_cm  
  * 100.0  
)
```

Thus, Equation S170 follows:

$$H_{SO}(\text{J}) = hcH_{SO}(\text{cm}^{-1}) \times 100 \quad (\text{S170})$$

The factor of 100 converts inverse centimeters to inverse meters.

4.8.11 Energy mask used for the LZ probability

The Landau-Zener probability is evaluated only for bins satisfying:


```
mask = (
    np.isfinite(E_grid_cm1)
    & np.isfinite(v_bohr_s_grid)
    & (E_grid_cm1 >= E_MECP_cm1)
    & (v_bohr_s_grid > velocity_floor)
)
```

Therefore, the active LZ bins must satisfy the condition in Equation S171:

$$E \geq V_{\text{MECP}} \quad (\text{S171})$$

The second active-bin condition is given in Equation S172:

$$v > v_{\text{floor}} \quad (\text{S172})$$

Because the physical velocity is zero at the exact MECP threshold, the first nonzero LZ probability generally occurs at the first grid point strictly above the MECP.

All subthreshold and zero-velocity bins remain assigned zero Landau-Zener probability, as given in Equation S173:

$$P_{\text{LZ}} = 0 \quad (\text{S173})$$

4.8.12 Time derivative of the diabatic energy difference

The rate of change of the surface-energy difference is calculated in Hartree per second as:

```
dDeltaE_dt_Eh_s = (
    gradient_gap_magnitude
    * v_bohr_s_grid
)
```

Equivalently, the expression can be written as Equation S174:

$$\left| \frac{d\Delta E}{dt} \right| = \Delta g_{\parallel} v \quad (\text{S174})$$

Here, $\Delta g_{\parallel}$ is expressed in Hartree per bohr and v is expressed in bohr per second, so their product has units of Hartree per second.

This value is converted to joules per second using:

```
dDeltaE_dt_J_s = (
    dDeltaE_dt_Eh_s
    * Eh_to_J
)
```

The Hartree-to-joule conversion factor used in this step is given by Equation S175:

$$1 E_h = 4.3597447222071 \times 10^{-18} \text{ J} \quad (\text{S175})$$

4.8.13 Implemented Landau-Zener parameter

The code evaluates:

```
gamma[mask] = (  
    4.0  
    * np.pi**2  
    * H_J**2  
    / (  
        h_SI  
        * dDeltaE_dt_J_s[mask]  
    )  
)
```

Therefore, the implemented energy-dependent parameter is given by Equation S176:

$$\gamma(E) = \frac{4\pi^2 H_{SO}^2}{h \Delta g_{\parallel} v(E)} \quad (\text{S176})$$

Because the velocity appears in the denominator, Equation S177 shows that:

$$\gamma(E) \propto \frac{1}{v(E)} \quad (\text{S177})$$

Accordingly, the LZ parameter is largest immediately above the crossing threshold and decreases as the excess energy and velocity increase.

4.8.14 Implemented Landau-Zener probability

The probability is evaluated as:

```
P[mask] = (  
    1.0  
    - np.exp(  
        -2.0 * gamma[mask]  
    )  
)
```

Equivalently, the expression can be written as Equation S178:

$$P_{LZ}(E) = 1 - \exp[-2\gamma(E)] \quad (\text{S178})$$

Substituting the implemented γ gives Equation S179:

$$P_{LZ}(E) = 1 - \exp\left[-\frac{8\pi^2 H_{SO}^2}{h \Delta g_{\parallel} v(E)}\right] \quad (\text{S179})$$

The numerical result is cleaned and restricted to the physically permitted range:

```
P[~np.isfinite(P)] = 0.0  
P = np.clip(P, 0.0, 1.0)
```

After numerical cleanup and clipping, the Landau-Zener probability satisfies Equation S180:

$$0 \leq P_{LZ} \leq 1 \quad (\text{S180})$$

The Landau-Zener expression implemented in SPINKS corresponds to a double passage through the crossing region. For a single passage, the transition probability is defined by Equation S181:

$$p_{LZ}(E) = 1 - \exp[-\gamma(E)] \quad (\text{S181})$$

The code evaluates the quantity in Equation S182:

$$\gamma(E) = \frac{4\pi^2 H_{SO}^2}{h \Delta g_{\parallel} v(E)} \quad (\text{S182})$$

During a double passage, the system has two independent opportunities to change spin. The total probability is therefore the probability of crossing during the first passage plus the probability of remaining on the original surface during the first passage and crossing during the second passage, as expressed in Equation S183:

$$P_{LZ}(E) = p_{LZ}(E) + [1 - p_{LZ}(E)]p_{LZ}(E) \quad (\text{S183})$$

This simplifies to Equation S184:

$$P_{LZ}(E) = 2p_{LZ}(E) - p_{LZ}^2(E) \quad (\text{S184})$$

Substituting the single-passage probability from Equation S185,

$$p_{LZ}(E) = 1 - \exp[-\gamma(E)] \quad (\text{S185})$$

into the preceding relation gives Equation S186:

$$P_{LZ}(E) = 2[1 - e^{-\gamma(E)}] - [1 - e^{-\gamma(E)}]^2 = 1 - e^{-2\gamma(E)} \quad (\text{S186})$$

Accordingly, the Python implementation is:

```
P[mask] = 1.0 - np.exp(-2.0 * gamma[mask])
```

The factor of 2.0 multiplying gamma therefore implements the incoherent double-passage probability given in Equation S187:

$$P_{LZ}(E) = 1 - \exp[-2\gamma(E)] = 2p_{LZ}(E) - p_{LZ}^2(E) \quad (\text{S187})$$

used in the downstream statistical treatment.

This form assumes that the two passages are treated as statistically independent and that coherent phase interference between the passages is not included. In other words, SPINKS applies the incoherent double-passage probability rather than a Stückelberg-interference expression.

4.8.15 Energy dependence of the LZ probability

For energies above the MECP, the velocity is given by Equation S188:

$$v(E) = \sqrt{\frac{2(E - V_{\text{MECP}})}{\mu}} \quad (\text{S188})$$

The implemented Landau-Zener parameter therefore varies according to Equation S189:

$$\gamma(E) \propto \frac{1}{\sqrt{E - V_{\text{MECP}}}} \quad (\text{S189})$$

Consequently:

- the probability is zero below and at the threshold because the code excludes zero-velocity bins;
- it reaches its largest evaluated value at the first positive-velocity grid point above the MECP; and
- it generally decreases as the total energy increases.

This behavior differs from a probability that would grow monotonically with excess energy.

4.8.16 Conversion of SOC and excess energy to atomic units

For the weak coupling expression, the SOC is converted from cm^{-1} to Hartree:

$$H_{\text{Eh}} = \text{float}(\text{abs}(H_{\text{cm}})) / \text{autocm}$$

Equivalently, the expression can be written as Equation S190:

$$H_{\text{SO}}(E_h) = \frac{H_{\text{SO}}(\text{cm}^{-1})}{219474.6313705} \quad (\text{S190})$$

The energy relative to the MECP is similarly converted:

$$E_{\text{excess_Eh}} = (E_{\text{grid_cm1}} - E_{\text{MECP_cm1}}) / \text{autocm}$$

Equivalently, the expression can be written as Equation S191:

$$\epsilon(E_h) = \frac{E - V_{\text{MECP}}}{219474.6313705} \quad (\text{S191})$$

Unlike the LZ calculation, the WC function does not apply an explicit above-threshold mask. The Airy-function expression is evaluated over the complete finite grid, including energies below the MECP.

4.8.17 Weak coupling prefactor

The energy-independent prefactor is calculated as:

```
prefactor = (  
    4.0  
    * np.pi**2  
    * H_Eh**2  
    * (  
        2.0 * mu_au  
        / (projected_gradient_mean * gradient_gap_magnitude)  
    ) ** (2.0 / 3.0)  
)
```

Therefore, Equation S192 follows:

$$A_{\text{WC}} = 4\pi^2 H_{\text{SO}}^2 \left(\frac{2\mu}{\bar{g} \Delta g_{\parallel}} \right)^{2/3} \quad (\text{S192})$$

This term depends on:

- the square of the SOC;
- the reaction-coordinate reduced mass;
- the projected gradient mean; and
- the gradient-difference magnitude.

For a fixed molecular system, the prefactor has the proportionality shown in Equation S193:

$$A_{\text{WC}} \propto H_{\text{SO}}^2 \quad (\text{S193})$$

4.8.18 Weak coupling Airy scale

The Airy scaling factor is calculated as:

```
scale = (  
    2.0  
    * mu_au  
    * gradient_gap_magnitude**2  
    / projected_gradient_mean**4  
) ** (1.0 / 3.0)
```

Equivalently, the expression can be written as Equation S194:

$$\alpha = \left(\frac{2\mu \Delta g_{\parallel}^2}{\bar{g}^4} \right)^{1/3} \quad (\text{S194})$$

The Airy argument is then:

```

airy_arg[mask] = (
    -E_excess_Eh[mask]
    * scale
)

```

The corresponding expression is given in Equation S195:

$$x(E) = -\epsilon(E)\alpha \quad (\text{S195})$$

At the exact MECP energy, Equation S196 gives:

$$\epsilon = 0 \quad (\text{S196})$$

This gives Equation S197:

$$x = 0 \quad (\text{S197})$$

Below the MECP, the relation in Equation S198 applies:

$$\epsilon < 0 \Rightarrow x > 0 \quad (\text{S198})$$

Above the MECP, the relation in Equation S199 applies:

$$\epsilon > 0 \Rightarrow x < 0 \quad (\text{S199})$$

4.8.19 Evaluation of the Airy function

The Airy function of the first kind is evaluated using:

```

Ai[mask] = airy(
    airy_arg[mask]
)[0]

```

This gives the Airy-function value:

$$\text{Ai}[x(E)]$$

The implemented weak coupling probability is:

```

P[mask] = (
    prefactor
    * Ai[mask] ** 2
)

```

Equivalently, the expression can be written as Equation S200:

$$P_{\text{WC}}(E) = 4\pi^2 H_{\text{SO}}^2 \left(\frac{2\mu}{\bar{g} \Delta g_{\parallel} \hbar^2} \right)^{2/3} \text{Ai}^2 \left[-\epsilon(E) \left(\frac{2\mu \Delta g_{\parallel}^2}{\bar{g}^4 \hbar^2} \right)^{1/3} \right] \quad (\text{S200})$$

The result is cleaned and clipped:

```
P[~np.isfinite(P)] = 0.0
P = np.clip(P, 0.0, 1.0)
```

This gives Equation S201:

$$0 \leq P_{WC} \leq 1 \quad (S201)$$

4.8.20 Subthreshold WC probabilities

The weak coupling expression is evaluated at energies below the ZPE-corrected MECP because the Airy-function treatment remains defined for positive Airy arguments.

Thus, Step 8 can produce small nonzero WC probabilities under the condition in Equation S202:

$$E < V_{MECP} \quad (S202)$$

This is intentional in the implemented formula and differs from the LZ calculation, which is explicitly masked to zero below the crossing threshold.

The later rate calculation determines how these probability values enter the effective number and density-of-states treatment.

4.8.21 Effective Landau-Zener probability

The effective LZ curve is calculated from the effective SOC using the implementation-specific helper function.

In the Local PC implementation:

```
P_LZ_effective, gamma_LZ_effective, v_bohr_s, v_m_s = (
    compute_LZ_for_HSO_cm(H_SO_cm)
)
```

In the HPC implementation:

```
P_LZ_effective, landau_zener_exponent_effective, v_bohr_s, v_m_s = (
    compute_landau_zener_probability(H_SO_cm)
)
```

The Local PC compatibility variables are:

```
P_LZ_micro
gamma_LZ
effective_LZ_probability
effective_LZ_gamma
```

The HPC compatibility variables are:

```
P_LZ_effective_alias
landau_zener_exponent
effective_LZ_probability_for_rates
landau_zener_exponent_eff_rate
```

Both implementations refer to the same effective Landau-Zener probability defined in Equation S203:

$$P_{LZ}^{\text{eff}}(E) = P_{LZ}[H_{SO}^{\text{eff}}, E] \quad (\text{S203})$$

4.8.22 Effective weak coupling probability

The effective WC curve is calculated from the effective SOC using the implementation-specific helper function.

In the Local PC implementation:

```
P_WC_effective, airy_arg_WC_effective, Ai_WC_effective = (
    compute_WC_for_HSO_cm(H_SO_cm)
)
```

In the HPC implementation:

```
P_WC_effective, airy_arg_WC_effective, Ai_WC_effective = (
    compute_weak_coupling_probability(H_SO_cm)
)
```

The Local PC compatibility variables are:

```
P_WC_micro
effective_WC_probability
effective_WC_airy_argument
effective_WC_airy_value
```

The HPC compatibility variables are:

```
P_WC_effective_alias
effective_WC_probability_for_rates
effective_WC_airy_argument_for_rates
effective_WC_airy_value_for_rates
```

The Airy argument and Airy-function values are retained so that the numerical WC calculation can be reproduced independently.

Thus, Equation S204 follows:

$$P_{WC}^{\text{eff}}(E) = P_{WC}[H_{SO}^{\text{eff}}, E] \quad (\text{S204})$$

4.8.23 Selection of the SOC matrix

Step 8 first checks whether only a scalar effective SOC is available:

```
SOC_EFFECTIVE_ONLY
SOC_MATRIX_CHANNELS_AVAILABLE
```


If the calculation is in effective-only mode, no channel-specific or intermediate curves are generated.

Otherwise, the preferred SOC matrix is:

`SOC_Ms_matrix_scaled_cm1`

when available. If no scaled matrix is present, Step 8 uses:

`SOC_Ms_matrix_cm1`

The selected matrix is stored as:

`SOC_channel_matrix_cm1`

and its source is recorded in:

`SOC_channel_matrix_label`

The scaled matrix therefore takes precedence when the effective SOC was modified or rescaled consistently after Step 2.

4.8.24 Validation of the M_S dimensions

The low- and high-spin magnetic quantum numbers are imported as:

`Ms_low`
`Ms_high`

and the corresponding total-spin values as:

`S_low`
`S_high`

The matrix shape must satisfy:

```
SOC_channel_matrix_cm1.shape == (
    len(Ms_low),
    len(Ms_high)
)
```

Equivalently, the expression can be written as Equation S205:

$$\dim(\mathbf{H}_{SO}) = (2S_{\text{low}} + 1) \times (2S_{\text{high}} + 1) \quad (\text{S205})$$

Step 8 terminates if the matrix dimensions do not match the imported M_S arrays.

4.8.25 Individual SOC channel definition

For every matrix element:

$H_{ij} = \text{SOC_channel_matrix_cm1}[i, j]$ $H_{\text{abs}} = \text{float}(\text{abs}(H_{ij}))$
--

The corresponding spin transition is:

$$|S_{\text{low}}, M_{S,\text{low}}\rangle \rightarrow |S_{\text{high}}, M_{S,\text{high}}\rangle$$

Matrix elements satisfying the threshold in Equation S206 are ignored:

$$|H_{ij}| \leq 10^{-10} \text{ cm}^{-1} \quad (\text{S206})$$

Only channels above this threshold are retained.

Each nonzero channel is assigned a unique label containing:

- the low-spin S value;
- the low-spin M_S value;
- the high-spin S value; and
- the high-spin M_S value.

For example:

$S_{0.5_M_S-0.5_S_{1.5_M_S+0.5}}$

The label identifies a transition from the state in Equation S207:

$$S = \frac{1}{2}, M_S = -\frac{1}{2} \quad (\text{S207})$$

to the state in Equation S208:

$$S = \frac{3}{2}, M_S = +\frac{1}{2} \quad (\text{S208})$$

4.8.26 Individual channel probability curves

For each nonzero matrix element, Step 8 evaluates both LZ and WC probabilities using the same absolute SOC magnitude.

In the Local PC implementation:

$P_{\text{lz_ij}}, \text{gamma_lz_ij}, _, _ = \text{compute_LZ_for_HSO_cm}(H_{\text{abs}})$ $P_{\text{wc_ij}}, \text{airy_arg_wc_ij}, \text{Ai_wc_ij} = \text{compute_WC_for_HSO_cm}(H_{\text{abs}})$

In the HPC implementation:

$P_{\text{lz_ij}}, \text{gamma_lz_ij}, _, _ = \text{compute_landau_zener_probability}(H_{\text{abs}})$ $P_{\text{wc_ij}}, \text{airy_arg_wc_ij}, \text{Ai_wc_ij} = \text{compute_weak_coupling_probability}(H_{\text{abs}})$
--

Thus, Equation S209 follows:

$$P_{\text{LZ}}^{M_S, M'_S}(E) = P_{\text{LZ}} \left[| H_{M_S, M'_S} |, E \right] \quad (\text{S209})$$

The second probability result is given in Equation S210:

$$P_{\text{WC}}^{M_S, M'_S}(E) = P_{\text{WC}} \left[| H_{M_S, M'_S} |, E \right] \quad (\text{S210})$$

The complete channel metadata are stored in:

[P_LZ_channels](#)
[P_WC_channels](#)

For each channel, the dictionaries retain:

- the probability array;
- the LZ parameter or Airy quantities;
- the complex SOC element;
- its absolute magnitude;
- both spin quantum numbers;
- both M_S values;
- the matrix indices; and
- the source SOC matrix.

4.8.27 No sum over M_S-specific channels

Step 8 does not calculate a sum over M_S-specific probability channels:

$$\sum_{M_S, M'_S} P^{M_S, M'_S}(E)$$

and does not multiply an individual probability curve by the number of symmetry-equivalent channels.

Every nonzero matrix element remains an independent probability channel in:

[P_LZ_channels](#)
[P_WC_channels](#)

Any later averaging, weighting, or statistical combination must therefore be performed explicitly in the subsequent rate treatment rather than being hidden in Step 8.

4.8.28 Dependence of individual curves on |H_SO|

Both probability expressions depend quadratically on the SOC magnitude. For the Landau-Zener probability:

$$P_{LZ} \sim f(|H_{ij}|^2),$$

For the weak-coupling probability:

$$P_{WC} \sim |H_{ij}|^2 A_i^2(\dots).$$

Therefore, channels with equal |H_{ij}| values produce numerically identical probability arrays, even if the underlying complex matrix elements differ in sign or phase.

This observation is used only to reduce visual duplication in the plots.

4.8.29 Grouping equal-|H_SO| channels for plotting

The two implementations use different helper and tolerance-variable names for the same representative-only grouping procedure.

In the Local PC implementation:

```
assign_channel_to_representative_group(...)
```

```
MS_CHANNEL_GROUP_ATOL_CM1
```

```
MS_CHANNEL_GROUP_RTOL
```

In the HPC implementation:

```
find_matching_hso_group(...)
```

```
hso_group_atol_cm1
```

```
hso_group_rtol
```

Both implementations use an absolute tolerance of $1.0 \times 10^{-8} \text{ cm}^{-1}$ and a relative tolerance of 1.0×10^{-10} . Within these tolerances, two channels are grouped when their SOC magnitudes satisfy Equation S211:

$$|H_i| \approx |H_j| \tag{S211}$$

Each group retains one representative probability array under the dictionary key P_{representative} while preserving all underlying individual-channel labels and metadata.

The grouping is representative-only and performs no probability summation, averaging, or multiplication by the number of equivalent channels.

4.8.30 Interpretation of grouped channel labels

When multiple channels share one plotted curve, their labels are separated with commas:

$$P^{M_{S,1},M'_{S,1}}, P^{M_{S,2},M'_{S,2}}, \dots$$

They are not separated by plus signs.

This notation indicates that the displayed line represents several identical individual curves, not a sum such as

$$P_1 + P_2.$$

The group multiplicity stored in the output is therefore a count of equivalent curves, not a probability weighting factor.

4.8.31 Grouping by |M_S|

The intermediate treatment groups rows of the SOC matrix according to the absolute lower-spin magnetic quantum number:

```
abs_ms_groups = sorted(
    set([
        round(abs(float(x)), 8)
        for x in Ms_low
    ])
)
```

Rows with M_S = +1/2 are represented by Equation S212:

$$M_S = +\frac{1}{2} \tag{S212}$$

Rows with M_S = -1/2 are represented by Equation S213:

$$M_S = -\frac{1}{2} \tag{S213}$$

These two rows belong to the same intermediate group, defined by Equation S214:

$$|M_S| = \frac{1}{2} \tag{S214}$$

Similarly, rows with $M_S = \pm 3/2$ are grouped according to Equation S215:

$$|M_S| = \frac{3}{2} \quad (\text{S215})$$

4.8.32 Row-resolved intermediate SOC magnitude

For each lower-spin row i , the SOC magnitude summed over all high-spin M_S' components is calculated as:

```
H_row = float(
    np.sqrt(
        np.sum(np.abs(row) ** 2)
    )
)
```

Equivalently, the expression can be written as Equation S216:

$$H_i^{\text{row}} = \sqrt{\sum_{M_S'} |H_{M_S, i, M_S'}|^2} \quad (\text{S216})$$

This is a root-sum-square SOC over all final M_S' components for one fixed initial M_S .

Only finite, nonzero row magnitudes are retained.

4.8.33 Intermediate SOC for an $|M_S|$ group

If an $|M_S|$ group contains multiple symmetry-related rows, their row magnitudes are combined using:

```
H_int = float(
    np.sqrt(
        np.mean(H_rows ** 2)
    )
)
```

Therefore, Equation S217 follows:

$$H_{\text{int}}(|M_S|) = \sqrt{\frac{1}{n} \sum_{i=1}^n (H_i^{\text{row}})^2} \quad (\text{S217})$$

The relation used here is stated in Equation S218:

$$(H_i^{\text{row}})^2 = \sum_{M_S'} |H_{M_S, i, M_S'}|^2 \quad (\text{S218})$$

Therefore, the implemented intermediate SOC is given by Equation S219:

$$H_{\text{int}}(|M_S|) = \sqrt{\frac{1}{n} \sum_{i \in |M_S|} \sum_{M_{S'}} |H_{M_{S,i}, M_{S'}}|^2} \quad (\text{S219})$$

The number of contributing rows is stored in the group metadata as:

"degeneracy": len(row_indices)

4.8.34 Intermediate probability curves

For every nonzero intermediate SOC value, Step 8 calculates an LZ and WC probability curve.

In the Local PC implementation:

P_lz_int, gamma_lz_int, _, _ = compute_LZ_for_HSO_cm(H_int)
P_wc_int, airy_arg_wc_int, Ai_wc_int = compute_WC_for_HSO_cm(H_int)

In the HPC implementation, the corresponding calculations are:

P_lz_int, gamma_lz_int, _, _ = compute_landau_zener_probability(H_int)
P_wc_int, airy_arg_wc_int, Ai_wc_int = compute_weak_coupling_probability(H_int)

The resulting intermediate Landau-Zener probability is given by Equation S220:

$$P_{\text{LZ}}^{|M_S|}(E) = P_{\text{LZ}}[H_{\text{int}}(|M_S|), E] \quad (\text{S220})$$

The second intermediate-SOC relation is given in Equation S221:

$$P_{\text{WC}}^{|M_S|}(E) = P_{\text{WC}}[H_{\text{int}}(|M_S|), E] \quad (\text{S221})$$

The results are stored in:

[P_LZ_intermediate](#)
[P_WC_intermediate](#)

Each record retains:

- the probability array;
- the LZ parameter or Airy quantities;
- the intermediate SOC;
- the contributing row SOC magnitudes;
- the absolute lower-spin M_S value;
- the degeneracy;
- the row indices; and
- the source matrix.

4.8.35 No weighted sum of intermediate curves

Step 8 calculates one intermediate curve for each unique lower-spin $|M_S|$ value.

It does not calculate a degeneracy-weighted sum such as:

$$\sum_{|M_S|} g_{|M_S|} P^{|M_S|}(E)$$

The degeneracy is stored as metadata, but it is not applied to the probability arrays in Step 8.

This preserves a clear distinction among:

- individual channel probabilities;
- intermediate $|M_S|$ -resolved probabilities; and
- the effective scalar-SOC probability.

4.8.36 Plotting ranges

The LZ plot begins no more than 1000 cm^{-1} below the MECP:

```
lz_plot_min_cm1 = max(  
    total_energy_grid_cm1[0],  
    E_MECP_cm1 - 1000.0  
)
```

The WC plot begins no more than 1500 cm^{-1} below the MECP:

```
wc_plot_min_cm1 = max(  
    total_energy_grid_cm1[0],  
    E_MECP_cm1 - 1500.0  
)
```

The wider subthreshold range for WC allows the Airy-function probability below the MECP to be displayed.

The plots use the actual Step 7 arrays. No interpolated plotting grid is introduced.

4.8.37 Landau–Zener probability plot

The LZ plot contains:

- the effective probability as a black dotted curve;
- intermediate curves as dashed lines;

- one representative line for each unique individual-channel $|H_{SO}|$ magnitude; and
- a vertical line at the ZPE-corrected MECP.

The plot is saved as:

Step8_LZ_probabilities.png

with the title:

Effective, Intermediate, and MS-specific LZ Probabilities

The effective, intermediate, and channel-specific curves are visually distinguished but are all evaluated on the same numerical grid.

4.8.38 Weak coupling probability plot

The WC plot is constructed analogously and contains:

- the effective WC curve;
- intermediate $|M_S|$ -resolved curves;
- representative equal- $|H_{SO}|$ channel curves; and
- the active MECP threshold.

The plot is saved as:

Step8_WC_probabilities.png

with the title:

Effective, Intermediate, and MS-specific WC Probabilities

Because the WC expression is evaluated below the barrier, the plotted WC curve can remain nonzero to the left of the MECP marker.

4.8.39 Step 8 summary file

The file:

Step8_probability_summary.txt

records:

- effective SOC;
- active MECP threshold;
- the active gradient-difference magnitude;

- the active projected gradient mean;
- reduced mass in amu, kg, and atomic units;
- energy spacing;
- maximum grid index;
- MECP bin;
- energy- and velocity-grid sources;
- minimum and maximum effective LZ probabilities;
- minimum and maximum effective WC probabilities;
- number of intermediate curves;
- number of nonzero individual channels;
- number of unique plotted channel curves; and
- paths to all Step 8 output files.

4.8.40 Effective probability grid file

The complete effective probability grid is written to:

Step8_effective_probability_grid.txt

The columns are:

total_energy_grid_cm1
E_minus_MECP_cm1
v_m_s
v_bohr_s
P_LZ_effective

The implementation-specific LZ-parameter column is:

In the Local PC implementation:

gamma_LZ_effective

In the HPC implementation:

landau_zener_exponent_effective

The remaining common columns are:

P_WC_effective
airy_arg_WC_effective
Ai_WC_effective

For each total-energy bin, the file records E, E - V_MECP, the velocity in both units, the effective LZ probability, the implemented LZ parameter, the effective WC probability, the Airy argument, and the Airy-function value.

This file provides all quantities needed to reproduce both effective probability curves.

4.8.41 Intermediate probability summary

The file:

Step8_intermediate_probability_summary.txt
--

records, for each intermediate group:

- its label;
- the absolute lower-spin M_S value;
- degeneracy;
- intermediate SOC value; and
- the contributing row SOC magnitudes.

A typical entry may correspond to the intermediate group defined in Equation S222:

$$|M_S| = \frac{1}{2} \quad (\text{S222})$$

For this example, the two $M_S = \pm 1/2$ rows, their individual row norms, and the root-mean-square intermediate SOC are recorded.

4.8.42 M_S -resolved channel summary

The file:

Step8_channel_probability_summary.txt

contains three sections.

1-LZ plotting groups

This section records:

- group key;
- common $|H_{SO}|$ magnitude;
- number of equivalent channels;
- channel symbols; and
- individual channel labels.

2-WC plotting groups

The same information is recorded independently for the WC curves.

3-Individual channels

Every nonzero matrix element is listed with its channel symbol and a reader-facing transition label. Internal labels such as `S0.0_Ms+0.0__S1.0_Ms+1.0` remain unchanged inside the code, while the text output converts them to a form such as $(S = 0, M_S = 0) \rightarrow (S' = 1, M_{S'} = +1)$.

- channel symbol;
- reader-facing spin/ M_S transition label;
- real part of the complex SOC matrix element;
- imaginary part of the complex SOC matrix element;
- $|H_{SO}|$;
- the low-spin S and M_S values;
- the high-spin S and M_S values;
- matrix indices i and j ; and
- the corresponding source-matrix information retained in the channel dictionaries.

Thus, grouping for plotting does not remove the underlying individual channel information.

4.8.43 Local and cluster output handling

The local output directory is:

```
local_step8 = os.path.join(
    local_base,
    "LZ_WC_probabilities"
)
```

In the Local PC implementation, the remote-directory compatibility alias points to the same local directory:

```
remote_step8 = local_step8
```

In the HPC implementation, the remote directory is constructed as:

```
remote_step8 = posixpath.join(remote_base, "LZ_WC_probabilities")
```

When an active SFTP connection is available in the HPC implementation, Step 8 transfers the summary file, effective grid, intermediate summary, channel summary, and both probability plots to the remote Step 8 directory.

- In the Local PC implementation, the corresponding remote-file compatibility variables are aliases to the same local files so that later shared workflow steps can use the same interface.

The local calculations and files remain valid even if remote transfer fails.

4.8.44 Effective probability variables and compatibility aliases

The common effective probability quantities exported by both implementations are:

```
P_LZ_effective  
P_WC_effective  
airy_arg_WC_effective  
Ai_WC_effective
```

The Local PC implementation additionally exports:

```
gamma_LZ_effective  
P_LZ_micro  
gamma_LZ  
P_WC_micro  
effective_LZ_probability  
effective_LZ_gamma  
effective_WC_probability  
effective_WC_airy_argument  
effective_WC_airy_value  
rate_velocity_bohr_s
```

The HPC implementation additionally exports:

```
landau_zener_exponent_effective  
P_LZ_effective_alias  
landau_zener_exponent  
P_WC_effective_alias  
effective_LZ_probability_for_rates  
landau_zener_exponent_eff_rate  
effective_WC_probability_for_rates  
effective_WC_airy_argument_for_rates  
effective_WC_airy_value_for_rates  
velocity_bohr_per_s_for_rates
```

These compatibility names point to the same Step 8 probability and velocity results; they do not introduce a second scientific calculation.

4.8.45 Intermediate and channel-specific variables

The intermediate probability dictionaries are:

<code>P_LZ_intermediate</code> <code>P_WC_intermediate</code>
--

The individual channel dictionaries are:

<code>P_LZ_channels</code> <code>P_WC_channels</code>
--

The representative plotting groups are:

<code>P_LZ_channel_groups</code> <code>P_WC_channel_groups</code>
--

The active SOC matrix and its source are retained as:

<code>SOC_channel_matrix_cm1</code> <code>SOC_channel_matrix_label</code>
--

These variables preserve the complete effective → intermediate → M_S-specific hierarchy.

4.8.46 Exported probability functions

The probability helper functions are retained globally for later workflow use.

In the Local PC implementation:

<code>compute_LZ_for_HSO_cm</code> <code>compute_WC_for_HSO_cm</code>
--

In the HPC implementation:

<code>compute_landau_zener_probability</code> <code>compute_weak_coupling_probability</code>

The two implementations evaluate the same LZ and WC equations; only the helper-function names differ. A custom LZ energy grid is accepted only when matching velocity arrays are supplied explicitly, preventing Step 8 from creating an independent velocity definition.

4.8.47 Summary of the step 8 algorithm

The complete Step 8 sequence begins from the common Step 7 total-energy and velocity arrays, validates the grid and threshold indexing, uses the active gradient-difference magnitude and projected gradient mean, evaluates effective LZ and clipped WC probabilities, and, when an M_S-resolved SOC matrix is available, evaluates intermediate and individual-channel probabilities without summing equivalent channels. The Local PC

and HPC implementations use the same scientific equations but retain several different compatibility-variable and helper-function names, as noted above.

The probability evaluation therefore uses the common total-energy and velocity arrays E_j and $v(E_j)$, together with H_{SO}^{eff} , $\Delta g_{||}$, $\bar{g}$, μ , and V_{MECP} .

For the Landau-Zener treatment, Equation S223 gives:

$$\gamma(E) = \frac{4\pi^2 H_{SO}^2}{\hbar \Delta g_{||} v(E)} \quad (S223)$$

The second Landau-Zener relation is given in Equation S224:

$$P_{LZ}(E) = 1 - \exp[-2\gamma(E)] \quad (S224)$$

for positive-velocity bins above the MECP.

For the weak-coupling treatment, Equation S225 gives:

$$P_{WC}(E) = 4\pi^2 H_{SO}^2 \left(\frac{2\mu}{\bar{g} \Delta g_{||}} \right)^{2/3} \text{Ai}^2 \left[-\epsilon(E) \left(\frac{2\mu \Delta g_{||}^2}{\bar{g}^4} \right)^{1/3} \right] \quad (S225)$$

All quantities entering the weak-coupling expression are evaluated in atomic units. Therefore, $\hbar = 1$ and does not appear explicitly in the implemented equations.

The relevant definition is given in Equation S226:

$$\epsilon(E) = E - V_{MECP} \quad (S226)$$

The effective probabilities use the effective SOC according to Equation S227:

$$H_{SO} = H_{SO}^{eff} \quad (S227)$$

For an individual channel, the SOC magnitude is defined by Equation S228:

$$H_{SO} = |H_{M_S, M_{S'}}| \quad (S228)$$

For an individual channel, this SOC magnitude generates the LZ and WC probability curves already defined in Equations S209 and S210.

For an intermediate $|M_S|$ group, the row SOC is given by Equation S229:

$$H_i^{row} = \sqrt{\sum_{M_{S'}} |H_{M_S, i, M_{S'}}|^2} \quad (S229)$$

The group SOC is given in Equation S230:

$$H_{int}(|M_S|) = \sqrt{\frac{1}{n} \sum_{i \in |M_S|} (H_i^{row})^2} \quad (S230)$$

The corresponding variable flow is:

H_SO_cm
GRADIENT_GAP_NORM_EH_PER_BOHR
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR
reduced_mass_amu
VaG_MECP_cm1
total_energy_grid_cm1
v_m_s
v_bohr_s

The common grid and velocity variables are validated before the effective probabilities are evaluated:

P_LZ_effective
P_WC_effective

SOC_Ms_matrix_scaled_cm1
SOC_Ms_matrix_cm1

Each nonzero matrix element H_{ij} is treated as an individual channel and produces:

P_LZ_channels
P_WC_channels

Channels with equal |H_{ij}| values may be grouped for plotting only.
No summation is performed.

For the intermediate treatment, SOC-matrix rows are grouped by |M_{S,low}|:

H_row
H_int
P_LZ_intermediate
P_WC_intermediate

Therefore, Step 8 transforms the effective and M_S-resolved SOC information into a complete hierarchy of energy-dependent spin-crossing probabilities:

$$\left\{ H_{\text{SO}}^{\text{eff}}, \mathbf{H}_{\text{SO}}^{M_S, M'_S}, E_j, v_j, \Delta g_{\parallel}, \bar{g}, \mu \right\} \rightarrow \left\{ P^{\text{eff}}, P^{|M_S|}, P^{M_S, M'_S} \right\}_{\text{LZ, WC}}$$

The effective probability arrays are subsequently passed directly to Step 9, where they are combined with the reactant density of states and crossing-point sum of states to calculate the Landau-Zener and weak coupling microcanonical rate constants.

4.9 Step 9: Microcanonical Rate Constants

Step 9 converts the spin-crossing probabilities obtained in Step 8 into microcanonical reaction rate constants. No additional electronic-structure calculations are performed. Instead, the step combines the previously calculated Landau-Zener and weak coupling probabilities with the reactant and MECP rovibrational densities of states to evaluate effective, intermediate, and individual M_s -specific microcanonical rate constants over the selected energy range.

The principal operations performed in Step 9 are

1. importing all quantities required from Steps 1–8,
2. selecting the active ZPE-corrected MECP threshold,
3. constructing the common energy grid,
4. evaluating reactant and MECP rovibrational densities of states,
5. calculating effective Landau-Zener and weak coupling microcanonical rates,
6. calculating intermediate-spin microcanonical rates,
7. calculating individual M_s -specific microcanonical rate constants,
8. grouping symmetry-equivalent channels for visualization, and
9. generating numerical output files and publication-quality figures.

4.9.1 Import of quantities from the preceding steps

Step 9 begins by verifying that the quantities required for the microcanonical calculations are available. The current implementation requires the Step 7/8 energy grid and active spin-crossing quantities together with the reactant and crossing-point vibrational/rotational data:

```
required_vars_step9 = [  
    "jobname",  
    "freq_reactant_real_cm1",  
    "freq_MECP_real_cm1",  
    "VaG_MECP_cm1",  
    "probability_energy_grid_cm1",  
    "H_SO_cm",  
    "GRADIENT_GAP_NORM_EH_PER_BOHR",  
    "reduced_mass_amu",  
    "reference_rotational_constants_cm1",  
    "crossing_rotational_constants_cm1",  
    "local_base",  
    "remote_base"  
]
```

```
for var in required_vars_step9:
    if var not in globals():
        raise RuntimeError(
            f"Required variable '{var}' is missing. Run Local PC Steps 1–8 first."
        )
```

In the HPC implementation, the corresponding validation message is:

```
f"Required variable '{var}' is missing. Run cluster Steps 1–8 first."
```

These variables provide all physical quantities required for the microcanonical rate calculations. The required variable list is identical in the Local PC and HPC implementations.

The reactant and MECP vibrational frequencies are stored in:

```
freq_reactant_real_cm1
freq_MECP_real_cm1
```

These two variables define the vibrational data used for the reactant minimum and the minimum-energy crossing point, respectively.

The active MECP energy threshold is stored in:

```
VaG_MECP_cm1
```

This value contains the MECP energy relative to the reference minimum. The common Step 7/8 probability-energy grid is supplied separately:

```
probability_energy_grid_cm1
```

This array contains the common energy grid established in Step 7 and used by Step 8.

The effective spin–orbit coupling is stored in:

```
H_SO_cm
```

This scalar coupling is selected in Step 2 and subsequently used to generate the effective Landau-Zener and weak coupling probabilities.

The gradient-gap magnitude used in the transition-probability calculations is stored in:

```
GRADIENT_GAP_NORM_EH_PER_BOHR
```

This quantity contains the gradient-difference magnitude evaluated in Step 6. The final reaction-coordinate reduced mass is stored in:

```
reduced_mass_amu
```

This value is the final reaction-coordinate reduced mass selected after the effective-Hessian analysis.

The rotational constants required for the density-of-states calculations are stored in:

reference_rotational_constants_cm1

The corresponding MECP rotational constants are stored in:

crossing_rotational_constants_cm1

Together, these two arrays provide the rotational constants required for the reactant and crossing-point rotational density-of-states calculations.

Before proceeding, SPINKS verifies that every required variable exists. If any quantity is missing, execution is terminated with an informative error message indicating that the previous workflow steps must first be completed. This prevents incomplete datasets from propagating into the rate calculations.

4.9.2 Selection of the active ZPE-corrected MECP threshold

Before constructing the common energy grid, Step 9 checks which MECP barrier should be used as the active spin-crossing threshold:

```
# =====
# Energy check and active threshold
# =====

section("STEP 9 ENERGY CHECK")

if "VaG_MECP_electronic_cm1" in globals():
    print(
        f"Electronic MECP barrier      : "
        f"{VaG_MECP_electronic_cm1:.6f} cm^-1"
    )

if "VaG_MECP_ZPE_corrected_cm1" in globals():
    VaG_MECP_cm1 = float(VaG_MECP_ZPE_corrected_cm1)
    E_MECP = VaG_MECP_cm1
    E_MECP_cm1 = VaG_MECP_cm1

    print(
        f"ZPE-corrected MECP barrier  : "
        f"{VaG_MECP_ZPE_corrected_cm1:.6f} cm^-1"
    )
    print(
        f"Forced Step 9 threshold      : "
        f"{VaG_MECP_cm1:.6f} cm^-1"
    )
else:
    E_MECP = float(VaG_MECP_cm1)
    E_MECP_cm1 = E_MECP

    print(
        f"Active MECP barrier           : "
        f"{VaG_MECP_cm1:.6f} cm^-1"
```

)

The first conditional statement checks whether the electronic MECP barrier is still available:

`if "VaG_MECP_electronic_cm1" in globals():`

When available, its value is printed for diagnostic purposes. This value represents the electronic energy difference between the MECP and the selected reference minimum before the zero-point energy correction is applied. It is retained so that the electronic and ZPE-corrected barriers can be compared directly.

The active threshold itself is selected by the second conditional statement:

`if "VaG_MECP_ZPE_corrected_cm1" in globals():`

The ZPE-corrected MECP barrier used here is the quantity generated in Step 7 according to Equation S231:

$$V_{\text{MECP}}^{\text{ZPE}} = V_{\text{MECP}}^{\text{electronic}} + [ZPE_{\text{MECP}}^{\text{effhess}} - ZPE_{\text{reference}}] \quad (\text{S231})$$

The corresponding code-level mapping is summarized by Equation S232:

$$\text{VaG_MECP_ZPE_corrected_cm1} = \text{VaG_MECP_electronic_cm1} + \text{Delta_ZPE_MECP_minus_REF_cm1}. \quad (\text{S232})$$

When this ZPE-corrected value exists, Step 9 explicitly replaces the general MECP-barrier variable with it:

`VaG_MECP_cm1 = float(VaG_MECP_ZPE_corrected_cm1)`

When the ZPE-corrected value exists, the active Step 9 barrier is assigned according to Equation S233:

$$\text{VaG_MECP_cm1} = V_{\text{MECP}}^{\text{ZPE}} \quad (\text{S233})$$

The same active threshold is then assigned to two additional aliases:

`E_MECP = VaG_MECP_cm1`
`E_MECP_cm1 = VaG_MECP_cm1`

The two Step 9 MECP-energy aliases then satisfy Equation S234:

$$\text{E_MECP} = \text{E_MECP_cm1} = \text{VaG_MECP_cm1} = \text{VaG_MECP_ZPE_corrected_cm1} \quad (\text{S234})$$

These aliases are used throughout the subsequent probability, density-of-states, convolution, and microcanonical-rate calculations. The assignments ensure that all parts of Step 9 employ exactly the same crossing threshold.

The printed statement:

```
print(
    f"Forced Step 9 threshold      : "
    f"{VaG_MECP_cm1:.6f} cm^-1"
)
```

makes this replacement explicit in the console output. The word Forced indicates that Step 9 does not continue using a provisional electronic barrier when the ZPE-corrected barrier from Step 7 is available.

The corresponding threshold-bin relation is given by Equation S235:

$$i_{MECP} = \left\lceil \frac{E_{MECP}}{\Delta E} \right\rceil \quad (\text{S235})$$

Here, i_{MECP} denotes the MECP threshold-bin index.

Consequently, changing the active MECP threshold changes the stored crossing-bin index and therefore the energy at which the Landau-Zener probability-weighted state count and rate begin. It also changes the placement of the shifted crossing-density copy used for plotting and diagnostics.

The else block provides a fallback:

```
else:
    E_MECP = float(VaG_MECP_cm1)
    E_MECP_cm1 = E_MECP
```

This fallback branch is used only when the ZPE-corrected barrier is unavailable. In that case, Step 9 retains the existing MECP barrier and assigns it to the two Step 9 energy aliases. This preserves compatibility with older workflow states or externally supplied datasets.

The resulting Step 9 variable flow is:

```
VaG_MECP_ZPE_corrected_cm1
```

↓

```
VaG_MECP_cm1
```

↓

```
E_MECP
E_MECP_cm1
```

↓

Step 9 energy-grid and microcanonical-rate calculations

Therefore, the active threshold used in Step 9 is, whenever available, the ZPE-corrected MECP barrier generated in Step 7. The original electronic barrier remains available only as a separately reported diagnostic quantity.

4.9.3 Construction of the common total-energy grid

After selecting the active ZPE-corrected MECP threshold, Step 9 initializes the physical constants and reconstructs the common energy grid that is used throughout all subsequent density-of-states, probability, and microcanonical rate calculations:

```
section("CONSTANTS AND GRID SETUP")

autocm = 219474.6313705
h_SI = 6.62607015e-34
c_SI = 2.99792458e8
Eh_to_J = 4.3597447222071e-18
planck_hsec = h_SI / Eh_to_J

amu_kg = 1.66053906660e-27
bohr_m = 5.29177210903e-11
c_cm_s = 2.99792458e10
kJmol_per_cm1 = 0.01196266

probability_energy_grid_cm1 = np.asarray(probability_energy_grid_cm1,
dtype=float).reshape(-1)

if probability_energy_grid_cm1.size == 0:
    raise RuntimeError("The probability energy grid is empty. Run Step 7 first.")

energy_step_cm1 = float(globals().get("energy_step_cm1", 1.0))
maximum_energy_bin_index = int(np.ceil(np.nanmax(probability_energy_grid_cm1) /
energy_step_cm1))
total_energy_grid_cm1 = np.arange(maximum_energy_bin_index + 1, dtype=float) *
energy_step_cm1

crossing_energy_bin_index = int(np.ceil(E_MECP / energy_step_cm1))
zero_point_bin_offset = 0

reactant_degeneracy = float(globals().get("reactant_degeneracy", 1.0))
```

The first part of this block defines the physical conversion constants required throughout the downstream calculations. These constants are used for the energy, mass, length, density-of-states, and rate conversions performed later in Step 9.

Next, the energy grid generated previously in Step 7 is recovered as a NumPy array:

```
probability_energy_grid_cm1 = np.asarray(probability_energy_grid_cm1,
dtype=float).reshape(-1)
```

The Step 9 rate grid therefore uses the same energy axis as the imported probability grid, as summarized in Equation S236:

$$E_{\text{rate}} = E_{\text{probability}} \quad (\text{S236})$$

This identity ensures complete consistency between Steps 8 and 9.

Before continuing, the code verifies that the imported energy grid is not empty:

```
if probability_energy_grid_cm1.size == 0:  
    raise RuntimeError("probability_energy_grid_cm1 is empty. Run Step 7 first.")
```

If no energy grid is available, execution is terminated immediately because the subsequent density-of-states and convolution calculations require a valid common energy axis.

The energy spacing is then fixed as:

```
energy_step_cm1 = float(globals().get("energy_step_cm1", 1.0))
```

The corresponding uniform energy increment is given by Equation S237:

$$\Delta E = 1 \text{ cm}^{-1} \quad (\text{S237})$$

The spacing is inherited from the earlier workflow when available; the current standard workflow supplies a 1 cm^{-1} grid. Step 9 therefore does not hard-code a different grid spacing.

The highest energy represented on the grid is determined from the imported Step 7 array:

```
maximum_energy_bin_index = int(np.ceil(np.nanmax(probability_energy_grid_cm1) /  
energy_step_cm1))
```

and the complete uniformly spaced energy grid is reconstructed as:

```
total_energy_grid_cm1 = np.arange(maximum_energy_bin_index + 1, dtype=float) *  
energy_step_cm1
```

The reconstructed uniform energy grid is defined by Equation S238:

$$E_i = i\Delta E, i = 0, 1, \dots, N_{\max} \quad (\text{S238})$$

Thus, the reconstructed total-energy array contains uniformly spaced bins from zero to the maximum energy represented by the imported Step 7 grid.

Once the common energy grid has been established, the location of the active MECP threshold on that grid is identified by:

```
crossing_energy_bin_index = int(np.ceil(E_MECP / energy_step_cm1))
```

The code assignment above implements the threshold-bin relation defined in Equation S235. For clarity, the mathematical threshold index and its implementation counterpart are related by Equation S239:

$$i_{\text{code}} = i_{\text{MECP}} \quad (\text{S239})$$

The implementation stores this threshold-bin index in the following variable:

```
crossing_energy_bin_index
```

The crossing energy is the active ZPE-corrected barrier selected in Section 4.9.2. This stored index marks the first MECP-related bin used by the Landau-Zener probability and rate-convolution logic.

The reactant zero-point reference is assigned as:

```
zero_point_bin_offset = 0
```

All Step 9 energies are referenced to the reactant/reference minimum. The internal crossing density remains on its own zero-based internal-energy grid, while a separately shifted copy is generated for plotting and diagnostics.

Finally, the reactant electronic degeneracy is recovered as:

```
reactant_degeneracy = float(globals().get("reactant_degeneracy", 1.0))
```

The reactant electronic degeneracy enters the RRKM expression for the microcanonical rate constant according to Equation S240:

$$k(E) = \frac{g N(E)}{h \rho_R(E)} \quad (\text{S240})$$

where the numerator contains the reactant electronic degeneracy and the probability-weighted number of crossing states, while the denominator contains Planck's constant and the reactant rovibrational density of states.

This section therefore reconstructs the common total-energy grid, identifies the location of the ZPE-corrected MECP threshold, and establishes the energy framework that is used consistently throughout all subsequent density-of-states and microcanonical rate calculations.

4.9.4 Evaluation of the reactant and MECP rovibrational densities of states

Before calculating the microcanonical rate constants, Step 9 prepares the vibrational frequencies and rotational constants obtained from the reactant minimum and the MECP calculations. These quantities are subsequently used to construct the harmonic vibrational, rotational, and rovibrational densities of states for both the reactant and the crossing point:

```
section("FREQUENCY AND ROTATIONAL DATA")

freq_R = np.asarray(freq_reactant_real_cm1, dtype=float).reshape(-1)
freq_X = np.asarray(freq_MECP_real_cm1, dtype=float).reshape(-1)

freq_R = freq_R[np.isfinite(freq_R) & (freq_R > 0.0)]
freq_X = freq_X[np.isfinite(freq_X) & (freq_X > 0.0)]

rot_R_cm1 = np.asarray(reference_rotational_constants_cm1, dtype=float).reshape(-1)
rot_X_cm1 = np.asarray(crossing_rotational_constants_cm1, dtype=float).reshape(-1)

rot_R_cm1 = rot_R_cm1[np.isfinite(rot_R_cm1) & (rot_R_cm1 > 0.0)]
rot_X_cm1 = rot_X_cm1[np.isfinite(rot_X_cm1) & (rot_X_cm1 > 0.0)]
```



```

if len(rot_R_cm1) != 3:
    raise RuntimeError("reference_rotational_constants_cm1 must contain 3 positive values.")

if len(rot_X_cm1) != 3:
    raise RuntimeError("crossing_rotational_constants_cm1 must contain 3 positive values.")

if len(freq_R) == 0:
    raise RuntimeError("No positive reactant frequencies were found.")

if len(freq_X) == 0:
    raise RuntimeError("No positive MECP effective-Hessian frequencies were found.")

```

The reactant and MECP vibrational frequencies are first converted into one-dimensional NumPy arrays using:

```

freq_R = np.asarray(freq_reactant_real_cm1, dtype=float).reshape(-1)
freq_X = np.asarray(freq_MECP_real_cm1, dtype=float).reshape(-1)

```

The subsequent filtering operation:

```

freq_R = freq_R[np.isfinite(freq_R) & (freq_R > 0.0)]
freq_X = freq_X[np.isfinite(freq_X) & (freq_X > 0.0)]

```

The filtering removes undefined, infinite, and nonpositive frequencies. For the crossing point, the filtered array contains the positive effective-Hessian frequencies exported by Step 6 after removal of the selected reaction-coordinate direction.

The rotational constants of the reactant minimum and the MECP are processed similarly:

```

rot_R_cm1 = np.asarray(reference_rotational_constants_cm1, dtype=float).reshape(-1)
rot_X_cm1 = np.asarray(crossing_rotational_constants_cm1, dtype=float).reshape(-1)

```

followed by removal of any invalid or nonpositive values. Since a nonlinear molecule possesses three principal moments of inertia, Step 9 verifies that exactly three positive rotational constants remain for each structure. If this condition is not satisfied, execution is terminated to prevent construction of an incorrect rotational density of states.

The rotational constants are subsequently converted into principal moments of inertia using:

```

def rot_constants_to_inertia_amu_bohr2(B_cm1):
    conv = h_SI / (8.0 * np.pi**2 * c_cm_s) / (amu_kg * bohr_m**2)
    return conv / np.asarray(B_cm1, dtype=float)

reference_principal_inertias = rot_constants_to_inertia_amu_bohr2(rot_R_cm1)
crossing_principal_inertias = rot_constants_to_inertia_amu_bohr2(rot_X_cm1)

```

The rigid-rotor relation between a rotational constant and its principal moment of inertia is given by Equation S241:

$$B = \frac{h}{8\pi^2 c I} \quad (\text{S241})$$

Rearrangement gives the principal moment of inertia in Equation S242:

$$I = \frac{h}{8\pi^2 c B} \quad (\text{S242})$$

where B is the rotational constant expressed in cm^{-1} . The resulting principal moments of inertia are stored in $\text{amu}\cdot\text{Bohr}^2$, matching the units used by the rotational density-of-states routine.

After the vibrational frequencies and moments of inertia have been prepared, Step 9 evaluates the vibrational and rotational densities of states:

```
vibrational_density_reference = build_harmonic_ladder_density(freq_R,
maximum_energy_bin_index, energy_step_cm1)
vibrational_density_crossing = build_harmonic_ladder_density(freq_X,
maximum_energy_bin_index, energy_step_cm1)

rotational_density_reference = build_classical_rotational_density(reference_principal_inertias,
total_energy_grid_cm1, autocrm)
rotational_density_crossing = build_classical_rotational_density(crossing_principal_inertias,
total_energy_grid_cm1, autocrm)
```

The vibrational helper constructs the discrete vibrational ladder density on the common energy grid. Each positive normal-mode frequency contributes counts at its harmonic excitations, while the zero-excitation reference contribution is retained on the first positive-energy bin. The resulting reactant and MECP vibrational densities are denoted $\rho_{\text{vib}}^{\text{R}}(E)$ and $\rho_{\text{vib}}^{\text{X}}(E)$, respectively.

The rotational helper evaluates the rigid-rotor density directly on the common total-energy grid from the product of the three principal moments of inertia.

For each positive energy bin, the rotational density is evaluated according to Equation S243:

$$\rho_{\text{rot}}(E) = \frac{4}{a_{\text{u} \rightarrow \text{cm}^{-1}}} \sqrt{2EI_A I_B I_C} \quad (\text{S243})$$

Here, the Hartree-to-wavenumber conversion factor is used explicitly, and the three principal moments of inertia enter through their product.

The reactant and internal crossing rovibrational densities are then formed with the zero-based convolution helper, which applies a discrete convolution to the positive-energy portions of the rotational and vibrational densities and scales the result by the energy-grid spacing.

```

reactant_state_density = convolve_internal_densities(
    rotational_density_reference, vibrational_density_reference, energy_step_cm1
)

crossing_internal_state_density = convolve_internal_densities(
    rotational_density_crossing, vibrational_density_crossing, energy_step_cm1
)

crossing_shifted_state_density = np.zeros_like(crossing_internal_state_density)

```

The positive-energy internal crossing density is then copied upward by the MECP threshold index to form a threshold-aligned density used for plotting and diagnostic output. The production rate engine uses the unshifted internal crossing density together with explicit probability and output start indices; the shifted density is not passed to the Landau-Zener rate convolution.

Step 9 therefore retains three rovibrational density arrays:

- Reactant rovibrational density of states:

```
reactant_state_density
```

- Internal crossing-point rovibrational density used by both rate-convolution helpers:

```
crossing_internal_state_density
```

- Threshold-shifted crossing-density copy retained for plotting and diagnostics:

```
crossing_shifted_state_density
```

These quantities provide the statistical basis for the probability-weighted state-count convolutions that generate $N(E)$ and the corresponding microcanonical rate constants.

4.9.5 Independent evaluation of the rovibrational densities of states

Step 9 evaluates the rovibrational state densities with an independently implemented zero-based discrete engine. The current implementation keeps the vibrational ladder construction, rotational density, and rovibrational convolution as separate helper functions.

The vibrational and rotational densities of states are first evaluated independently using dedicated helper functions:

```

def build_harmonic_ladder_density(frequencies_cm1, maximum_bin_index,
energy_spacing_cm1):
    ...
    return density / energy_spacing_cm1

def build_classical_rotational_density(principal_inertias_amu_bohr2, energy_grid_cm1,
hartree_per_wavenumber):
    ...
    return density

vibrational_density_reference = build_harmonic_ladder_density(freq_R,
maximum_energy_bin_index, energy_step_cm1)
vibrational_density_crossing = build_harmonic_ladder_density(freq_X,
maximum_energy_bin_index, energy_step_cm1)

rotational_density_reference = build_classical_rotational_density(reference_principal_inertias,
total_energy_grid_cm1, autocm)
rotational_density_crossing = build_classical_rotational_density(crossing_principal_inertias,
total_energy_grid_cm1, autocm)

```

The vibrational routine assigns each positive normal mode its independent harmonic ladder on the common zero-based grid; it does not recursively enumerate simultaneous multimode occupation combinations. The resulting reactant and MECP vibrational densities are denoted $\rho_{\text{vib}}^{\text{R}}(E)$ and $\rho_{\text{vib}}^{\text{X}}(E)$, respectively.

The classical rotational density used by the implementation is summarized in Equation S244:

$$\rho_{\text{rot}}(E) = \frac{4}{\text{autocm}} \sqrt{2E I_A I_B I_C} \quad (\text{S244})$$

where the three principal moments of inertia are expressed in $\text{amu} \cdot \text{Bohr}^2$ and the Hartree-to-wavenumber factor provides the required energy conversion.

The current implementation does not shift the separate vibrational and rotational densities before convolution. Instead, it first constructs the internal crossing rovibrational density and only afterward creates a shifted copy:

```

reactant_state_density = convolve_internal_densities(rotational_density_reference,
vibrational_density_reference, energy_step_cm1)

crossing_internal_state_density = convolve_internal_densities(rotational_density_crossing,
vibrational_density_crossing, energy_step_cm1)
crossing_shifted_state_density = np.zeros_like(crossing_internal_state_density)

available_shifted_bins = maximum_energy_bin_index - crossing_energy_bin_index

if available_shifted_bins > 0:
    crossing_shifted_state_density[crossing_energy_bin_index + 1:crossing_energy_bin_index + 1
+ available_shifted_bins] = (
        crossing_internal_state_density[1:1 + available_shifted_bins]
    )

```

Thus, the stored MECP threshold index places a diagnostic copy of the internal crossing density on the total-energy axis without altering the internal density used by the rate helpers.

The internal-density helper performs the rotational-vibrational discrete convolution on the positive-energy array elements, reserves element zero as the threshold origin, and restores the result to the zero-based array with the appropriate energy-spacing factor.

```

def convolve_internal_densities(rotational_density, vibrational_density, energy_spacing_cm1):
    rotational_positive = np.asarray(rotational_density, dtype=float)[1:]
    vibrational_positive = np.asarray(vibrational_density, dtype=float)[1:]

    convolution = np.convolve(rotational_positive, vibrational_positive, mode="full")
    result = np.zeros_like(rotational_density, dtype=float)

    usable = min(result.size - 1, convolution.size)
    if usable > 0:
        result[1:usable + 1] = convolution[:usable] * energy_spacing_cm1

    return result

```

The rotational-vibrational convolution used to construct the internal rovibrational densities is summarized by Equation S245:

$$\rho(E) = \sum_j \rho_{\text{rot}}(E_j) \rho_{\text{vib}}(E - E_j) \Delta E \quad (\text{S245})$$

Here, ΔE denotes the energy-grid spacing. The standard workflow uses 1 cm^{-1} , but Step 9 inherits the available grid spacing.

This procedure produces the following stored rovibrational density arrays:

- Reactant rovibrational density:

```
reactant_state_density
```

- Internal crossing-point density used in both probability-weighted rate convolutions:

```
crossing_internal_state_density
```

- Threshold-shifted crossing-point density used only for plotting and diagnostics:

```
crossing_shifted_state_density
```

Both the LZ and WC wrappers convolve their probabilities with the same internal crossing density. Their different onset behavior is imposed through different probability and output start indices, not through different crossing-density arrays.

Finally, the maximum values of each density are printed as a diagnostic:

```
print(f"Reactant DOS max      : {np.nanmax(reactant_state_density):.6e}")
print(f"Crossing internal DOS max : {np.nanmax(crossing_internal_state_density):.6e}")
print(f"Crossing shifted LZ DOS max : {np.nanmax(crossing_shifted_state_density):.6e}")
```

These diagnostics provide a convenient verification that the reactant and MECP densities of states have been constructed successfully before they are employed in the convolution expressions used to calculate the effective number of crossing states and the corresponding microcanonical rate constants in the subsequent sections.

4.9.6 Evaluation of the Landau-Zener transition probability

Following construction of the rovibrational densities of states, Step 9 defines a general routine for evaluating the energy-dependent Landau-Zener transition probability for an arbitrary spin-orbit coupling. The same routine is used for effective, intermediate, and individual M_s -specific LZ calculations.

```
def compute_landau_zener_probability_for_coupling(H_cm, E_grid_cm1):
    E_grid_cm1 = np.asarray(E_grid_cm1, dtype=float).reshape(-1)

    H_cm = float(abs(H_cm))
    gradient_gap_local = abs(float(GRADIENT_GAP_NORM_EH_PER_BOHR))
    mu_kg_local = float(reduced_mass_amu) * amu_kg

    H_J = h_SI * c_SI * H_cm * 100.0
    E_excess_J = (
        (E_grid_cm1 - E_MECP)
        * 100.0
        * h_SI
        * c_SI
    )

    v_m_s_local = np.zeros_like(E_grid_cm1)
    mask_energy = E_excess_J > 0.0

    v_m_s_local[mask_energy] = np.sqrt(
        2.0 * E_excess_J[mask_energy] / mu_kg_local
```

)

```
v_bohr_s_local = v_m_s_local / bohr_m
```

The spin-orbit coupling enters the Landau-Zener adiabaticity parameter quadratically, as summarized in Equation S246:

$$\gamma_{\text{LZ}} \propto H_{\text{SO}}^2 \quad (\text{S246})$$

```
gamma = np.full_like(E_grid_cm1, np.nan, dtype=float)
```

```
P = np.zeros_like(E_grid_cm1, dtype=float)
```

```
mask = (
    np.isfinite(E_grid_cm1)

    & np.isfinite(v_bohr_s_local)

    & (E_grid_cm1 >= E_MECP)

    & (v_bohr_s_local > 1.0e-12)

)
```

The spin-orbit coupling is converted from cm^{-1} to joules according to Equation S247:

$$H_{\text{SO}}(\text{J}) = hc H_{\text{SO}}(\text{cm}^{-1}) \quad (\text{S247})$$

```
energy_gap_sweep_rate_Eh_s = gradient_gap_local * v_bohr_s_local
energy_gap_sweep_rate_J_s = energy_gap_sweep_rate_Eh_s * Eh_to_J

gamma[mask] = (
    4.0 * np.pi**2 * H_J**2
    / (h_SI * energy_gap_sweep_rate_J_s[mask])
)

P[mask] = 1.0 - np.exp(-2.0 * gamma[mask])
P = np.clip(P, 0.0, 1.0)
return P, gamma, v_bohr_s_local
```

The accessible-energy condition is given by Equation S248:

$$E > E_{\text{MECP}} \quad (\text{S248})$$

For accessible energies, the reaction-coordinate velocity is given by Equation S249:

$$v = \sqrt{\frac{2(E - E_{\text{MECP}})}{\mu}} \quad (\text{S249})$$

In Equation S249, the excess energy above the MECP is understood after conversion from cm^{-1} to joules, consistent with the implementation; μ is the reaction-coordinate reduced mass.

The diabatic energy-gap sweep rate and Landau-Zener parameter are then evaluated using Equations S250 and S251:

$$\frac{d\Delta E}{dt} = \Delta g \, v \quad (\text{S250})$$

$$\gamma = \frac{4\pi^2 H_{\text{SO}}^2}{h (d\Delta E/dt)} \quad (\text{S251})$$

The Landau-Zener transition probability is evaluated using Equation S252:

$$P_{\text{LZ}} = 1 - e^{-2\gamma} \quad (\text{S252})$$

The probability is subsequently constrained to the physical interval defined in Equation S253:

$$0 \leq P_{\text{LZ}} \leq 1 \quad (\text{S253})$$

The function returns three arrays:

- Landau-Zener transition probability:

P

- Landau-Zener adiabaticity parameter:

gamma

- Reaction-coordinate velocity in Bohr s^{-1} :


```
v_bohr_s_local
```

Because the routine accepts an arbitrary spin-orbit coupling as input, the same implementation generates the effective, intermediate, and individual M_s -specific Landau-Zener probabilities without duplicating the computational procedure.

4.9.7 Calculation of the effective number of crossing states and microcanonical rate constants

Step 9 next defines a shared probability-weighted convolution helper together with dedicated LZ and WC wrappers. Both wrappers use the same internal crossing density; their distinction is the index at which the probability segment and the output array begin.

```
def compute_landau_zener_microcanonical_rate(probability):
    cumulative_states = convolve_probability_with_crossing_density(
        probability=probability,
        crossing_density=crossing_internal_state_density,
        probability_start_index=crossing_energy_bin_index + 1,
        output_start_index=crossing_energy_bin_index + 1
    )

    rate = microcanonical_rate_from_cumulative_states(
        cumulative_states,
        reactant_state_density,
        reactant_degeneracy
    )

    return cumulative_states, rate
```

The Landau-Zener wrapper passes the probability array to the shared convolution helper together with the internal crossing density and the MECP-dependent start indices.

With the MECP threshold index denoted by i_{MECP} , the corresponding discrete LZ state-count relation is summarized by Equation S254:

$$N_{\text{LZ}}(E_i) = \sum_{j=i_{\text{MECP}}+1}^i P_{\text{LZ}}(E_j) \rho_X(E_i - E_j) \Delta E \quad (\text{S254})$$

The implementation performs this calculation with a zero-based discrete convolution. The probability slice begins one bin above the MECP threshold, while the crossing-density slice begins at its first positive-energy bin.

The shared convolution helper forms the two positive-index segments and evaluates their convolution as:

```
probability_segment = probability[probability_start_index:]
density_segment = crossing_density[1:]
convolution = np.convolve(probability_segment, density_segment, mode="full")
```

The available convolution values are then written to the total-energy array and scaled by the energy spacing and the Hartree-to-wavenumber conversion factor.

The resulting probability-weighted state-count array is passed to the common microcanonical-rate helper.

The common RRKM-like rate conversion implemented in Step 9 is:

```
rate[valid] = (  
    reactant_degeneracy_factor * cumulative_states[valid]  
    / (reactant_density[valid] * planck_hsec)  
)
```

This implementation corresponds to the statistical relationship in Equation S255:

$$k_{LZ}(E) = \frac{g_R N_{LZ}(E)}{h \rho_R(E)} \quad (\text{S255})$$

In Equation S255, the numerator contains the reactant electronic degeneracy and the effective LZ number of crossing states, while the denominator contains Planck's constant and the reactant rovibrational density of states.

The effective number of crossing states, $N(E)$, is a dimensionless energy-dependent statistical quantity rather than a literal integer count. It represents the probability-weighted number of crossing-region states accessible at total energy E .

The weak-coupling wrapper uses the same internal crossing density but starts both the probability segment and output at index 1. The energy dependence associated with the crossing region is therefore carried by the WC probability itself.

```
def compute_weak_coupling_microcanonical_rate(probability):  
    cumulative_states = convolve_probability_with_crossing_density(  
        probability=probability,  
        crossing_density=crossing_internal_state_density,  
        probability_start_index=1,  
        output_start_index=1  
    )  
  
    rate = microcanonical_rate_from_cumulative_states(  
        cumulative_states,  
        reactant_state_density,  
        reactant_degeneracy  
    )  
  
    return cumulative_states, rate
```

The corresponding weak-coupling number of crossing states is summarized by Equation S256:

$$N_{WC}(E_i) = \sum_{j=1}^i P_{WC}(E_j) \rho_{TP}(E_i - E_j) \Delta E \quad (S256)$$

The WC wrapper begins at index 1, whereas the LZ wrapper begins one bin above the stored MECP threshold. Thus, both theories use the same internal crossing density but different probability/output start indices.

The weak coupling microcanonical rate constant is calculated from the same statistical relationship, as shown in Equation S257:

$$k_{WC}(E) = \frac{g_R N_{WC}(E)}{h \rho_R(E)} \quad (S257)$$

Thus, both theories employ the same statistical rate conversion while differing in their transition probabilities and convolution onset.

Both dedicated wrappers remain available for explicit use throughout the remainder of Step 9.

4.9.8 Calculation of the effective Landau-Zener and weak coupling microcanonical rate constants

After defining the helper routines for evaluating the Landau-Zener and weak coupling microcanonical rate constants, Step 9 calculates the effective microcanonical rates using the effective spin-orbit coupling determined in Step 2. These effective quantities represent the principal microcanonical rate constants that are subsequently used throughout the remainder of the SPINKS workflow:

```
section("EFFECTIVE LZ/WC RATE CALCULATION")

effective_LZ_probability, effective_LZ_exponent, effective_velocity_bohr_per_s = (
    compute_landau_zener_probability_for_coupling(H_SO_cm, total_energy_grid_cm1)
)

effective_LZ_cumulative_states, effective_LZ_rate =
compute_landau_zener_microcanonical_rate(effective_LZ_probability)

probLZ = effective_LZ_probability
gamma_LZ_rate = effective_LZ_exponent
LZ_cumulative_states = effective_LZ_cumulative_states
LZ_rate = effective_LZ_rate
```

The calculation begins by evaluating the Landau-Zener probability over the entire Step 9 energy grid using the effective spin-orbit coupling obtained in Step 2. The helper returns three quantities:

- Effective Landau-Zener probability:

```
effective_LZ_probability
```

- Corresponding Landau-Zener exponent:

```
effective_LZ_exponent
```

- Reaction-coordinate velocity:

```
effective_velocity_bohr_per_s
```

The effective spin-orbit coupling is therefore transformed into an effective probability, a Landau-Zener exponent, and a reaction-coordinate velocity on the common Step 9 grid.

The effective Landau-Zener probability is then passed directly to the microcanonical rate helper:

```
effective_LZ_cumulative_states, effective_LZ_rate =  
compute_landau_zener_microcanonical_rate(effective_LZ_probability)
```

The effective Landau-Zener probability is then passed directly to the microcanonical rate helper, which evaluates the effective number of crossing states followed by the effective microcanonical rate constant.

The resulting effective LZ calculation follows the sequence shown below:

$$H_{SO}^{\text{eff}} \rightarrow P_{LZ}^{\text{eff}}(E) \rightarrow N_{LZ}^{\text{eff}}(E) \rightarrow k_{LZ}^{\text{eff}}(E)$$

For backward compatibility, the effective LZ quantities are also assigned to the following variables:

```
problZ = effective_LZ_probability  
gamma_LZ_rate = effective_LZ_exponent  
LZ_cumulative_states = effective_LZ_cumulative_states  
LZ_rate = effective_LZ_rate
```

These aliases preserve compatibility with earlier SPINKS variable names while retaining the same underlying effective LZ results.

The effective weak coupling microcanonical rate is evaluated through the following fallback logic:

```
if "compute_weak_coupling_probability" in globals():  
    effective_WC_probability, effective_WC_airy_argument, effective_WC_airy_value =  
    compute_weak_coupling_probability(  
        H_SO_cm,  
        total_energy_grid_cm1  
    )  
  
elif "P_WC_effective" in globals():  
    effective_WC_probability = np.asarray(P_WC_effective, dtype=float).reshape(-1)
```

```

...
else:
    raise RuntimeError("WC probability is missing. Run Step 8 first.")

effective_WC_cumulative_states, effective_WC_rate =
compute_weak_coupling_microcanonical_rate(effective_WC_probability)

probWC = effective_WC_probability
WC_cumulative_states = effective_WC_cumulative_states
WC_rate = effective_WC_rate

```

When the cluster Step 8 weak-coupling helper is available, Step 9 recalculates the clipped effective WC probability on the Step 9 total-energy grid. Otherwise, the stored Step 8 effective WC probability is reused after a grid-size check. In the Local PC workflow, this fallback is normally used because Step 8 exports a differently named WC helper.

The effective WC probability is then converted into the corresponding number of crossing states and microcanonical rate through the sequence shown below:

$$H_{SO}^{\text{eff}} \rightarrow P_{WC}^{\text{eff}}(E) \rightarrow N_{WC}^{\text{eff}}(E) \rightarrow k_{WC}^{\text{eff}}(E)$$

The corresponding rate conversion is performed by:

```

effective_WC_cumulative_states, effective_WC_rate =
compute_weak_coupling_microcanonical_rate(effective_WC_probability)

```

For backward compatibility, the effective WC quantities are also assigned to:

```

probWC = effective_WC_probability
WC_cumulative_states = effective_WC_cumulative_states
WC_rate = effective_WC_rate

```

These aliases preserve compatibility throughout the remainder of the workflow.

Finally, Step 9 reports the numerical ranges of the effective probabilities and microcanonical rate constants:

```

print(f"P_LZ effective range : ...")
print(f"k_LZ effective range : ...")
print(f"P_WC effective range : ...")
print(f"k_WC effective range : ...")

```

These diagnostics verify that both theories have been evaluated before Step 9 proceeds to the intermediate and individual M_s -specific microcanonical rates.

4.9.9 Calculation of the intermediate microcanonical rate constants

In addition to the effective microcanonical rates, Step 9 evaluates rates corresponding to the intermediate spin-orbit couplings generated in Step 8. These intermediate quantities retain the kinetics associated with the grouped $|M_s|$ manifolds without being summed to form the effective rate.

Landau-Zener Intermediate Rate Constants:

```
section("INTERMEDIATE RATE CALCULATION")

k_LZ_intermediate = {}
N_LZ_intermediate = {}
P_LZ_intermediate_rate_grid = {}

if "P_LZ_intermediate" in globals() and len(P_LZ_intermediate) > 0:
    for int_label, int_data in P_LZ_intermediate.items():
        H_int = float(int_data["H_int_cm1"])

        P_int, gamma_int, _ = compute_landau_zener_probability_for_coupling(
            H_int,
            total_energy_grid_cm1
        )

        N_int, k_int = compute_landau_zener_microcanonical_rate(P_int)

        P_LZ_intermediate_rate_grid[int_label] = P_int
        N_LZ_intermediate[int_label] = N_int
        k_LZ_intermediate[int_label] = {
            "H_int_cm1": H_int,
            "abs_Ms_low": float(int_data["abs_Ms_low"]),
            "degeneracy": int(int_data["degeneracy"]),
            "k": k_int,
            "N": N_int,
            "P": P_int
        }
```

The intermediate Landau-Zener calculations begin by creating dictionaries that will store the transition probabilities, effective numbers of crossing states, and microcanonical rate constants for every intermediate spin manifold generated in Step 8.

The calculation then loops over every intermediate coupling:

```
for int_label, int_data in P_LZ_intermediate.items():
```

where each intermediate is identified by its corresponding spin-orbit coupling:

```
H_int = float(int_data["H_int_cm1"])
```

Rather than reusing the probabilities stored in Step 8, SPINKS recalculates the Landau-Zener probability over the common Step 9 energy grid:

```
P_int, gamma_int, _ = compute_landau_zener_probability_for_coupling(
    H_int,
    total_energy_grid_cm1
)
```

This ensures that every intermediate LZ probability is evaluated on the same Step 9 energy grid with the same active MECF threshold, reduced mass, and gradient-gap magnitude.

The resulting probability is subsequently converted into the corresponding effective number of crossing states and microcanonical rate constant:

```
N_int, k_int = compute_landau_zener_microcanonical_rate(P_int)
```

The resulting sequence is intermediate SOC → LZ probability → LZ number of crossing states → LZ microcanonical rate.

Each intermediate-rate record retains the corresponding intermediate SOC magnitude, the absolute lower-spin projection, the grouped degeneracy, and the associated probability, number-of-states, and rate arrays.

The intermediate weak coupling rate constants are calculated using an analogous procedure:

```
k_WC_intermediate = {}
N_WC_intermediate = {}
P_WC_intermediate_rate_grid = {}

if "P_WC_intermediate" in globals() and len(P_WC_intermediate) > 0:
    for int_label, int_data in P_WC_intermediate.items():
        H_int = float(int_data["H_int_cm1"])

        if "compute_weak_coupling_probability" in globals():
            P_int, _ = compute_weak_coupling_probability(
                H_int,
                total_energy_grid_cm1
            )
        else:
            P_int = np.asarray(
                int_data["P"],
                dtype=float
            ).reshape(-1)

        N_int, k_int = compute_weak_coupling_microcanonical_rate(P_int)

    ...
```

For each intermediate weak-coupling channel, the cluster workflow recalculates the probability when the Step 8 cluster WC helper is available. Otherwise, including the normal Local PC path, the clipped probability stored in Step 8 is reused after a grid-size check.

Each probability array is then transformed through the sequence intermediate SOC → WC probability → WC number of crossing states → WC microcanonical rate using:

```
N_int, k_int = compute_weak_coupling_microcanonical_rate(P_int)
```

The resulting intermediate quantities are retained together with the corresponding projection and grouped-degeneracy metadata from Step 8.

After all intermediate spin manifolds have been processed, Step 9 reports the total number of intermediate Landau-Zener and weak coupling microcanonical rate curves generated:

```
print(f"Computed LZ intermediate rates for {len(k_LZ_intermediate)} curves.")
```

```
...
```

```
print(f"Computed WC intermediate rates for {len(k_WC_intermediate)} curves.")
```

These intermediate rate curves preserve the kinetic behavior associated with the grouped $|M_s|$ manifolds. They are diagnostic channel-resolved results and are not summed to construct the effective microcanonical rate.

4.9.10 Calculation of M_s -specific microcanonical rate constants

The final stage of the rate evaluation extends the statistical treatment to every individual M_s -resolved spin-crossing channel identified in Step 8. Each allowed channel is treated independently using its corresponding absolute spin-orbit coupling magnitude. No averaging or summation of channels is performed at this stage.

The implementation begins by allocating separate dictionaries for the M_s -specific Landau-Zener probabilities, effective numbers of crossing states, and microcanonical rate constants:

```
# =====  
# MS-specific channel rates  
# =====  
  
section("MS-SPECIFIC CHANNEL RATE CALCULATION")  
  
k_LZ_channels = {}  
N_LZ_channels = {}  
P_LZ_channels_rate_grid = {}  
  
if "P_LZ_channels" in globals() and len(P_LZ_channels) > 0:  
    for ch_label, ch_data in P_LZ_channels.items():  
        H_abs = float(ch_data["H_abs_cm1"])  
  
        P_ch, gamma_ch, _ = compute_landau_zener_probability_for_coupling(  
            H_abs,  
            total_energy_grid_cm1  
        )
```



```
N_ch, k_ch = compute_landau_zener_microcanonical_rate(P_ch)
```

```
P_LZ_channels_rate_grid[ch_label] = P_ch
```

```
N_LZ_channels[ch_label] = N_ch
```

```
k_LZ_channels[ch_label] = k_ch
```

Because the transition-probability expressions depend on the square of the coupling magnitude, their channel dependence is summarized by Equation S258:

$$P = f(H_{SO}^2) \quad (S258)$$

Thus, channels differing only in the sign of a spin-orbit matrix element produce identical transition probabilities and identical microcanonical rate constants.

Using the channel-specific coupling, the Step 9 Landau-Zener helper evaluates the complete M_s -specific probability curve on the common total-energy grid.

The corresponding M_s -specific LZ number of crossing states follows Equation S259:

$$N_{LZ}^{(M_s)}(E) = \sum_j P_{LZ}^{(M_s)}(E_j) \rho_X(E - E_j) \Delta E \quad (S259)$$

The M_s -specific microcanonical Landau-Zener rate constant is then obtained using Equation S260:

$$k_{LZ}^{(M_s)}(E) = \frac{g_R N_{LZ}^{(M_s)}(E)}{h \rho_R(E)} \quad (S260)$$

Thus, every spin channel is propagated independently through the complete nonadiabatic statistical framework, producing a distinct energy-dependent microcanonical rate constant whenever its SOC magnitude differs.

The resulting probability, number-of-states, and rate arrays are stored separately for every transition channel:

```
P_LZ_channels_rate_grid[ch_label] = P_ch
```

```
N_LZ_channels[ch_label] = N_ch
```

```
k_LZ_channels[ch_label] = k_ch
```

The separate dictionaries preserve the identity of every M_s -resolved transition throughout the calculation, allowing direct visualization and comparison without premature averaging or channel combination.

An analogous procedure is subsequently performed for the weak coupling formalism:

```

k_WC_channels = {}
N_WC_channels = {}
P_WC_channels_rate_grid = {}

if "P_WC_channels" in globals() and len(P_WC_channels) > 0:
    for ch_label, ch_data in P_WC_channels.items():
        H_abs = float(ch_data["H_abs_cm1"])

        if "compute_weak_coupling_probability" in globals():
            P_ch, _, _ = compute_weak_coupling_probability(H_abs, total_energy_grid_cm1)
        else:
            P_ch = np.asarray(ch_data["P"], dtype=float).reshape(-1)

        N_ch, k_ch = compute_weak_coupling_microcanonical_rate(P_ch)

        P_WC_channels_rate_grid[ch_label] = P_ch
        N_WC_channels[ch_label] = N_ch
        k_WC_channels[ch_label] = k_ch

```

The channel-specific WC probability is recalculated when the cluster weak-coupling helper is available; otherwise the corresponding clipped Step 8 channel probability is reused. The resulting number of crossing states is converted to the M_s -specific WC microcanonical rate using Equation S261:

$$k_{WC}^{(M_s)}(E) = \frac{g_{RN_{WC}}^{(M_s)}(E)}{h \rho_R(E)} \quad (\text{S261})$$

The resulting arrays are stored independently for every M_s -resolved transition. As in the Landau-Zener treatment, no channel summation or symmetry reduction is applied during this stage.

4.9.11 Construction of symmetry-collapsed M_s -specific rate curves

Many independently evaluated M_s -specific channels have identical absolute spin-orbit coupling magnitudes, as expressed by Equation S262:

$$|H_{SO,i}| = |H_{SO,j}| \quad (\text{S262})$$

Such channels generate identical probability and rate curves. SPINKS therefore retains one representative curve for visualization while preserving all individual channel arrays. This grouping is only a presentation reduction: no probabilities, numbers of crossing states, or rate constants are summed, averaged, weighted, or degeneracy-scaled.

The corresponding implementation begins by constructing new dictionaries that store representative Landau-Zener rate curves for each symmetry-equivalent group:

```

# =====
# Symmetry-collapsed MS-specific channel rates
# =====

section("SYMMETRY-COLLAPSED MS-SPECIFIC RATE CURVES")

k_LZ_channel_groups = {}
N_LZ_channel_groups = {}
P_LZ_channel_groups_rate_grid = {}

if "P_LZ_channel_groups" in globals() and len(P_LZ_channel_groups) > 0:
    for group_key, group_data in P_LZ_channel_groups.items():

```

Each Step 8 symmetry group contains the labels of equivalent spin-transition channels together with a representative channel. If the predefined representative is unavailable, Step 9 selects the first valid member and copies its probability, number-of-states, and rate arrays.

```

k_rep = np.asarray(
    k_LZ_channels[representative_label],
    dtype=float
).copy()

N_rep = np.asarray(
    N_LZ_channels[representative_label],
    dtype=float
).copy()

P_rep = np.asarray(
    P_LZ_channels_rate_grid[representative_label],
    dtype=float
).copy()

```

All members of a representative group have the same SOC magnitude, as shown in Equation S263:

$$|H_{SO,1}| = |H_{SO,2}| = \dots = |H_{SO,n}| \quad (\text{S263})$$

Their transition probabilities are therefore identical according to Equation S264:

$$P_1(E) = P_2(E) = \dots = P_n(E) \quad (\text{S264})$$

The corresponding numbers of crossing states are identical according to Equation S265:

$$N_1(E) = N_2(E) = \dots = N_n(E) \quad (\text{S265})$$

The microcanonical rate curves are consequently identical according to Equation S266:

$$k_1(E) = k_2(E) = \dots = k_n(E) \quad (\text{S266})$$

Retaining one representative curve therefore removes only graphical duplication while preserving the complete numerical information for every individual channel.

The representative curve together with the complete list of equivalent transition labels is stored as:

```
k_LZ_channel_groups[group_key] = {
  "H_abs_cm1": float(group_data["H_abs_cm1"]),
  "labels": labels,
  "channel_symbols": list(
    group_data.get("channel_symbols", [])
  ),
  "degeneracy": len(labels),
  "representative_label": representative_label,
  "k_representative": k_rep,
  "N_representative": N_rep,
  "P_representative": P_rep,
  "combination_rule": "representative_only_no_sum"
}
```

The group metadata explicitly records the no-summation rule:

```
"combination_rule": "representative_only_no_sum"
```

This field documents that the representative curve is used only for visualization and reporting. Every original M_s -specific channel remains available through the individual arrays generated in Section 4.9.10.

The same representative-only procedure is applied to the WC treatment, so both LZ and WC use the same symmetry-collapsing strategy while retaining complete channel-specific data.

Finally, the total numbers of representative Landau-Zener and weak coupling curves are reported together with the grouping rule:

```
print(f"Representative LZ MS curves      : {len(k_LZ_channel_groups)}")
print(f"Representative WC MS curves      : {len(k_WC_channel_groups)}")
print("MS-specific combination rule      : representative only; no summation")
```

This diagnostic confirms that SPINKS performs only a graphical reduction of symmetry-equivalent M_s -specific channels. The underlying microcanonical rates remain unchanged.

4.9.12 Generation of final rate arrays for canonical calculations

Following evaluation of the effective, intermediate, and M_s -specific microcanonical rate constants, Step 9 constructs standardized output arrays for the canonical calculations in Step 10. The zero-energy element used internally by the discrete engine is omitted from the exported rate grids, logarithmic arrays are generated for positive rates, and backward-compatible aliases are retained. No new physical approximation is introduced in this stage.

The implementation begins by extracting the physical energy grid together with the corresponding rovibrational densities of states:

```
# =====
# Final arrays and backward-compatible names
# =====

section("FINAL RATE ARRAYS")

E_rate_LZ_cm1 = total_energy_grid_cm1[1:]
E_rate_WC_cm1 = total_energy_grid_cm1[1:]

rho_reactant_cm = reactant_state_density[1:]
rho_MECP_internal_cm = crossing_internal_state_density[1:]
rho_MECP_shifted_plot = crossing_shifted_state_density[1:]
```

The zero-energy element is omitted from the standardized exported arrays. The reactant density is passed forward for canonical averaging, while the internal and shifted crossing densities are retained for diagnostics, plotting, and compatibility.

The effective Landau-Zener probability, adiabaticity parameter, reaction velocity, number of crossing states, and microcanonical rate constants are then assembled into standardized arrays:

```
P_LZ_rate_grid = effective_LZ_probability[1:]
gamma_LZ_rate_grid = effective_LZ_exponent[1:]
effective_velocity_bohr_per_s_grid = effective_velocity_bohr_per_s[1:]

N_MECP_LZ = effective_LZ_cumulative_states[1:]
k_LZ_micro_s_inv = effective_LZ_rate[1:]

k_LZ_effective_micro_s_inv = k_LZ_micro_s_inv
N_MECP_LZ_effective = N_MECP_LZ
```

The corresponding weak coupling quantities are prepared in an analogous manner:

```
P_WC_rate_grid = effective_WC_probability[1:]
N_MECP_WC = effective_WC_cumulative_states[1:]
k_WC_micro_s_inv = effective_WC_rate[1:]

k_WC_effective_micro_s_inv = k_WC_micro_s_inv
N_MECP_WC_effective = N_MECP_WC
```

These standardized arrays provide a consistent interface for the canonical kinetic calculations in Step 10 while retaining compatibility with earlier SPINKS variable names.

For visualization purposes, logarithmic representations of the effective microcanonical rate constants are also generated. Because the logarithm is defined only for positive values, the code first identifies all positive rate constants:

```
positive_k = k_LZ_micro_s_inv > 0.0
log_k_LZ_raw = np.full_like(k_LZ_micro_s_inv, np.nan)
log_k_LZ_raw[positive_k] = np.log10(k_LZ_micro_s_inv[positive_k])
```

The same positive-rate masking procedure is applied to WC. Zero-valued rates are represented by NaN in the logarithmic arrays so that undefined logarithms are not plotted.

Finally, aliases are assigned to the rovibrational densities of states and effective microcanonical quantities:

```

reactant_state_density_independent = reactant_state_density
crossing_shifted_state_density_independent = crossing_shifted_state_density
crossing_internal_state_density_independent = crossing_internal_state_density

LZ_cumulative_states_independent = effective_LZ_cumulative_states
LZ_rate_independent = effective_LZ_rate

WC_cumulative_states_independent = effective_WC_cumulative_states
WC_rate_independent = effective_WC_rate

```

These aliases preserve backward compatibility while the standardized arrays are passed to Step 10. Step 9 then generates the effective, intermediate, and M_s -specific LZ/WC plots and writes the numerical summary, effective-rate grid, intermediate-rate summary, channel-rate summary, and DOS grid.

4.9.13 Summary of the step 9 algorithm

For plotting and diagnostics, the threshold-shifted crossing density is defined by Equation S267:

$$\rho_X(E) = \begin{cases} 0, & E < E_{\text{MECP}} \\ \rho_{TP}(E - E_{\text{MECP}}), & E \geq E_{\text{MECP}} \end{cases} \quad (\text{S267})$$

The production LZ and WC rate convolutions do not use this shifted copy. Both use the internal crossing density, with their different onset behavior controlled by the explicit start indices in the two wrappers.

The effective Landau-Zener and weak coupling probabilities are transformed into probability-weighted numbers of crossing states according to Equation S268:

$$N_{\text{LZ}}(E) = \sum_j P_{\text{LZ}}(E_j) \rho_X(E - E_j) \Delta E, N_{\text{WC}}(E) = \sum_j P_{\text{WC}}(E_j) \rho_{TP}(E - E_j) \Delta E \quad (\text{S268})$$

The corresponding effective microcanonical rate constants are evaluated using Equation S269:

$$k_{\text{LZ}}(E) = \frac{g_R N_{\text{LZ}}(E)}{h \rho_R(E)}, k_{\text{WC}}(E) = \frac{g_R N_{\text{WC}}(E)}{h \rho_R(E)} \quad (\text{S269})$$

The same statistical procedure is repeated for every intermediate spin-orbit coupling generated in Step 8, producing the complete hierarchy of intermediate LZ and WC microcanonical rate constants.

The algorithm is then applied independently to every M_s -resolved spin-crossing channel using the corresponding absolute SOC magnitude.

Channels with identical absolute spin-orbit couplings satisfy Equation S270:

$$|H_{\text{SO},1}| = |H_{\text{SO},2}| = \dots \quad (\text{S270})$$

These channels consequently have identical microcanonical rate curves, as summarized by Equation S271:

$$k_1(E) = k_2(E) = \dots \quad (\text{S271})$$

Only one representative curve is retained for plotting each such group, while the complete list of equivalent channel labels and every individual numerical array are preserved. No probabilities, state counts, or rates are summed.

The corresponding computational sequence is:

Reactant/MECP frequencies, rotational constants, and ZPE-corrected MECP energy

↓

SPINKS rovibrational densities of states

↓

Effective LZ/WC probabilities

↓

Effective LZ/WC microcanonical rate constants

↓

Intermediate microcanonical rates

↓

M_s -specific microcanonical rates

↓

Symmetry-collapsed representative M_s curves

↓

Final energy/rate arrays, diagnostics, outputs, and exported variables

↓

Step 10 canonical rate calculations

Therefore, Step 9 transforms the clipped effective, intermediate, and M_s -resolved spin-crossing probabilities into the corresponding hierarchy of LZ and WC microcanonical rate constants. Symmetry-equivalent M_s channels are represented by one plotted curve without summation, and the standardized energy/rate arrays provide the direct input to Step 10.

4.10 Step 10: Canonical Rate Constants

Following the construction of the effective and M_S-resolved microcanonical rate constants in Step 9, Step 10 converts these energy-dependent quantities into canonical reaction rate constants through Boltzmann averaging over the reactant density of states. This transformation enables direct comparison with experimentally measured temperature-dependent rate constants while maintaining the same Landau-Zener and weak coupling formalisms developed in the preceding step.

The canonical averaging procedure is performed separately for the effective Landau-Zener and weak coupling rate constants, the intermediate LZ/WC channels associated with each unique |M_S| group, and the representative M_S-specific channels retained after symmetry reduction. In every case, the microcanonical rate constants are weighted by the reactant density of states and the Boltzmann factor before normalization by the canonical reactant partition function.

The general canonical rate expression is given by Equation S272:

$$k(T) = \frac{\int k(E) \rho_R(E) e^{-E/(k_B T)} dE}{\int \rho_R(E) e^{-E/(k_B T)} dE} \quad (\text{S272})$$

where $k(E)$ is the microcanonical rate constant, $\rho_R(E)$ is the reactant density of states, k_B is the Boltzmann constant, and T is the absolute temperature. Within SPINKS, these integrals are evaluated numerically on the common energy grid established in Step 9 using the discrete energy spacing ΔE .

In addition to computing the effective canonical Landau-Zener and weak coupling rate constants, Step 10 evaluates canonical rate constants for every intermediate and representative M_S-specific spin-crossing channel. The resulting temperature-dependent rate constants are exported as numerical tables and graphical plots over the user-defined temperature range.

4.10.1 Import of step 9 microcanonical quantities

Step 10 imports the standardized Step 9 energy grids, reactant rovibrational density, and effective microcanonical LZ/WC rates, verifies their availability, and then performs canonical Boltzmann averaging. Intermediate and representative M_S-specific dictionaries are used when channel-resolved SOC information is available.

The corresponding implementation first defines the complete list of variables required for the canonical calculations:


```

required_vars_step10 = [
    "jobname",
    "E_rate_LZ_cm1",
    "E_rate_WC_cm1",
    "rho_reactant_cm",
    "k_LZ_effective_micro_s_inv",
    "k_WC_effective_micro_s_inv",
    "local_base",
    "remote_base"
]

for var in required_vars_step10:
    if var not in globals():
        raise RuntimeError(
            f"Required variable '{var}' is missing. Run Step 9 first."
        )

```

The required-variable block therefore contains the two Step 9 rate-energy grids, the reactant rovibrational density of states, the effective LZ/WC microcanonical rate arrays, and the base directories needed for output handling.

The Local PC and HPC implementations use the same scientific inputs. Their output-directory handling differs only because the Local PC remote paths are compatibility aliases, whereas the HPC implementation creates and transfers artifacts to a remote directory.

The default execution-mode assignment differs between the two implementations:

In the Local PC implementation:

```
RUN_MODE = globals().get("RUN_MODE", "LOCAL")
```

In the HPC implementation:

```
RUN_MODE = globals().get("RUN_MODE", "CLUSTER")
```

The remaining workflow and SOC-mode flags are common to both implementations:

```

workflow_mode = "MECP_ONLY"

SOC_EFFECTIVE_ONLY = bool(
    globals().get("SOC_EFFECTIVE_ONLY", False)
)

SOC_MATRIX_CHANNELS_AVAILABLE = bool(
    globals().get("SOC_MATRIX_CHANNELS_AVAILABLE", True)
)

if SOC_EFFECTIVE_ONLY or not SOC_MATRIX_CHANNELS_AVAILABLE:
    print("\nEffective-only SOC mode active.")
    print("Only effective LZ/WC rates will be computed/plotted.")

```

When effective-only SOC mode is active, Step 10 evaluates and plots only the effective canonical LZ/WC rates. Otherwise, it also canonical-averages the intermediate and symmetry-collapsed representative MS-specific rates.

To facilitate the remaining calculations, Step 10 also defines helper routines for section formatting, validated numerical input, text output, figure generation, and construction of mathematically formatted labels for M_S-specific transition channels:

```
def section(title):
    ...
def ask_float(...):
    ...
def ask_int(...):
    ...
def write_local_text(...):
    ...
def save_current_figure(...):
    ...
def ms_tex_value(...):
    ...
def make_canonical_group_legend(...):
    ...
```

These utility functions standardize user input and output handling and generate consistent labels for representative M_S-specific canonical rate curves. In particular, the group-legend helper identifies symmetry-equivalent spin-transition channels with identical absolute spin-orbit couplings and labels their coincident curves without implying a summed contribution.

4.10.2 Construction of the canonical temperature grid

Unlike the microcanonical calculations of Step 9, which evaluate reaction rates as functions of total internal energy, the canonical treatment averages over a thermal ensemble of reactant states. Step 10 therefore constructs a user-defined temperature grid for the canonical Landau-Zener and weak coupling rate constants.

The discrete canonical-averaging expression used by SPINKS is given by Equation S273:

$$k(T) = \frac{\sum k(E) \rho_R(E) e^{-E/(k_B T)} \Delta E}{\sum \rho_R(E) e^{-E/(k_B T)} \Delta E} \quad (\text{S273})$$

where $k(E)$ denotes the microcanonical reaction rate constant obtained in Step 9, $\rho_R(E)$ is the reactant rovibrational density of states, k_B is the Boltzmann constant, and T is the absolute temperature.

The implementation begins by defining the Boltzmann constant expressed in wavenumber units:

```
kB_cm1_per_K = 0.695034800
```

This value allows the Boltzmann factor to be evaluated directly with energies expressed in cm^{-1} . The user then specifies the temperature interval and the number of temperature points:

```

T_min = ask_float(
    "Minimum temperature for canonical rates, K, e.g. 100: ",
    minimum=1.0e-12 )

T_max = ask_float(
    "Maximum temperature for canonical rates, K, e.g. 1000: ",
    minimum=1.0e-12 )

nT = ask_int(
    "Number of temperature points, e.g. 100: ",
    minimum=2 )

```

The lower and upper temperatures define the range over which the canonical rate constants are computed, while the third input defines the number of uniformly spaced temperature values. A consistency check requires the upper temperature to exceed the lower temperature:

```

if T_max <= T_min:
    raise RuntimeError(
        "Invalid temperature range..."
    )

```

The complete temperature grid is then generated as:

```

T_grid_K = np.linspace(
    T_min,
    T_max,
    nT
)

```

The resulting temperature sequence is given by Equation S274:

$$T_i = T_{\min} + (i - 1) \frac{T_{\max} - T_{\min}}{n_T - 1}, i = 1, \dots, n_T \quad (\text{S274})$$

This sequence forms the common temperature grid used throughout the subsequent canonical calculations.

The Step 9 arrays are then converted to one-dimensional NumPy arrays before canonical averaging:

```

E_rate_LZ_cm1 = np.asarray(E_rate_LZ_cm1, dtype=float).reshape(-1)
E_rate_WC_cm1 = np.asarray(E_rate_WC_cm1, dtype=float).reshape(-1)
rho_reactant_cm = np.asarray(rho_reactant_cm, dtype=float).reshape(-1)

k_LZ_effective_micro_s_inv = np.asarray(k_LZ_effective_micro_s_inv,
dtype=float).reshape(-1)
k_WC_effective_micro_s_inv = np.asarray(k_WC_effective_micro_s_inv,
dtype=float).reshape(-1)

```

Additional consistency checks verify that the energy grids, reactant density of states, and effective microcanonical rate constants have matching dimensions. The average spacing of each rate-energy grid is then evaluated:

```
dE_LZ = float(np.mean(np.diff(E_rate_LZ_cm1))) if E_rate_LZ_cm1.size > 1 else 1.0
```

```
dE_WC = float(np.mean(np.diff(E_rate_WC_cm1))) if E_rate_WC_cm1.size > 1 else 1.0
```

These two spacings provide the integration step used in the corresponding discrete Boltzmann sums.

4.10.3 Canonical Boltzmann averaging

The central objective of Step 10 is to transform the energy-dependent microcanonical reaction rate constants obtained in Step 9 into temperature-dependent canonical rate constants. Unlike the microcanonical treatment, where the reaction rate is evaluated at a fixed total energy, the canonical formalism assumes that the reactant population is distributed among all accessible rovibrational states according to the Boltzmann distribution. Consequently, the observable reaction rate at a given temperature is obtained as the Boltzmann-weighted average of the microcanonical rate constants over the entire reactant energy manifold.

The corresponding implementation is performed through the helper routine:

The helper receives an energy grid, reactant state density, microcanonical rate array, temperature grid, and energy spacing. These inputs correspond respectively to E , $\rho_R(E)$, $k(E)$, T , and ΔE in the canonical expressions below.

```
def boltzmann_average_microcanonical_rate(
    energy_cm1,
    state_density,
    microcanonical_rate,
    temperature_grid_K,
    energy_spacing_cm1
):
    energy_cm1 = np.asarray(energy_cm1, dtype=float).reshape(-1)
    state_density = np.asarray(state_density, dtype=float).reshape(-1)
    microcanonical_rate = np.asarray(
        microcanonical_rate,
        dtype=float
    ).reshape(-1)
    temperature_grid_K = np.asarray(
        temperature_grid_K,
        dtype=float
    ).reshape(-1)

    if (
        energy_cm1.shape != state_density.shape
        or energy_cm1.shape != microcanonical_rate.shape
    ):
        raise RuntimeError(
            "Energy, state-density, and microcanonical-rate arrays "
            "must have the same shape."
```

```
)
```

```
thermal_energy_cm1 = kB_cm1_per_K * temperature_grid_K
```

```
canonical_rate = np.zeros_like(temperature_grid_K, dtype=float)

normalization_integral = np.zeros_like(
    temperature_grid_K,
    dtype=float
)
```

At each temperature, the Boltzmann weight is evaluated according to Equation S275:

$$w(E, T) = \exp\left(-\frac{E_i}{k_B T}\right) \quad (\text{S275})$$

```
for temperature_index, thermal_energy in enumerate(
    thermal_energy_cm1
):
    boltzmann_weight = np.exp(-energy_cm1 / thermal_energy)

    denominator = np.sum(
        state_density * boltzmann_weight
    ) * energy_spacing_cm1
```

The discrete reactant normalization integral is given by Equation S276:

$$Q_R(T) = \sum_i \rho_R(E_i) \exp\left(-\frac{E_i}{k_B T}\right) \Delta E \quad (\text{S276})$$

```
numerator = np.sum(
    microcanonical_rate
    * state_density
    * boltzmann_weight
) * energy_spacing_cm1
```

The corresponding Boltzmann-weighted numerator is given by Equation S277:

$$N(T) = \sum_i k(E_i) \rho_R(E_i) \exp\left(-\frac{E_i}{k_B T}\right) \Delta E \quad (\text{S277})$$

```
normalization_integral[temperature_index] = denominator
if denominator > 0.0 and np.isfinite(denominator):
```

```

        canonical_rate[temperature_index] = numerator / denominator

    else:

        canonical_rate[temperature_index] = 0.0

    canonical_rate[~np.isfinite(canonical_rate)] = 0.0

    canonical_rate = np.clip(canonical_rate, 0.0, None)

    return canonical_rate, normalization_integral

```

Combining the numerator with the normalization integral gives the canonical rate in Equation S278:

$$k(T) = \frac{N(T)}{Q_R(T)} \quad (\text{S278})$$

The function stores the normalization integral and the canonical rate separately. If the denominator is nonpositive or nonfinite, the canonical rate is set to zero; any remaining nonfinite values are replaced with zero, and negative numerical roundoff is clipped. The returned arrays are the canonical rate and the reactant normalization integral.

4.10.4 Evaluation of the effective canonical Landau-Zener and weak coupling rate constants

The first application of the canonical averaging procedure is the calculation of the effective canonical Landau-Zener and weak coupling reaction rate constants. These quantities represent the thermal averages of the effective microcanonical rate constants obtained in Step 9 and therefore provide the primary temperature-dependent kinetic predictions of the SPINKS workflow. Since the effective spin-orbit coupling already incorporates the complete electronic coupling information for the spin-crossing event, these canonical rate constants constitute the principal theoretical predictions for comparison with experiment or other kinetic models.

The corresponding implementation applies the canonical averaging routine independently to the effective Landau-Zener and weak coupling microcanonical rate constants:

```

# =====
# Effective canonical rates
# =====

```

```
section("EFFECTIVE CANONICAL LZ/WC RATES")
```

```
k_LZ_canonical_s_inv, LZ_canonical_normalization = boltzmann_average_microcanonical_rate(
    E_rate_LZ_cm1,
    rho_reactant_cm,
    k_LZ_effective_micro_s_inv,
    T_grid_K,
    dE_LZ
)
```

```
k_WC_canonical_s_inv, WC_canonical_normalization =
boltzmann_average_microcanonical_rate(
    E_rate_WC_cm1,
    rho_reactant_cm,
    k_WC_effective_micro_s_inv,
    T_grid_K,
    dE_WC
)
```

For the Landau-Zener treatment, the effective Step 9 microcanonical rate is converted to its canonical counterpart according to Equation S279:

$$k_{LZ}(T) = \frac{\sum_i k_{LZ}(E_i) \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}} \quad (\text{S279})$$

The effective weak-coupling microcanonical rate is converted analogously according to Equation S280:

$$k_{WC}(T) = \frac{\sum_i k_{WC}(E_i) \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}} \quad (\text{S280})$$

Both effective canonical rate constants therefore use the same Boltzmann-averaging procedure and differ only in the underlying Step 9 microcanonical rate curve.

The calculated canonical rate arrays are also assigned the following compatibility aliases:

```
kcanon_LZ = k_LZ_canonical_s_inv
kcanon_WC = k_WC_canonical_s_inv
```

The Landau-Zener and weak-coupling normalization integrals are retained as separate diagnostic arrays. Because both use the same reactant density of states, they should agree when evaluated on identical energy grids.

```
LZ_canonical_normalization
WC_canonical_normalization
```

Finally, the minimum and maximum values of both canonical rate curves are reported:

```

print(
    f"k_LZ canonical range : "
    f"{np.nanmin(k_LZ_canonical_s_inv):.6e} "
    f"to "
    f"{np.nanmax(k_LZ_canonical_s_inv):.6e} s^-1"
)

print(
    f"k_WC canonical range : "
    f"{np.nanmin(k_WC_canonical_s_inv):.6e} "
    f"to "
    f"{np.nanmax(k_WC_canonical_s_inv):.6e} s^-1"
)

```

These diagnostics verify completion of the effective canonical averaging before the intermediate and M_S-specific canonical calculations are performed.

4.10.5 Evaluation of the intermediate canonical Landau-Zener and weak coupling rate constants

After the effective canonical rates are obtained, Step 10 applies the same Boltzmann averaging routine independently to every intermediate microcanonical rate dictionary produced in Step 9. Each stored intermediate curve is canonical-averaged as its own rate curve; no interpolation, summation, or averaging between different intermediate SOC groups is performed.

The implementation begins by allocating dictionaries that will store the intermediate canonical rate constants:

```

# =====
# Intermediate canonical rates
# =====

section("CANONICAL INTERMEDIATE RATES")

k_LZ_intermediate_canonical = {}
k_WC_intermediate_canonical = {}

```

When the intermediate Landau-Zener dictionary from Step 9 is available, each stored microcanonical rate curve is extracted and canonically averaged:


```

for key, data in k_LZ_intermediate.items():

    k_E = np.asarray(
        data["k"][1:],
        dtype=float
    ).reshape(-1)

    k_T, _ = boltzmann_average_microcanonical_rate(
        E_rate_LZ_cm1,
        rho_reactant_cm,
        k_E,
        T_grid_K,
        dE_LZ
    )

```

The corresponding intermediate Landau-Zener canonical rate is given by Equation S281:

$$k_{\text{LZ}}^{(i)}(T) = \frac{\sum_j k_{\text{LZ}}^{(i)}(E_j) \rho_R(E_j) e^{-E_j/(k_B T)} \Delta E}{\sum_j \rho_R(E_j) e^{-E_j/(k_B T)} \Delta E} \quad (\text{S281})$$

Each intermediate coupling therefore produces its own temperature-dependent canonical rate curve on the common temperature grid.

Each intermediate canonical Landau-Zener rate is stored together with the corresponding physical quantities:

```

k_LZ_intermediate_canonical[key] = {
    "H_int_cm1": data["H_int_cm1"],
    "abs_Ms_low": data["abs_Ms_low"],
    "degeneracy": data["degeneracy"],
    "k_T": k_T
}

```

The stored record therefore preserves the intermediate SOC magnitude, the absolute lower-spin projection, the associated degeneracy, and the complete temperature-dependent canonical rate array.

Retaining these quantities together preserves the physical identity of each intermediate spin-crossing pathway throughout the remainder of the workflow.

An entirely analogous procedure is subsequently performed for the weak coupling formalism:

```

for key, data in k_WC_intermediate.items():

    k_E = np.asarray(
        data["k"][1:],
        dtype=float
    ).reshape(-1)

    k_T, _ = boltzmann_average_microcanonical_rate(
        E_rate_WC_cm1,

```

```

    rho_reactant_cm,
    k_E,
    T_grid_K,
    dE_WC
)

k_WC_intermediate_canonical[key] = {
    "H_int_cm1": data["H_int_cm1"],
    "abs_Ms_low": data["abs_Ms_low"],
    "degeneracy": data["degeneracy"],
    "k_T": k_T
}

```

The weak-coupling intermediate canonical rate constants are obtained with the same canonical averaging expression, differing only in the underlying weak-coupling microcanonical rates from Step 9.

The same Boltzmann-averaging form is applied to the weak-coupling intermediate curves.

Upon completion of the averaging procedure, the total numbers of intermediate canonical Landau-Zener and weak coupling rate curves are reported:

```

print(
    f"LZ intermediate canonical curves : "
    f"{len(k_LZ_intermediate_canonical)}"
)

print(
    f"WC intermediate canonical curves : "
    f"{len(k_WC_intermediate_canonical)}"
)

```

These diagnostics confirm that the intermediate spin-orbit-coupling groups from Step 9 have been propagated through the canonical averaging procedure before the representative M_S-specific canonical rates are evaluated.

4.10.6 Evaluation of the symmetry-collapsed M_S-specific canonical rate constants

The final canonical kinetic calculations extend the Boltzmann averaging procedure to the symmetry-collapsed M_S-specific spin-crossing channels constructed in Step 9. Step 9 retains one representative microcanonical rate curve for each set of channels with identical absolute spin-orbit coupling magnitude. Step 10 canonical-averages each representative curve directly. Because all channels within a group have identical microcanonical and canonical rate curves, no summation or degeneracy scaling is performed.

The implementation begins by allocating dictionaries that will contain the representative canonical Landau-Zener and weak coupling rate curves:

```
# =====
# Symmetry-collapsed MS-specific canonical rates
# =====

section("CANONICAL SYMMETRY-COLLAPSED MS-SPECIFIC RATES")

k_LZ_channel_groups_canonical = {}
k_WC_channel_groups_canonical = {}
```

For every representative Landau-Zener channel group generated in Step 9, the implementation iterates over the stored group dictionary and extracts the representative microcanonical rate curve:

```
for key, data in k_LZ_channel_groups.items():
    k_group_E = np.asarray(
        data["k_representative"][1:],
        dtype=float
    ).reshape(-1)
```

The extracted representative curve is then converted to its canonical counterpart:

```
k_group_T,_ = boltzmann_average_microcanonical_rate(
    E_rate_LZ_cm1,
    rho_reactant_cm,
    k_group_E,
    T_grid_K,
    dE_LZ
)
```

The corresponding representative Landau-Zener canonical rate is given by Equation S282:

$$k_{LZ}^{(\text{group})}(T) = \frac{\sum_i k_{LZ}^{(\text{group})}(E_i) \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}} \quad (\text{S282})$$

Here, the group rate denotes the representative microcanonical rate for one symmetry-equivalent family of M_S-specific spin-transition channels.

Equal absolute spin-orbit coupling magnitudes within a group satisfy Equation S283:

$$|H_{SO,1}| = |H_{SO,2}| = \dots = |H_{SO,n}| \quad (\text{S283})$$

The corresponding canonical rates are therefore identical, as expressed by Equation S284:

$$k_1(T) = k_2(T) = \dots = k_n(T) \quad (\text{S284})$$

Canonical averaging of a single representative microcanonical curve therefore describes the temperature dependence shared by the entire symmetry-equivalent group.

The resulting representative canonical rate curve is then stored together with the associated physical information describing the symmetry group:

```

k_LZ_channel_groups_canonical[key] = {
    "H_abs_cm1": float(data["H_abs_cm1"]),
    "degeneracy": int(data.get("degeneracy", 1)),
    "channel_symbols": list(
        data.get("channel_symbols", [])
    ),
    "labels": list(data.get("labels", [])),
    "representative_label": data.get(
        "representative_label"
    ),
    "combination_rule": "representative_only_no_sum",
    "k_T": k_group_T
}

```

The stored record also retains the absolute SOC magnitude, equivalent-channel count, transition symbols, all channel labels, and the representative-channel identity. The combination-rule entry in the code block above explicitly records that the representative curve is not a summed channel contribution.

The representative canonical curve is retained for group-level plotting and reporting. It is not multiplied by the group degeneracy, and no canonical channel rates are summed or averaged.

The weak-coupling treatment uses the same representative-only procedure. The implementation iterates over the weak-coupling channel-group dictionary, extracts each representative microcanonical curve, and canonical-averages it:

```

for key, data in k_WC_channel_groups.items():
    k_group_E = np.asarray(data["k_representative"])[1:], dtype=float).reshape(-1)
    k_group_T, _ = boltzmann_average_microcanonical_rate(
        E_rate_WC_cm1,
        rho_reactant_cm,
        k_group_E,
        T_grid_K,
        dE_WC
    )

```

The resulting canonical curve is stored in the corresponding representative weak-coupling dictionary:

```

k_WC_channel_groups_canonical[key] = {
    "H_abs_cm1": float(data["H_abs_cm1"]),
    "degeneracy": int(data.get("degeneracy", 1)),
    "combination_rule": "representative_only_no_sum",
    "k_T": k_group_T
}

```

Thus, both the Landau-Zener and weak-coupling formalisms preserve the representative-only symmetry grouping established in Step 9.

Finally, the program reports the total numbers of representative canonical rate curves generated for each theoretical treatment:

```
print(
    f"Representative LZ MS-specific canonical curves : "
    f"{len(k_LZ_channel_groups_canonical)}"
)

print(
    f"Representative WC MS-specific canonical curves : "
    f"{len(k_WC_channel_groups_canonical)}"
)

print(
    "MS-specific canonical combination rule : "
    "no summation"
)
```

These diagnostic messages confirm that the canonical workflow preserves the same symmetry-grouping philosophy introduced in Step 9. The reported representative curves are intended only to eliminate redundant overlapping plots arising from symmetry-equivalent spin-transition channels. The underlying canonical reaction kinetics remain completely unchanged, and no physical contributions are combined during this stage of the calculation.

4.10.7 Generation of canonical rate plots

Step 10 generates two canonical-rate figures. The first plots $k(T)$ directly versus temperature. The second uses $1000/T$ on the horizontal axis and $\log_{10}[k(T)/s^{-1}]$ on the vertical axis. Both figures contain the effective, intermediate, and representative M_S-specific LZ/WC curves.

The program first creates a dedicated output directory for the canonical calculations and registers the filenames that will be used to store the generated figures:

```
local_step10 = os.path.join(
    local_base,
    "Canonical_rates"
)

os.makedirs(local_step10, exist_ok=True)

def register_plot(name):
    ...
```

Output-directory handling differs between the Local PC and HPC implementations:

In the Local PC implementation:

```
remote_step10 = local_step10
```

In the HPC implementation:

```
remote_step10 = posixpath.join(remote_base, "Canonical_rates")
```

The first figure presents the canonical reaction rate constants directly as functions of temperature:

```
plt.figure(figsize=(9.2,5.2))

plt.plot(
    T_grid_K,
    k_LZ_canonical_s_inv,
    ...
)

plt.plot(
    T_grid_K,
    k_WC_canonical_s_inv,
    ...
)
```

The effective Landau-Zener and weak-coupling canonical rates are displayed together with the intermediate canonical curves and representative M_S-specific canonical curves. Intermediate curves are distinguished by their intermediate SOCs, whereas representative M_S-specific curves are labeled with the corresponding spin-transition symbols. Only one curve is plotted for each symmetry-equivalent group.

The first figure therefore compares the effective, intermediate, and representative M_S-specific canonical rate families over the complete user-defined temperature range.

The second figure is generated as a log-scale reciprocal-temperature plot using the positive-rate masks:

```
mask_lz = k_LZ_canonical_s_inv > 0.0

plt.plot(
    1000.0 / T_grid_K[mask_lz],
    np.log10(k_LZ_canonical_s_inv[mask_lz]),
    ...
)
mask_wc = k_WC_canonical_s_inv > 0.0
plt.plot(
    1000.0 / T_grid_K[mask_wc],
    np.log10(k_WC_canonical_s_inv[mask_wc]),
    ...
)
```

Only positive canonical rate constants are included before applying log10. The horizontal axis is $1000/T$ (K^{-1}), and the vertical axis is $\log_{10}[k(T)/s^{-1}]$. The plotted title is "Log-scale Canonical Effective, Intermediate, and MS-specific LZ/WC Rates."

Together, the linear $k(T)$ figure and the log-scale reciprocal-temperature figure provide complementary views of the same canonical rate data without introducing an additional kinetic model.

4.10.8 Export of canonical results and output files

After the canonical calculations and plots are complete, Step 10 writes four numerical/text outputs and exports the canonical variables required by the later workflow.

The principal output-file paths are defined explicitly as:

```
canonical_rate_file = os.path.join(local_step10, "Step10_canonical_LZ_WC_rates.txt")
summary_file_step10 = os.path.join(local_step10, "Step10_canonical_rate_summary.txt")
intermediate_canonical_file_step10 = os.path.join(local_step10,
"Step10_intermediate_canonical_rates.txt")
representative_canonical_file_step10 = os.path.join(local_step10,
"Step10_representative_MS_canonical_rates.txt")
```

These four files contain the effective canonical table, a calculation summary, the intermediate canonical rates, and the representative M_S-specific canonical rates, respectively.

The effective canonical table is assembled from the following columns:

```
table_columns = [
    T_grid_K, 1000.0 / T_grid_K, k_LZ_canonical_s_inv, k_WC_canonical_s_inv,
    LZ_canonical_normalization, WC_canonical_normalization
]
canonical_rate_table = np.column_stack(table_columns)
```

The resulting table contains temperature, reciprocal temperature, the effective LZ/WC canonical rates, and the two normalization arrays. It is written to disk using:

```
np.savetxt(
    canonical_rate_file,
    canonical_rate_table,
    ...
)
```

providing a complete numerical description of the effective canonical reaction kinetics over the selected temperature interval.

The intermediate and representative-group files preserve the SOC magnitude and identifying metadata together with the canonical-rate ranges. The representative-group output explicitly records that no summation is performed.

The Local PC implementation performs no SFTP transfer. The HPC implementation constructs remote target paths and transfers the four text files and each generated plot:

```
remote_canonical_rate_file = posixpath.join(remote_step10,
os.path.basename(canonical_rate_file)) # HPC
sftp.put(canonical_rate_file, remote_canonical_rate_file) # HPC
for local_path in plot_files_step10.values():
    remote_path = posixpath.join(remote_step10, os.path.basename(local_path)) # HPC
    sftp.put(local_path, remote_path) # HPC
```

The principal canonical quantities exported for the later workflow include:

```
T_grid_K
k_LZ_canonical_s_inv
k_WC_canonical_s_inv
k_LZ_intermediate_canonical
k_WC_intermediate_canonical
k_LZ_channel_groups_canonical
k_WC_channel_groups_canonical
```

The exported workspace also retains the normalization arrays, helper functions, output paths, and plotting metadata required by the subsequent threshold-truncation and reporting steps.

4.10.9 Summary of the step 10 algorithm

Step 10 transforms the Step 9 microcanonical reaction-rate hierarchy into temperature-dependent canonical rates by Boltzmann averaging over the complete reactant rovibrational manifold.

For summary purposes, the reactant normalization integral is written in Equation S285:

$$Q_R(T) = \sum_i \rho_R(E_i) \exp\left(-\frac{E_i}{k_B T}\right) \Delta E \quad (\text{S285})$$

The corresponding canonical reaction rate is written in Equation S286:

$$k(T) = \frac{\sum_i k(E_i) \rho_R(E_i) e^{-E_i/(k_B T)} \Delta E}{Q_R(T)} \quad (\text{S286})$$

This averaging procedure is applied independently to the effective and intermediate Landau-Zener and weak coupling microcanonical rate curves from Step 9.

Representative M_S-specific canonical rate curves retain the symmetry grouping introduced in Step 9.

For each group of channels with equal absolute spin-orbit coupling magnitude, the stored group curve is the selected representative-channel curve, as summarized by Equation S287:

$$k_{group}(T) = k_{rep}(T) \quad (S287)$$

Only one representative canonical curve is retained for each symmetry-equivalent group. No summation, averaging, weighting, or degeneracy scaling of M_S-specific canonical rate constants is performed.

The computational sequence implemented in Step 10 is summarized schematically below:

Step 9 microcanonical rates

↓
 Temperature grid
 (minimum temperature, maximum temperature, number of points)
 ↓
 Boltzmann weighting
 $\exp(-E/kBT)$
 ↓
 Reactant partition function
 $Q_R(T)$
 ↓
Effective canonical
LZ/WC rates
 ↓
Intermediate
canonical rates
 ↓
 Representative M_S-specific
 canonical rates
canonical rates
 ↓
Canonical plots
 ($k(T)$, log-scale reciprocal-temperature)
 ↓
Output tables,
summary files,
exported variables,
 and HPC transfer when applicable

Step 10 therefore converts the Step 9 microcanonical hierarchy into temperature-dependent canonical rates using the same Boltzmann-weighting routine for effective, intermediate, and representative M_S-specific LZ/WC curves. No symmetry-equivalent channel rates are summed, and the resulting arrays are passed to the subsequent WC threshold-truncation and final-reporting steps.

4.11 Step 11: Conditional WC Threshold-Truncation Correction

Step 11 provides a conditional correction procedure for weak coupling probabilities that reach or exceed unity. The weak coupling expression used in Step 8 is an analytical approximation and may produce values greater than or equal to one under sufficiently strong coupling or within particular energy regions. Because a transition probability cannot physically exceed unity, SPINKS evaluates the underlying, unclipped weak coupling probability and determines whether the threshold-truncation rule must be applied.

When the raw weak coupling probability remains below unity over the complete energy grid, Step 11 performs no correction and preserves all Step 8–10 results unchanged. When the raw probability reaches or exceeds unity, the probability is set to zero beginning at the first unphysical energy point and continuing through the remainder of the energy grid. The corrected probability arrays are then propagated through the complete weak coupling kinetic treatment, including the effective, intermediate, symmetry-collapsed M_s -specific, microcanonical, and canonical rate calculations. All recomputed quantities use the CORRECTED_ prefix, while the original clipped Step 8–10 results remain available for direct comparison.

4.11.1 Import of required quantities and conditional workflow setup

Step 11 begins by verifying the quantities required to reconstruct the raw weak coupling probability and recompute the corresponding rate constants. The Local PC implementation requires the following variables; the HPC implementation requires the same set plus one remote-path variable:

```
required_vars_step105 = [  
    "jobname",  
    "total_energy_grid_cm1",  
    "E_MECP",  
    "H_SO_cm",  
    "autocm",  
    "mu_au",  
    "projected_gradient_mean",  
    "gradient_gap_magnitude",  
    "compute_weak_coupling_microcanonical_rate",  
    "effective_WC_probability",  
    "effective_WC_rate",  
    "effective_WC_cumulative_states",  
    "E_rate_WC_cm1",  
    "rho_reactant_cm",  
    "local_base",  
]
```

Additional required variable in the HPC implementation:

```
remote_base
```

```
for var in required_vars_step105:  
    if var not in globals():  
        raise RuntimeError(  
            f'{var} is missing. Run Steps 8–10 first.'  
        )
```

The imported quantities include the common total-energy grid, the MECP threshold, the effective spin-orbit coupling, the reduced mass, the projected-gradient and gradient-gap quantities, the previously calculated clipped weak coupling probability and rate arrays, and the reactant density of states required for canonical averaging.

The workflow and execution modes are then recovered:

In the Local PC implementation:

```
RUN_MODE = globals().get("RUN_MODE", "LOCAL")
```

In the HPC implementation:

```
RUN_MODE = globals().get("RUN_MODE", "CLUSTER")
```

Both implementations use:

```
workflow_mode = globals().get("workflow_mode", "MECP_ONLY")
```

The execution-mode difference affects path and transfer bookkeeping only; the Step 11 scientific procedure is the same in both implementations.

Step 11 additionally defines helper routines for section formatting, file writing, figure saving, spin-projection notation, and construction of representative-channel legends:

```
def section(title):  
    ...  
  
def write_local_text(path, text):  
    ...  
  
def save_current_figure(path):  
    ...  
  
def ms_tex_value(x):  
    ...  
  
def make_corrected_group_legend(  
    prefix,  
    group_data  
):  
    ...
```

The legend helper preserves the same symmetry-grouping convention used in Steps 9 and 10. Multiple symbols indicate coincident curves arising from channels with identical $|H_{SO}|$ values, and no summation is implied.

4.11.2 Reconstruction of the raw weak coupling probability

The correction decision is based on the raw, unclipped weak coupling probability rather than the probability array previously constrained to the physical interval. Step 11 therefore reconstructs the underlying weak coupling expression directly from the effective spin–orbit coupling and the common energy grid:

```
def compute_raw_weak_coupling_probability(
    coupling_cm1,
    energy_grid_cm1
):
    energy_grid_cm1 = np.asarray(
        energy_grid_cm1,
        dtype=float
    ).reshape(-1)

    coupling_Eh = float(abs(coupling_cm1)) / float(autocm)

    excess_energy_Eh = (
        energy_grid_cm1 - float(E_MECP)
    ) / float(autocm)
```

The absolute spin–orbit coupling is converted from cm^{-1} to Hartree according to Equation S288.

$$H_{\text{SO}}^{\text{Eh}} = \frac{|H_{\text{SO}}^{\text{cm}^{-1}}|}{c_{\text{Eh} \rightarrow \text{cm}^{-1}}} \quad (\text{S288})$$

The denominator in this conversion is the Hartree-to-wavenumber conversion factor used by the code.

The energy relative to the MECP is similarly converted to Hartree according to Equation S289:

$$E_{\text{excess}} = \frac{E - E_{\text{MECP}}}{c_{\text{Eh} \rightarrow \text{cm}^{-1}}} \quad (\text{S289})$$

The raw weak coupling probability is evaluated using the Airy-function expression implemented in the code:

```
prefactor = (
    4.0
    * np.pi**2
    * coupling_Eh**2
    * (
        2.0 * float(mu_au)
        / (
            float(projected_gradient_mean)
            * float(gradient_gap_magnitude)
        )
    )
```

```

    )
  ) ** (2.0 / 3.0)
)

scale = (
  2.0
  * float(mu_au)
  * float(gradient_gap_magnitude)**2
  / float(projected_gradient_mean)**4
) ** (1.0 / 3.0)

```

The corresponding prefactor is given by Equation S290.

$$A_{WC} = 4\pi^2 H_{SO}^2 \left[\frac{2\mu}{\bar{g} \Delta g_{\parallel}} \right]^{2/3} \quad (S290)$$

Here, μ is the reduced mass, $\bar{g}$ is the projected-gradient mean, and $\Delta g_{\parallel}$ is the gradient-gap magnitude.

All quantities entering the raw weak-coupling expression are evaluated in atomic units. Therefore, $\hbar = 1$ and does not appear explicitly in Equations S290–S292.

The Airy-function argument is defined by Equation S291.

$$x(E) = -(E - E_{MECP}) \left[\frac{2\mu \Delta g_{\parallel}^2}{\bar{g}^4} \right]^{1/3} \quad (S291)$$

The code evaluates the Airy function as:

```

airy_argument[valid] = -excess_energy_Eh[valid] * scale

airy_value[valid] = airy(airy_argument[valid])[0]

raw_probability[valid] = (
  prefactor * airy_value[valid] ** 2
)

```

The raw weak coupling probability is evaluated according to Equation S292:

$$P_{WC}^{\text{raw}}(E) = A_{WC} \text{Ai}^2[x(E)] \quad (S292)$$

Any non-finite numerical entries are then replaced with zero:

```
raw_probability[~np.isfinite(raw_probability)] = 0.0
```

No clipping to unity is performed at this stage. This is essential because the purpose of Step 11 is to determine whether the analytical expression itself reaches or exceeds the physical probability limit.

The routine returns:

```
return raw_probability, airy_argument, airy_value
```

including the raw probability, the Airy-function argument, and the Airy-function values. Retaining these intermediate quantities allows the complete mathematical behavior of the weak coupling expression to be inspected and exported.

4.11.3 Application of the weak coupling threshold-truncation rule

The threshold-truncation rule is implemented by a dedicated helper routine:

```
def apply_probability_threshold_truncation(
    raw_probability,
    energy_grid_cm1,
    threshold=1.0
):
```

The routine first verifies that the probability and energy arrays have identical lengths:

```
if raw_probability.size != energy_grid_cm1.size:
    raise RuntimeError(
        "raw_probability and energy_grid_cm1 must have the same length."
    )
```

The indices at which the raw probability reaches or exceeds the selected threshold are then identified:

```
threshold_indices = np.flatnonzero(
    raw_probability >= float(threshold)
)
```

The probability threshold is fixed at unity, as shown in Equation S293:

$$P_{\text{threshold}} = 1 \quad (\text{S293})$$

The correction flag is then set according to whether any threshold-crossing indices are found:

```
correction_needed = threshold_indices.size > 0
```

The first unphysical point is the first index i satisfying Equation S294:

$$P_{\text{WC}}^{\text{raw}}(E_i) \geq 1 \quad (\text{S294})$$

If no such point exists, the original clipped workflow remains active. If one or more such points exist, the first offending energy is recorded and the threshold-truncation rule is applied:

```

if correction_needed:
    first_bad_index = int(threshold_indices[0])
    first_bad_energy = float(
        energy_grid_cm1[first_bad_index]
    )
    first_bad_value = float(
        raw_probability[first_bad_index]
    )

    corrected_probability[first_bad_index:] = 0.0

```

The corrected probability is therefore given by Equation S295.

$$P_{WC}^{corr}(E) = \begin{cases} P_{WC}^{raw}(E), & E < E_{first} \\ 0, & E \geq E_{first} \end{cases} \quad (S295)$$

The threshold energy is the first energy at which Equation S296 is satisfied:

$$P_{WC}^{raw}(E) \geq 1 \quad (S296)$$

The result is subsequently restricted to the physical interval:

```

corrected_probability[
    ~np.isfinite(corrected_probability)
] = 0.0
corrected_probability = np.clip(
    corrected_probability,
    0.0,
    1.0)

```

The routine returns a structured dictionary containing the corrected probability, the correction flag, the first unphysical index, the first unphysical energy, the corresponding raw probability, and the threshold:

```

return {
    "corrected_probability": corrected_probability,
    "correction_needed":
        correction_needed,
    "first_bad_index":
        first_bad_index,
    "first_bad_energy":
        first_bad_energy,
    "first_bad_value":
        first_bad_value,
    "threshold":
        float(threshold),
}

```

This design allows the same convention to be applied consistently to the effective, intermediate, and M_s -specific weak coupling channels.

4.11.4 Conditional evaluation of the threshold-corrected effective weak coupling rate

The raw effective weak coupling probability is first reconstructed using the effective spin–orbit coupling:

```
P_WC_raw_effective, airy_arg_WC_raw_effective, Ai_WC_raw_effective = (  
    compute_raw_weak_coupling_probability(H_SO_cm, total_energy_grid_cm1)  
)
```

The threshold-truncation rule is then tested:

```
effective_correction_result = apply_probability_threshold_truncation(  
    P_WC_raw_effective,  
    total_energy_grid_cm1,  
    threshold=1.0  
)
```

The principal diagnostic values are extracted:

```
WC_threshold_correction_needed = effective_correction_result["correction_needed"]  
  
WC_threshold_first_bad_index = effective_correction_result["first_bad_index"]  
  
WC_threshold_first_bad_energy = effective_correction_result["first_bad_energy"]  
  
WC_threshold_first_bad_value = effective_correction_result["first_bad_value"]
```

These values are exported immediately so that later workflow stages can determine whether the correction was activated.

No-correction branch

When the raw effective probability never reaches unity, the code terminates the correction workflow without recomputing rates:


```

if not WC_threshold_correction_needed:

    print(
        "Raw WC probability "
        "never reached 1."
    )

    print(
        "threshold-corrected WC correction "
        "is NOT needed."
    )

    print(
        "No CORRECTED-prefixed "
        "variables were created."
    )
print("No WC recomputation, plotting, saving, or uploading was performed.")

```

In this branch:

- no CORRECTED-prefixed probability arrays are generated,
- no weak coupling microcanonical rates are recomputed,
- no canonical rates are recomputed,
- no figures are produced,
- no correction files are saved,
- and no remote uploads are attempted.

This preserves the original Step 8–10 results exactly when the analytical weak coupling expression remains physically acceptable.

Correction branch

When the raw effective probability reaches or exceeds unity, the corrected effective probability is obtained as:

```
CORRECTED_P_WC_effective = effective_correction_result["corrected_probability"]
```

The corresponding effective number of crossing states and microcanonical rate constant are recomputed:

```

CORRECTED_N_WC_effective, CORRECTED_k_WC_effective = (
    compute_weak_coupling_microcanonical_rate(CORRECTED_P_WC_effective)
)

```

The corrected cumulative number of crossing states is given by Equation S297:

$$N_{WC}^{corr}(E) = \sum_j P_{WC}^{corr}(E_j) \rho_{TP}(E - E_j) \Delta E \quad (S297)$$

The corresponding corrected microcanonical rate constant is given by Equation S298:

$$k_{WC}^{corr}(E) = \frac{g_R N_{WC}^{corr}(E)}{h \rho_R(E)} \quad (S298)$$

The corrected arrays are assigned several compatible names:

```
CORRECTED_k_WC_effective_micro_s_inv = CORRECTED_k_WC_effective
```

```
CORRECTED_rate_WC_effective = CORRECTED_k_WC_effective
```

```
CORRECTED_states_WC_effective = CORRECTED_N_WC_effective
```

The code then reports the maximum original clipped probability, maximum raw probability, maximum corrected probability, original microcanonical rate, and corrected microcanonical rate. This provides an immediate numerical comparison between the original clipped treatment and the threshold-truncation treatment.

4.11.5 Correction of intermediate weak coupling probabilities and rates

When effective correction is required, Step 11 applies the same procedure to all intermediate weak coupling channels generated in Steps 8 and 9:

```
CORRECTED_P_WC_intermediate = {}
CORRECTED_N_WC_intermediate = {}
CORRECTED_k_WC_intermediate = {}
```

For each intermediate coupling:

```
for key, data in \
    k_WC_intermediate.items():

    H_int = float(
        data["H_int_cm1"]
    )
```

the raw probability is reconstructed:

```
P_raw, airy_arg_raw, Ai_raw = (
    compute_raw_weak_coupling_probability(
        H_int,
        total_energy_grid_cm1
    )
)
```

The threshold-truncation rule is then applied independently:

```

result = apply_probability_threshold_truncation(
    P_raw,
    total_energy_grid_cm1,
    threshold=1.0
)

```

```
corrected_probability = result["corrected_probability"]
```

The corrected number of crossing states and microcanonical rate constant are evaluated through the same weak coupling RRKM routine:

```

corrected_states, corrected_rate = (
    compute_weak_coupling_microcanonical_rate(
        corrected_probability
    )
)

```

For each intermediate channel, the correction sequence is:

$$P_{WC,i}^{\text{raw}}(E) \rightarrow P_{WC,i}^{\text{corr}}(E) \rightarrow N_{WC,i}^{\text{corr}}(E) \rightarrow k_{WC,i}^{\text{corr}}(E).$$

The probability dictionary stores both corrected and uncorrected quantities:

```

CORRECTED_P_WC_intermediate[key] = {
    "H_int_cm1": H_int,
    "abs_Ms_low":
        float(data["abs_Ms_low"]),
    "degeneracy":
        int(data["degeneracy"]),
    "P_raw": P_raw,
    "corrected_probability": corrected_probability,
    "airy_arg_raw": airy_arg_raw,
    "Ai_raw": Ai_raw,
    "correction_needed":
        result["correction_needed"],
    "first_bad_index":
        result["first_bad_index"],
    "first_bad_energy":
        result["first_bad_energy"],
    "first_bad_value":
        result["first_bad_value"],
}

```

The corrected kinetic quantities are stored separately:

```

CORRECTED_k_WC_intermediate[key] = {
    "H_int_cm1": H_int,
    "abs_Ms_low":
        float(data["abs_Ms_low"]),
    "degeneracy":
        int(data["degeneracy"]),
    "corrected_states": corrected_states,
    "corrected_rate": corrected_rate,
}

```

```

"corrected_probability": corrected_probability,
"P_raw": P_raw,
"correction_needed":
    result["correction_needed"],
"first_bad_index":
    result["first_bad_index"],
"first_bad_energy":
    result["first_bad_energy"],
"first_bad_value":
    result["first_bad_value"],
}

```

Importantly, the effective probability determines whether Step 11 is activated globally, but each intermediate channel retains its own correction flag. Because intermediate couplings may differ in magnitude, some intermediate channels may reach unity while others may remain below it. The code therefore applies the correction independently to every intermediate coupling rather than assuming identical behavior.

4.11.6 Correction of symmetry-collapsed M_s -specific weak coupling channels

Step 11 next extends the downstream statistical treatment to the symmetry-collapsed M_s -specific weak coupling groups constructed in Step 9:

```

CORRECTED_P_WC_channel_groups = {}
CORRECTED_N_WC_channel_groups = {}
CORRECTED_k_WC_channel_groups = {}

```

For each representative group, the absolute spin-orbit coupling and the corresponding channel metadata are retrieved:

```

for group_key, group_data in \
    k_WC_channel_groups.items():

    H_abs = float(
        group_data["H_abs_cm1"]
    )

    labels = list(
        group_data.get("labels", [])
    )

    symbols = list(
        group_data.get(
            "channel_symbols",
            []
        )
    )

    representative_label = (
        group_data.get(

```

```

        "representative_label"
    )
)

```

The raw weak coupling probability is then reconstructed using the group-specific coupling:

```

P_raw, airy_arg_raw, Ai_raw = (
    compute_raw_weak_coupling_probability(
        H_abs,
        total_energy_grid_cm1
    )
)

```

The threshold-truncation rule is applied independently to the representative group:

```

result = apply_probability_threshold_truncation(
    P_raw,
    total_energy_grid_cm1,
    threshold=1.0
)

corrected_probability = result["corrected_probability"]

```

The representative corrected number of crossing states and rate constant are recomputed:

```

corrected_representative_states, \
corrected_representative_rate = (
    compute_weak_coupling_microcanonical_rate(
        corrected_probability
    )
)

```

For each representative group, the correction sequence is:

$$P_{WC,g}^{\text{raw}}(E) \rightarrow P_{WC,g}^{\text{corr}}(E) \rightarrow N_{WC,g}^{\text{corr}}(E) \rightarrow k_{WC,g}^{\text{corr}}(E).$$

The corrected representative data are stored together with all channel labels and symmetry information:

```

CORRECTED_k_WC_channel_groups[
    group_key
] = {
    "H_abs_cm1": H_abs,
    "degeneracy": int(
        group_data.get(
            "degeneracy",
            len(labels)
        )
    ),
    "channel_symbols": symbols,
    "labels": labels,
    "representative_label":
        representative_label,
    "corrected_representative_states":

```

```

    corrected_representative_states,
    "corrected_representative_rate":
        corrected_representative_rate,
    "corrected_probability": corrected_probability,
    "P_raw": P_raw,
    "correction_needed":
        result["correction_needed"],
    "first_bad_index":
        result["first_bad_index"],
    "first_bad_energy":
        result["first_bad_energy"],
    "first_bad_value":
        result["first_bad_value"],
    "combination_rule":
        "representative_only_no_sum",
}

```

As in Steps 9 and 10, channels possessing identical $|H_{SO}|$ values generate identical probability and rate curves. Therefore, one representative corrected curve is retained for each group. The explicit rule is:

```
representative_only_no_sum
```

confirms that no probabilities, numbers of states, microcanonical rates, or canonical rates are summed or multiplied by the number of equivalent channels.

4.11.7 Recalculation of canonical weak coupling rate constants

When the Step 10 canonical averaging routine and temperature variables are available, Step 11 recomputes the corrected canonical weak coupling rates:

```

CORRECTED_k_WC_canonical_s_inv = None
CORRECTED_WC_canonical_normalization = None

CORRECTED_k_WC_intermediate_canonical = {}
CORRECTED_k_WC_channel_groups_canonical = {}

```

The corrected effective microcanonical rate is canonically averaged using the same expression introduced in Step 10:

```

CORRECTED_k_WC_canonical_s_inv, \
CORRECTED_WC_canonical_normalization = (
    boltzmann_average_microcanonical_rate(
        E_rate_WC_cm1,
        rho_reactant_cm,
        CORRECTED_k_WC_effective[1:],
        T_grid_K,
        dE_WC
    )
)

```

The corrected canonical rate is therefore given by Equation S299.

$$k_{\text{WC}}^{\text{corr}}(T) = \frac{\sum_i k_{\text{WC}}^{\text{corr}}(E_i) \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}}{\sum_i \rho_R(E_i) e^{-E_i/(k_B T) \Delta E}} \quad (\text{S299})$$

The first element of the corrected microcanonical array is removed before canonical averaging so that its dimensions match the Step 10 physical energy grid.

Every corrected intermediate microcanonical curve is treated analogously:

```

for key, data in \
    CORRECTED_k_WC_intermediate.items():

    k_T, _ = (
        boltzmann_average_microcanonical_rate(
            E_rate_WC_cm1,
            rho_reactant_cm,
            data["corrected_rate"][1:],
            T_grid_K,
            dE_WC
        )
    )

```

The resulting dictionary retains the intermediate coupling, spin projection, degeneracy, corrected canonical rate, correction flag, and first unphysical energy.

Representative M_s -specific groups are also canonically averaged:

```

for key, data in \
    CORRECTED_k_WC_channel_groups.items():

    k_T, _ = (
        boltzmann_average_microcanonical_rate(
            E_rate_WC_cm1,
            rho_reactant_cm,
            data[
                "corrected_representative_rate"
            ][1:],
            T_grid_K,
            dE_WC
        )
    )

```

Each representative canonical entry preserves

- the absolute SOC magnitude,
- equivalent-channel count,
- symbols,
- labels,
- representative identity,
- correction status,
- first bad energy,
- and the no-summation rule.

If Step 10 variables are unavailable, canonical correction is skipped while the corrected probability and microcanonical results remain available.

4.11.8 Diagnostics, output files, plots, and exported variables

Selected-energy diagnostic

Step 11 prints a direct comparison at selected energies:

```
selected_energies = [  
  3000,  
  3500,  
  3560,  
  3566,  
  3600,  
  4000,  
  5000,  
  6000,  
  8000  
]
```

For each selected energy, the code reports

- raw unclipped weak coupling probability,
- original clipped probability,
- corrected threshold-truncated probability,
- original clipped microcanonical rate,
- and corrected microcanonical rate.

This diagnostic makes the onset and consequences of the correction directly visible at representative energy points.

Output files

When correction is required, Step 11 creates the common local directory:

```
local_step11 = os.path.join(local_base, "WC_threshold_truncation_correction")
```

In the Local PC implementation:

```
remote_step11 = local_step11
```

In the HPC implementation:

```
remote_step11 = posixpath.join(remote_base, "WC_threshold_truncation_correction")
```

Thus, the Local PC version uses a local compatibility alias, whereas the HPC version creates and uses a separate remote directory when the remote utilities are available. Four principal text files are generated:

```
summary_file_step11 = os.path.join(local_step11, "Step11_CORRECTED_WC_summary.txt")
effective_grid_file_step11 = os.path.join(local_step11,
"Step11_CORRECTED_WC_effective_grid.txt")
intermediate_file_step11 = os.path.join(local_step11,
"Step11_CORRECTED_WC_intermediate_summary.txt")
channel_file_step11 = os.path.join(local_step11,
"Step11_CORRECTED_WC_representative_channel_summary.txt")
```

The summary file records the effective SOC, MECP energy, correction flag, first unphysical energy and probability, maximum original and corrected probabilities, maximum microcanonical rates, maximum canonical rates, and counts of corrected intermediate and representative channels.

The effective-grid file contains the full energy-dependent comparison:

$$E, P_{WC}^{raw}, P_{WC}^{clipped}, P_{WC}^{corr}, \\ N_{WC}^{clipped}, N_{WC}^{corr}, k_{WC}^{clipped}, k_{WC}^{corr}.$$

The intermediate summary records each coupling, spin projection, degeneracy, correction status, first unphysical energy, first unphysical probability, and maximum corrected rate.

The representative M_s -specific summary records each group coupling, equivalent-channel count, channel symbols, representative label, correction status, first unphysical point, and maximum representative corrected rate.

Probability and rate plots

Step 11 generates a probability comparison plot containing:

```
plt.plot(
    total_energy_grid_cm1,
    effective_WC_probability,
    label=rf"Clipped  $P_{\{WC\}}^{\{eff\}}$  ( $H_{\{\mathrm{SO}\}}^{\{eff\}} = H_{SO\_cm:.1f}$ )"
)

plt.plot(
    total_energy_grid_cm1,
    P_WC_raw_effective,
    label=rf"Raw WC unclipped ( $H_{\{\mathrm{SO}\}}^{\{eff\}} = H_{SO\_cm:.1f}$ )"
)

plt.plot(
    total_energy_grid_cm1,
    CORRECTED_P_WC_effective,
    label=rf"Threshold-truncated  $P_{\{WC\}}^{\{eff\}}$  ( $H_{\{\mathrm{SO}\}}^{\{eff\}} = H_{SO\_cm:.1f}$ )"
)
```

The plot additionally marks

- $P = 1$,
- the MECP energy,
- and the first energy at which the raw probability reaches unity.

A microcanonical-rate comparison plot is also generated:

```
plt.plot(
    total_energy_grid_cm1[mask_py],
    np.log10(effective_WC_rate[mask_py]),

    label=rf"Clipped  $k_{\{WC\}}^{\{eff\}}(E)$  ( $H_{\{\mathrm{SO}\}}^{\{eff\}} = H_{SO\_cm:.1f}$ )"
)

plt.plot(
    total_energy_grid_cm1[mask_corrected],
    np.log10(
        CORRECTED_k_WC_effective[mask_corrected]),

    ),
    label=rf"Threshold-truncated  $k_{\{WC\}}^{\{eff\}}(E)$  ( $H_{\{\mathrm{SO}\}}^{\{eff\}} = H_{SO\_cm:.1f}$ )"
)
```

Only positive rates are plotted logarithmically.

When canonical correction is available, two additional figures are produced:

1. $k_{WC}(T)$ versus temperature;
2. $\log_{10}[k_{WC}(T)]$ versus $1000/T$.

These plots compare the original clipped canonical weak coupling rate with the threshold-truncated canonical rate. The current code also writes both linear- and log-scale microcanonical and canonical comparison figures.

Remote transfer

For cluster calculations, all text files and figures are transferred using SFTP to the corresponding remote correction directory. If the transfer fails, the local files remain available and the remote-path variables are reset to None.

Exported variables

The complete corrected data set is exported through:

```
globals().update({  
    ...  
})
```

Important exported quantities include:

```
P_WC_raw_effective  
CORRECTED_P_WC_effective  
CORRECTED_N_WC_effective  
CORRECTED_k_WC_effective  
CORRECTED_k_WC_effective_micro_s_inv  
CORRECTED_k_WC_intermediate  
CORRECTED_k_WC_channel_groups  
CORRECTED_k_WC_canonical_s_inv  
CORRECTED_k_WC_intermediate_canonical  
CORRECTED_k_WC_channel_groups_canonical  
WC_threshold_correction_needed  
WC_threshold_first_bad_index  
WC_threshold_first_bad_energy  
WC_threshold_first_bad_value
```

The original clipped Step 8–10 variables are not overwritten. This permits Step 12 to select explicitly between the original clipped weak coupling results and the threshold-truncated results.

4.11.9 Summary of the step 11 algorithm

Step 11 conditionally applies the threshold-truncation rule when the analytical weak coupling probability becomes unphysical. The raw weak coupling probability is reconstructed from the spin–orbit coupling, reduced mass, projected-gradient mean, gradient difference, MECP energy, and Airy-function relations given in Equations S288–S292.

The first threshold-crossing point and its energy are identified according to Equations S293–S296.

When no such energy exists, Step 11 terminates without modifying the Step 8–10 probability or rate arrays. No corrected rates, figures, or files are generated.

When the threshold is reached, the corrected probability is defined by Equation S295.

The corrected effective number of crossing states and microcanonical rate constant are then recomputed according to Equations S297 and S298.

The same correction is applied independently to every intermediate weak coupling channel and every symmetry-collapsed M_s -specific representative channel.

Channels with identical $|H_{SO}|$ values retain a single representative corrected curve. No channel probabilities, numbers of states, microcanonical rates, or canonical rates are summed.

When Step 10 variables are available, all corrected microcanonical rates are further transformed into canonical rate constants according to Equation S299.

The complete Step 11 workflow can therefore be summarized as:

Raw WC probability from the Airy expression

↓

Does the raw WC probability reach or exceed unity?

↓

No → preserve the original clipped WC results; no corrected rates or files are generated

Yes → identify the first energy at which $P_{WC}^{raw} \geq 1$

↓

Set the corrected WC probability to zero from that energy onward

↓

Recompute the effective WC microcanonical rate

↓

Recompute intermediate WC probabilities and rates

↓

Recompute representative M_s -specific WC rates without summation

↓

Perform canonical averaging when Step 10 variables are available

↓

Generate comparison plots, diagnostics, output files, and CORRECTED_-prefixed variables Accordingly, Step 11 serves as a conditional consistency layer between the analytical weak coupling probability and the threshold-truncation rule. It preserves the original clipped treatment whenever the raw probability remains below unity; when the threshold is reached, it constructs a complete parallel hierarchy of corrected effective, intermediate, and M_s -specific weak coupling probabilities, microcanonical rates, and canonical rates without overwriting the original results.

4.12 Step 12: Final Results Reporting, Diagnostics, and Optional NAST Interoperability Export

Step 12 completes the SPINKS workflow by consolidating the principal physical quantities and kinetic results generated throughout the preceding stages into a final human-readable report. It also provides an interactive energy-resolved diagnostic for direct inspection of the Landau-Zener and weak coupling probabilities, numbers of states, and microcanonical rate constants at user-selected energies.

A second major function of Step 12 is the automatic generation of two complete NAST input files. One input uses the electronic MECP barrier together with internal NAST zero-point-energy treatment, whereas the second uses the externally calculated ZPE-corrected MECP barrier and disables additional NAST ZPE correction. The two files therefore provide equivalent but independently testable representations of the barrier convention.

Step 12 also determines whether the original clipped weak coupling results from Steps 8–10 or the threshold-corrected weak coupling results from Step 11 must be used in the final report. The original arrays are retained when no correction was required, whereas the corrected Step 11 arrays are selected automatically when Step 11 identified a threshold crossing.

4.12.1 Initialization and helper functions

Both implementations import the common file-system and numerical packages used for reporting and array processing. The HPC implementation additionally imports `posixpath` for remote path construction:

```
import os
import numpy as np
```

In the HPC implementation:

```
import posixpath

print("\n===== STEP 12: FINAL RESULTS REPORTING, DIAGNOSTICS, AND EXPORT
=====\\n")
```

The energy-conversion constants used throughout the final report are then defined:

```

hartree_to_cm = float(
    globals().get(
        "hartree_to_cm",
        219474.6313705
    )
)

hartree_to_kjmol = 2625.499638
cm_to_kjmol = 0.01196266

```

The Hartree-to-wavenumber conversion is given by Equation S305:

$$1 E_h = 219474.6313705 \text{ cm}^{-1} \quad (\text{S305})$$

The second conversion is given in Equation S306:

$$1 E_h = 2625.499638 \text{ kJ mol}^{-1} \quad (\text{S306})$$

An additional wavenumber-to-energy conversion factor is defined, although the final energy blocks in this implementation primarily convert directly from Hartree.

Several general-purpose helper functions are constructed. The section-formatting function generates standardized console headings:

```

def section(title):
    line = (
        "\n"
        + "=" * 72
        + f"\n {title}\n"
        + "=" * 72
        + "\n"
    )
    print(line)
    return line

```

The scalar-formatting helper safely handles missing, nonfinite, and nonnumeric quantities:

```

def fmt_scalar(x, digits=12):
    try:
        if x is None:
            return "None"

        x = float(x)

        if not np.isfinite(x):
            return "nan"

        return f"{x:.{digits}g}"

    except Exception:
        return str(x)

```

This function prevents the final report from failing when an optional quantity is unavailable and ensures consistent numerical formatting.

Two additional routines retrieve arrays or scalar values from the global workspace using ordered lists of possible variable names:

```
def as_array_or_none(*names):
    for name in names:
        if name in globals():
            arr = np.asarray(
                globals()[name],
                dtype=float
            ).reshape(-1)

            if arr.size > 0:
                return arr, name

    return None, None
```

A corresponding scalar helper is defined as:

```
def scalar_or_none(*names):
    for name in names:
        if name in globals():
            try:
                return (
                    float(globals()[name]),
                    name
                )
            except Exception:
                pass

    return None, None
```

This design provides compatibility among different code versions that may use alternative variable names for the same physical quantity. The selected source name is returned together with the numerical value and later written explicitly to the final report.

The energy-formatting helper converts a Hartree-valued quantity into Hartree, wavenumber, and kJ mol⁻¹ representations:

```
def energy_block(label, value_Eh):
    if (
        value_Eh is None
        or not np.isfinite(value_Eh)
    ):
        return [
            f"{label}_hartree = nan",
            f"{label}_cm1 = nan",
            f"{label}_kJmol = nan",
        ]

    return [
        f"{label}_hartree = "
        f"{value_Eh:.15f}",
```

```

f"{label}_cm1 = "
f"{value_Eh * hartree_to_cm:.8f}",

f"{label}_kJmol = "
f"{value_Eh * hartree_to_kjmol:.8f}",
]

```

For a quantity E expressed in Hartree, the wavenumber conversion is given by Equation S307:

$$E_{\text{cm}^{-1}} = E_{E_h} C_{E_h \rightarrow \text{cm}^{-1}} \quad (\text{S307})$$

The second Hartree-based relation is given in Equation S308:

$$E_{\text{kJ mol}^{-1}} = E_{E_h} C_{E_h \rightarrow \text{kJ mol}^{-1}} \quad (\text{S308})$$

The NAST-array formatting helper divides long arrays into multiple namelist-compatible lines:

```

def format_namelist_array(
    name,
    values,
    per_line=10
):
    values = np.asarray(
        values,
        dtype=float
    ).reshape(-1)

    chunks = []

    for i in range(
        0,
        len(values),
        per_line
    ):
        chunk = ", ".join(
            f"{x:.8g}"
            for x in values[
                i:i + per_line
            ]
        )

        if i == 0:
            chunks.append(
                f"{name} = {chunk}"
                + (
                    ", "
                    if i + per_line
                    < len(values)
                    else ""
                )
            )
        else:
            chunks.append(
                f"    {chunk}"
            )

```



```

        + (
            " "
            if i + per_line
            < len(values)
            else ""
        )
    )

```

```

return "\n".join(chunks)

```

The remaining helpers write text files and simultaneously print and retain console messages:

```

def write_text(path, text):
    os.makedirs(
        os.path.dirname(path),
        exist_ok=True
    )

    with open(
        path,
        "w",
        encoding="utf-8",
        newline="\n"
    ) as f:
        f.write(text)
def print_and_store(
    lines,
    text=""
):
    print(text)
    lines.append(str(text))

```

The latter is important because the canonical table and interactive diagnostic output are later embedded directly in the final report.

4.12.2 Automatic selection of the active weak coupling results

Step 12 determines whether the original clipped weak coupling results from Steps 8–10 or the threshold-corrected results from Step 11 must be used:

```

use_threshold_corrected_WC = (
    "WC_threshold_correction_needed" in globals()
    and WC_threshold_correction_needed is True
    and "CORRECTED_P_WC_effective" in globals()
    and "CORRECTED_k_WC_effective_micro_s_inv" in globals()
    and "CORRECTED_N_WC_effective" in globals()
)

```

The correction is activated only when all of the following conditions are satisfied:

1. the Step 11 correction flag exists;

2. that flag is explicitly True;
3. the corrected weak coupling probability is available;
4. the corrected weak coupling microcanonical rate is available; and
5. the corrected weak coupling cumulative number of states is available.

When these conditions are met, the active weak coupling quantities are assigned from the Step 11 arrays:

```
if use_threshold_corrected_WC:
    active_probWC = np.asarray(
        CORRECTED_P_WC_effective,
        dtype=float
    ).reshape(-1)
    active_rateWC = np.asarray(
        CORRECTED_k_WC_effective_micro_s_inv,
        dtype=float
    ).reshape(-1)
    active_nosWC = np.asarray(
        CORRECTED_N_WC_effective,
        dtype=float
    ).reshape(-1)
    active_WC_label = "Threshold-corrected WC"
```

Otherwise, the original weak coupling quantities are retained:

```
else:
    active_probWC = np.asarray(effective_WC_probability, dtype=float).reshape(-1)
    active_rateWC = np.asarray(effective_WC_rate, dtype=float).reshape(-1)
    active_nosWC = np.asarray(effective_WC_cumulative_states, dtype=float).reshape(-1)
    active_WC_label = "Original clipped WC"
```

This conditional assignment ensures that all subsequent reporting and diagnostics use a single internally consistent weak coupling probability, cumulative-state array, and microcanonical rate.

The selected source is immediately recorded:

```
step12_console_lines = []
debug_records = []

print_and_store(
    step12_console_lines,
    f"WC source: {active_WC_label}"
)
```

The original Step 8–10 arrays are not overwritten. The active variables serve only as aliases used for final reporting and diagnostics.

4.12.3 Reporting of canonical rate constants

Step 12 first verifies that the canonical temperature grid and both effective canonical rate arrays are available:

```
required_canonical_vars = [  
    "T_grid_K",  
    "k_LZ_canonical_s_inv",  
    "k_WC_canonical_s_inv",  
]  
  
missing_canonical = [  
    v  
    for v in required_canonical_vars  
    if v not in globals()  
]
```

If one or more quantities are missing, a warning is stored and canonical weak coupling reporting is disabled:

```
if missing_canonical:  
    print_and_store(  
        step12_console_lines,  
        "\nWARNING: Canonical Step 10 "  
        "variables not found."  
    )  
  
    print_and_store(  
        step12_console_lines,  
        f"Missing: {missing_canonical}"  
    )  
  
    active_k_WC_canonical = None
```

When the canonical arrays are available, Step 12 selects the corrected canonical weak coupling rate whenever Step 11 was activated:

```
else:  
    if (  
        use_threshold_corrected_WC  
        and  
        "CORRECTED_k_WC_canonical_s_inv"  
        in globals()  
    ):  
        active_k_WC_canonical = (  
            CORRECTED_k_WC_canonical_s_inv  
        )  
    else:  
        active_k_WC_canonical = (  
            k_WC_canonical_s_inv  
        )
```

The Landau-Zener rate always remains the Step 10 result because Step 11 corrects only the weak coupling treatment.

The canonical results are then printed and stored in a formatted table:

```
for T, k_lz, k_wc in zip(
    T_grid_K,
    k_LZ_canonical_s_inv,
    active_k_WC_canonical
):
    log_lz = (
        np.log10(k_lz)
        if k_lz > 0
        else np.nan
    )

    log_wc = (
        np.log10(k_wc)
        if k_wc > 0
        else np.nan
    )
```

For each temperature, the table reports the temperature, 1000/T, the Landau-Zener and active weak coupling canonical rate constants, and their base-10 logarithms.

The output formatting is implemented as:

```
line = (
    f"{T:8.2f} "
    f"{1000.0/T:10.6f} "
    f"{k_lz:18.10e} "
    f"{log_lz:14.8f} "
    f"{k_wc:18.10e} "
    f"{log_wc:14.8f}"
)
```

Logarithmic values are reported only for positive rate constants. Zero or negative values are represented as nan, avoiding undefined logarithmic operations.

4.12.4 Interactive microcanonical diagnostic

Step 12 next verifies the availability of the quantities required for energy-resolved microcanonical analysis:

```

required_micro_vars = [
    "total_energy_grid_cm1",
    "energy_step_cm1",
    "crossing_energy_bin_index",
    "E_MECP",
    "reactant_state_density",
    "crossing_shifted_state_density",
    "crossing_internal_state_density",
    "effective_LZ_probability",
    "effective_LZ_cumulative_states",
    "effective_LZ_rate",
]

```

If any of these variables are missing, execution stops:

```

missing_micro = [
    v
    for v in required_micro_vars
    if v not in globals()
]

if missing_micro:
    raise RuntimeError(
        "Missing microcanonical variables: "
        f"{missing_micro}"
    )

```

The user is then prompted to enter one or more total energies relative to the reference minimum:

```

while True:
    answer = input(
        "Energy in cm-1, or 'stop': "
    ).strip()

```

The loop terminates when the user enters stop, s, q, quit, or exit.

For a numerical requested energy, the corresponding discrete-grid index is obtained as:

```

energy_index = int(
    round(requested_energy_cm1 / energy_step_cm1)
)

```

Equivalently, the mathematical expression is given in Equation S309:

$$i_{\text{dbg}} = \text{round}\left(\frac{E_{\text{requested}}}{\Delta E}\right) \quad (\text{S309})$$

The diagnostic index in Equation S309 is stored in the code variable:

```
energy_index
```

The requested energy is therefore mapped to the nearest available energy bin according to Equation S310:

$$E_{\text{bin}} = i_{\text{dbg}} \Delta E \quad (\text{S310})$$

The program checks that the selected index lies within the available energy range. If not, the allowed interval is reported.

The transition-state internal density of states is selected using an index measured relative to the MECP threshold:

```
internal_crossing_index = (  
    energy_index - crossing_energy_bin_index  
)  
if (  
    internal_crossing_index > 0  
    and internal_crossing_index < crossing_internal_state_density.size  
):  
    crossing_internal_density_value = float(  
        crossing_internal_state_density[internal_crossing_index]  
    )  
else: crossing_internal_density_value = np.nan
```

Thus, the internal transition-state density is evaluated using the energy measured above the crossing threshold according to Equation S311:

$$E_{\text{internal}} = E_{\text{bin}} - E_{\text{MECP}} \quad (\text{S311})$$

When the selected energy does not correspond to a valid internal crossing-state bin, the internal crossing density is reported as nan.

A complete diagnostic record is then constructed using the current code variables:

```

record = {
  "requested_energy_cm1": requested_energy_cm1,
  "grid_energy_cm1": float(total_energy_grid_cm1[energy_index]),
  "energy_index": int(energy_index),
  "crossing_energy_bin_index": crossing_energy_bin_index,
  "E_MECP_cm1": float(E_MECP),
  "WC_source": active_WC_label,
  "reactant_state_density": float(reactant_state_density[energy_index]),
  "crossing_shifted_state_density": float(crossing_shifted_state_density[energy_index]),
  "crossing_internal_state_density": crossing_internal_density_value,
  "P_LZ": float(effective_LZ_probability[energy_index]),
  "P_WC": float(active_probWC[energy_index]),
  "N_LZ": float(effective_LZ_cumulative_states[energy_index]),
  "N_WC": float(active_nosWC[energy_index]),
  "k_LZ_s_inv": float(effective_LZ_rate[energy_index]),
  "k_WC_s_inv": float(active_rateWC[energy_index]),
  "log10_k_LZ": float(np.log10(effective_LZ_rate[energy_index]))
  if effective_LZ_rate[energy_index] > 0.0 else np.nan,
  "log10_k_WC": float(np.log10(active_rateWC[energy_index]))
  if active_rateWC[energy_index] > 0.0 else np.nan,
}

```

This record includes the selected energy, energy-bin index, MECP position, reactant and crossing-state densities, effective probabilities, cumulative numbers of states, microcanonical rates, and logarithmic rates.

When the threshold correction is active and the raw probability is available, the uncorrected raw value is additionally included:

```

if (
  use_threshold_corrected_WC
  and "P_WC_raw_effective" in globals()
):
  raw_wc = np.asarray(P_WC_raw_effective, dtype=float).reshape(-1)
  if raw_wc.size == expected_grid_size:
    record["P_WC_raw"] = float(raw_wc[energy_index])

```

The diagnostic output is divided into five conceptual blocks:

1. energy-grid information;
2. density-of-states information;
3. effective probabilities;
4. effective numbers of states;
5. microcanonical rates.

The printed result therefore provides an energy-specific comparison of the effective LZ/WC probabilities, cumulative numbers of states, and microcanonical rates.

Every selected-energy record is retained so that all user-requested diagnostics can be reproduced later in the final output file.

4.12.5 Construction of the final output directory

After the interactive diagnostic has been completed, Step 12 creates the final output directory:

```
final_output_dir = os.path.join(
    local_base,
    "final_output_and_interoperability_exports"
)

os.makedirs(
    final_output_dir,
    exist_ok=True
)
```

The Local PC and HPC implementations differ only in how the corresponding remote-output path is handled.

In the Local PC implementation:

```
remote_final_output_dir = final_output_dir
```

In the HPC implementation, a separate remote directory is constructed when a remote base path is available:

```
remote_final_output_dir = None
if "remote_base" in globals():
    remote_final_output_dir = posixpath.join(
        remote_base,
        "final_output_and_interoperability_exports"
    )
if "remote_mkdir_p" in globals() and "sftp" in globals():
    try: remote_mkdir_p(sftp, remote_final_output_dir)
    except Exception: remote_final_output_dir = None
```

In the HPC implementation, failure to create the remote directory does not prevent local file generation.

This directory becomes the principal folder for the final SPINKS deliverables and contains

- the complete final output report;
- the electronic-barrier NAST interoperability input;
- the ZPE-corrected-barrier NAST interoperability input.

4.12.6 Collection of final physical quantities

Step 12 gathers the physical quantities required for the final report and NAST input. For each quantity, several possible variable names are searched in a fixed priority order.

Reactant and MECP frequencies:

```
reference_frequencies, reference_frequencies_source = (  
    as_array_or_none(  
        "reference_vibrational_frequencies_cm1",  
        "freq_reactant_real_cm1"  
    )  
)  
crossing_frequencies, crossing_frequencies_source = (  
    as_array_or_none(  
        "freq_MECP_effhess_real_cm1",  
        "freq_MECP_eff_real_cm1"  
    )  
)
```

The selected reactant and effective-Hessian MECP frequency arrays are retained together with their source names for traceability.

Rotational or inertial quantities:

```
reference_inertias, reference_inertias_source = (  
    as_array_or_none(  
        "reference_principal_inertias",  
        "Reference_moments_of_inertia_amu_bohr2"  
    )  
)  
crossing_inertias, crossing_inertias_source = (  
    as_array_or_none(  
        "crossing_principal_inertias",  
        "MECP_moments_of_inertia_amu_bohr2"  
    )  
)
```

These arrays contain the reactant and crossing-state principal inertias selected from the current workflow variables. The exact source name is retained for traceability.

Reduced mass, spin-orbit coupling, and gradient quantities:

```
redmass, redmass_source = (  
    scalar_or_none(  
        "reduced_mass_amu"  
    )  
)  
soc_eff, soc_source = (  
    scalar_or_none(  
        "H_SO_cm"  
    )  
)
```

```

)
gradient_gap, gradient_gap_source = (
    scalar_or_none(
        "GRADIENT_GAP_NORM_EH_PER_BOHR",
        "gradient_gap_magnitude"
    )
)

projected_gradient_mean_value, projected_gradient_mean_source = (
    scalar_or_none(
        "PROJECTED_GRADIENT_MEAN_EH_PER_BOHR",
        "projected_gradient_mean_Eh_per_Bohr",
        "projected_gradient_mean"
    )
)

```

Step 12 preferentially selects the finalized gradient quantities used by the probability and rate calculations:

```

GRADIENT_GAP_NORM_EH_PER_BOHR
PROJECTED_GRADIENT_MEAN_EH_PER_BOHR

```

when available. This ensures that the same validated values used in the probability and rate calculations are reported and passed to NAST.

Absolute electronic energies and electronic barrier:

```

Ele_R_abs_Eh, Ele_R_source = (
    scalar_or_none(
        "Ele_REF_hartree",
        "E_ref_hartree"
    )
)

Ele_X_abs_Eh, Ele_X_source = (
    scalar_or_none(
        "Ele_MECP_hartree"
    )
)

```

The preferred electronic barrier is obtained directly from:

```

Ele_barrier_Eh, \
Ele_barrier_source = (
    scalar_or_none(
        "E_MECP_electronic_hartree"
    )
)

```

If this quantity is unavailable, it is reconstructed from the absolute electronic energies:

```

if (
  Ele_barrier_Eh is None
  and
  Ele_R_abs_Eh is not None
  and
  Ele_X_abs_Eh is not None
):
  Ele_barrier_Eh = (
    Ele_X_abs_Eh
    - Ele_R_abs_Eh
  )

```

Therefore, Equation S312 follows:

$$V_{\text{MECP}}^{\text{elec}} = E_{\text{MECP}}^{\text{elec}} - E_R^{\text{elec}} \quad (\text{S312})$$

Reactant and MECP zero-point energies

The reactant ZPE is searched using several possible source variables:

```

ZPE_R_Eh, ZPE_R_source = (
  scalar_or_none(
    "ZPE_REF_freq_hartree",
    "ZPE_REF_from_freq_hartree",
    "ZPE_REF_hartree",
    "ZPE_REF_thermo_hartree"
  )
)

```

The MECP ZPE is similarly selected:

```

ZPE_X_Eh, ZPE_X_source = (
  scalar_or_none(
    "ZPE_MECP_effhess_hartree",
    "ZPE_X_hartree",
    "ZPE_MECP_hartree",
    "ZPE_MECP_from_freq_hartree"
  )
)

```

If the MECP ZPE is unavailable but the MECP frequencies are present, it is reconstructed as:

```

ZPE_X_Eh = (
  0.5
  * float(np.sum(crossing_frequencies))
  / hartree_to_cm
)

```

The corresponding expression is given in Equation S313:

$$\text{ZPE}_X = \frac{1}{2} \sum_i hc \nu_{X,i} \quad (\text{S313})$$

The equivalent Hartree expression is given by Equation S314:

$$\text{ZPE}_X(E_h) = \frac{\frac{1}{2} \sum_i v_{X,i}}{c_{E_h \rightarrow \text{cm}^{-1}}} \quad (\text{S314})$$

The ZPE-corrected barrier is then evaluated:

```
ZPE_corrected_barrier_Eh = None

if (
    Ele_barrier_Eh is not None
    and
    ZPE_R_Eh is not None
    and
    ZPE_X_Eh is not None
):
    ZPE_corrected_barrier_Eh = (
        Ele_barrier_Eh
        + (
            ZPE_X_Eh
            - ZPE_R_Eh
        )
    )
```

Thus, Equation S315 follows:

$$V_{\text{MECP}}^0 = V_{\text{MECP}}^{\text{elec}} + \text{ZPE}_X - \text{ZPE}_R \quad (\text{S315})$$

NAST temperature and energy-grid parameters

The first and last temperatures of the Step 10 grid are assigned as the NAST temperature limits:

```
export_temperature_min_K = float(
    np.asarray(
        T_grid_K,
        dtype=float
    )[0]
) if "T_grid_K" in globals() else 300.0
export_temperature_max_K = float(
    np.asarray(
        T_grid_K,
        dtype=float
    )[-1]
) if "T_grid_K" in globals() else 350.0
```

The energy spacing is obtained from:

```
export_energy_step_cm1 = float(
    globals().get(
        "energy_step_cm1",
        1.0
    )
)
```

The maximum total energy is taken from the Step 7 maximum-energy value. The code explicitly requires this quantity:

```

if "E_max_cm1" not in globals():
    raise RuntimeError(
        "Cannot determine the interoperability-export "
        "maximum bin because E_max_cm1 from Step 7 is missing."
    )

```

The number of NAST energy bins is then calculated as:

```

export_maximum_bin_index = int(
    np.ceil(
        float(E_max_cm1)
        / export_energy_step_cm1
    )
)

```

Equivalently, the expression can be written as Equation S316:

$$\text{maxn} = \left\lceil \frac{E_{\text{max}}}{\Delta E} \right\rceil \quad (\text{S316})$$

The actual maximum energy represented by the integer NAST grid is:

```

export_maximum_energy_cm1 = (
    export_maximum_bin_index
    * export_energy_step_cm1
)

```

This gives Equation S317:

$$E_{\text{max}} = \text{maxn} \times \Delta E \quad (\text{S317})$$

4.12.7 Determination of global probability and rate extrema

Before constructing the final report, Step 12 defines a helper that extracts the minimum and maximum finite values from an array:

```

def finite_min_max(values):
    if values is None:
        return np.nan, np.nan

    arr = np.asarray(
        values,
        dtype=float
    ).reshape(-1)

    finite = arr[
        np.isfinite(arr)
    ]

    if finite.size == 0:
        return np.nan, np.nan

    return (

```

```

float(np.min(finite)),
float(np.max(finite))
)

```

This function ignores NaN and infinite values and prevents invalid entries from distorting the reported extrema.

The effective Landau-Zener probability extrema are obtained from:

```

P_LZ_min, P_LZ_max = finite_min_max(
    globals().get("effective_LZ_probability", None)
)

```

The raw weak coupling probability is selected preferentially from the unclipped Step 11 array:

```

P_WC_raw_array = globals().get(
    "P_WC_raw_effective",
    None
)

```

If the raw Step 11 weak coupling array is unavailable, the original effective weak coupling probability from Step 8 is used as the fallback source:

```

if P_WC_raw_array is None:
    P_WC_raw_array = (
        globals().get(
            "effective_WC_probability",
            None
        )
    )

    P_WC_raw_source = "effective_WC_probability"
else:
    P_WC_raw_source = "P_WC_raw_effective"

```

This distinction is important because, when Step 11 has been executed, the report can separately show the raw weak coupling probability $P_{WC}^{raw}(E)$ and the final downstream probability $P_{WC}^{final}(E)$.

The final weak coupling extrema are always calculated from the active downstream probability:

```

P_WC_final_min, \
P_WC_final_max = (
    finite_min_max(
        active_probWC
    )
)

```

The report source is explicitly identified:

```

if use_threshold_corrected_WC:
    P_WC_final_source = (
        "Threshold-corrected WC "
        "probability from Step 11"
    )
else:
    P_WC_final_source = (
        "Original clipped WC probability; "
        "no Step 11 correction applied"
    )

```

The Landau-Zener microcanonical extrema are calculated from:

```
effective_LZ_rate
```

The weak coupling microcanonical extrema are calculated from:

```
active_rateWC
```

The Landau-Zener canonical extrema are calculated from:

```
k_LZ_canonical_s_inv
```

The weak coupling canonical extrema are calculated from:

```
active_k_WC_canonical
```

Consequently, the final output reports the global minimum and maximum values of P_{LZ} , P_{WC}^{raw} , P_{WC}^{final} , $k_{LZ}(E)$, $k_{WC}(E)$, $k_{LZ}(T)$, and $k_{WC}(T)$.

4.12.8 Collection or reconstruction of intermediate spin–orbit couplings

Step 12 compiles the intermediate spin–orbit couplings introduced in Step 8 for inclusion in the final report:

```
intermediate_soc_records = []
```

The preferred source is the intermediate SOC dictionary constructed in Step 8:

```

if (
    "P_LZ_intermediate" in globals()
    and isinstance(P_LZ_intermediate, dict)
    and P_LZ_intermediate
):
    for label, data in sorted(
        P_LZ_intermediate.items(),
        key=lambda item: float(item[1].get("abs_Ms_low", np.inf))
    ):
        intermediate_soc_records.append({
            "label": str(label),
            "abs_Ms_low": float(data["abs_Ms_low"]),
            "degeneracy": int(data.get("degeneracy", 1)),
            "H_int_cm1": float(data["H_int_cm1"]),

```

```

"H_rows_cm1": np.asarray(
    data.get("H_rows_cm1", []), dtype=float
).reshape(-1),
"source": "grouped SOC row norms from Step 8",
})

```

If the Step 8 intermediate dictionary is unavailable, Step 12 reconstructs the intermediate SOC_s from the M_s-resolved SOC matrix:

```

elif (
    "SOC_Ms_matrix_cm1" in globals()
    and "Ms_low" in globals()
):
    soc_matrix_for_intermediate = np.asarray(SOC_Ms_matrix_cm1, dtype=complex)
    ms_low_for_intermediate = np.asarray(Ms_low, dtype=float).reshape(-1)
    if soc_matrix_for_intermediate.shape[0] != len(ms_low_for_intermediate):
        raise RuntimeError(
            "Cannot construct intermediate SOC values because the number of SOC-
            matrix rows does not match the number of low-spin Ms values."
        )

    unique_abs_ms = sorted(
        set(round(abs(float(ms)), 8) for ms in ms_low_for_intermediate)
    )
    for abs_ms in unique_abs_ms:
        row_indices = [
            i for i, ms in enumerate(ms_low_for_intermediate)
            if abs(abs(float(ms)) - abs_ms) < 1.0e-8
        ]
        row_soc_values = []
        for i in row_indices:
            row = soc_matrix_for_intermediate[i, :]
            row_soc = float(np.sqrt(np.sum(np.abs(row) ** 2)))

```

For each |M_s| group, the norm of each SOC row is evaluated according to Equation S318:

$$H_i^{\text{row}} = \sqrt{(\sum_j |H_{ij}|^2)} \quad (\text{S318})$$

The intermediate SOC is then obtained as the root-mean-square of the equivalent +M_s and –M_s row norms:

```

H_int_cm1 = float(
    np.sqrt(np.mean(row_soc_values ** 2))
)

```

Thus, Equation S319 follows:

$$H_{\text{int}} = \sqrt{[(1/n) \sum_{i=1}^n (H_i^{\text{row}})^2]} \quad (\text{S319})$$

For a pair of symmetry-equivalent rows, the intermediate SOC is given by Equation S320:

$$H_{\text{int}} = \sqrt{\{[(H_{+M_s}^{\text{row}})^2 + (H_{-M_s}^{\text{row}})^2]/2\}} \quad (\text{S320})$$

This fallback reproduces the same intermediate-SOC definition used in Step 8 and ensures that the final report can still contain intermediate couplings even when the previously constructed dictionary is unavailable.

4.12.9 Construction of the final SPINKS report

The main final report begins with the project identity and output directory:

```
report.append(
    "FINAL SPINKS OUTPUT"
)

report.append(
    "=" * 72
)

report.append(
    f"jobname = {jobname}"
)

report.append(
    "final_output_dir = "
    f"{final_output_dir}"
)
```

The report is then divided into several scientific sections.

All M_s -specific SOC channels

When available, every complex SOC matrix element is written:

```
for key, value in H_SO_channels_cm1.items():
    channel_label = format_soc_channel_key_for_report(key)
    report.append(
        f"{channel_label:45s} = "
        f"{value.real: .10f} {value.imag:+.10f}i cm^-1 ; "
        f"H_SO = {abs(value):.10f}"
    )
```

Each channel entry therefore includes the formatted spin-projection label, the real and imaginary SOC components, and the SOC magnitude.

If no channel-resolved matrix is available, the report explicitly states this.

Classified SOC matrix components

The classified components from earlier SOC analysis are reported separately:

```
if (
  "H_SO_intermediate_components_cm1"
  in globals()
  and
  H_SO_intermediate_components_cm1
);
```

These quantities are individual complex SOC matrix elements classified according to their spin-projection relationships. They are distinct from the intermediate SOC values, which are subsequently constructed from row norms and RMS combinations.

Intermediate SOC values

The report explains that intermediate SOC values are calculated separately for each unique $|M_S, \text{low}|$ value.

```
report.append(
  "Intermediate SOC values are "
  "calculated separately for "
  "each unique  $|M_S, \text{low}|$  value "
)
```

For each intermediate coupling, the report includes $|M_S, \text{low}|$, the number of equivalent low-spin rows, the intermediate SOC, the contributing row SOC values, and the source.

If no intermediate SOC values exist, the report notes that the workflow may be operating in effective-only SOC mode.

Effective SOC

The effective spin-orbit coupling is reported with its selected source:

```
report.append(
  "H_SO_eff = {fmt_scalar(soc_eff)} cm-1 ; source = {soc_source}"
)
```

When available, the report additionally includes the following ORCA effective-SOC variable:

```
H_SO_ORCA_effective_cm
```

The corresponding source identifier is stored in:

```
H_SO_source
```

when available. This permits direct comparison between the final effective SOC used by SPINKS and the value obtained directly from the ORCA analysis.

Gradient quantities

The two gradient quantities used by the probability expressions are reported explicitly:

```

report.append(
    "the norm of the gradient parallel to the reaction coordinate / "
    "GRADIENT_GAP_NORM_EH_PER_BOHR = "
    f"{fmt_scalar(gradient_gap)} Eh/Bohr ; "
    f"source = {gradient_gap_source}"
)
report.append(
    "mean gradient / PROJECTED_GRADIENT_MEAN_EH_PER_BOHR = "
    f"{fmt_scalar(projected_gradient_mean_value)} Eh/Bohr ; "
    f"source = {projected_gradient_mean_source}"
)

```

Thus, the final report distinguishes the gradient-difference magnitude, Δg , from the projected mean gradient, $\bar{g}$.

The exact source variable is preserved, demonstrating which finalized quantity was passed to the NAST input.

Rotational data, reduced mass, and frequencies

The report records both reactant and MECP rotational or inertial arrays, the reduced mass, and the complete frequency arrays.

For the frequencies, both the source and number of modes are reported:

```

report.append(
    f"reference frequency count = "
    f"{0 if reference_frequencies is None else len(reference_frequencies)}"
)
report.append(
    f"crossing frequency count = "
    f"{0 if crossing_frequencies is None else len(crossing_frequencies)}"
)

```

This provides a direct consistency check of the reactant and crossing-state vibrational degrees of freedom.

Energy quantities

The following energy blocks are generated:

```

Electronic_energy_reactant_absolute
Electronic_energy_MECP_absolute
Electronic_barrier
ZPE_reactant
ZPE_MECP
ZPE_corrected_barrier

```

Each energy block is reported in Hartree, cm⁻¹, and kJ mol⁻¹.

The report also records the source of the electronic barrier, reactant ZPE, and MECP ZPE.

Global probability and rate extrema

The final report contains a dedicated section summarizing the global minimum and maximum values of all principal effective probabilities and rates.

The weak coupling probability is divided into two distinct entries:

1. raw weak coupling probability;
2. final weak coupling probability used downstream.

This distinction allows the final output to report, for example, the quantity defined by Equation S321:

$$\max P_{WC}^{\text{raw}} > 1 \quad (\text{S321})$$

while simultaneously showing whether the final weak coupling probability is the original clipped array or the threshold-corrected Step 11 array.

The report also identifies whether the weak coupling microcanonical and canonical extrema were obtained from the original clipped or threshold-corrected results.

Canonical and interactive output

All stored console messages are appended to the report:

```
report.extend(  
    step12_console_lines  
)
```

Thus, the final file contains the complete canonical rate table and all interactive energy diagnostics printed during execution.

Every selected-energy record is additionally written as a key-value block:

```
for rec in debug_records:  
    for key, value in rec.items():  
        report.append(  
            f"{key} = {value}"  
        )
```

This preserves the exact numerical data independently of the formatted console representation.

The complete report is finally assembled and written:

```

final_report_text = (
    "\n".join(report)
    + "\n"
)

final_report_file = os.path.join(
    final_output_dir,
    "FINAL_OUTPUT.txt"
)

write_text(
    final_report_file,
    final_report_text
)

```

4.12.10 Validation of quantities required for NAST input

Before constructing the NAST files, Step 12 verifies that every required physical quantity is available:

```

if reference_frequencies is None:
    raise RuntimeError(
        "Cannot write the NAST interoperability input: "
        "reference frequencies were not found."
    )

```

Equivalent checks are performed for the crossing frequencies, reactant and crossing inertias, reduced mass, effective SOC, gradient-gap magnitude, projected mean gradient, electronic MECP barrier, and ZPE-corrected MECP barrier.

These checks prevent incomplete or physically inconsistent NAST input files from being written.

4.12.11 Construction of the NAST input template

The complete NAST input is generated by:

```

def build_nast_interoperability_input(
    zpe_value,
    crossing_energy_hartree
):

```

The function produces three namelist sections:

```

&keys
&inputdata
&probability

```

NAST key block:

```
&keys
zpe = {int(zpe_value)}
printmore = .true.
&end
```

The zpe flag determines whether NAST applies its own zero-point-energy treatment. The printmore flag requests detailed NAST output.

Frequency and rotational data

The reactant and MECP frequency arrays are inserted as:

```
{format_namelist_array(
  "freR",
  reference_frequencies
)}

{format_namelist_array(
  "freX",
  crossing_frequencies
)}
```

The rotational or inertial quantities are similarly included:

```
{format_namelist_array(
  "inertR",
  reference_inertias
)}

{format_namelist_array(
  "inertX",
  crossing_inertias
)}
```

These arrays define the rovibrational properties used by NAST to construct reactant densities of states and crossing-state sums of states.

Energy reference and grid

The reactant energy is set to zero:

```
enR = 0.0
```

The crossing energy is supplied through:

```
enX = {crossing_energy_hartree:.15f}
```

The energy grid is defined by:

```
maxn = {export_maximum_bin_index}
Estep = {export_energy_step_cm1:.8g}
```

and the canonical temperature interval by:

```
T1 = {export_temperature_min_K:.8g}
T2 = {export_temperature_max_K:.8g}
```

Thus, the number of points in the common energy grid is given by Equation S322:

$$E_n = n\Delta E, n = 0, \dots, \max n \quad (\text{S322})$$

Probability block

The quantities required by the NAST spin-crossing probability expressions are written as:

```
&probability
redmass = {redmass:.15f}
soc = {soc_eff:.15f}
grad = {gradient_gap:.15e}
gradmean = {projected_gradient_mean_value:.15e}
&end
```

These fields correspond to the reduced mass, effective SOC, gradient-gap magnitude, and projected mean gradient required by the NAST probability block.

The final NAST template is therefore:

```
def build_nast_interoperability_input(
    zpe_value,
    crossing_energy_hartree
):
    return f""""&keys
zpe = {int(zpe_value)}
printmore = .true.
&end

&inputdata
{format_namelist_array("freR", reference_frequencies)}

{format_namelist_array("freX", crossing_frequencies)}

{format_namelist_array("inertR", reference_inertias)}
{format_namelist_array("inertX", crossing_inertias)}

enR = 0.0
enX = {crossing_energy_hartree:.15f}

maxn = {export_maximum_bin_index}
Estep = {export_energy_step_cm1:.8g}
T1 = {export_temperature_min_K:.8g}
T2 = {export_temperature_max_K:.8g}
&end

&probability
redmass = {redmass:.15f}
soc = {soc_eff:.15f}
grad = {gradient_gap:.15e}
gradmean = {projected_gradient_mean_value:.15e}
&end
""""
```

4.12.12 Generation of the two NAST barrier conventions

Two separate NAST input files are generated.

Electronic-barrier input with *zpe* = 1:

```
nast_export_zpe1_text = build_nast_interoperability_input(  
    zpe_value=1,  
    crossing_energy_hartree=Ele_barrier_Eh  
)
```

The electronic-barrier NAST convention is summarized by Equation S323:

$$zpe = 1, enX = V_{MECP}^{elec} \quad (S323)$$

The barrier supplied to enX is the electronic barrier defined previously by Equation S312.

NAST subsequently accounts for the reactant and crossing-state ZPEs from the supplied frequency arrays.

Externally ZPE-corrected input with *zpe* = 0:

```
nast_export_zpe0_text = build_nast_interoperability_input(  
    zpe_value=0,  
    crossing_energy_hartree=ZPE_corrected_barrier_Eh  
)
```

The externally ZPE-corrected NAST convention is summarized by Equation S324:

$$zpe = 0, enX = V_{MECP}^0 \quad (S324)$$

The barrier supplied to enX is the ZPE-corrected barrier defined previously by Equation S315.

This avoids applying the zero-point-energy correction twice.

When the SPINKS implementation and the externally calculated ZPE correction use equivalent conventions, the resulting kinetic predictions should be consistent.

The two files are assigned descriptive names:

```
nast_export_zpe1_file = os.path.join(  
    final_output_dir,  
    "NAST_zpe1_electronic_barrier.inp"  
)  
  
nast_export_zpe0_file = os.path.join(  
    final_output_dir,  
    "NAST_zpe0_ZPE_corrected_barrier.inp"  
)
```

Both files are then written to disk.

4.12.13 Optional cluster upload and global export

In the HPC implementation, when a remote output directory and SFTP connection are available, Step 12 uploads:

- the final SPINKS report;
- the zpe = 1 NAST input;
- the zpe = 0 NAST input.

```
sftp.put(  
    final_report_file,  
    posixpath.join(  
        remote_final_output_dir,  
        os.path.basename(  
            final_report_file  
        )  
    )  
)  
)
```

Equivalent HPC transfer commands are used for both NAST files. Failure to upload the files produces a warning but does not invalidate the locally generated outputs.

In the Local PC implementation, no remote transfer is performed. The remote-file compatibility aliases point to the corresponding local files:

```
remote_final_report_file = final_report_file  
remote_nast_export_zpe1_file = nast_export_zpe1_file  
remote_nast_export_zpe0_file = nast_export_zpe0_file
```

Finally, both implementations export the principal Step 12 results and active weak coupling quantities to the global workspace:

```
globals().update({  
    "final_output_dir": final_output_dir,  
    "final_report_file": final_report_file,  
    "nast_interoperability_zpe1_file": nast_export_zpe1_file,  
    "nast_interoperability_zpe0_file": nast_export_zpe0_file,  
    "debug_records_step12": debug_records,  
    "ZPE_corrected_barrier_Eh_final": ZPE_corrected_barrier_Eh,  
    "Electronic_barrier_Eh_final": Ele_barrier_Eh,  
    "active_probWC": active_probWC,  
    "active_rateWC": active_rateWC,  
    "active_nosWC": active_nosWC,  
    "active_k_WC_canonical": active_k_WC_canonical,  
    "active_WC_label": active_WC_label,  
    "use_threshold_corrected_WC": use_threshold_corrected_WC,  
})
```

The Local PC implementation additionally exports its four local-path compatibility aliases:

`remote_final_output_dir`, `remote_final_report_file`, `remote_nast_export_zpe1_file`,
`remote_nast_export_zpe0_file`

These exports preserve the final report and NAST interoperability paths, interactive diagnostic records, finalized barrier energies, and the active weak coupling arrays selected for reporting.

4.12.14 Summary of the step 12 algorithm

Step 12 is the final consolidation and external-interface stage of SPINKS. The active weak coupling quantities are selected according to Equation S325:

$$(P_{WC}, N_{WC}, k_{WC})_{\text{active}} = \begin{cases} (P_{WC}, N_{WC}, k_{WC})_{\text{original}}, & \text{no Step 11 correction} \\ (P_{WC}, N_{WC}, k_{WC})_{\text{corrected}}, & \text{Step 11 correction active} \end{cases} \quad (\text{S325})$$

The selected weak coupling data are used consistently in both the canonical summary and the interactive microcanonical diagnostic.

For each user-selected energy, the nearest grid point is identified according to Equation S309.

The corresponding diagnostic reports the reactant and crossing densities, effective probabilities, cumulative numbers of states, and microcanonical rates at that energy.

The final report consolidates the complete set of physically important workflow quantities, including

- all M_s -specific complex SOC channels;
- classified SOC components;
- intermediate SOC values;
- effective SOC;
- gradient-gap magnitude;
- mean gradient;
- rotational or inertial data;
- reduced mass;
- reactant and MECP frequencies;
- absolute electronic energies;
- electronic MECP barrier;
- reactant and MECP ZPEs;

- ZPE-corrected MECP barrier;
- global minimum and maximum probabilities;
- global minimum and maximum microcanonical rates;
- global minimum and maximum canonical rates;
- canonical temperature table;
- user-selected microcanonical diagnostics.

The electronic barrier used by Step 12 is defined by Equation S312.

The ZPE-corrected barrier is defined by Equation S315.

Intermediate SOC values are preferentially imported from Step 8. When reconstruction is required, the SOC row norm is evaluated according to Equation S318 and the grouped RMS value according to Equation S319.

The NAST maximum energy-bin index is determined according to Equation S316.

The electronic-barrier NAST convention is summarized by Equation S323.

The externally ZPE-corrected NAST convention is summarized by Equation S324.

The first allows NAST to calculate the ZPE correction from the supplied frequencies, whereas the second supplies the externally calculated ZPE-corrected barrier directly and disables additional NAST ZPE treatment.

The overall Step 12 workflow can therefore be summarized as:

Results from Steps 1–11

↓

Select the active WC source: original clipped or threshold-corrected

↓

Print the canonical LZ/WC rate table

↓

Perform interactive energy selection

↓

Report densities of states, probabilities, numbers of states, and microcanonical rates

↓

Collect final SOC, gradient, mass, frequency, rotational, and energy data

↓

Determine global minimum and maximum probabilities and rates

↓

Construct FINAL_OUTPUT.txt

↓

Build NAST input 1: zpe = 1 with the electronic barrier

↓

Build NAST input 2: zpe = 0 with the ZPE-corrected barrier

↓

Save locally and, when available, upload to the cluster

Step 12 therefore converts the complete internal SPINKS data structure into a transparent and reproducible final deliverable. It preserves the provenance of every important physical quantity, records whether weak coupling correction was required, enables direct energy-resolved kinetic inspection, and produces ready-to-run NAST input files for independent validation of the SPINKS microcanonical and canonical rate calculations.

References

- (1) Neese, F. Software update: The ORCA program system, version 5.0. *WIREs Comput. Mol. Sci.* **2022**, 12, e1606.
- (2) Dergachev, V. D.; Rooein, M.; Dergachev, I. D.; Lykhin, A. O.; Mauban, R. C.; Varganov, S. A. NAST: Nonadiabatic Statistical Theory Package for Predicting Kinetics of Spin-Dependent Processes. *Top. Curr. Chem.* **2022**, 380, 15.